

A Survey of Limit-Order-Book Models, Simulators and Hardware Acceleration

Vincent Maciejewski
M2 Technologies
mayeski@gmail.com

Draft — October 4, 2026

Abstract

The limit order book (LOB) is the principal observable representation of liquidity in electronic markets. A large literature extracts predictive information from it: order-flow imbalance, queue imbalance and the micro-price; stochastic queue and first-passage models; Hawkes and neural point processes; and convolutional, recurrent, attention-based and spatial-temporal Transformer models, pretrained message-level encoders, generative models and hybrids that combine statistical structure with learned representations. A substantial part of that literature evaluates models primarily as statistical forecasters, and large independent re-tests find that many published deep models lose much of their accuracy on new data. This survey asks a different question: *how does information contained in the limit order book become an executable trading decision, and how should prediction, queue dynamics, simulation and execution be evaluated together?*

We organise the literature along four axes—representation, model class, prediction objective and economic role—and follow a single chain throughout: LOB state → representation → prediction → decision → queue → execution → P&L. Each link can destroy information or add uncertainty. We distinguish models that predict prices or events from models that estimate execution outcomes—fill probability, queue depletion, adverse selection—and argue that residual queue position and conditional execution value form the missing translation layer between LOB prediction research and trading. For the learned models we explain, from first principles, how they are built and trained, take apart meta-models and structural hybrids that combine them with classical models, and show how much reported accuracy can come from the construction of the label and the data split rather than from the network, with a small PyTorch example on a synthetic market.

The last part turns to the infrastructure that execution research depends on. We survey order-book simulators—historical replay, agent-based, model-based and generative—and compare them on the properties execution questions need: per-order queue position, explicit latency, determinism and a shared code path to live trading. The Kaspar system—a venue-independent system whose CME handlers are implemented—serves as the worked example of a deterministic, queue-exact replay in which simulated orders enter the reconstructed historical first-in-first-out queue and are filled only by recorded executions that reach an order behind them, and we show how an order-book model is added to it as an actor that publishes its predictions to a strategy. We then explain why order-book models are slow at inference—book building, sequential recurrence, attention whose cost grows with the square of the sequence length, and memory-bound evaluation at batch size one—and which stages can be accelerated on FPGAs, GPUs and other processors.

Two criteria run through the survey: *tradeability*, under which predictive performance is judged after latency, spread, fees, queue position and adverse selection; and *Occam’s razor*, under which each additional layer of complexity—model capacity and hardware—must earn incremental out-of-sample economic value. This is a survey and a methodological paper. Its

numerical examples are stylised, and its only computations are illustrations on synthetic data; it reports no new results on market data.

Keywords: limit order book, market microstructure, order-flow imbalance, micro-price, Hawkes process, queue position, first-passage time, adverse selection, deterministic simulation, deep learning, spatial-temporal Transformer, foundation models, uncertainty quantification, multiple testing, Occam’s razor, market simulation, hardware acceleration, FPGA.

Contents

I	Background and Scope	7
1	Introduction	7
1.1	The limit order book as a dynamic market state	7
1.2	Notation	11
1.3	A short probability primer	12
1.4	Prediction versus tradeability	13
1.5	Prediction, execution and simulation are different problems	14
1.6	Two fixed positions	15
1.7	Development of the field	15
1.8	Positioning and taxonomy	16
2	Survey methodology	19
2.1	Scope: which markets and which questions	20
2.2	Source selection	20
2.3	Strength of evidence for a claim	21
2.4	Conditions for comparing two LOB studies	21
2.5	A reader’s checklist	22
II	Models of the Limit Order Book	23
3	Stylised facts a model should reproduce	23
4	Order-flow imbalance and queue imbalance	24
4.1	Order-flow imbalance	24
4.2	Evidence and extensions	26
4.3	Queue imbalance and richer book features	27
4.4	The shape of the book	29
4.5	The micro-price	30
4.6	Order flow across instruments	39
5	Events rather than snapshots	40
6	Queue depletion and first-passage models	43
6.1	The Cont–Stoikov–Talreja model	45
6.1.1	The model	45
6.1.2	Estimating the model	47
6.1.3	From the whole book to two counters racing to zero	48

6.1.4	Three probabilities a trader needs	50
6.1.5	Stylised facts reproduced by the model	55
6.1.6	Limitations and descendants	56
6.2	The Cont-de Larrard model	56
6.3	The queue-reactive model	61
6.4	Summary: what the queue models tell a trader	70
7	Hawkes processes	71
7.1	Limitations of the Poisson process	72
7.2	The four ideas in pictures	74
7.3	The language of point processes	77
7.4	Two simpler models: Poisson and renewal processes	78
7.5	The simplest Hawkes model: one event type	79
7.6	The branching ratio	79
7.7	Simulated Hawkes paths	80
7.8	Telling Poisson, renewal and Hawkes processes apart	80
7.9	The multivariate model	82
7.10	Reading the branching matrix	85
7.11	From events to prices	86
7.12	Fitting Hawkes models in practice	88
7.13	Empirical findings from fitted Hawkes models	92
7.14	Near-criticality	92
7.15	Marks: events with sizes and labels	94
7.16	State-dependent Hawkes models	96
7.17	Regimes: when the parameters change over time	99
7.18	Variants of the Hawkes model	101
7.19	Neural point processes	102
7.19.1	Recurrent memory: RMTTP and the neural Hawkes process	106
7.19.2	Attention: the Transformer and self-attentive Hawkes processes	106
7.19.3	State-space memory: the Mamba Hawkes process	107
7.19.4	Learning only the kernel	108
7.19.5	Applications to financial events and order books	108
7.19.6	Benefits and costs	109
7.20	Uses of Hawkes models	111
7.20.1	Using the branching ratio	112
7.20.2	From intensities to decisions	114
7.20.3	Execution and market making	115
8	Meta-models: learning on top of the classical models	116
8.1	The architecture	117
8.2	Inputs supplied by the classical models	117
8.3	Learners used in the literature	117
8.4	State models and event models measure different things	119
8.5	Combining forecasts: methods borrowed from the general forecasting literature	120
8.5.1	Attention as a combiner	125
8.6	Published evidence	128
8.7	Training protocols in the literature	131
8.8	Summary	131

9	Structural hybrids	132
9.1	Structural hybrids versus meta-models	133
9.2	Two ways to build classical structure in	134
9.3	A structural hybrid in detail: the deep queue-reactive model	135
9.4	A prediction example: the state-dependent parallel neural Hawkes process	139
9.5	A third example: a neural Hawkes simulator for market making	142
9.6	Smaller designs for combining classical structure with a network	146
9.7	Other published hybrids and gaps in the literature	149
10	Deep learning for limit order books	151
10.1	Definition and design choices of a deep LOB model	151
10.2	Inputs and outputs of a deep LOB model	153
10.2.1	The three input formats	154
10.2.2	Inputs and the classical models that use the same information	155
10.2.3	Measured effect of the input choice	155
10.2.4	Prediction targets	156
10.2.5	The FI-2010 data set	157
10.3	The families of network	157
10.4	Training a deep LOB model: a worked example	161
10.5	Scoring, label construction, data splits and targets	167
10.6	Convolutional and recurrent models	170
10.6.1	DeepLOB in detail	171
10.6.2	Lighter models, normalisation and variants	174
10.7	Attention and Transformers for a single asset	175
10.7.1	Attention and Transformers from the beginning	175
10.7.2	Attention on the book, in detail	180
10.8	Tokenisation: how vocabularies of market events are built	185
10.9	State-space models	188
10.10	Pretrained encoders, foundation models and uncertainty	188
10.10.1	LOBERT in detail	188
10.10.2	Confidence and uncertainty	192
10.11	Multi-asset and universal models	193
10.12	Benchmark and re-test results	195
10.13	Representation versus architecture	196
10.14	Fill-probability models	198
10.15	Generative models	199
10.16	Contribution of deep learning relative to the classical models	200
III	Simulation and Hardware Acceleration	202
11	Mechanics a simulator must capture	202
11.1	Passive and aggressive execution	202
11.1.1	Signal value and execution value	203
11.2	Queue position and fill probability	204
11.2.1	Residual queue position	204
11.3	Adverse selection	204
11.3.1	Conditioning on the fill	204

11.3.2	Queue position as an information variable	205
11.4	Latency and signal decay	205
12	Order-book simulators	206
12.1	Naive backtests and queue-exact replay	206
12.1.1	Assumptions of a naive backtest	206
12.1.2	Requirements of a queue-exact replay	206
12.1.3	Limits of replay fidelity	207
12.2	Existing simulators compared	207
12.3	Kaspar HFT	212
12.3.1	Queue-exact simulated fills	213
12.3.2	Deterministic replay	215
12.3.3	Replay, paper and live trading from the same code	215
12.3.4	A model-free execution benchmark	216
12.3.5	Latency and slippage measurement	216
12.4	Adding an order-book model to Kaspar	217
13	Hardware acceleration of order-book models	219
13.1	Stages of the per-event pipeline	219
13.2	Sources of per-event latency in recurrent networks and Transformers	220
13.3	Audited benchmarks	222
13.4	Stages that need acceleration	223
13.5	Determinism as a risk-control property	223
13.6	Commercial FPGA products for trading	223
13.7	Accelerators between FPGAs and GPUs	224
13.8	Transformers on FPGAs	224
13.9	GPUs for training, simulation and single-event inference	225
13.10	Networks and distance	225
13.11	Placing workloads on hardware	225
13.12	Hardware cannot extend signal lifetime	225
14	Research agenda	226
14.1	Models	226
14.2	Execution and simulation	227
14.3	Hardware	228
15	Conclusion	228
A	Symbols and acronyms	231
A.1	Symbols	231
A.2	Acronyms	233
B	Glossary	233
B.1	Markets and trading	233
B.2	Exchanges and market data	235
B.3	Statistics and evaluation	236
B.4	Machine learning	238
B.5	Probability and stochastic processes	240
B.6	Systems and hardware	241

B.7 Economics	242
-------------------------	-----

Part I

Background and Scope

About this part. Part I gives the background the rest of the survey relies on and states its scope. It explains what a limit order book is and how it is built from exchange messages, fixes the notation and the probability used throughout, and makes the survey’s central distinction: predicting the next price move is not the same as making money from it. Section 1 covers the market, the notation and the two principles that organise the survey; Section 2 explains how the literature was chosen and how strongly each kind of claim should be believed.

1 Introduction

This section introduces the object of study and the language used to describe it. It explains how an electronic market matches buyers and sellers through a limit order book, what the book looks like as data, and how exchanges publish it; then fixes the notation and the few ideas from probability that the rest of the survey needs. It ends with the survey’s main argument—prediction, execution and simulation are different problems—and the two principles and four axes used to organise the literature.

1.1 The limit order book as a dynamic market state

Modern electronic markets operate as *continuous double auctions*: buyers and sellers may submit orders at any moment, and a trade happens whenever a buy price meets a sell price. The study of how these trading mechanisms turn orders and information into prices is called *market microstructure* (O’Hara, 1995; Harris, 2002; Hasbrouck, 2007). *Liquidity*—how easily one can trade without moving the price—is supplied by resting limit orders and consumed by aggressive orders, and the visible limit order book records the outcome at a resolution that ranges from individual messages to aggregated price levels (Gould et al., 2013; Bouchaud et al., 2018).

Limit order books match buyers and sellers in more than half of the world’s financial markets (Gould et al., 2013). In their survey, Gould et al. (2013) review the empirical regularities of order books and the models proposed to explain them, and conclude that many of those models resemble real books poorly and that several well-established facts have yet to be reproduced. The economic theory of why quotes are set where they are is older. In the model of Glosten and Milgrom (1985), a market maker who trades with a mixture of better-informed and uninformed traders quotes a positive spread between bid and ask even with no transaction costs and no expected profit: the spread is what the market maker needs to recover, from the uninformed, what it loses to the informed. The idea that order flow carries information about value is the economic reason the book is worth modelling at all: its stream of orders, cancellations and trades—thousands of events a second in active markets—explains a large part of short-horizon price changes (Cont et al., 2014), and the predictive models of this survey try to extract what it says about the next few seconds (Sirignano and Cont, 2019; Zhang et al., 2019a; Lucchese et al., 2024).

This survey is written for readers with a computer-science or electrical-engineering background and no prior finance. Market, statistical and machine-learning terms are explained where they first appear, and Appendix B collects them with references. Most examples use *futures*: standardised exchange-traded contracts to buy or sell an asset at a set future date (Hull, 2022). The running examples are the E-mini S&P 500 future (ES, on an index of 500 large US stocks), the E-mini

Nasdaq-100 future (NQ) and the 10-year US Treasury note future (ZN), all traded on CME Group’s electronic platform. Each such contract is an *instrument*.

Orders and order types. An *order* is an instruction to buy or sell a quantity of a contract. The quantity is counted in *lots* (one futures contract, or one share), and prices are restricted to multiples of the *tick*, the smallest allowed price step. The main order types are the following (Harris, 2002).

- A *limit order* says “buy (or sell) up to this many lots, at this price or better”. Whatever cannot trade immediately waits in the book. Waiting buy orders are *bids*; waiting sell orders are *asks* (or *offers*).
- A *market order* says “buy (or sell) this many lots now, at whatever price is available”. It trades immediately against waiting orders and never waits itself. A limit order priced to trade immediately (a buy at or above the best ask) behaves the same way; both are called *aggressive*, and the trader is said to *cross the spread*. A trader whose limit order waits in the book is *passive*.
- A *cancel* removes a waiting order, completely or partly; a *modify* (or *replace*) changes its price or quantity.
- *Time-in-force* qualifiers say how long an order may wait: a *day* order waits until the session ends; an *immediate-or-cancel* (IOC) order trades what it can at once and cancels the rest; a *fill-or-kill* (FOK) order trades in full at once or not at all.
- An *iceberg* order shows only part of its quantity; when the visible part trades, more appears. A fully *hidden* order shows nothing. Both trade but are partly or wholly invisible in the book.

The book is a data structure. The *limit order book* (LOB) is the set of all waiting orders for one instrument. Each side is naturally implemented as an ordered map from price to a queue of orders: bids sorted from highest price down, asks from lowest price up, with a hash map from order identifier to order so that a cancel can find its order in constant time. The *best bid* is the highest bid price, the *best ask* the lowest ask price. When an aggressive buy arrives, the exchange’s *matching engine* trades it against the best ask, then the next price up, and so on until the buy is filled or its limit price is reached; trading through several price levels is called *walking the book*. Figure 1 shows the structure.

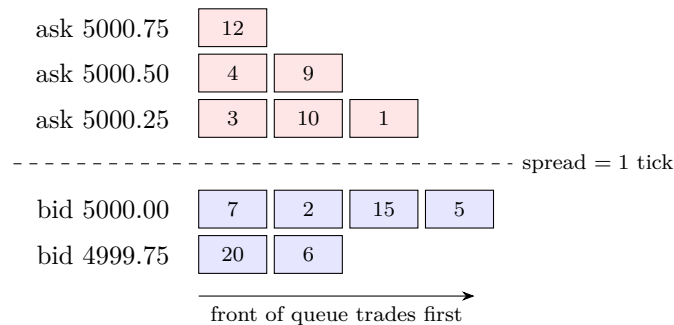


Figure 1: A small book. Each box is one waiting order and the number is its size in lots. Orders at the same price form a first-in-first-out queue, front on the left. The best bid is 5000.00 with $7 + 2 + 15 + 5 = 29$ lots; the best ask is 5000.25 with 14 lots; the spread is one tick of 0.25.

Queues and priority. Which waiting order trades first is decided by the venue’s *priority rule*. Most venues use *price-time priority*: better prices trade first, and among orders at the same price the one that arrived first trades first. Each price level is then exactly a FIFO queue. Some products use *pro-rata* allocation instead, in which an incoming order is split among the waiting orders in proportion to their sizes; at CME, for example, most futures match first-in-first-out while SOFR interest-rate futures use a pro-rata or allocation algorithm (CME Group, n.d.b). Under price-time priority an order’s place in the queue matters, and the rules for keeping it matter too: on CME Globex, decreasing an order’s quantity keeps its place, while increasing the quantity or changing the price sends it to the back of the queue (CME Group, n.d.b). The volume waiting ahead of an order is its *queue position*; much of this survey is about why that number matters.

Contents of a market-data feed. Exchanges publish the book through market-data feeds at three levels of detail. *Level 1* gives only the best bid and ask and their sizes. *Level 2*, or *market by price* (MBP), gives the total size at each of the best few price levels. *Level 3*, or *market by order* (MBO), reports every individual order: each addition, execution, cancellation and modification, with an order identifier. MBO data provide the information needed to reconstruct the displayed order-level queues, and therefore to estimate queue positions, subject to the venue’s matching rules and feed semantics; hidden liquidity and feed gaps remain outside what any feed shows.

Example: building the book from Nasdaq ITCH. Nasdaq’s TotalView-ITCH 5.0 feed is a widely used MBO protocol and a clear example of how a book is built from messages (Nasdaq, 2015). Every message is a small fixed-layout binary record with a one-character type, a timestamp in nanoseconds since midnight, and, for order messages, an 8-byte *order reference number* that identifies the order for the rest of the day. The order messages are:

Type	Name	Effect on the book
A, F	Add Order	new order (side, shares, price) joins the back of its queue
E	Order Executed	order traded; reduce its shares by the executed amount
C	Order Executed With Price	as E, at a price different from the order’s own
X	Order Cancel	partial cancel; reduce its shares
D	Order Delete	order cancelled; remove it
U	Order Replace	remove the old order; add a new one with a new reference number
P	Trade (non-cross)	an execution against a non-displayed order; no book change

Building the book is a short event loop over these messages:

```
book = {'B': SortedDict(), 'S': SortedDict()} # price -> deque of orders
orders = {} # reference number -> order

def on_message(m):
    if m.type in 'AF': # add: join the back of the queue
        o = Order(m.ref, m.side, m.price, m.shares)
        book[o.side].setdefault(o.price, deque()).append(o)
        orders[o.ref] = o
    elif m.type in 'ECX': # execution or partial cancel
        o = orders[m.ref]
        o.shares -= m.shares
        if o.shares == 0: remove(o)
    elif m.type == 'D': # full cancel
        remove(orders[m.ref])
    elif m.type == 'U': # replace: remove, re-add at back
        old = orders[m.old_ref]
```

```

        remove(old)
        on_message(Add(m.new_ref, old.side, m.price, m.shares))

def remove(o):
    q = book[o.side][o.price]
    q.remove(o)                                # O(1) with a linked list
    if not q: del book[o.side][o.price]
    del orders[o.ref]

```

Three details matter later in this survey. First, an execution message names the waiting order that traded, so the feed reveals *which* order in the queue was hit—the information needed to reconstruct queue positions (Section 12). Second, a replace carries a new reference number, so in the reconstructed book the order leaves its place and rejoins at the back. Third, hidden orders produce only P messages and never enter the book: the book a feed lets you build is the *displayed* book. ITCH is delivered over a sequenced UDP multicast protocol (MoldUDP64), so a receiver can detect lost packets by gaps in the sequence numbers. CME’s MDP 3.0 market-by-order feed, used by the Kaspar system of Section 12.3, follows the same pattern with its own binary encoding (CME Group, n.d.c).

At time t , an L -level description of the book is

$$\mathcal{L}_t = \left\{ p_{\ell,t}^b, v_{\ell,t}^b, p_{\ell,t}^a, v_{\ell,t}^a \right\}_{\ell=1}^L, \quad (1)$$

where $p_{\ell,t}^b$ and $v_{\ell,t}^b$ are the price and displayed volume at the ℓ -th best bid level ($\ell = 1$ is the best bid), and $p_{\ell,t}^a$, $v_{\ell,t}^a$ the same on the ask side.

In words: this is the market-by-price view of the book: a table with L rows per side, each row a price and the total number of lots waiting there. It can be computed from the order-level book by summing each queue. It is the state of the market, and it changes every time an order arrives, trades or is cancelled.

The mid-price and spread,

$$m_t = \frac{1}{2} \left(p_{1,t}^a + p_{1,t}^b \right), \quad s_t = p_{1,t}^a - p_{1,t}^b, \quad (2)$$

In words: the mid m_t is halfway between the best price anyone will buy at and the best price anyone will sell at; it is the usual single-number “price” of the market. The spread s_t is the gap between them: buying and immediately selling back costs one spread.

These are convenient summaries that discard most of the book: the distribution of liquidity across levels, the sequence of events that produced the current state, and the queue position of any individual order.

Throughout, numerical examples use stylised values chosen to make the arithmetic visible. They are not empirical estimates. The only computations the paper reports are illustrations on synthetic data—simulated point processes, a simulated random-walk market, and the small PyTorch example of Section 10.4; it reports no new results on market data.

Example 1.1 (Same mid, different market). Two E-mini S&P 500 (ES) books have the same mid (5000.125) and the same one-tick spread (tick = 0.25). In the first, 12 lots rest at the best ask; in the second, 900. Buying 100 lots aggressively costs several ticks more in the first book than in the second. Mid and spread do not distinguish them.

Three properties make LOB modelling unusually hard. The data are *event-driven*: orders arrive, execute and cancel at irregular times. The state is *high-dimensional* even when only a few levels

are observed. And the statistical properties are *non-stationary* across time of day, assets, volatility regimes and market-structure changes. A model that performs well for one representation, horizon or market is therefore not, by that fact, a model of order-book dynamics.

1.2 Notation

This subsection fixes the symbols used throughout, and explains what each one means for a trader watching the book. Symbols used in only one section are defined where they first appear. To make the symbols concrete, each group is followed by its value in one running example: the book of Figure 1, where at time t we decide to place a passive buy order for one lot at 5000.00.

Time and events

t is time. Book events (additions, cancellations, executions) are numbered $n = 1, 2, \dots$ and happen at times $t_1 \leq t_2 \leq \dots$. A quantity written with subscript t is its value just after everything that happened up to and including t ; $t-$ means “just before t ”. $H > 0$ is a horizon—how far ahead we look—and δ is the tick size.

What it means: the market is a stream of events, and we look at it as a sequence of snapshots taken whenever we like. H is the question “how long am I willing to wait?”

Example: $\delta = 0.25$ (one ES tick), and we look $H = 1$ s ahead.

Book

$p_{\ell,t}^b, v_{\ell,t}^b, p_{\ell,t}^a, v_{\ell,t}^a$, the mid m_t and the spread s_t are as defined in Section 1. The change in the mid over the next H seconds is $\Delta m_{t,H} = m_{t+H} - m_t$.

What it means: $\Delta m_{t,H}$ is “how much did the price go up (or down, if negative) while I waited”. Almost every prediction model in the literature targets some function of it.

Example: $p_{1,t}^b = 5000.00, v_{1,t}^b = 29, p_{1,t}^a = 5000.25, v_{1,t}^a = 14$, so $m_t = 5000.125$ and $s_t = 0.25$. If one second later the best bid and ask are 4999.75 and 5000.00, then $m_{t+H} = 4999.875$ and $\Delta m_{t,H} = -0.25$: the price fell one tick.

Information and prediction

X_t is everything observable at t that a model may use: the book, recent events, and anything derived from them. Y_{t+H} is the quantity we want to predict, which becomes known only at $t + H$; $\hat{Y}_t$ is a model’s prediction of it, made at t .

What it means: X_t is the model’s input vector and $\hat{Y}_t$ its output. The rule is that X_t may contain only what was really known at t —no peeking at the future.

Example: X_t could be the sizes of the five best levels on each side; Y_{t+H} could be “did the mid rise within one second?”, and $\hat{Y}_t = 0.6$ a model’s estimate that it will.

Actions and orders

a_t is what we do at t : place an order, keep it, cancel it, or cross the spread. d is the side of an order, $d = +1$ for a buy and $d = -1$ for a sell. For a waiting order, Q_t^{ahead} is the displayed volume ahead of it in its queue, and q stands for a particular value of Q_t^{ahead} .

What it means: d lets one formula serve both buyers and sellers, since a rising price is good for a buyer and bad for a seller. Q_t^{ahead} is “how many people are in line in front of me”.

Example: a_t = place a bid at 5000.00, $d = +1$. The new order joins the back of the 5000.00 queue, so $Q_t^{\text{ahead}} = 29$.

Fills

τ_F is how long after t the order is filled ($\tau_F = \infty$ if it never is), $t_F = t + \tau_F$ is the clock time of the fill, and p^{fill} is the price received. Fill is the event “the order was filled within the horizon”,

$\{\tau_F \leq H\}$, and $\mathbb{1}_{\text{Fill}}$ is 1 if that happened and 0 otherwise.

What it means: a passive order is a bet that someone will come and trade with it; these symbols record whether, when and at what price they did.

Example: sellers trade through the whole 5000.00 queue 0.4 s later and reach our order: $\tau_F = 0.4$ s, $p^{\text{fill}} = 5000.00$, and since $0.4 \leq 1$, $\mathbb{1}_{\text{Fill}} = 1$.

Profit

Π is the profit per lot of an order decided at t , in price points unless stated otherwise (some examples use ticks); it is defined precisely in Section 11.1. A passive order that is never filled makes $\Pi = 0$. $h > 0$ is the *mark-out horizon*: how long after a fill we wait before scoring it.

What it means: Π is the number we actually care about; every other symbol is a step on the way to it.

Example: with $h = 1$ s, if the mid one second after our fill is 4999.875, our buy at 5000.00 is worth $4999.875 - 5000.00 = -0.125$: we were filled, and then the price moved against us. This is *adverse selection* (Section 11.3), and it is why a fill is not automatically good news.

Models

M denotes a model, M_0 a baseline (usually the simplest reasonable one) and M_1 a candidate. $V(M)$ is how much money-relevant value M delivers *out of sample* (OOS)—on data that were not used to build it, as opposed to *in sample*—for example net profit after costs, or a Sharpe ratio (average return divided by the standard deviation of return (Sharpe, 1994)). $\Delta V = V(M_1) - V(M_0)$.

What it means: ΔV answers “is the new model worth it, compared with the simple one, on data it has never seen?”

Example: if a linear model earns 0.010 points per trade after costs and a neural network earns 0.012, then $\Delta V = 0.002$ points per trade; Principle 2 below asks whether that is real and whether it pays for the extra complexity.

Operators

$\mathbb{P}$ is “the probability of”, $\mathbb{E}$ is “the average of”, and a vertical bar means “given”. Section 1.3 explains these in plain terms.

1.3 A short probability primer

Most equations in this paper are statements about probabilities and averages. The few ideas needed are the following; a first course in probability covers all of them.

$\mathbb{P}(A)$, “the probability of A ”

A number between 0 and 1: the fraction of times event A would happen if the situation were repeated many times. Example: $\mathbb{P}(\Delta m_{t,H} > 0)$ is the fraction of moments after which the mid is higher H seconds later.

$\mathbb{P}(A | B)$, “the probability of A given B ”

The same fraction, but counted only over the cases where B is true. The vertical bar is read “given”. Example:

$$\mathbb{P}(\Delta m_{t,H} > 0 \mid \text{bid queue much larger than ask queue})$$

asks: of all the moments when buyers were queued much more heavily than sellers, what fraction were followed by a rise? Writing $| X_t$ means “given everything we can observe at time t ”—it is what a prediction model computes: a function from the current inputs X_t to a probability.

$\mathbb{E}[Z]$, “the expected value of Z ”

The long-run average of a random number Z . If a trade makes +2 with probability 0.6 and -1 with probability 0.4, then $\mathbb{E}[Z] = 0.6 \times 2 + 0.4 \times (-1) = 0.8$.

$\mathbb{E}[Z \mid B]$

The average of Z over only the cases where B is true.

$\mathbb{I}_A$, the indicator of A

Equal to 1 if A happens and 0 otherwise, like a Boolean cast to an integer. Its average is the probability: $\mathbb{E}[\mathbb{I}_A] = \mathbb{P}(A)$. A useful consequence, used in Section 11.1, is $\mathbb{E}[Z \mathbb{I}_A] = \mathbb{P}(A) \mathbb{E}[Z \mid A]$: the average of “ Z if A happened, else 0” equals how often A happens times the average of Z when it does.

$\inf\{u > 0 : \text{condition}\}$

“The earliest time u after now at which the condition holds”—the time a `while` loop that polls the condition would exit.

A hat, as in $\hat{Y}_t$

An estimate produced by a model, as opposed to the true value.

1.4 Prediction versus tradeability

Suppose a model estimates $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$, the probability that the mid rises over the next H . That number describes directional predictability. It does not say whether a trader can monetise it. An aggressive buy pays the offer and must overcome the half spread and fees. A passive buy avoids the spread but may never execute, and the event that makes it execute may itself carry information about the next price move. The economically relevant sequence is

$$\begin{aligned} X_t &\rightarrow \hat{Y}_t \rightarrow a_t \rightarrow \text{placement} \rightarrow \text{queue position} \\ &\rightarrow \text{execution} \rightarrow \text{realised profit and loss (P\&L)}. \end{aligned} \tag{3}$$

In words: data go into a model; the model makes a forecast; the forecast is turned into a decision (buy, sell, wait, cancel); the decision becomes an order; the order takes a place in a queue; whether and when it trades depends on that place; and only then is money made or lost. A forecast can be right and still be lost at any later arrow.

Example 1.2 (Accurate but untradeable). In ES (tick 0.25 points, one-tick spread), a classifier is right on the sign of the next mid move 55% of the time, and the typical move is half a tick (0.125 points). The expected move per trade in the predicted direction is

$$(0.55 - 0.45) \times 0.125 = 0.0125 \text{ points,}$$

while an aggressive entry costs the half spread, 0.125 points, before fees. The expected directional edge is one tenth of the half-spread cost, before fees, so it does not cover even the cost of entry. The accuracy is real, and the strategy loses money. The same signal used to decide *which resting passive order to keep and which to cancel* pays no spread; whether it helps is an execution question (Part III), not an accuracy question.

A related line of work asks not only for a forecast but for a measure of *when the forecast can be trusted*. UQ-LOB (Edward Manoharan et al., 2026) attaches an uncertainty module to a pretrained LOB encoder and outputs a *calibrated* distribution—one whose stated probabilities match observed frequencies, so that events given 70% happen about 70% of the time (Guo et al., 2017)—with a

scalar confidence. Restricting attention to the most confident tenth (*decile*) of predictions raises directional macro-F1 by 0.11–0.15 in its regression variant (F1 is the harmonic mean of precision and recall; *macro*-F1 averages it over the classes “up”, “down” and “stationary”), across seven cryptocurrency assets at 5, 10 and 15 second horizons. Selective prediction changes the tradeability arithmetic directly: if a confident tier moved accuracy from 55% to 70% on a tenth of the signals, the expected edge per trade in Example 1.2 would rise from 0.10 to 0.40 of the typical move, with one tenth as many trades. Whether that clears the half spread still depends on the size of the moves in the confident tier. UQ-LOB itself stops at directional reliability: its authors state that they evaluate reliability, not profitability, and that a trading simulation with realistic execution and fees is required before any claim about returns (Edward Manoharan et al., 2026). That is the step this survey is about.

This is the central question of the survey:

Which information extracted from the limit order book survives the complete translation from statistical prediction to executable trading?

It changes how models should be compared. A model that improves classification accuracy by a fraction of a percentage point is not more useful than a linear model if the improvement vanishes after latency and costs. A model with modest directional accuracy may still be valuable if it improves the timing or selection of passive executions.

1.5 Prediction, execution and simulation are different problems

LOB research frequently mixes up three different questions. Each is written below as a probability or an expected value (Section 1.3). The big $\mathbb{P}$ is “the probability of”, $\mathbb{E}$ is “the average of”, and the vertical bar is read “given”: everything to the right of the bar is what we already know, everything to the left is what we want to know.

$$\text{Prediction: } \mathbb{P}(Y_{t+H} \mid X_t), \quad (4)$$

$$\text{Execution: } \mathbb{P}(\text{Fill} \mid X_t, Q_t^{\text{ahead}}, a_t) \text{ or } \mathbb{E}[\Pi \mid X_t, Q_t^{\text{ahead}}, a_t], \quad (5)$$

$$\text{Simulation: } \mathbb{P}(\{X_u\}_{u \in (t, t+H]} \mid X_t). \quad (6)$$

In words:

- *Prediction* (4): given what we can see now, how likely is each possible value of some future quantity Y —for example, “will the mid be higher in one second?” It is a question about the market, and our own orders play no part in it.
- *Execution* (5): given what we can see now, how much volume is ahead of our order, and what we chose to do, how likely is our order to be filled within H seconds—or, in the second form, how much do we expect to earn? It is a question about *our* order, and its answer changes with our action a_t .
- *Simulation* (6): given what we can see now, how likely is each possible *whole future path* of the market over the next H seconds? A simulator draws sample paths from this distribution. The difference from (4) is the difference between a weather simulator, which produces whole plausible tomorrows, and a forecast of tomorrow’s maximum temperature.

Two different activities are both called “simulation” in this literature, and the survey keeps them apart. *Generative market simulation* draws new market paths from a model of (6): Hawkes

simulators, agent-based markets and the generative networks of Section 10.15. *Historical replay* does not draw paths at all: it reconstructs the one path that was recorded, event by event, and evaluates the counterfactual outcome of the researcher’s own orders against it—an estimate of (5) on observed data. The Kaspar system of Section 12.3 is a replay system, not a generative simulator; Section 12 compares the kinds of simulator. They are related but not interchangeable. A Hawkes process can generate realistic clustered order flow without producing directional alpha. A survival model can give accurate fill probabilities while playing no role in price prediction. A Transformer can predict short-horizon returns without modelling queue dynamics. Each model must be evaluated against the problem it solves.

Table 1: One event stream (a day of ES market-by-order data), three models, three evaluation criteria. Using the wrong column is a common source of misleading comparisons.

Model	Target	Appropriate metric		Inappropriate metric
Hawkes, 4 event types	event intensity	point-process likelihood; time-rescaling residual test	log-likelihood	directional accuracy
Survival model	$\mathbb{P}(\tau_F \leq 1 \text{ s} \mid Q_t^{\text{ahead}} = q)$	survival likelihood; calibration by queue bucket		mid-move AUC
Classifier	$\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$	log loss, AUC, then net P&L		fill rate

1.6 Two fixed positions

Two positions recur throughout the paper.

Principle 1 (The market is observed, not generated). Recorded exchange data are the ground truth. A replay simulator reconstructs the historical book from those data; it does not invent market behaviour. The only counterfactual it produces is the execution outcome of the researcher’s own orders.

Principle 2 (Occam’s razor: complexity is an empirical hypothesis). Among models that deliver the same out-of-sample economic value, prefer the simplest. Each added layer—model capacity, specialised hardware, and the breadth of the search that produced the model—is a hypothesis that it adds value, and must earn incremental out-of-sample economic value that survives realistic execution assumptions and the multiple-testing burden of its own selection.

Occam’s razor is applied here to economic value, not to statistical fit alone: a more complex model is not rejected for being complex, but it carries the burden of showing that its extra value is real.

1.7 Development of the field

The modelling of order books has passed through five overlapping stages.

1. **Zero-intelligence models.** Orders arrive at random, with no strategy at all, and the matching rules do the rest (Smith et al., 2003). The surprise was how much this explains: on eleven London stocks, a model with one free parameter accounts for 96% of the variance of the bid–ask spread (Farmer et al., 2005). Market structure, not trader intelligence, explains much of what the book looks like.
2. **Stochastic queue models.** The best queues are treated as random walks driven by order arrivals, cancellations and executions, giving closed-form results for the time to the next price

change and the probability of its direction (Cont et al., 2010; Cont and de Larrard, 2013), later with rates that depend on queue sizes (Huang et al., 2015).

3. **Self-exciting point processes.** Hawkes models capture the clustering of events and the fact that many events are triggered by earlier ones; estimated branching ratios on E-mini S&P data range from about 0.7–0.8 to about one (Bacry et al., 2015; Filimonov and Sornette, 2012; Hardiman et al., 2013).
4. **Hand-engineered signals.** Order-flow imbalance (Cont et al., 2014), queue imbalance (Gould and Bonart, 2016) and the micro-price (Stoikov, 2018) compress the book into a few numbers with clear economic meaning.
5. **Learned models.** From 2017, convolutional and recurrent networks (Tsantekidis et al., 2017a; Zhang et al., 2019a), then attention and spatial-temporal Transformers (Wallbridge, 2020; Berti and Kasneci, 2025), then pretrained message-level encoders and large generative models (Linna et al., 2025; Zhang et al., 2026), together with hybrids that put point-process structure inside networks (Shi and Cartlidge, 2022; Bodor and Carlier, 2025).

Each stage added capacity. Whether each also added *economic* value is the question this survey keeps asking.

1.8 Positioning and taxonomy

The literature has three historical traditions. The first is econometric and reduced-form modelling of prices, order flow and impact (Kyle, 1985; Glosten and Milgrom, 1985; Cont et al., 2014). The second is continuous-time point-process and queueing models: Hawkes processes preserve the event-driven structure and model excitation between event types (Hawkes, 1971; Bacry et al., 2015), while queue-reactive and first-passage models describe displayed liquidity and its depletion (Cont et al., 2010; Huang et al., 2015). The third is machine learning, from hand-engineered features through CNNs, recurrent networks, Transformers, state-space models and, most recently, pretrained message-level encoders (Tsantekidis et al., 2017a; Zhang et al., 2019a; Wallbridge, 2020; Linna et al., 2025).

Position relative to existing surveys. Existing surveys and books cover parts of this ground well (Table 2): the LOB as a trading mechanism and its empirical literature (Gould et al., 2013; Tripathi et al., 2020), machine-learning prediction from the book (Zaznov et al., 2022), generative simulation of order books (Jain et al., 2024b), and, more recently, large language models and autonomous agents in financial trading (Ding et al., 2024; Dong et al., 2025; Hua et al., 2026). These works provide taxonomies of models and applications, but generally treat LOB modelling, trading prediction, execution and queue position, market simulation and the hardware that runs the models as separate topics. We are not aware of a survey that brings these strands together around the question of how information extracted from the LOB survives the complete translation from statistical prediction to executable trading. That question is also this survey’s criterion for selecting literature: a model or method is included when it bears on one link of the chain—representation, prediction, calibration, decision, queue position, execution, simulation, inference hardware and economic value. The individual subjects are not new; the integration, and the criterion that connects them, are. One distinction follows from it and recurs throughout: historical replay is distinguished from generative market simulation.

Generative LOB simulation versus historical replay. The closest neighbouring review, on LOB simulation (Jain et al., 2024b), classifies, by methodology, models that *generate* order-book dynamics, and evaluates them by their ability to reproduce stylised facts and price impact. That is one of two legitimate ways of using a simulator, and they answer different questions:

- *generative simulation* asks whether a model can produce plausible market trajectories, including ones never observed, such as stress scenarios;
- *historical replay* asks what would have happened to a particular strategy’s orders against one observed trajectory.

To decide whether an order would have executed against the historical market, the market does not need to be generated, only reconstructed. This survey is concerned primarily with the second question; generative models are covered in Section 10.15 as tools for simulation and stress testing.

Language models. The surveys of LLMs in trading review systems in which the language model reads market information, reasons about it and produces trading decisions that are then backtested (Ding et al., 2024; Hua et al., 2026). This survey does not review that architecture.

The purpose here is therefore not another catalogue of architectures, but an organisation of the literature around the translation from information to decision. Three questions recur: what representation contains the useful information; what model extracts it; and does the result produce incremental economic value after execution constraints?

Table 2: Existing surveys, books and benchmark studies, and what this survey adds.

Work	Focus	Not covered there
Gould et al. (2013)	empirical facts and models of LOBs	deep learning; execution value; queue position in practice
Abergel et al. (2016), Bouchaud et al. (2018)	books on empirical properties, models and simulation of order books and impact	recent deep and generative models; model-search statistics
Tripathi et al. (2020)	systematic review of 103 articles (1981–2018) on the LOB as a trading mechanism and its implications for market efficiency, classified by sub-area, market, method and LOB measure	deep-learning prediction (named there only as a future direction); execution value; queue position
Jain et al. (2024b)	review of LOB simulation models and the stylised facts used to validate them	prediction and trading value
Ding et al. (2024), Dong et al. (2025), Hua et al. (2026)	surveys of LLMs as trading and financial agents: architectures, inputs, workflows, benchmarks, backtests, deployment	LOB microstructure; execution
Zaznov et al. (2022)	survey of price-change prediction from the LOB	classical point-process and queue models; execution
Prata et al. (2024), Briola et al. (2025a)	benchmark re-tests of deep models; transaction-level evaluation	classical models; queue position; search statistics
This survey	information-to-decision chain: classical and deep models, queue position, simulators and deterministic replay, hardware acceleration	–

Contributions. The survey offers:

1. a single chain—representation, prediction, decision, queue, execution, profit—that places classical and learned models in one framework;
2. a treatment of queue position, fill probability and adverse selection as modelling targets in their own right, with the signal-value/execution-value distinction (Section 11.1.1);
3. an account of deep LOB models from first principles—inputs, targets, network families, training and scoring, with a runnable PyTorch example—and of how label construction and data splits can inflate reported accuracy (Section 10);
4. a treatment of the two ways of combining classical and learned models, meta-models and structural hybrids, with published examples taken apart step by step (Sections 8 and 9);
5. a survey of order-book simulators, with queue-exact deterministic replay as the evaluation instrument for execution claims, and a description of how a model is added to one as an actor (Part III);
6. an account of why order-book models are slow at inference and what hardware can accelerate (Part III);
7. a reader-oriented apparatus: notation, a probability primer, plain-language explanations of every equation, a glossary, and a from-scratch introduction to point processes (Section 7).

Reading order. Readers new to markets should read Part I in full and use the glossary. Readers interested mainly in models should read Part II, then Section 10.12. Practitioners concerned with execution, simulation or hardware should read Part III. Readers new to point processes should start Section 7 at its beginning, which introduces them from scratch.

Four axes for locating a piece of work

Every model or paper in this survey answers four separate questions, and we describe it by its answer to each. Think of the four answers as the four fields of a record; a work is a point in a four-dimensional space.

Representation: what does the model see?

The input format: raw book snapshots (the table of Equation (1) at sampled times); event or message streams (every add, cancel and trade, in order); order-flow aggregates such as OFI (Section 4); queue state (sizes and positions in the queues); or the state of several instruments at once.

Model class: what kind of function maps input to output?

Linear and econometric models; queue and first-passage models; point processes such as Hawkes; state-space models; convolutional and recurrent networks; Transformers; pretrained encoders; or a hybrid of these.

Objective: what quantity does it output?

Price direction; return; event intensity; volatility; queue depletion time; fill probability; execution cost; or a simulated future.

Economic role: what decision is the output for?

Aggressive trading (crossing the spread); passive trading (resting orders); market making (quoting both sides); execution of a given order; risk measurement; or simulation for research. A paper that evaluates only statistical accuracy and names no decision has an *unspecified* economic role.

The axes are used in three ways.

First, to decide what can be compared. Two results are comparable only if they share an objective and an economic role, and in addition the data fields of Section 2.4. Representation and model class are the things a study varies; objective and role are the things it must hold fixed. A Hawkes model scored on event-intensity likelihood and a Transformer scored on direction accuracy are not competitors, however often they appear in the same table.

Second, to organise the paper. Part II moves along the model-class axis, from linear models to pretrained encoders, and Section 5 along the representation axis. Part III moves along the objective axis from direction to fill probability and adverse selection, and asks how simulators and hardware support the economic role of passive execution.

Third, to expose a gap. Table 3 places the main works discussed in this survey on the four axes. The economic-role column is where the literature is thinnest: many of the learned models are evaluated on accuracy alone, and the decision their output would drive is left unspecified. The rest of the paper is about filling that column.

Table 3: Selected works placed on the four axes. “Unspecified” means the work evaluates statistical accuracy without stating or evaluating a trading decision. SHADOW-PPOV (Shadow Passive Percentage-of-Volume) is the model-free execution algorithm of Section 12.3.4.

Work	Representation	Model class	Objective	Economic role
OFI (Cont et al., 2014)	order-flow aggregate	linear	contemporaneous price change	price impact (execution)
Book dynamics (Cont et al., 2010)	queue state	queueing / first passage	direction; queue depletion	market making, execution
Queue-reactive (Huang et al., 2015)	queue state	queueing (Markov)	event intensity	simulation
Hawkes models (Bacry et al., 2015)	event stream	point process	event intensity	simulation, risk
Queue value (Moallemi and Yuan, 2017)	queue state	queueing	value of a queue position	passive trading, market making
DeepLOB (Zhang et al., 2019a)	raw snapshots	CNN + recurrent	direction	aggressive trading (simple trading simulation)
TransLOB (Wallbridge, 2020)	raw snapshots	Transformer	direction	unspecified
Deep OFI (Kolm et al., 2023)	order-flow aggregate	recurrent / feed-forward	return, several horizons	unspecified (scored by out-of-sample variance explained)
LOBERT (Linna et al., 2025)	event stream	pretrained encoder	direction; next message	unspecified
UQ-LOB (Edward Manoharan et al., 2026)	event stream	encoder + uncertainty module	direction with confidence	unspecified (stated by the authors)
SHADOW-PPOV (Maciejewski, 2026d)	event stream	none (rule)	execution cost	passive execution

2 Survey methodology

A survey makes many claims, drawn from many papers that were written for different purposes. This section explains what the survey covers, how its sources were chosen, how strongly each kind of claim should be believed, and what has to be known about two studies before their numbers can be compared. The last part is a practical checklist for reading any paper in this field.

2.1 Scope: which markets and which questions

The survey covers limit order books in four kinds of market. They share the mechanism of Section 1 but differ in ways that change what a model can learn and whether a result carries over from one to another.

Table 4: The markets covered, and how they differ in ways that matter for LOB models.

Market	How it trades	Typical data	What it means for models
Futures (e.g. ES, NQ, ZN)	Each contract trades on one exchange, in nearly continuous sessions	Exchange MBO feeds such as CME MDP 3.0	One book holds all the liquidity, so the book a model sees is the whole market.
US equities	The same stock trades on many exchanges and off-exchange venues at once	Exchange feeds such as ITCH, one venue at a time	One venue’s book is only part of the market; flow elsewhere moves prices too.
Cryptocurrencies	Many independent exchanges, open 24 hours a day, every day	Public exchange APIs, often snapshots plus updates	Data are easy to obtain, but fees, participants and trading hours differ from traditional markets.
Fixed income	Government bond futures on futures exchanges; cash bonds on electronic inter-dealer platforms	Varies by venue	Treasury futures are large-tick instruments: the price rarely moves by more than a tick, queues are long, and queue position matters.

Within these markets the survey includes work whose main object is one of four things: *prediction* (forecasting prices or events), *signal extraction* (building features from the book), *execution* (deciding how to place, keep and cancel orders), or *simulation* (reproducing the market for research). Equilibrium models, which derive prices from the optimal behaviour of rational traders, are deliberately left out except where they help explain an LOB model. Agent-based models, which simulate many rule-following traders, are covered as simulators (Section 12), not as models fitted to the data. Synthetic market data produced by learned generative models are discussed in Section 10.15, but, following Principle 1, they play no part in the methodology the survey recommends.

2.2 Source selection

This is a *narrative* survey, not a *systematic review*. A systematic review fixes a search query and a list of databases in advance, screens every result against written criteria, and reports how many papers were kept at each step, so another team could repeat the search exactly. A narrative survey instead selects work that illustrates and tests an argument. That is appropriate here, because the aim is to organise the field around one question—what survives the translation from prediction to execution—rather than to count papers. It also has a known weakness: the selection may favour work that fits the argument. To limit that, the survey includes work that cuts against its own position where such work exists, for example evidence that deep models given suitable inputs reach state-of-the-art predictive accuracy (Section 10).

A work was included if it made at least one of the following contributions: a representation of

the order book; a predictive model; a queue or execution model; an evaluation method; a public data set or benchmark; or a design for simulation or research infrastructure. Preference was given to peer-reviewed work and to widely cited preprints; recent preprints are included when they are the only source for an idea, and are identified as preprints in the bibliography. Every reference was checked against the publisher, the preprint server or the official documentation.

2.3 Strength of evidence for a claim

Not all statements in a survey deserve the same confidence. We distinguish four kinds, and the reader should ask of every claim which kind it is.

Established result

Supported directly by a cited study, or true by mathematics. *Example:* a linear Hawkes process is stationary if and only if the cascade of events it triggers dies out (Section 7). This is a theorem; no data can overturn it.

Empirical synthesis

A conclusion drawn by comparing several studies, none of which tested it directly. *Example:* “the input representation can matter as much as the architecture” (Section 10). It rests on a pattern across papers, so it can be wrong for a market those papers did not study.

Methodological recommendation

A proposed standard, argued for rather than proved. *Example:* every result should report how many model configurations were tried (White, 2000; Harvey et al., 2016). Its justification is a statistical argument, not a measurement.

Open hypothesis

Plausible, but the evidence is incomplete or conflicting. *Example:* adding Hawkes intensities to a large network that already sees the same events adds little value (Section 9). The survey argues for it; nobody has shown it.

The distinction matters because LOB studies differ in assets, horizons, targets and data, and a result established under one set of conditions is at best a hypothesis under another. Table 57 in Section 15 classifies the main claims of this paper in these terms.

2.4 Conditions for comparing two LOB studies

Papers in this field routinely report a single headline number—an accuracy, an F1 score, a Sharpe ratio—and readers routinely compare those numbers across papers. Such comparisons are meaningful only when the studies agree on the conditions that produced the numbers. Table 5 lists the conditions that matter most and explains how each one can change the headline number on its own, with no change to the model.

Example 2.1 (Accuracy without knowledge). A three-class target labels each sample “up”, “down” or “stationary” according to whether the mid moves by more than a threshold within the horizon. Suppose that, with a short horizon on a large-tick future, 80% of samples are “stationary”, because the mid rarely moves at all. A model that ignores its input and always answers “stationary” is then 80% accurate. A genuinely useful model might be 75% accurate and still be better, because it is right on some of the up and down moves that are the only cases a trader cares about. Accuracy alone cannot show this; balanced accuracy, macro-F1 or the confusion matrix (the table of predicted against actual classes) can.

Table 5: Fields a study must report before its results can be compared with another’s.

Field	Why it changes the result
Asset, exchange, period, trading hours	Markets differ (Table 4); the same market differs between calm and volatile years and between the open, midday and the close.
Tick size, typical spread, depth	A large tick makes mid changes rarer and queues longer, which changes both what is predictable and what is tradeable.
Message rate	A busy instrument produces many events per second; a horizon of ten events is a few milliseconds in one market and minutes in another.
Data level (L1, L2, L3)	Only L3 data reveal the causes of book changes and queue positions (Section 5).
Sampling clock	An <i>event clock</i> samples after a fixed number of events, a <i>calendar clock</i> after a fixed number of seconds. The event clock over-represents busy periods, which behave differently.
Horizon and target	Predicting the sign of the next mid change is a different problem from predicting a move larger than a threshold, or a return over ten seconds (Section 10).
Class balance	If most samples belong to one class, a model that always predicts that class scores high accuracy while knowing nothing (Example 2.1).
Train/test protocol	A random split lets the model train on data from after its test period; a chronological split does not (Section 10.5).
Costs, latency and execution model	The same forecast is profitable or unprofitable depending on whether the spread, fees and delays are counted (Section 1.4).
Number of configurations tried	The best of many tries looks good by chance (White, 2000; Harvey et al., 2016).

Example 2.2 (Two “65% accuracy” results). One study reports 65% on FI-2010 (Ntakaris et al., 2018): five Nasdaq Nordic stocks, ten trading days, a horizon of 10 in the data set’s own sampling units, three classes. Another reports 65% on a cryptocurrency pair sampled on a one-second calendar clock with two classes. With two balanced classes, 50% is what guessing achieves; with three classes, guessing achieves anywhere from 33% to the share of the largest class. The two numbers sit on different scales and share almost none of the fields of Table 5. Their equality carries no comparative information.

2.5 A reader’s checklist

The following questions can be asked of any paper in this field, including this one. A study that cannot answer them has not yet shown that its result is useful for trading.

1. **What exactly is predicted, over what horizon, on what clock?** (Section 10)
2. **What does a trivial model score on the same data?** Always guessing the most common class, or a linear model on order-flow imbalance (Section 4).
3. **Was the test period strictly after the training period?** (Section 10.5)
4. **How many models or settings were tried before this one was reported?** The best of many tries looks good by chance (White, 2000; Harvey et al., 2016).
5. **What decision would the prediction drive, and was that decision evaluated after spread, fees and latency?** (Sections 1.4 and 11.1)
6. **If the decision uses passive orders, how were fills decided?** A fill rule that ignores the queue overstates results (Section 12.1).

Part II

Models of the Limit Order Book

About this part. Part II surveys the models of the order book, from the simplest to the most complex. It starts with the empirical facts any model must respect (Section 3); moves through hand-built signals computed from the book (order-flow imbalance, queue imbalance, the micro-price; Section 4); then to models of the queues themselves and the time until they empty (Section 6); to Hawkes processes, which model how events trigger further events (Section 7). It then adds learning in three steps of decreasing structure: meta-models, which feed the outputs of the classical models to a learner (Section 8); structural hybrids, which build classical structure inside a network (Section 9); and deep networks that learn from the book with little imposed structure (Section 10). Information from related instruments is treated with the signals (Section 4.6), with Hawkes models and with deep models, rather than in a separate section. Every model ends with a model card answering the same four questions.

3 Stylised facts a model should reproduce

Before turning to models, it helps to list what any adequate model of the book must be consistent with. These regularities have been documented repeatedly across markets and periods.

Events cluster

Order-book events arrive in bursts separated by quiet periods, and a large share of events are triggered by earlier ones (Section 7).

Long waits are common

Times between price changes have heavy tails; in a simple model of exactly balanced queues the expected waiting time is even infinite (Cont and de Larrard, 2013).

Order flow has long memory

The signs of successive market orders are positively correlated over very long lags, with a Hurst exponent of about 0.7, mainly because large orders are split into many small ones (Lillo and Farmer, 2004; Lillo et al., 2005).

Prices nevertheless diffuse

Despite persistent order flow, prices are close to unpredictable beyond short horizons, because limit orders absorb the pressure: the market is held near a critical balance between the two (Bouchaud et al., 2004).

Short-horizon mean reversion

Measured volatility is higher over very short windows than over longer ones (bid–ask bounce), and correlations between assets fall when measured over very short windows (the Epps effect) (Bacry et al., 2013).

The average book is humped

Average depth is largest a few ticks away from the best price, not at it (Bouchaud et al., 2002).

Impact is concave

The price impact of a large order executed in pieces grows less than linearly with its size, close to a square-root law (Bouchaud et al., 2018); Hawkes models reproduce its concave, relaxing shape (Bacry and Muzy, 2014).

Order flow explains price changes

Over short windows, price changes are approximately linear in order-flow imbalance, with a slope inversely related to depth (Cont et al., 2014).

Not only trades move prices

Roughly half of price-changing events are limit orders placed inside the spread (Eisler et al., 2012), and the book beyond the best quotes carries part of the price information (Cao et al., 2009).

Activity is seasonal

Intensities and volatility follow intraday patterns, so models fitted with a constant baseline over a non-stationary period overstate how self-exciting the market is (Filimonov and Sornette, 2015).

Generative models are judged by how many of these they reproduce (Section 10.15); predictive models are judged by whether they exploit them after costs.

Presentation of each model. Every model in this part ends with a *model card* that answers the same four questions: what has to be calibrated (the numbers estimated from data), how it is calibrated, what the fitted model is then used for, and a picture that makes it easy to remember and to teach. Descriptive features that are not models in their own right (Section 4.4) get a *feature card* with the same four questions instead. The cards summarise the section they close; the detail and the citations are in the text above each one.

4 Order-flow imbalance and queue imbalance

The simplest useful models of the book are single numbers computed directly from it. This section covers the two most important: *order-flow imbalance*, which measures the net buying pressure from the stream of book events, and *queue imbalance*, which measures how lopsided the two best queues are at an instant. It then describes further features of the book’s shape beyond the best level, and shows how a few such features can be mapped to a state from which the expected future price is calibrated—the micro-price, a better estimate of the “true” price than the mid or the weighted mid. It ends with order flow across related instruments and the lead–lag between futures and cash. These signals are cheap, interpretable and hard to beat, which makes them the baselines for everything that follows.

Snapshot models. Everything in this section is built from *snapshots* of the book: the sizes and prices visible at the best levels at a moment, or the change in them between one moment and the next. Queue imbalance, the weighted mid, the micro-price and the shape features use a single snapshot; order-flow imbalance uses the differences between consecutive snapshots. None of them uses *why* the book changed—whether lots left a queue because they were traded or because they were cancelled, or which order did what. That information exists only in the stream of individual events, and the next section turns to it: Section 5 explains what event-level data add, and the models that follow—queue depletion (Section 6), Hawkes processes (Section 7) and the message-level networks of Section 10—are *event models*, built on that stream.

4.1 Order-flow imbalance

The idea. Why does the price move? Look again at Figure 1. The best ask is 5000.25 with 14 lots waiting. If buyers take all 14 lots, or the sellers waiting there cancel, the best ask becomes 5000.50

and the mid rises. If instead more sellers join at 5000.25, the queue grows and a rise becomes harder. The same holds in mirror image on the bid side. So anything that *grows the bid queue or shrinks the ask queue* pushes the price up, and anything that *shrinks the bid queue or grows the ask queue* pushes it down. Order-flow imbalance (OFI) simply keeps a running score of these pushes (Cont et al., 2014).

Measuring one event. Visible changes at the best prices come from three event types: limit orders arriving, aggressive orders executing against waiting ones, and cancellations. For event number n , define

$$\Delta v_n^b = v_{1,t_n}^b - v_{1,t_n-}^b, \quad \Delta v_n^a = v_{1,t_n}^a - v_{1,t_n-}^a.$$

Recall that $v_{1,t}^b$ is the number of lots waiting at the best bid at time t , and that t_n- means “just before event n ”. So Δv_n^b is *the size of the best-bid queue just after event n minus its size just before*: how many lots event n added to the best-bid queue (positive) or removed from it (negative). Δv_n^a is the same for the best-ask queue. In the book of Figure 1:

- a new 20-lot buy order at 5000.00 gives $\Delta v_n^b = +20$ (29 lots become 49), and $\Delta v_n^a = 0$;
- a sell market order for 10 lots trades against the front of the bid queue and gives $\Delta v_n^b = -10$;
- a seller cancelling 3 lots at 5000.25 gives $\Delta v_n^a = -3$ (14 lots become 11).

Scoring and adding up. While the best prices do not change, the score of event n and the order-flow imbalance over the last w seconds are

$$e_n = \Delta v_n^b - \Delta v_n^a, \quad \text{OFI}_t^{(w)} = \sum_{n: t_n \in (t-w, t]} e_n. \quad (7)$$

In words: e_n is positive when event n grew the bid queue or shrank the ask queue—a push up—and negative for a push down; subtracting Δv_n^a is what turns a shrinking ask queue into a positive score. The sum runs over every event n whose time t_n falls in the last w seconds, so $\text{OFI}_t^{(w)}$ is the net buying pressure over that window, like the net flow into a tank: lots in on the buy side and out on the sell side count as inflow, the reverse as outflow.

When a best price *does* change, the full definition of Cont et al. (2014) counts whole queues. If the best bid rises to a new, higher price, the entire new queue counts as added buying interest; if the best-bid level is emptied and the best bid falls, the entire old queue counts as removed. The ask side is treated symmetrically. This keeps the score meaningful across price changes.

From pressure to price. The associated short-horizon relation is a straight line,

$$m_t - m_{t-w} = \beta \text{OFI}_t^{(w)} + \varepsilon_t, \quad (8)$$

where β is the price-impact coefficient and ε_t a residual.

In words: over a short window, the mid moves roughly in proportion to the net buying pressure. β converts lots of pressure into price: if β is 0.01 points per lot, a net 100 lots of pressure moves the mid by about one point. ε_t is whatever the line does not explain. Fitting β is an ordinary linear regression of price change on OFI.

Cont et al. (2014) find that β is inversely related to market depth: in a thin book a few lots empty a queue and move the price, while in a deep book the same lots barely dent it. OFI is low-dimensional, cheap to compute and easy to interpret, which makes it the natural baseline against which more complex models should be judged.

Example 4.1 (OFI by hand). Five events at unchanged best prices, all inside one window of length w :

n	event	Δv_n^b	Δv_n^a	e_n
1	bid limit order, 20 lots	+20	0	+20
2	ask cancel, 15 lots	0	-15	+15
3	buy market order hits ask, 10 lots	0	-10	+10
4	ask limit order, 30 lots	0	+30	-30
5	bid cancel, 5 lots	-5	0	-5
OFI _t ^(w)				+10

Events 2 and 3 enter through identical arithmetic, yet one is a cancellation and the other an aggressive trade. OFI does not distinguish them; Section 5 returns to this.

4.2 Evidence and extensions

Explanatory power of OFI for price changes. On 50 US stocks, Cont et al. (2014) find that the linear relation explains mid-price changes over short intervals with an average R^2 of about 65%— R^2 , the coefficient of determination, being the fraction of the variance of price changes that the fitted line explains—against about 32% for the imbalance of *trades* alone; adding trade imbalance to OFI raises the average only to about 67%. The slope is close to inversely proportional to average depth, $\beta \propto 1/\text{depth}$. The fit varies a great deal across stocks—from above 90% for some to below 15% for others—which is itself a warning against quoting a single number for “the” predictability of order flow. The lesson that limit orders and cancellations carry as much price information as trades is one of the most robust in the field.

Looking deeper than the best level. Information also sits beyond the best quotes. On the Australian Stock Exchange, Cao et al. (2009) attribute about 22% of price discovery to the book beyond the best bid and offer, and find that imbalances deeper in the book predict short-term returns even after controlling for the autocorrelation of returns, the inside spread and the imbalance of trades. *Multi-level OFI* computes the imbalance separately at each of the first L levels, giving a vector rather than a number; for six Nasdaq stocks, Xu et al. (2018) find that the out-of-sample fit improves with every additional level included. Cont et al. (2023) combine the level-wise imbalances into a single *integrated OFI*, the first principal component of the multi-level vector, which captures over 89% of its variance. On the 100 largest S&P 500 stocks over 2017–2019 the integrated OFI raises the average out-of-sample R^2 of the contemporaneous price-impact regression from about 65% (best level only) to about 84%.

Horizon and reversal. The predictive content of order imbalance depends on the horizon. In Chinese equities, Zhang et al. (2025) find that order imbalance predicts returns positively from 5 to 30 minutes ahead and negatively from 60 to 120 minutes ahead, with no reversal for large or heavily traded stocks. The usual interpretation is that part of the price pressure from imbalance is temporary and is undone as liquidity is replenished. A signal must be used over the horizon on which it has the sign one assumes.

Estimating the slope in real time. Because β moves with depth, it is re-estimated continuously in practice: by a rolling-window regression, by normalising OFI by recent average depth (a *depth-normalised* OFI), or by treating β_t as a hidden state that drifts over time and tracking it with a

Kalman filter (Kalman, 1960), the standard recursive estimator for linear systems observed with noise.

Uses of OFI. The same quantity serves several decisions. As a directional signal, it predicts the sign of the next short-horizon move. For a market maker, it sets the *skew* of quotes—leaning bids and offers in the direction of pressure—and informs inventory control (Cartea et al., 2015). For execution, it indicates when resting passively is likely to be rewarded and when the order book is about to move away.

Model card: Order-flow imbalance

What you calibrate. The price-impact slope β for a chosen window w (and, for multi-level OFI, one slope per level or the weights of the integrated OFI).

How. An ordinary least-squares regression of mid changes on $\text{OFI}_t^{(w)}$. Because β falls as depth rises, it is re-estimated on a rolling window, absorbed by dividing OFI by recent depth, or tracked as a drifting hidden state with a Kalman filter. Integrated OFI takes the first principal component of the level-wise OFIs.

What you do with it. A baseline directional signal; the skew of a market maker’s quotes; a warning, for a passive order, that the book is about to move away from it.

How to picture it. Net flow into a tank: lots joining the bid or leaving the ask are inflow, lots joining the ask or leaving the bid are outflow; the level of the tank leads the price.

4.3 Queue imbalance and richer book features

Queue imbalance compresses the top of book into one number,

$$\text{QI}_t = \frac{v_{1,t}^b - v_{1,t}^a}{v_{1,t}^b + v_{1,t}^a} \in [-1, 1]. \quad (9)$$

In words: the difference between the best-bid and best-ask queue sizes, divided by their total so the result does not depend on how busy the market is. It is +1 when only buyers are queued, −1 when only sellers are, and 0 when the two queues are equal.

Queue imbalance and the next price move. The mid moves when one of the two best queues is used up. If the book is skewed—many lots on one side, few on the other—the thin queue needs far less trading and cancelling to disappear than the thick one, so it is likely to be emptied first; later sections call this “the thin side breaks first”. The next price move is therefore more likely to be *towards the thin side*: a thin ask is likely to be taken out, and the price ticks up; a thin bid is likely to be hit, and the price ticks down. Think of two piles of sand being removed at roughly the same rate: the smaller pile is gone first. Stochastic models of the book make this precise, computing the probability of an up move as a function of the two queue sizes (Cont et al., 2010).

Example 4.2 (Reading the imbalance in Figure 1). The best bid has 29 lots and the best ask 14, so $\text{QI}_t = (29 - 14)/(29 + 14) \approx 0.35$: the book leans towards the buyers. The ask queue is the thinner one and is likely to break first, so the next change in the mid is more likely to be up than down. For a trader the skew has two consequences. A new passive buy order would join the back of the long 29-lot queue and probably wait while the price moves away from it. A new passive sell order would join the short ask queue and probably be filled—just before the up move that the imbalance predicted, which is adverse selection (Section 11.3).

The size-weighted mid-price. The same intuition gives a better single-number price than the mid. The *weighted mid* weights each side’s price by the size on the *other* side,

$$m_t^{\text{wmid}} = \frac{v_{1,t}^b p_{1,t}^a + v_{1,t}^a p_{1,t}^b}{v_{1,t}^b + v_{1,t}^a}, \quad (10)$$

In words: when the bid queue is much larger than the ask queue, the price is pulled towards the ask, the side expected to break, and vice versa. With equal queues it equals the mid. In Figure 1 it is $5000.00 + 0.25 \times 29/43 \approx 5000.169$, above the mid of 5000.125.

Limits of the intuition. Displayed size is not committed size. A thin queue may be refilled immediately by new orders; hidden or iceberg orders may make the thin side thicker than it looks; and orders placed only to be cancelled (spoofing, below) can create a skew that is not real. Imbalance is therefore a strong but noisy signal.

Spoofing and layering. Displayed liquidity is not necessarily committed liquidity. Manipulative orders do not falsify OFI—OFI still measures observed changes—but they alter the mapping from displayed flow to future executable prices, and they distort queue-imbalance, depth and depletion estimates.

Documented cases. Manipulation of the order book is not hypothetical. On the Korea Exchange, Lee et al. (2013) document traders placing large orders away from the market to create imbalance and withdrawing them. A trader in the E-mini S&P 500 was charged with layering over five years, conduct the CFTC alleged contributed to the conditions of the May 2010 flash crash (Commodity Futures Trading Commission, 2015). In 2020 JPMorgan paid \$920 million over spoofing in precious-metals and US Treasury futures over at least eight years (Commodity Futures Trading Commission, 2020). In a spoofing episode in Ultra Treasury bond futures, Debie et al. (2023) measure ask-side liquidity costs falling from about 10.4 to about 8 basis points when the spoof order was placed and recovering after it was cancelled—a displayed improvement in liquidity that was not real. Cartea et al. (2020) model spoofing as an optimal-control problem in which the spoofer trades the gain from skewing the order imbalance against the expected regulatory penalty. Quote stuffing—bursts of order messages—affected over 74% of US-listed securities at least once in 2010, with lower liquidity and higher short-term volatility during episodes (Egginton et al., 2016).

Table 6: How contamination affects order-book signals.

Form	Signature in the data	Effect on signals
Spoofing	large orders away from the touch, cancelled quickly	OFI and imbalance point the wrong way
Layering	several non-genuine orders at successive levels	distorts depth, slope and multi-level OFI
Quote stuffing	bursts of submissions and cancellations	inflates event counts; breaks event-clock assumptions
Icebergs and hidden orders	repeated executions at one price with stable visible size	displayed depth understates liquidity; depletion and fill estimates biased

Mitigations include down-weighting orders that are cancelled within a short time, features built from cancellation rates and from the ratio of trades to orders, and detection of iceberg refills. They reduce but do not remove the problem.

Evidence. Gould and Bonart (2016) fit a logistic regression of the direction of the next mid move on queue imbalance for ten Nasdaq stocks. The relationship is strongly significant for every stock, but its value depends on the tick size: out of sample, queue imbalance improves binary classification over a null model by about 50–60% for large-tick stocks and only about 10–30% for small-tick stocks, and probabilistic classification by about 20–30% against about 2–6%. Queue imbalance is most informative exactly where queues are long and the price rarely moves—the large-tick regime that ES and ZN futures also belong to.

Example 4.3 (Same imbalance, different history). Bid 300, ask 100 gives $QI_t = (300 - 100)/(300 + 100) = 0.5$. In book A the ask fell from 400 to 100 because a 300-lot market buy lifted it. In book B it fell because 300 lots were cancelled. Both books have the same snapshot and the same QI_t . A trader who saw the messages knows that A contains an aggressive buyer and B does not.

Model card: Queue imbalance and weighted mid

What you calibrate. Nothing for QI_t or m_t^{wmid} themselves. To turn QI_t into a probability, the coefficients of a logistic regression of the next mid-move direction on QI_t (Gould and Bonart, 2016).

How. Count, over historical data, which way the mid moved next after each observed value of the imbalance, and fit the logistic curve by maximum likelihood; report it separately for large-tick and small-tick instruments, where its value differs.

What you do with it. Predict the direction of the next mid change; decide which side a passive order should rest on and when it is likely to be adversely selected.

How to picture it. Two piles of sand removed at about the same rate: the smaller pile is gone first, and the price moves towards the side whose pile disappeared.

4.4 The shape of the book

Unlike the subsections around it, this one does not present a model. It lists *features*: numbers computed from the book that describe how liquidity is spread across price levels. A feature makes no statement about what happens next; it becomes predictive only as an input to a model—a regression, a classifier, a queue model or a network of the later sections—and its value has to be shown there. The one entry that involves a model, resilience, is a quantity read off a fitted Hawkes model (Section 7), not a model of its own.

Beyond the best level, several features summarise how liquidity is distributed:

Multi-level imbalance

the queue imbalance computed with sizes summed over the best ℓ levels on each side, for several ℓ .

Book slope

how fast cumulative depth grows with distance from the best price, usually the fitted slope of cumulative size against price distance, separately for each side. A steep slope means a deep book that absorbs large orders.

Curvature

the change of that slope with distance—whether depth is concentrated near the touch or further out. Its empirical relevance is tied to the average shape of the book, which in equities is humped: depth is usually largest a few ticks away from the best price, not at it (Bouchaud et al., 2002).

Depth-weighted and volume-weighted prices

averages of the prices at the first ℓ levels, weighted by their sizes; the weighted mid is the one-level case.

Resilience

how quickly the book refills after a large trade. [Large \(2007\)](#) measures it with a ten-variate Hawkes model on an LSE stock and finds that when the book does refill, the impulse responses have half-lives under 20 seconds, but that a resilient response follows a large trade in under 40% of cases.

Depth-normalised quantities

order-flow measures divided by recent average depth, motivated by the inverse relation between price impact and depth.

Imbalance and spread are features too. The quantities of the previous subsection belong on the same list: queue imbalance, its $[0, 1]$ form ι_t , and the spread s_t are also numbers computed from a snapshot of the book. The next subsection uses two of them to build a model.

Feature card: Book-shape features

What you compute. Imbalance summed over the first ℓ levels; the slope and curvature of cumulative depth against distance from the best price; depth-weighted prices; depth-normalised order flow; for resilience, the impulse response of a fitted Hawkes model.

How. Directly from each snapshot of the book: sums and weighted averages over levels, and least-squares fits of cumulative size against price distance, per side. Resilience requires fitting the Hawkes model of Section 7 first ([Large, 2007](#)). The only choices are the number of levels and the fitting window.

What you do with it. Feed them as inputs to a predictive or execution model, normalise other signals by depth, and judge how far a large order would walk the book. Any predictive value must be demonstrated by the model that uses them.

How to picture it. The profile of a hillside on each side of a valley: a steep hillside holds a lot of liquidity close to the price, a shallow one makes large orders walk far.

4.5 The micro-price

The idea: from book features to a state, and from a state to a forecast. The micro-price turns the features of the previous subsection into a forecast in three steps.

1. **Describe the book by a few features.** Out of everything visible in the book, keep a small number of features that matter for where the price goes next. The micro-price keeps two: the imbalance ι_t and the spread s_t .
2. **Map the features to a state.** Cut each feature into a few ranges—imbalance below 0.2, between 0.2 and 0.4, and so on; spread of one tick or two—so that every possible book falls into one of a finite list of *states*. Two books that differ in detail but fall in the same ranges are treated as the same state. This is a deliberate compression: thousands of possible books become a few dozen boxes.
3. **Calibrate, for each state, what the price does next.** Go through historical data, and every time the book was in a given state, record what the mid did afterwards. The average is the expected price change from that state. Adding it to the current mid gives the forecast.

In words: map today's book to a box, look up what the price did on average the many other times the book was in that box, and add that to the mid. The table of boxes and their average price changes is the whole model.

There are two ways to do step 3, and both are calibration to market data:

Direct

For each state, average the mid change observed over the next k price changes, every time the book was in that state. This measures the outcome directly and needs no further assumption, but each state sees relatively few complete k -change histories, so the averages are noisy, and the horizon k must be chosen.

Chained (Stoikov)

Calibrate only what happens on the *next* step from each state—to which state the book moves, and whether and how far the mid moves. Such one-step transitions are plentiful in the data. Then chain them: if the state evolves as a Markov chain, the expected price change over k steps, and in the limit over all future steps, follows from the one-step frequencies by matrix algebra. The formulas below do exactly this; they compute the expectation from calibrated one-step probabilities, not from first principles.

The chained method uses the data more efficiently and gives the long-run limit without choosing a horizon; it is exact only to the extent that the Markov assumption holds. The direct method is the check: if the two disagree for a state, the chain is missing something.

Two features are a choice, not a requirement. Any of the shape features of Section 4.4—second-level depth, slope—or recent order flow could be added to the state. Each addition multiplies the number of states and leaves fewer observations per state, so the estimates become noisier: the same trade-off between richer description and evidence that runs through this survey (Principle 2).

The weighted mid has three defects (Stoikov, 2018). It is not a *martingale*—its expected future value is not its current value, so it is a biased estimate of where the price is going. It is noisy, especially when the spread widens. And it can move the wrong way: if the last lot on a thin ask is cancelled, the ask moves up a tick, yet the weighted mid can fall. The *micro-price* of Stoikov (2018) replaces the ad hoc weighting by an estimate of the expected future mid itself.

Definition. Let $\eta_1 < \eta_2 < \dots$ be the successive times after t at which the mid changes, and let s_t be the spread. The micro-price measures imbalance as the bid side's share of the volume at the two best prices,

$$\iota_t = \frac{v_{1,t}^b}{v_{1,t}^b + v_{1,t}^a} \in [0, 1]. \quad (11)$$

In words: $v_{1,t}^b$ and $v_{1,t}^a$ are the numbers of lots waiting at the best bid and the best ask, so ι_t is the fraction of those lots that are on the buying side. In Figure 1, with 29 lots bid and 14 offered, $\iota_t = 29/43 \approx 0.67$: two thirds of the top-of-book volume is waiting to buy. It is 0.5 when the two queues are equal, near 1 when almost all the volume is on the bid, and near 0 when almost all of it is on the ask.

This is the same information as the *queue imbalance* QI_t of Equation (9), the difference between the two best queue sizes divided by their sum, which runs from -1 to 1 instead of from 0 to 1 . The two are related by

$$\iota_t = \frac{1 + \text{QI}_t}{2},$$

because

$$1 + \text{QI}_t = ((v_{1,t}^b + v_{1,t}^a) + (v_{1,t}^b - v_{1,t}^a)) / (v_{1,t}^b + v_{1,t}^a) = 2v_{1,t}^b / (v_{1,t}^b + v_{1,t}^a).$$

In the example, $\text{QI}_t \approx 0.35$ and $(1 + 0.35)/2 \approx 0.67$. Stoikov (2018) works with the $[0, 1]$ form, which reads directly as a share.

With these ingredients, the micro-price is the limit of the expected mid after more and more price changes,

$$m_t^{\text{micro}} = \lim_{k \rightarrow \infty} \mathbb{E}[m_{\eta_k} \mid \iota_t, s_t]. \quad (12)$$

In words: start from the current imbalance and spread, and ask where the mid will be after the next price change, after the one after that, and so on. The answers settle down to a value; that value is the micro-price. By construction it is the best estimate, given the top of the book, of where the price is heading.

Reading the definition piece by piece. The equation packs four ideas into one line.

The times $\eta_1 < \eta_2 < \dots$

The mid does not move continuously. It stays flat while orders join and leave the two best queues, and jumps when one of them is emptied or an order is placed inside the spread. η_1 is the moment of the next such jump after now, η_2 the jump after that, and so on; m_{η_1} is “the mid just after the next price change”. Counting in price changes rather than in seconds means the definition does not depend on how busy the market is.

The imbalance ι_t

the bid side’s share of the top-of-book volume, defined above.

The conditioning “ $\mid \iota_t, s_t$ ”

The expectation uses only two numbers: the imbalance and the spread. Deeper levels and the history of messages are deliberately left out. This is what “given the top of the book” means, and it is a modelling choice, not a law.

The limit $k \rightarrow \infty$

For each k , $\mathbb{E}[m_{\eta_k} \mid \iota_t, s_t]$ is the average mid after k price changes, over all the ways the book could evolve from today’s state. As k grows, today’s imbalance tells us less and less about each further change, so these averages stop moving. The value they settle on is the micro-price.

Limits of a one-step expectation. The expected mid after one change, $\mathbb{E}[m_{\eta_1} \mid \iota_t, s_t]$, already uses the imbalance, and it is tempting to call it the fair price. But price changes are partly undone: after the thin ask breaks and the mid ticks up, the new book is often lopsided the other way, and the next change is more likely to be down (the bid–ask bounce of Section 3). The one-step answer overshoots. Following the book through further changes adds these predictable corrections, and the limit is a price with no predictable drift left in it: its expected value after any number of further changes is itself. That is the *martingale* property the weighted mid lacks, and it is why the micro-price, in contrast to the weighted mid, is an unbiased estimate of where the price is going—under the model’s assumptions.

Two assumptions make it computable: the pair (ι_t, s_t) evolves as a Markov chain, and the dynamics do not depend on the price level itself. Then

$$m_t^{\text{micro}} = m_t + \sum_{k=1}^{\infty} g^{(k)}(\iota_t, s_t), \quad (13)$$

$$g^{(1)}(\iota, s) = \mathbb{E}[m_{\eta_1} - m_t \mid \iota, s], \quad g^{(k+1)}(\iota, s) = \mathbb{E}[g^{(k)}(\iota_{\eta_1}, s_{\eta_1}) \mid \iota, s].$$

In words: the first term is the expected size of the next price change; each further term is the expected next change, averaged over the states the book can be in after the previous one. The micro-price is the mid plus the sum of all these expected adjustments.

The recursion is the step-by-step reading of the limit. $g^{(1)}(\iota, s)$ answers “how far, on average, does the mid move at the next change, starting from state (ι, s) ?”. To get $g^{(2)}$, look at every state $(\iota_{\eta_1}, s_{\eta_1})$ the book can be in just after that first change, ask the same one-step question from each of them, and average the answers, weighting each state by how likely the book is to land there. $g^{(3)}$ repeats this one change further out. Because the chain does not depend on the price level, the same function $g^{(1)}$ can be reused from every state; only the starting point moves.

Example 4.4 (The micro-price of Figure 1, step by step). The book of Figure 1 has 29 lots at the best bid (5000.00), 14 at the best ask (5000.25), a one-tick spread and a mid of $m_t = 5000.125$, so $\iota_t = 29/43 \approx 0.67$. The probabilities below are stylised; in practice they are the counts of the estimation procedure that follows.

1. *First change.* Suppose that, in the data, books in this state see the mid move up next 70% of the time and down 30% of the time. With a one-tick spread the mid moves by half a tick, 0.125, either way: if the ask queue empties, the ask rises one tick and the mid rises 0.125; if the bid queue empties, the mid falls 0.125. So

$$g^{(1)} = 0.7 \times 0.125 + 0.3 \times (-0.125) = 0.05.$$

2. *Second change.* After an up move the new book tends to lean the other way, and suppose the expected next change from those states is -0.02 ; after a down move suppose it is $+0.02$. Weighting by how likely each first move was,

$$g^{(2)} = 0.7 \times (-0.02) + 0.3 \times (+0.02) = -0.008.$$

This is the bounce: part of the expected first move is expected to be undone.

3. *Third change and beyond.* Repeating the procedure gives smaller and smaller terms, say $g^{(3)} = +0.002$ and the rest negligible.
4. *Add them up.*

$$m_t^{\text{micro}} \approx 5000.125 + 0.05 - 0.008 + 0.002 = 5000.169.$$

The successive estimates of the future mid are 5000.175 after one change, 5000.167 after two and 5000.169 after three: they oscillate and settle. In this stylised case the result happens to be close to the weighted mid (5000.169); in general the two differ, because the micro-price takes its weights from transitions observed in the data rather than from a fixed formula.

Estimation. In practice the imbalance is split into n_ι buckets (for example by quantiles) and the spread into n_s values, so the bucketed state (ι, s) takes $n_\iota n_s$ values. From data one counts three kinds of transition from each state: to another state with the mid unchanged, collected in a matrix $\mathbf{U}$; to each possible size of mid change, in a matrix $\mathbf{J}^{(1)}$; and to another state with the mid changed, in a matrix $\mathbf{J}^{(2)}$. With Δ the vector of possible mid changes and

$$\mathbf{B} = (\mathbf{I} - \mathbf{U})^{-1} \mathbf{J}^{(2)},$$

$$g^{(1)} = (\mathbf{I} - \mathbf{U})^{-1} \mathbf{J}^{(1)} \Delta, \quad g^{(k+1)} = \mathbf{B} g^{(k)}.$$

The factor $(\mathbf{I} - \mathbf{U})^{-1}$ sums over all the steps in which the book changes without the mid moving—the same geometric-series device used for the Hawkes model in Section 7. The data are symmetrised before estimation (each observation is added again with bid and ask swapped and the price change negated), which guarantees that the series converges. Stoikov reports that a handful of terms suffices in practice, and that the micro-price predicts short-term price changes better than either the mid or the weighted mid, for both a large-tick and a small-tick stock.

Computing it in practice. The computation splits into an offline part, done once on historical data, which produces a table of adjustments, and an online part, done at every book update, which is a table look-up. Write

$$g^*(\iota, s) = \sum_{k \geq 1} g^{(k)}(\iota, s)$$

for the total adjustment, so that

$$m_t^{\text{micro}} = m_t + g^*(\iota_t, s_t).$$

Offline: estimate the table g^ .*

1. **Define the states.** The chain needs a finite list of states, so the two continuous inputs are turned into categories.

- *Imbalance.* Recall that $\iota = v_1^b / (v_1^b + v_1^a)$ is the share of the lots at the two best prices that sit on the bid: a number between 0 and 1. Compute it for every historical observation of the book. Then choose a number of buckets n_ι and split $[0, 1]$ into that many intervals. The simplest split is by equal width; with $n_\iota = 5$ the buckets are $[0, 0.2)$, $[0.2, 0.4)$, $[0.4, 0.6)$, $[0.6, 0.8)$ and $[0.8, 1]$. Stoikov’s reference notebook splits by *quantiles* instead: sort all the historical values of ι and place the cut points so that each bucket holds the same number of observations, which avoids buckets that are almost empty. It uses ten buckets for a large-tick stock and four for a small-tick one. Because the symmetrisation step mirrors every observation ($\iota \rightarrow 1 - \iota$), the cut points should be symmetric around 0.5—for quantile buckets, compute the quantiles on the values of ι together with their mirrors $1 - \iota$ —so that the mirror of a bucket is again a bucket.
- *Spread.* Measure the spread in ticks, s/δ , and keep only the values that occur often—for example one and two ticks for a large-tick contract—discarding the rare observations with wider spreads.
- *State.* A state is a pair (imbalance bucket, spread in ticks); with n_ι buckets and n_s spread values there are $n_\iota n_s$ states, which can be numbered $0, 1, \dots, n_\iota n_s - 1$, for instance as bucket $+$ $n_\iota \times$ (spread in ticks $- 1$).

For the book of Figure 1, $\iota = 29/43 \approx 0.67$ and the spread is one tick; with the five equal-width buckets above, ι falls in the fourth bucket, $[0.6, 0.8)$ (number 3 when counting from 0), so the state is number $3 + 5 \times 0 = 3$.

2. **Understand what a transition carries.** The state contains no price: it is only the imbalance bucket and the spread. The price is tracked alongside the chain: every transition from one state to the next carries a change in the mid, which is zero on most steps. Leaving the price level out of the state is the second modelling assumption above—the dynamics do not depend on the level, only on changes. Two kinds of step occur.

- *The book reshuffles at unchanged prices.* Orders join or leave the two best queues, or trade against them without emptying them. The best prices stay put, so the mid does not change, but the queue sizes do, and ι may move to another bucket. These steps are the transitions in $\mathbf{U}$.
- *A best price changes.* A best queue is emptied, or an order is placed inside the spread. The mid jumps, and the best queues are now *different* queues—for example the old second level of the ask becomes the new best ask—so ι and the spread jump to new values as well. These steps are recorded in $\mathbf{J}^{(1)}$ (how far the mid moved) and $\mathbf{J}^{(2)}$ (which state the book landed in).

In the book of Figure 1 (state 3: $\iota \approx 0.67$, one-tick spread), suppose the 14 lots at the best ask are bought or cancelled and the next ask level, 5000.50, holds 40 lots. The best ask is now 5000.50, the best bid is still 5000.00 with 29 lots, the spread is two ticks, the mid has risen from 5000.125 to 5000.25, and $\iota = 29/69 \approx 0.42$. The chain has gone from state (bucket 3, one tick) to state (bucket 2, two ticks), and the transition carries a mid change of +0.125. If a buyer then places an order at 5000.25, inside the spread, the spread returns to one tick and the mid rises again, to 5000.375: another transition, carrying another +0.125.

3. **Record transitions.** For each pair of consecutive observations of the book, record the state before, the state after, and the change in the mid between them. Most changes are zero; the others are multiples of half a tick (the notebook keeps changes of at most one tick).
4. **Symmetrise.** Append a mirrored copy of every record: imbalance bucket $\iota \rightarrow$ its mirror $(1 - \iota)$, same spread, mid change negated. Buying and selling are then treated alike, and the sum of adjustments converges.
5. **Count and normalise.** For each starting state, divide the counts of each kind of transition by the total number of transitions out of that state. This gives $\mathbf{U}$ (moves to another state, mid unchanged), $\mathbf{J}^{(1)}$ (mid moves by each possible amount) and $\mathbf{J}^{(2)}$ (moves to another state, mid changed). Each row of $\mathbf{U}$ and $\mathbf{J}^{(1)}$ together sums to one.
6. **Matrix algebra.** Compute $(I - \mathbf{U})^{-1}$, then

$$g^{(1)} = (I - \mathbf{U})^{-1} \mathbf{J}^{(1)} \Delta$$

and

$$\mathbf{B} = (I - \mathbf{U})^{-1} \mathbf{J}^{(2)},$$

then $g^{(2)} = \mathbf{B}g^{(1)}$, $g^{(3)} = \mathbf{B}g^{(2)}$ and so on, and add them. The notebook stops after six terms. The result is one number per state.

Online: at every book update. Compute ι_t and s_t from the two best queues, find the state they fall in, and add the stored adjustment to the mid:

$$m_t^{\text{micro}} = m_t + g^*(\iota_t, s_t).$$

The cost is one division, one look-up and one addition.

Example 4.5 (The matrix computation with two states). Take a one-tick spread throughout and two imbalance buckets: L ($\iota < 0.5$, more volume at the ask) and H ($\iota \geq 0.5$). The mid moves by $\Delta = (-0.125, +0.125)$. Suppose the counts give, with rows and columns ordered (L, H),

$$\mathbf{U} = \begin{pmatrix} 0.6 & 0.2 \\ 0.2 & 0.6 \end{pmatrix}, \quad \mathbf{J}^{(1)} = \begin{pmatrix} 0.15 & 0.05 \\ 0.05 & 0.15 \end{pmatrix} \text{ (columns: down, up),}$$

so from L the next step is a down move with probability 0.15 and an up move with probability 0.05, and the reverse from H; each row of $\mathbf{U}$ and $\mathbf{J}^{(1)}$ together sums to one.

•

$$I - \mathbf{U} = \begin{pmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{pmatrix},$$

with determinant 0.12, so

$$(I - \mathbf{U})^{-1} = \frac{1}{0.12} \begin{pmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{pmatrix} \approx \begin{pmatrix} 3.33 & 1.67 \\ 1.67 & 3.33 \end{pmatrix}.$$

•

$$\mathbf{J}^{(1)}\mathbf{\Delta} = (0.15(-0.125) + 0.05(0.125), 0.05(-0.125) + 0.15(0.125)) = (-0.0125, +0.0125),$$

the expected mid change on the very next step.

•

$$g^{(1)} = (I - \mathbf{U})^{-1}\mathbf{J}^{(1)}\mathbf{\Delta} \approx (-0.0208, +0.0208) :$$

the expected size of the next mid change, counting all the steps on which the book reshuffles before it.

Now suppose that after a down move the new book is in H with probability 0.6, and after an up move it is in L with probability 0.6—the bounce. Then from L,

$$\mathbf{J}_{L \rightarrow L}^{(2)} = 0.15 \times 0.4 + 0.05 \times 0.6 = 0.09$$

and

$$\mathbf{J}_{L \rightarrow H}^{(2)} = 0.15 \times 0.6 + 0.05 \times 0.4 = 0.11,$$

and symmetrically from H, so

$$\mathbf{J}^{(2)} = \begin{pmatrix} 0.09 & 0.11 \\ 0.11 & 0.09 \end{pmatrix}, \quad \mathbf{B} = (I - \mathbf{U})^{-1}\mathbf{J}^{(2)} \approx \begin{pmatrix} 0.483 & 0.517 \\ 0.517 & 0.483 \end{pmatrix}.$$

Each row of $\mathbf{B}$ sums to one: it is the distribution of the state just after the next mid change. Then

$$g^{(2)} = \mathbf{B}g^{(1)} \approx (+0.0007, -0.0007),$$

opposite in sign to $g^{(1)}$ because of the bounce, and $g^{(3)} \approx (-0.00002, +0.00002)$. Adding,

$$g^* \approx (-0.0202, +0.0202).$$

A book in state H has a micro-price about 0.02 points—a sixth of a half-tick—above the mid; a book in state L, the same amount below.

A minimal implementation of the offline part, with the states already numbered and the data already symmetrised:

```
import numpy as np

def fit_microprice(x, y, dm, n_states, jumps, n_terms=6):
    # x, y: state before / after each step; dm: mid change;
    # jumps: sorted nonzero mid changes
    U = np.zeros((n_states, n_states))
    J1 = np.zeros((n_states, len(jumps)))
    J2 = np.zeros((n_states, n_states))
    for xi, yi, d in zip(x, y, dm):
        if d == 0:
            U[xi, yi] += 1
        else:
            J1[xi, np.searchsorted(jumps, d)] += 1
            J2[xi, yi] += 1
    total = U.sum(1) + J1.sum(1) # all transitions out of each state
    U, J1, J2 = (M / total[:, None] for M in (U, J1, J2))
    # A sums over steps with no mid change
    A = np.linalg.inv(np.eye(n_states) - U)
```

```

g = A @ J1 @ jumps          # g^(1)
B = A @ J2
g_star = g.copy()
for _ in range(n_terms - 1):
    g = B @ g                # g^(k+1) = B g^(k)
    g_star += g
return g_star                # one adjustment per state

# online, at every book update:
# micro = mid + g_star[state(iota, spread)]

```

The bucket counts, the six-term truncation and the symmetrisation follow Stoikov's reference notebook and talk slides; the published paper may present them differently.

The matrix formula as a sum over all paths. The conditional expectation $\mathbb{E}[\cdot \mid \iota_t, s_t]$ is not read off the current book directly. The current book only picks the starting state x ; the expectation is an average over every way the chain can evolve from x , with each path weighted by its probability, and the probabilities are transition frequencies counted in historical data. The matrix products enumerate those paths exactly. The next mid change can happen on the first step, or after one step on which the book reshuffles without the mid moving, or after two such steps, and so on:

$$g^{(1)}(x) = [\mathbf{J}^{(1)}\Delta](x) + [\mathbf{U}\mathbf{J}^{(1)}\Delta](x) + [\mathbf{U}^2\mathbf{J}^{(1)}\Delta](x) + \dots = [(I - \mathbf{U})^{-1}\mathbf{J}^{(1)}\Delta](x).$$

In words: entry (x, y) of $\mathbf{U}^j$ is the total probability of all paths that go from x to y in j steps without the mid moving, and the inverse

$$(I - \mathbf{U})^{-1} = I + \mathbf{U} + \mathbf{U}^2 + \dots$$

adds every possible number of such steps at once.

In Example 4.5, starting from H, the mid moving at once contributes 0.0125; one reshuffle first contributes

$$0.2 \times (-0.0125) + 0.6 \times 0.0125 = 0.005;$$

two reshuffles contribute 0.002; each further term is 0.4 times the previous one, and the total is

$$0.0125/(1 - 0.4) \approx 0.0208 = g^{(1)}(\text{H}).$$

The matrix $\mathbf{B}$ then carries the same bookkeeping across successive price changes.

The same expectation by Monte Carlo. Because the micro-price is an average over random paths, it can also be estimated by simulating those paths and averaging, instead of summing them in closed form:

1. start a path in the current state x , with the mid at zero;
2. at each step, draw the next event from row x of the estimated transition probabilities: a move to another state with the mid unchanged (probabilities in $\mathbf{U}$), or a mid change by Δ_j together with the state the book lands in (probabilities in $\mathbf{J}^{(2)}$, split by the size of the change);
3. continue until the mid has changed k times, for k large enough that further changes no longer move the average (six, as in the closed form), and record the cumulative mid change;
4. repeat for N_{paths} independent paths and average the recorded changes; the average estimates $g^*(x)$.

```

import numpy as np
rng = np.random.default_rng(0)

def microprice_mc(x0, U, Jk, jumps, n_changes=6, n_paths=100_000):
    # U[x, y]: P(go to y, mid unchanged)
    # Jk[j][x, y]: P(mid changes by jumps[j], go to y)
    n = U.shape[0]
    P = np.hstack([U] + list(Jk))    # row x: all possible next steps from x
    total = 0.0
    for _ in range(n_paths):
        x, dm, changes = x0, 0.0, 0
        while changes < n_changes:
            c = rng.choice(P.shape[1], p=P[x])
            # block 0: no mid change; block j: jump jumps[j-1]
            block, x = divmod(c, n)
            if block > 0:
                dm += jumps[block - 1]
                changes += 1
        total += dm
    return total / n_paths          # estimate of g_star[x0]

```

Simulation needs one extra estimate the closed form does not: the joint probability of a jump of a given size *and* the state the book lands in, which is $\mathbf{J}^{(2)}$ split by jump size. The two methods estimate the same number. The closed form is exact for the fitted chain and costs a matrix inverse; the Monte Carlo estimate carries sampling noise whose standard deviation falls like $1/\sqrt{N_{\text{paths}}}$, so halving the error needs four times as many paths. On the two-state chain of Example 4.5, starting from H, the closed form gives 0.0202; one run of 40,000 simulated paths gave 0.0231, and one run of a million paths gave 0.0198 with a standard error of 0.0003. For the small chain of the micro-price—a few dozen states—the closed form is the better choice. Neither method removes *estimation* noise: both use transition probabilities counted from finite data, and that noise passes into the result.

Simulation becomes the right tool when the closed form is out of reach. If the state is enlarged with history—recent OFI, the sign of the last move, deeper levels—the number of states multiplies and the matrices become too large to invert or too sparse to estimate. And if the model is not a finite Markov chain at all, as for the Hawkes and neural point processes of Section 7, whose intensities depend on the whole history, there is no transition matrix to invert: the expected mid after k changes is then estimated by simulating event paths from the current history, for example by thinning (Section 7.12), and averaging. This is the same choice Section 6.1 describes for queue models, where Cont, Stoikov and Talreja compute their probabilities exactly with Laplace transforms rather than by simulating the book.

Model card: Micro-price

What you calibrate. The transition matrices $\mathbf{U}$, $\mathbf{J}^{(1)}$, $\mathbf{J}^{(2)}$ between imbalance–spread buckets, after choosing the numbers of buckets n_l and n_s .

How. Bucket the imbalance and spread, symmetrise the data, and count transitions; the adjustments $g^{(k)}$ then follow by matrix algebra, and a handful of terms suffices.

What you do with it. A fair-value estimate to quote around in place of the mid, and a better reference than the mid or weighted mid for predicting short-term price changes.

How to picture it. A weather forecast that starts from today’s state and averages over every path the next few changes can take: the micro-price is where those paths land on average.

4.6 Order flow across instruments

The signals above use the book of one instrument. Related instruments—an index future and its constituents, ES and NQ, futures and the cash market, the points of the Treasury curve—trade on related information, so the book of one can carry information about another. This subsection extends OFI to several instruments, summarises the evidence on stocks and on the Treasury curve, and then turns to lead-lag between futures and cash, the form in which cross-instrument information is mainly used in execution. Cross-asset Hawkes models are in Section 7.18 and multi-asset deep models in Section 10.11.

Cross-impact regression. A simple cross-impact form is

$$m_t^{(\kappa)} - m_{t-w}^{(\kappa)} = \sum_{\kappa'} \beta_{\kappa\kappa'} \text{OFI}_t^{(w,\kappa')} + \varepsilon_t^{(\kappa)}, \quad (14)$$

where the superscripts (κ) and (κ') index instruments, $m_t^{(\kappa)}$ is the mid of instrument κ , $\text{OFI}_t^{(w,\kappa')}$ is the order-flow imbalance of instrument κ' , $\beta_{\kappa\kappa'}$ is the cross-impact of instrument κ' 's flow on instrument κ 's mid, and $\varepsilon_t^{(\kappa)}$ is a residual.

In words: the same straight-line fit as before, except that each instrument's price change may respond to the buying pressure in *every* instrument. The coefficients form a matrix; the diagonal is each instrument's own impact and the off-diagonal entries are the cross effects.

The form covers index constituents, ETF and constituents, futures and cash, correlated equities, related fixed-income contracts and cross-venue cryptocurrency markets.

Example 4.6 (Coefficients multiply). For NQ,

$$m_t^{(\text{NQ})} - m_{t-w}^{(\text{NQ})} = \beta_{\text{NQ,NQ}} \text{OFI}_t^{(w,\text{NQ})} + \beta_{\text{NQ,ES}} \text{OFI}_t^{(w,\text{ES})} + \varepsilon_t^{(\text{NQ})}$$

has two coefficients. Ten related instruments, each predicted from all ten OFIs at five lags, gives $10 \times 10 \times 5 = 500$ coefficients. At a 5% level about 25 are “significant” under the null alone.

Evidence on stocks. Cont et al. (2023), on the 100 largest S&P 500 stocks over 2017–2019, ask whether the order flow of *other* stocks helps. Two answers are needed. For the *contemporaneous* price change, once integrated OFI is used, adding other stocks' order flow does not improve the out-of-sample fit. For *forecasting* the next minute's return, lagged order flow of other stocks does help significantly, although the out-of-sample R^2 of all the forecasting models remains slightly negative and the effect decays within minutes. Explaining the present and predicting the future are, again, different problems.

The Treasury curve. For interest-rate futures the related instruments are the points of the yield curve. On daily inter-dealer data from 1992 to 1999, order-flow imbalances explain up to 26% of day-to-day yield changes on days without major announcements, and most of that effect comes from a factor common to all maturities: net buying across the whole curve says more about the level of yields than buying in one maturity (Brandt and Kavajecz, 2004). At one-minute resolution, Tomas et al. (2022) compare cross-impact models on 10-year Treasury note futures and E-mini futures jointly and find that models built on the covariance of returns and order flow, such as a multivariate Kyle model, fit out of sample far better than a model with only direct (own-instrument) impact. We found no published intraday cross-impact study covering the full Treasury futures curve.

Lead–lag between futures and cash. The clearest cross-asset structure in futures is that one market moves first. For the S&P 500 and Nasdaq-100, most price discovery happens in the E-mini futures rather than in the cash index or ETF (Hasbrouck, 2003). At high frequency the lead is measured in milliseconds and is bounded by the physical distance between data centres. Between Chicago, where CME matches, and the New Jersey equity venues, the response of SPY to E-mini price changes took 7.25–7.95 ms in April 2010, fell after a straighter fibre route was built, and was estimated at 4.2–5.2 ms over microwave links, against a speed-of-light minimum of 3.93 ms; about 20% of SPY trading could be identified as a direct response to E-mini moves (Laughlin et al., 2014). Measuring how often trades in two markets occur at a given time offset, Dobrev and Schaumburg (2017) find activity peaks at ± 5 ms between 10-year Treasury futures and cash Treasuries, with leadership running both ways, a clear lead of the E-mini over SPY at +5 ms, and a lead of the E-mini over cash Treasuries but not the reverse. The order-flow link between SPY and the E-mini is “positive, but short-lived”: beyond one second the correlations have little economic significance (Garrison et al., 2019).

Estimating the lag. Two markets trade at different, irregular times, so the usual correlation of returns sampled on a common grid is biased towards zero (the Epps effect). The Hayashi–Yoshida estimator avoids the grid by summing products of returns over every pair of overlapping intervals; shifting one series by a candidate lag and choosing the lag that maximises the estimated correlation gives a consistent lead–lag estimator (Hoffmann et al., 2013). On European data, Huth and Abergel (2014) find with this method that the CAC 40 future leads a large constituent stock by about 0.6 seconds on average, and that the leader’s past alone predicts the direction of the lagger’s next mid-quote move about 60% of the time—yet a naive strategy that crosses the spread on that forecast does not make money.

Cross-asset models should therefore report in-sample and out-of-sample improvement, incremental economic value, added latency, parameter count and stability across regimes.

Model card: Cross-asset models

What you calibrate. The cross-impact matrix $\beta_{\kappa\kappa'}$ and the lags at which it acts; for a cross-asset Hawkes model, the cross-excitation kernels between instruments.

How. Regression of each instrument’s price change on the order flow of all (with shrinkage or a factor structure to control the number of coefficients); lead–lag by maximising a Hayashi–Yoshida correlation over candidate lags; Hawkes cross-kernels by maximum likelihood.

What you do with it. Use the leader’s move to anticipate the lagger—mainly to protect resting orders in the lagging market from being picked off—and hedge across related contracts.

How to picture it. Thunder after lightning: the futures move is the flash and the cash market the sound, a few milliseconds later; the lag is set by distance.

The signals at a glance. Table 7 summarises the signals of this section.

The central empirical question is not whether richer representations improve in-sample fit, but whether the improvement survives out-of-sample evaluation and execution costs.

5 Events rather than snapshots

The previous section was about *snapshot models*: signals computed from the sizes and prices visible in the book at a moment, or from the change between two moments. This section marks the turn to *event models*, which work with the individual events that produce those changes—each order

Table 7: Classical order-book signals at a glance. Horizons are indicative of how the signals are used in the cited literature, not guarantees.

Signal	What it measures	Typical horizon	How it is fitted	Known failure modes
OFI, best level (Cont et al., 2014)	net buying pressure at the touch	seconds to minutes (contemporaneous fit)	linear regression; slope re-estimated with depth	weak in thin, noisy names; distorted by spoofing
Multi-level / integrated OFI (Xu et al., 2018; Cont et al., 2023)	pressure across the first levels	same	regression; principal components	more inputs to monitor; depth beyond the touch is less committed
Queue imbalance (Gould and Bonart, 2016)	skew of the two best queues	next mid change	logistic regression	weak for small-tick names; misled by hidden size
Weighted mid	price leaning toward the thin side	next mid change	none	noisy; not a martingale
Micro-price (Stoikov, 2018)	expected future mid given imbalance and spread	a few mid changes	Markov chain on imbalance and spread buckets	assumes Markov dynamics; needs symmetrised data
Book slope, curvature	how liquidity is spread across levels	longer	descriptive	sensitive to transient deep orders
Resilience (Large, 2007)	speed of refill after trades	seconds	Hawkes impulse response	varies strongly across regimes
Cross-asset OFI (Cont et al., 2023)	pressure in related instruments	lagged: about a minute	regression with shrinkage	many coefficients; gain decays within minutes
Lead-lag (Huth and Abergel, 2014)	which market moves first	milliseconds to seconds	Hayashi–Yoshida correlation over lags	one test: not profitable after crossing the spread

added, cancelled, modified or executed—and therefore know not only *what* changed in the book but *why*. The queue models, Hawkes processes and message-level networks of the following sections are all event models.

The same reduction in displayed volume has different information content depending on its cause. A snapshot representation can make two states identical even when their histories differ (Example 4.3). Market-by-order (L3) data carry the cause: every add, modify, cancel and execution, with an order identifier. On CME’s MDP 3.0 feed, for instance, book updates for individual orders and trade summaries arrive as separate message templates, so the two books in Example 4.3 produce visibly different message sequences.

Treating the three event types jointly is not optional. Price changes are often attributed to market orders, but in Nasdaq data roughly half of all price-changing events are limit orders placed *inside* the spread, which improve the best price rather than consume it (Eisler et al., 2012). A model that sees only trades misses half of what moves the price.

The case for message-level representation has recently been made directly in deep learning. LOBERT (Linna et al., 2025) adapts the BERT encoder to LOB data by treating each complete multi-dimensional message as a single token, while keeping continuous representations of price, volume and time. That sentence needs unpacking.

Tokens and encoders. Language models do not read text letter by letter. They first cut it into *tokens*—words or pieces of words—and replace each token by an integer from a fixed vocabulary; the model then learns a vector (an embedding) for each vocabulary entry and processes the sequence of vectors. BERT (Devlin et al., 2019) is an *encoder*: a Transformer that reads a whole sequence of tokens and produces, for each position, a vector summarising that token in its context. It is pretrained by hiding some tokens and asking the network to fill them in, and then adapted (*fine-tuned*) to specific tasks. To apply the same machinery to an order book, one must decide what a “token” of market data is.

Fields of a message. One exchange message describes one event, and it has several fields: its type (new order, modification, cancellation, execution), its side (buy or sell), its price, its size, and the time since the previous message. That is what “multi-dimensional” means: a message is a small record, not a single number. There are two ways to turn such records into tokens. One can split each message into several tokens—one for the type, one for the side, one for the price, and so on—so that a sequence of n messages becomes several times n tokens. Or one can map the *whole* message to one token. LOBERT does the latter, which its authors report shortens sequences by roughly an order of magnitude, so that the same context window of 512 tokens covers far more market history.

Encoding a message as one token. A single vocabulary entry cannot hold every possible price and size, so the fields are first coarsened into a few categories. In LOBERT the price is measured as the distance in ticks from the best quote on the opposite side and rounded into a few bins (0, 1, 2, 3, 5 and 10 ticks); the size is rounded into a few bins (0, 50, 100 and 200 shares) with a flag for round lots; and these are combined with the side and the type. Each combination that occurs in the training data gets one integer; with a few special tokens this gives a vocabulary of 293 message tokens. A new sell order for 100 shares one tick above the best bid, for example, becomes a single integer meaning “new / sell / 1 tick from the opposite quote / about 100 shares”. Rounding loses detail, so the exact price, size and time gap are also fed to the network as continuous numbers alongside the token. The token says what *kind* of event happened; the continuous values say exactly where, how large and when.

Pretraining and reported results. LOBERT is pretrained by *masked message modelling*: messages are hidden and the network predicts them, as BERT predicts hidden words. It is designed as a general-purpose, fine-tunable foundation model, and is trained and evaluated on Nasdaq ITCH message data for four large US stocks in 2015, with millisecond timestamps. Its results on next-message and mid-price prediction are discussed in Section 10.10.

A message-level encoder sees the difference between a cancellation and an execution in Example 4.3; a snapshot encoder, by construction, does not.

Plan of the following sections. LOBERT is mentioned here because it shows, in deep learning, the case this section makes: the individual event carries information that a snapshot loses. The next three sections are not about Transformers or tokens. They return to classical models that work directly with order-book events—orders arriving, cancelling and trading at the best queues—and describe them with probability: how long a queue survives before it is emptied (Section 6) and how one event makes further events more likely (Section 7). Deep networks, including message-level encoders such as LOBERT, are taken up again in Section 10.

6 Queue depletion and first-passage models

The idea: a queue as a bucket that fills and drains. Think of the best-ask queue as a bucket of lots. New sell limit orders pour lots in; buy market orders that trade against it, and sellers who cancel, drain lots out. Each event adds or removes a random amount at a random time, so the level of the bucket wanders up and down like a random walk. The question this section asks is: *how long until the bucket is empty?* That moment matters for three reasons.

1. When the best-ask queue empties, the best ask moves up a tick, and so does the mid. Price changes *are* queue depletions.
2. The direction of the next price move is a race between the two best queues: whichever empties first decides it. This is the formal version of “the thin side breaks first” (Section 4).
3. For a resting order, the wait until it is filled is the time until the volume *ahead of it* is used up—the same question, asked about part of a queue (Section 11.2).

The definition. The depletion time of the best ask is the first-passage time

$$\tau^a = \inf\{u > 0 : v_{1,t+u}^a \leq 0\}, \quad (15)$$

where u runs over time after t ; τ^b is defined in the same way for the bid.

In words: τ^a is how long from now until the best-ask queue is empty for the first time—the time at which a loop that polls the queue size would first see zero. “First passage” is the general name for the first time a wandering quantity reaches a level.

Reading the notation. The definition is written in set notation, read from the inside out.

$v_{1,t+u}^a$

the number of lots at the best ask (level 1, side a) at time $t + u$, that is, u seconds after now. As u grows, this traces the queue’s size into the future.

$$v_{1,t+u}^a \leq 0$$

the condition “the queue is empty at time $t + u$ ”. It is written “ ≤ 0 ” rather than “ $= 0$ ” only as a safeguard; a queue size cannot be negative, so the two mean the same thing.

$$\{u > 0 : \dots\}$$

read “the set of all future times u such that $\dots$ ”—here, the set of all moments after now at which the queue is empty. The colon is read “such that”.

inf

the *infimum*, the greatest lower bound of the set; for a set of times it is simply the earliest one. It is written inf rather than min for two technical reasons: a set of real times need not contain its earliest point, and if the queue never empties the set is empty, and the convention $\inf \emptyset = \infty$ gives $\tau^a = \infty$ —the queue never depletes.

So the whole expression reads: *the earliest time after now at which the best-ask queue is empty*. In code, it is the loop

```
u = 0.0
while ask_size(t + u) > 0:      # ask_size(.) = v^a_1 at that time
    u = time_of_next_event_after(t + u) - t
tau_a = u                      # first time the queue is seen empty
```

run on the future of the book—which is why τ^a is a random quantity, not a number known now: the models of this section give its probability distribution.

Example 6.1 (One depletion time). In Figure 1 the best ask holds 14 lots at 5000.25. Suppose that, from now ($u = 0$), a buy market order takes 6 lots at $u = 0.4$ s (8 left), a seller adds 3 lots at $u = 0.9$ s (11 left), a seller cancels 5 lots at $u = 1.3$ s (6 left), and a buy market order takes 6 lots at $u = 2.0$ s (0 left). The set of times at which the queue is empty begins at $u = 2.0$ s, so $\tau^a = 2.0$ s; at that moment the best ask moves up to 5000.50 and the mid rises. On another path of the future—more sellers joining, fewer buyers—the same queue might last a minute. The distribution of τ^a across all such paths is what the models below estimate.

The race between the two queues is then

$$\mathbb{P}(\text{next mid move is up} \mid X_t) = \mathbb{P}(\tau^a < \tau^b \mid X_t), \quad (16)$$

In words: the price goes up next exactly when the ask queue empties before the bid queue does.

A first estimate. If lots drain out at an average rate of r_{out} lots per second and pour in at r_{in} , the bucket loses $r_{\text{out}} - r_{\text{in}}$ lots per second on average, and a best-ask queue of $v_{1,t}^a$ lots takes on average

$$\mathbb{E}[\tau^a] \approx \frac{v}{r_{\text{out}} - r_{\text{in}}} \quad (r_{\text{out}} > r_{\text{in}})$$

seconds to empty: size divided by net drain rate, as for any bucket. Two warnings come with this formula. First, it is an average; the actual time is random and often far from it. Second, if inflow keeps up with outflow ($r_{\text{in}} \geq r_{\text{out}}$), the bucket has no net leak and the expected time can be very long or infinite.

Models. Stochastic models of the book (Cont et al., 2010) treat arrivals, cancellations and executions as random event streams (point processes) and derive the distribution of first-passage times and the probability of a mid move in either direction, including the race above. Queue-reactive

models (Huang et al., 2015) go one step further and let the rates themselves depend on the current queue sizes—for example, orders may join or cancel at different rates when a queue is short than when it is long. That reproduces many observed regularities of large-tick assets.

6.1 The Cont–Stoikov–Talreja model

The model of Cont et al. (2010) is the foundation on which most later queue-based models of the order book are built, and it repays careful study. It was one of the first models of the whole book to satisfy three requirements at once: its parameters can be estimated directly from order-book data with simple counting; it reproduces important empirical features of the book without being tuned to do so; and it is simple enough that the probabilities a trader actually needs—will the price go up next, will my order be filled before the price moves—can be *computed*, quickly and exactly, rather than estimated by simulation. Its central idea is that a limit order book is a queueing system: limit orders wait in line to be served by market orders, or leave the line by cancelling.

6.1.1 The model

The model at a glance. The whole model has four parts; the paragraphs below take them one at a time.

1. **State:** the number of orders resting at each price tick on each side of the book, counted in units of the average order and measured by distance from the opposite best quote.
2. **Events:** on each side, three kinds of event arrive as independent Poisson streams—limit orders at each distance ℓ , at rate $r_L(\ell)$; market orders at the best quote, at rate r_M ; and cancellations, each resting order at distance ℓ being cancelled at rate $r_c(\ell)$.
3. **Price mechanics:** when the best bid or best ask queue is emptied, that price moves one tick and the mid moves with it; distances are then measured from the new quotes.
4. **Consequence:** because every rate depends only on the current queue sizes, the book is a continuous-time Markov chain, and the probabilities a trader needs follow from the two best queues alone.

In words: the consequence is the payoff of the first three choices, and it has two halves. *Why Markov:* a Poisson stream has no memory—the chance of an arrival in the next instant is the same however long ago the last one was—and every rate in the model depends only on how many orders sit in each queue *now*. So to know how the book will evolve, it is enough to know its current queue sizes; how it got there adds nothing. That is what “Markov” means, and it is what makes the model computable: the future can be worked out from the present state with no record of the past. *Why only the two best queues:* the next price change happens when the best bid or best ask queue is emptied (or an order lands inside the spread). Orders deeper in the book do not touch either best queue until the price has already moved, so for questions about the *next* move—will it be up, will my order at the best bid fill first—the deeper levels can be ignored. The large book collapses to two counters racing to zero, which is what the rest of this subsection analyses.

Parameters to estimate. The model is fully specified once a handful of numbers are known. Each is a *rate*: an average number of events per unit of time (per minute in the paper’s tables), for one kind of event at one place in the book.

The unit of size

Before anything is counted, sizes are converted into “orders”: a queue of 300 shares, when the average limit order is 100 shares, counts as 3. This is a choice of units, not a quantity with economic meaning, but every rate below is expressed in it.

Limit-order rates $r_L(\bar{\ell})$, summarised by r_0 and Ξ

How often a new limit order arrives at distance $\bar{\ell}$ ticks from the opposite best quote: one rate for $\bar{\ell} = 1$, one for $\bar{\ell} = 2$, and so on. Rather than estimating each separately, the model assumes they follow the power law $r_L(\bar{\ell}) = r_0/\bar{\ell}^\Xi$ introduced below, so two numbers describe them all. r_0 is the scale—the rate at distance one—and Ξ says how fast the rate falls with distance. In the paper’s data, $r_0 \approx 1.92$ per minute and $\Xi \approx 0.52$, a pure number (Table 8): about 1.9 new orders a minute one tick away, about $1.92/2^{0.52} \approx 1.3$ two ticks away.

One market-order rate r_M

How often a market order arrives and removes one unit from the best quote on the other side. Market orders only ever hit the best price, so a single number suffices; in the paper’s data, $r_M \approx 0.94$ per minute.

Cancellation rates $r_c(\bar{\ell})$, one per distance

The chance per minute that *any one* resting order at distance $\bar{\ell}$ is cancelled. Because it is per order, a level holding 10 orders loses about $10 r_c(\bar{\ell})$ of them a minute to cancellation, and a level holding 2 orders about $2 r_c(\bar{\ell})$. These, in contrast to the limit-order rates, are estimated separately at each distance; in the paper’s data they lie between about 0.47 and 0.81 per minute.

For five price levels that is eight numbers in all: r_0 , Ξ , r_M and five cancellation rates. Most of them are book-keeping—how busy the market is. Two of the modelling choices behind them, however, encode how traders behave, and these carry the economic content of the model: *where* new orders are placed (mostly near the price, the power law), and *how* cancellation depends on queue size (in proportion to it, which keeps queues from growing without bound). They are discussed next.

State. Prices live on a grid of ticks. The state of the book is the number of orders resting at every price on the grid, counted in units of the average limit-order size. Because most activity happens near the current price, the model measures position relative to the quotes: let $Q_{\bar{\ell}}^{\text{bid}}(t)$ be the number of buy orders $\bar{\ell}$ ticks below the best ask, and $Q_{\bar{\ell}}^{\text{ask}}(t)$ the number of sell orders $\bar{\ell}$ ticks above the best bid. Note the convention: distance is measured from the *opposite* best quote. With a one-tick spread, the best bid is at distance $\bar{\ell} = 1$ from the best ask.

Events and their rates. Six kinds of event change the book, and each is modelled as an independent Poisson stream (Section 7 explains what that assumption means and where it fails):

- a **limit buy** order arrives $\bar{\ell}$ ticks below the best ask, and a **limit sell** order $\bar{\ell}$ ticks above the best bid, each at rate $r_L(\bar{\ell})$;
- a **market buy** order removes one unit from the best ask, and a **market sell** order one unit from the best bid, each at rate r_M ;
- each resting order $\bar{\ell}$ ticks from the opposite quote is **cancelled** independently at rate $r_c(\bar{\ell})$, so a level holding Q orders loses orders to cancellation at total rate $r_c(\bar{\ell}) Q$.

The two choices with economic content are these. First, limit orders arrive more often near the current price; following earlier empirical work (Bouchaud et al., 2002), the arrival rate is a power

law in distance,

$$r_L(\bar{\ell}) = \frac{r_0}{\bar{\ell}^\Xi}, \quad (17)$$

with a scale r_0 and exponent Ξ fitted to data.

In words: the farther a price is from where trading happens, the fewer orders are placed there, and the fall-off follows a power law: doubling the distance divides the rate by 2^Ξ .

Second, cancellations are proportional to queue size: each order has the same chance per second of being cancelled, so a long queue sheds orders faster than a short one. This single assumption is what keeps queues from growing without bound.

Under these assumptions the book is a continuous-time Markov chain: its future depends only on the current queue sizes, not on how they were reached. The authors show that a book that starts with the bid below the ask always keeps that property, and that if every cancellation rate is positive the chain is *ergodic*—it has a unique long-run distribution, so averages over a long simulated history converge to well-defined values, such as the average shape of the book.

6.1.2 Estimating the model

The estimation is deliberately elementary, which is part of the model's appeal. Every rate is estimated the same way: *count how many times the event happened, and divide by how long you watched*. If a bus stop sees 30 buses in 10 hours, the estimated rate is 3 buses an hour. The only complications are working out, from the data, which changes in the book were which kind of event, and, for cancellations, accounting for the fact that each resting order is cancelled separately.

One detail of the data explains the method. The authors observe the sizes at the first five price levels and the trades, but not individual orders (market-by-price data, Section 1). They therefore cannot see a limit order arrive; they see a queue get bigger. So an increase in a queue's size is counted as a limit-order arrival, a decrease that coincides with a trade as a market order, and a decrease with no trade as a cancellation. All sizes are converted to the unit of the average limit order.

Write T_{obs} for the total trading time observed, and $N_L(\bar{\ell})$ for the number of times the queue $\bar{\ell}$ ticks from the opposite best quote increased during that time. A hat on a symbol marks an estimate from data. The three kinds of rate are then:

Limit orders, at each distance

$$\hat{r}_L(\bar{\ell}) = \frac{N_L(\bar{\ell})}{T_{\text{obs}}}.$$

In words: the number of new orders seen at that distance, divided by the time watched. If over $T_{\text{obs}} = 600$ minutes of trading the queue one tick from the opposite quote grew 1,110 times, then $\hat{r}_L(1) = 1,110/600 = 1.85$ new orders a minute.

Doing this at distances one to five gives five rates. The power law $r_L(\bar{\ell}) = r_0/\bar{\ell}^\Xi$ is then fitted to them, so that the rate can be extrapolated to distances further out: taking logarithms gives $\ln r_L = \ln r_0 - \Xi \ln \bar{\ell}$, a straight line in $\ln \bar{\ell}$, and least squares on the five points gives the intercept $\ln r_0$ and the slope $-\Xi$.

Market orders

the number of market orders that changed the best quotes, converted to units of the average limit order, divided by T_{obs} .

In words: if 470 market orders hit the best quotes in 600 minutes and a market order is on average 1.2 times the size of a limit order, they removed $470 \times 1.2 = 564$ units, and $\hat{r}_M = 564/600 = 0.94$ a minute.

Cancellations, at each distance

the number of cancelled units at that distance, divided by T_{obs} , and *also divided by the average number of orders resting there*. The extra division is needed because the model’s cancellation rate is per order: a level holding three orders sees three times as many cancellations as a level holding one, and dividing by the average queue size undoes that.

In words: if 1,278 units were cancelled one tick from the opposite quote in 600 minutes, that is 2.13 a minute in total; if the queue there held three orders on average, each order was cancelled at a rate of $2.13/3 = 0.71$ a minute.

The worked numbers are chosen to reproduce the paper’s estimates in Table 8; the counts behind them are not reported in the paper.

The authors use five levels of quotes and trades for Tokyo Stock Exchange stocks over 125 trading days (August to December 2006) and report estimates for Sky Perfect Communications (Table 8).

Table 8: Estimated Cont–Stoikov–Talreja parameters for Sky Perfect Communications, Tokyo Stock Exchange, 2006 (Cont et al., 2010). Rates are per minute, in units of the average limit-order size.

Distance from opposite quote $\bar{\ell}$	1	2	3	4	5
Limit-order rate $\hat{r}_L(\bar{\ell})$	1.85	1.51	1.09	0.88	0.77
Per-order cancellation rate $\hat{r}_c(\bar{\ell})$	0.71	0.81	0.68	0.56	0.47
Market-order rate $\hat{r}_M = 0.94$; power-law fit $\hat{r}_0 = 1.92$, $\hat{\Xi} = 0.52$					

6.1.3 From the whole book to two counters racing to zero

Purpose of this subsection. The full model tracks every queue in the book, which is too much to compute with directly. But a trader’s questions—will the price go up next, will my order fill before the price moves—concern only the *next* price change, and until that change the best prices stay where they are. During that time only two things matter: the number of orders at the best bid and the number at the best ask. Each goes up when an order joins and down when one is traded or cancelled, and the price moves when either reaches zero. So the whole book can be replaced, for these questions, by two counters racing to zero. This subsection describes those counters and why they can be treated separately.

Justification of the simplification. Three observations make it work, and each follows from the assumptions of the model.

1. **Only the next price change matters.** The questions are all about what happens *first*: which side breaks, and whether a resting order is reached before it does. Whatever the book does after the price has moved is irrelevant to them, so the model only has to be followed up to that moment.
2. **Until then, nothing else in the book can interfere.** While the best prices stay put, the distances of all queues from the quotes stay put, so every rate in the model stays constant.

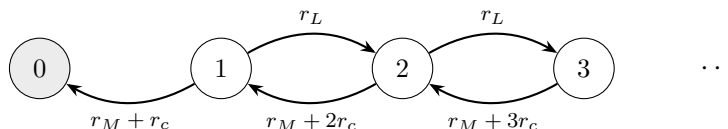
Orders arriving or cancelling deeper in the book change those deeper queues, but they do not touch the best bid or best ask queue. The deeper levels only come into play once a best queue is empty—which is exactly the moment we stop following the book.

3. **The two best queues do not affect each other.** In the model, the orders that join, trade against or cancel from the best bid are independent of those at the best ask: separate Poisson streams, with rates that depend only on each queue’s own size. Knowing that the ask queue just lost an order says nothing about what the bid queue does next. So the two counters can be analysed one at a time, and the probability that one wins the race combined from the two separately.

The payoff is large. A book with many levels on each side, whose state is a long list of queue sizes, reduces to two numbers, $v_{1,t}^b$ and $v_{1,t}^a$, each moving up and down on its own.

Role of the deeper levels. For questions about the *next* price change, yes, and within the model this is not an approximation: by points 2 and 3, nothing deeper in the book can change either best queue before one of them empties, so dropping the deeper levels changes no answer. Two qualifications apply. First, when the spread is wider than one tick, a new limit order placed *inside* the spread also moves the mid; the model accounts for it through the limit-order rates at those distances, which act as additional competing clocks, not through any deeper queue. Second, the deeper levels matter as soon as the price has moved: the old second level becomes the new best quote, so questions about several price changes—the average shape of the book, or volatility over a day—need the full model, and the authors answer them by simulating it (Section 6.1, “What the model reproduces”). Outside the model, ignoring depth is a real loss: the book beyond the best quotes carries part of the price information (Cao et al., 2009), and multi-level order flow improves price-impact fits (Section 4); the model cannot use that information because it assumes the deeper queues have no influence on the best ones. Problems about one counter hitting zero have known exact solutions, which is what lets the model give closed-form answers rather than simulations. The cost is the realism of point 3: in real markets the two sides do react to each other—the race of Section 6 is partly a reaction, not two independent clocks—and that dependence is exactly what later models, such as the queue-reactive model of Section 6.3, put back in.

Each counter is a birth–death process. The mathematical name for such a counter is a *birth–death process*: a count that goes up by one at a “birth” rate and down by one at a “death” rate. The names come from population models, where births add individuals and deaths remove them. Here a birth is a new limit order joining the queue, and a death is an order leaving it, by trade or cancellation. The birth rate is constant; the death rate depends on the current count, because the more orders are waiting, the more of them can be cancelled. If the spread is s_t/δ ticks, the best queues sit at distance $\bar{\ell} = s_t/\delta$ from the opposite quote, and a queue holding x orders evolves as



with birth rate $r_L(s_t/\delta)$ and death rate $r_M + x r_c(s_t/\delta)$; reaching state 0 (shaded) means the queue has been used up and the price moves, so the count stops there. The authors prove that, up to the first price change, the two best queues can be replaced by *independent* birth–death processes with these rates. Every question about the next price move becomes a question about which of two

independent counters hits zero first—the race of Section 6, now with exact rates. Figure 2 shows three such races.

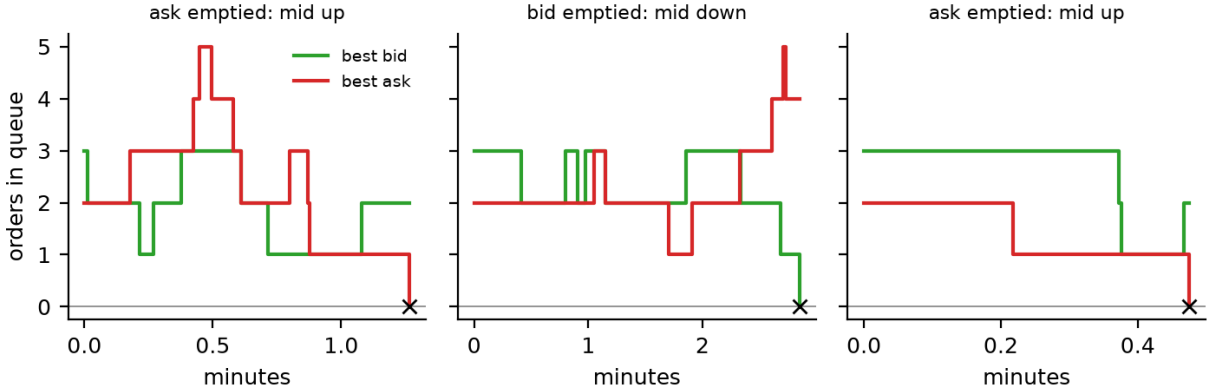


Figure 2: Three simulated races between the best-bid and best-ask counters, each starting from 3 orders at the bid and 2 at the ask, with the estimated rates of Table 8 at distance one. Each step up is a limit order joining; each step down is a market order or a cancellation. The race ends (cross) when one counter reaches zero, and that side decides the next price move. The same starting book can end either way, and after very different waiting times: the model’s answers are probabilities. Stylised simulation for illustration; not data.

Use of Laplace transforms. The time for a birth–death process starting at x to reach zero is a sum of x independent pieces: the time to go from x to $x - 1$, then from $x - 1$ to $x - 2$, and so on. Distributions of sums are awkward, but the *Laplace transform* of a waiting time τ , $\mathbb{E}[e^{-\varpi\tau}]$ as a function of a variable ϖ , turns sums of independent waiting times into products of their transforms. For birth–death processes each one-step transform has a known closed form as a continued fraction (Cont et al., 2010), so the transform of the whole first-passage time is a product of known pieces. Probabilities such as “queue A empties before queue B” are then obtained by numerically inverting a transform built from these products—a computation that takes far less time than simulating thousands of paths of the book, and gives full distributions rather than only averages.

In words: instead of simulating the book many times and counting how often the price went up, the model writes the answer as a formula in transform space and evaluates it directly. The transform is a book-keeping device that makes adding independent waiting times as easy as multiplying numbers.

6.1.4 Three probabilities a trader needs

The paper computes three conditional probabilities, each given the current number of orders at the best bid, $v_{1,t}^b$, and at the best ask, $v_{1,t}^a$, and the spread. This subsection first states what each probability is and what it shows; the equations that produce the numbers are collected at the end, in “How the probabilities are computed”.

1. Probability that the next price move is up. With a one-tick spread, the mid rises next if the ask queue empties before the bid queue. With a wider spread, a new order can also arrive inside the spread and move the mid first; the formula adds these arrivals as competing exponential clocks. Table 9 compares the model’s answer with the empirical frequency in the data. The model captures the main pattern well—the probability rises with the bid queue and falls with the ask

queue—though it overstates how strongly a large ask queue pushes the price down: wherever the ask queue is large, the model’s probability of an up move is below the empirical one. Some cells differ noticeably (for example 0.548 against an empirical 0.714 with four orders at the ask and five at the bid). Figure 3 draws the table as curves.

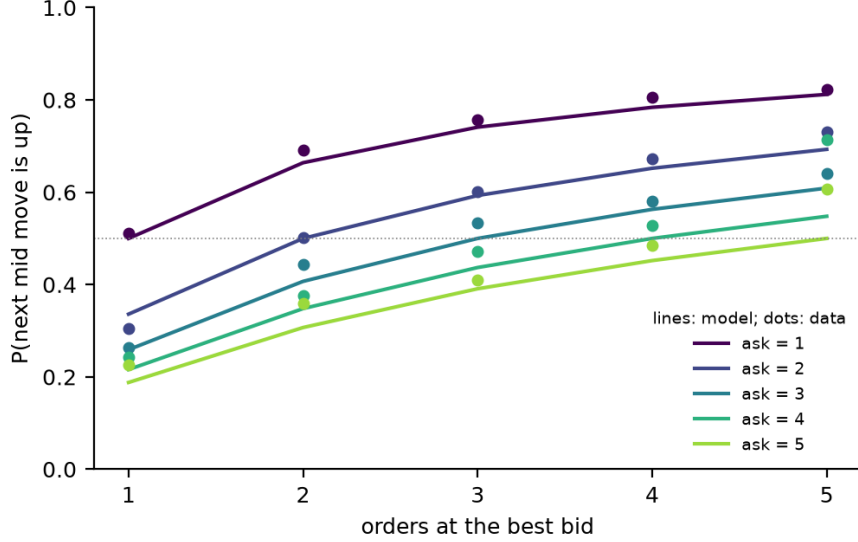


Figure 3: Probability that the next mid move is up, against the number of orders at the best bid, one curve per size of the best-ask queue: Cont–Stoikov–Talreja model (lines) and empirical frequency (dots), from Table 9. More orders at the bid, or fewer at the ask, make an up move more likely—the thin side breaks first. For large ask queues the dots sit above the lines: the model makes a large ask queue push the price down harder than it does in the data.

Table 9: Probability that the next mid move is up, one-tick spread, Sky Perfect Communications: empirical frequency / Cont–Stoikov–Talreja model (Laplace-transform method) (Cont et al., 2010). Queue sizes in units of the average limit order.

Orders at the best bid, $v_{1,t}^b$	Orders at the best ask, $v_{1,t}^a$				
	1	2	3	4	5
1	0.512 / 0.500	0.304 / 0.336	0.263 / 0.259	0.242 / 0.216	0.226 / 0.188
2	0.691 / 0.664	0.502 / 0.500	0.444 / 0.407	0.376 / 0.348	0.359 / 0.307
3	0.757 / 0.741	0.601 / 0.593	0.533 / 0.500	0.472 / 0.437	0.409 / 0.391
4	0.806 / 0.784	0.672 / 0.652	0.580 / 0.563	0.529 / 0.500	0.484 / 0.452
5	0.822 / 0.812	0.731 / 0.693	0.640 / 0.609	0.714 / 0.548	0.606 / 0.500

2. Probability that a bid order fills before the price moves. Suppose an order is placed at the back of the best bid, so that the bid queue holds $v_{1,t}^b$ orders including our own, and that it is never cancelled. It is filled when all orders ahead of it have been removed—by market sells, or by cancellation of the orders ahead—which is a pure-death process: with x orders in line including our own, the line shortens at rate $r_M + (x - 1)r_c$, since our own order is not cancelled. The order is filled before the price moves if this line empties before the ask queue does. Table 10 gives the model’s answers. The pattern is the execution problem of Part III in numbers: an order alone at the bid, facing five orders at the ask, is filled before the price moves about 78% of the time; the last of five orders at the bid, facing one at the ask, about 12% (Figure 4).

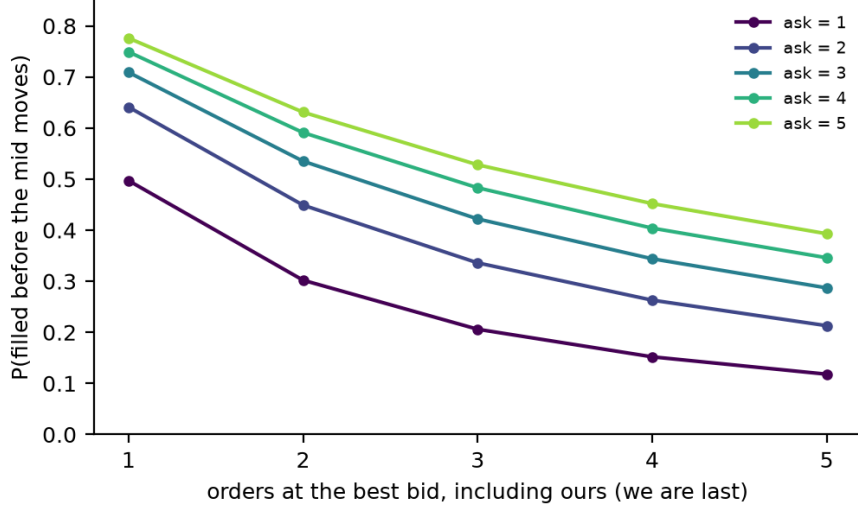


Figure 4: Probability that an order at the back of the best bid is filled before the mid moves, against the length of the bid queue (our order included and last), one curve per size of the best-ask queue; model values from Table 10. Every order ahead of ours lowers the chance; a long ask queue raises it, because it makes the price less likely to move up and away before we are reached.

Putting the first two together: adverse selection. Plotting the two model probabilities against each other, one point per book (Figure 5), shows something neither table shows alone. The points lie almost on a straight downward line (correlation -0.99): the books in which a resting bid is most likely to be filled are exactly the books in which the next price move is most likely to be *down*. An order at the bid gets filled mostly when the bid side is about to break—when it is buying just before the price falls. This is the adverse selection of Section 11.3, produced here by nothing more than two counters racing to zero; Section 6.2 returns to it. A fill probability on its own therefore says little about whether resting an order is a good idea: what matters is the fill probability together with what the price does after the fill.

The authors note that their data do not identify individual orders, so these probabilities could not be checked against data—exactly the gap that order-level data and queue-exact replay fill (Section 12.3).

Table 10: Probability that an order placed at the best bid is executed before the mid moves, one-tick spread, Cont–Stoikov–Talreja model (Laplace-transform method) (Cont et al., 2010).

Orders at the best bid, $v_{1,t}^b$	Orders at the best ask, $v_{1,t}^a$				
	1	2	3	4	5
1	0.497	0.641	0.709	0.749	0.776
2	0.302	0.449	0.535	0.591	0.631
3	0.206	0.336	0.422	0.483	0.528
4	0.152	0.263	0.344	0.404	0.452
5	0.118	0.213	0.287	0.346	0.393

3. Probability of making the spread. A market maker places one order at the best bid and one at the best ask and earns the spread if both are filled before the price moves. The model computes this probability too. Its most interesting feature is that it is *not* monotone: for a fixed

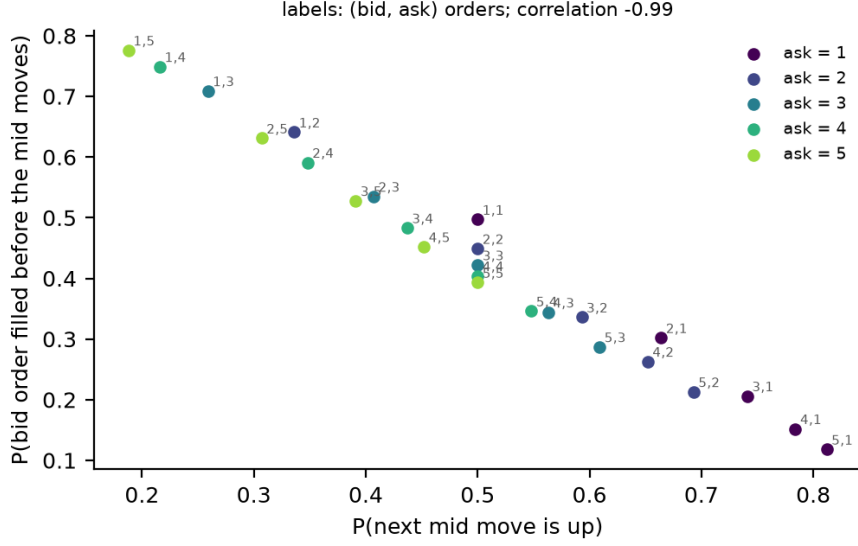


Figure 5: For each combination of queue sizes (labels: orders at the bid, orders at the ask), the model’s probability that the next move is up (horizontal) against the probability that an order at the back of the bid is filled before the mid moves (vertical); values from Tables 9 and 10. High fill probability comes with a likely down move: the fills a resting bid gets are disproportionately the bad ones.

number of orders at the ask, the probability first rises and then falls as the bid queue grows. With one order at the ask, it is about 0.27 with one order at the bid, peaks near 0.31 with two or three, and falls back to about 0.29 with five. A longer bid queue protects the bid from being exhausted, which helps, but also makes the bid order wait longer, which hurts; the balance gives an interior optimum.

Computation of the probabilities. All three answers come from one piece of mathematics: the time it takes a counter to reach zero. We follow [Cont et al. \(2010\)](#), Section 3, in the notation of this survey, for a one-tick spread. At the best quote a queue of x orders gains an order at rate $r_{\text{in}} = r_L$ and loses one at rate $r_{\text{out}}(x) = r_M + x r_c$ —the bucket of Section 6, now with exact rates.

Step 1: one step down. Let τ_x be the time the queue takes to go from x orders to $x - 1$. We describe its distribution by its Laplace transform

$$\mathcal{L}_x(\varpi) = \mathbb{E}[e^{-\varpi \tau_x}]$$

(Section 6.1, “Why Laplace transforms”). Look at the first event. It happens after an exponential waiting time with rate $r_L + r_{\text{out}}(x)$. With probability $r_{\text{out}}(x)/(r_L + r_{\text{out}}(x))$ it is a departure, and the queue is at $x - 1$: done. With probability $r_L/(r_L + r_{\text{out}}(x))$ it is an arrival, the queue is at $x + 1$, and it must first come back down to x (time τ_{x+1}) and then go down to $x - 1$ (a fresh copy of τ_x). In transform form, this “first-step analysis” reads

$$\begin{aligned} \mathcal{L}_x(\varpi) &= \frac{r_{\text{out}}(x)}{r_L + r_{\text{out}}(x) + \varpi} + \frac{r_L}{r_L + r_{\text{out}}(x) + \varpi} \mathcal{L}_{x+1}(\varpi) \mathcal{L}_x(\varpi), \\ \text{so } \mathcal{L}_x(\varpi) &= \frac{r_{\text{out}}(x)}{r_L + r_{\text{out}}(x) + \varpi - r_L \mathcal{L}_{x+1}(\varpi)}. \end{aligned} \tag{18}$$

In words: the transform of a waiting time that is an exponential delay followed by more waiting is the product of their transforms, and the two branches are weighted by how likely each is. Solving for

$\mathcal{L}_x$ expresses it through $\mathcal{L}_{x+1}$; substituting the same formula for $\mathcal{L}_{x+1}$, and so on, gives a fraction inside a fraction inside a fraction—a *continued fraction*, which converges quickly and is evaluated numerically (Abate and Whitt, 1999).

Step 2: the whole queue. To empty a queue of v orders it must step down from v to $v - 1$, then from $v - 1$ to $v - 2$, and so on to zero. These steps take independent times, and the transform of a sum of independent times is the product of their transforms:

$$\mathbb{E}[e^{-\varpi\tau^a}] = \prod_{x=1}^{v_{1,t}^a} \mathcal{L}_x(\varpi), \quad \mathbb{E}[e^{-\varpi\tau^b}] = \prod_{x=1}^{v_{1,t}^b} \mathcal{L}_x(\varpi). \quad (19)$$

In words: this is the whole reason for using transforms: adding waiting times is hard, multiplying their transforms is easy.

Step 3: the race (probability 1). The next move is up when the ask empties first, $\mathbb{P}(\tau^a < \tau^b) = \mathbb{P}(\tau^a - \tau^b < 0)$: the probability that the difference of the two times is negative, which is the distribution function of that difference evaluated at zero. Because the two queues are independent (Section 6.1, “From the whole book to two counters”), the transform of the difference is the product of the transform of τ^a at ϖ and that of τ^b at $-\varpi$; dividing by ϖ turns the transform of a density into the transform of its distribution function. So

$$\mathbb{P}(\text{next mid move is up}) = \left[\text{inverse transform of } \frac{1}{\varpi} \mathbb{E}[e^{-\varpi\tau^a}] \mathbb{E}[e^{+\varpi\tau^b}] \right] \text{ evaluated at } 0. \quad (20)$$

In words: build the transform of “ask time minus bid time” from the two queues, turn it back into a probability numerically, and read off the chance that the difference is negative.

The transform here is two-sided, because the difference can be positive or negative, and the inversion is done numerically. One check is immediate: with equal queues the two times have the same distribution, so the probability is exactly one half—the diagonal of Table 9.

Step 4: our own order (probability 2). An order at the back of the bid is filled when every order ahead of it is gone. New orders join *behind* it and cannot delay it, so for this line there are no arrivals at all: it is a pure-death process, and with x orders in line (ours included) it shortens at rate $r_M + (x - 1)r_c$ —our own order is never cancelled. Without arrivals, Step 1 has no second branch and each step down is a plain exponential, so

$$\mathbb{E}[e^{-\varpi\tau^{\text{fill}}}] = \prod_{x=1}^{v_{1,t}^b} \frac{r_M + (x - 1)r_c}{r_M + (x - 1)r_c + \varpi},$$

where τ^{fill} is the time until our order is filled. The order is filled before the price moves if this time beats the ask’s depletion time: the probability is $\mathbb{P}(\tau^{\text{fill}} < \tau^a)$, computed exactly as in Step 3 with τ^{fill} in the place of τ^a and τ^a in the place of τ^b . (The bid queue itself cannot empty first, because our order is in it and is filled before the queue is gone.)

Step 5: a wider spread, and the third probability. With a spread of several ticks the mid can also move because a limit order lands inside the spread. Such arrivals are independent exponential clocks with rates $r_L(\ell)$ for the in-spread distances, so they enter as extra competitors in the race; their transforms are again simple fractions, and the same inversion applies. The probability of making the spread combines two such races—for the bid order and the ask order—and is computed with the same tools; we do not reproduce its formula.

None of these steps needs simulation: each probability is a short product of fractions and one numerical inversion, which is why the model can answer a trader’s questions as fast as the book updates.

6.1.5 Stylised facts reproduced by the model

Without any tuning for these purposes, the fitted model reproduces several features of the real book:

- the **hump-shaped average book**: simulated average depth peaks two ticks from the opposite quote, as observed empirically (Bouchaud et al., 2002);
- the **order of magnitude of volatility**: the volatility of the simulated price is close to the realised volatility of the real stock over the same number of trades—computed from order-flow rates alone;
- the **one-step behaviour of queues**: the probability that a queue grows at its next change, as a function of its size and distance, is close to the empirical frequencies at all five distances.

Mechanism behind the reproduced facts. None of the three features was used to fit the model. Its parameters are only event counts—how often orders arrive, trade and cancel at each distance (Section 6.1, “Estimating the model”). The features *emerge* when the fitted model is run forward: the authors simulate the Markov chain for a long time, event by event, exactly as in Figure 2 but for the whole book, and measure the average book, the price path and the behaviour of the queues in the simulated history. That is the sense in which the model “reproduces” them. Why each one comes out right can be understood without the simulation.

The hump

Consider one price level deep in the book, far from any trading. Orders arrive at rate $r_L(\bar{\ell})$, and each one, independently, is cancelled at rate $r_c(\bar{\ell})$. A level fed in this way settles around $r_L(\bar{\ell})/r_c(\bar{\ell})$ orders: arrivals and departures balance when the queue is big enough that $r_c(\bar{\ell})$ times its size equals $r_L(\bar{\ell})$. With the estimates of Table 8 this ratio is about 2.6, 1.9, 1.6, 1.6 and 1.6 orders at distances one to five—largest at distance one, with no hump. Whatever produces the hump in the simulated book therefore comes from the interaction between the levels as the price moves—the best quote being drained by market orders and emptied, and every queue’s distance from the opposite quote changing after each price move—not from the rates of any one level. The authors obtain the shape by simulating the whole book; we know of no simple argument that predicts the peak at distance two, and do not offer one.

Volatility

In the model every price change is a queue depletion: when a best queue empties, the mid moves half a tick or so, up or down. The mid is therefore a random walk whose steps are the outcomes of successive races (Figure 2), and its variability over a period depends on how often races end—which depends on how fast orders arrive, trade and cancel, and how large the queues are when a race starts. The simulation runs for the same number of trades as the real day and measures how far the simulated price wanders; the result is close to the real figure. Nothing about volatility was put in; it follows from the order-flow rates. The Cont-de Larrard model of the next section makes the same link in closed form, with volatility proportional to $\delta\sqrt{r_L/\Theta}$.

One-step behaviour of queues

When a queue of x orders at the best quote next changes, it grows if a limit order arrives before any of the competing events; with independent exponential clocks this happens with probability $r_L/(r_L + r_M + x r_c)$, and at deeper levels, where there are no market orders, $r_L/(r_L + x r_c)$. The probability falls as the queue grows, because a longer queue offers more orders to cancel. This is the weakest of the three tests, since the rates were estimated from the same data; what it checks is the *form* of the model—in particular that cancellations grow in proportion to queue size.

6.1.6 Limitations and descendants

The authors are explicit about what the model leaves out. Order arrivals are independent Poisson streams, so there is no clustering of events and none of the long memory of order flow (Section 3); orders all have the same size; rates do not change through the day or with market conditions; and there is no strategic behaviour. The authors also derive a simple trading rule from the model’s up-move probability, but evaluate it only on paths simulated from the model itself and without transaction costs, so it says nothing about profitability on real markets—an instance of the gap between prediction and tradable value that Part III addresses. Each limitation defines a line of later work:

- [Cont and de Larrard \(2013, 2012\)](#) reduce the book to the two best queues and obtain closed forms and a diffusion limit (Section 6.2);
- queue-reactive models let all rates depend on the current queue sizes ([Huang et al., 2015](#)), and their neural extensions learn that dependence ([Bodor and Carlier, 2025](#));
- Hawkes models replace the Poisson streams by self-exciting ones (Section 7);
- the micro-price ([Stoikov, 2018](#)) keeps the Markov-chain logic but estimates the transitions directly from data rather than from a parametric model;
- queue-position valuation ([Moallemi and Yuan, 2017](#)) and fill-probability models ([Arroyo et al., 2024](#); [Zhong et al., 2025a](#)) extend the second probability above to individual orders, with adverse selection.

Finally, because the Laplace-transform method yields full distributions rather than single probabilities, it can answer latency questions directly—for example, the probability that the ask queue empties more than a given delay after the bid queue, which is what matters when an order takes time to reach the exchange ([Cont et al., 2010](#)).

Model card: Cont–Stoikov–Talreja

What you calibrate. The limit-order rates $r_L(\bar{\ell})$, summarised by r_0 and Ξ ; the market-order rate r_M ; the per-order cancellation rates $r_c(\bar{\ell})$; the unit of size (the average limit order).

How. Counting: increases, decreases not caused by trades, and market orders at each distance, divided by trading time (and, for cancellations, by average queue size); a least-squares power-law fit for r_L .

What you do with it. Compute, from the current two best queues, the probability that the next move is up, that a resting order fills before the price moves, and that a market maker earns the spread—fast and exactly, via Laplace transforms.

How to picture it. Two counters racing to zero, each ticking up when an order joins and down when one is traded or cancelled; the first to hit zero decides the next price move.

6.2 The Cont–de Larrard model

Motivation: closed-form answers. The Cont–Stoikov–Talreja model answers its questions exactly, but only numerically: each probability needs a continued fraction and a numerical Laplace inversion, so the answers are tables, not formulas. One cannot look at them and see *how* the probability of an up move depends on the queues, or how volatility depends on order flow. [Cont and de Larrard \(2013\)](#) trade some realism for formulas. They simplify the model until the answers can be written down in closed form, and every simplification is chosen to make one piece of mathematics tractable.

The simplifying assumptions, and what each buys.

1. **Only the two best queues are modelled.** The state is just the best bid price and the sizes of the two best queues. Nothing deeper in the book is tracked. *What it buys:* a two-number state instead of a whole book.
2. **The spread is always one tick.** This holds over 98% of the time for the large-tick US stocks they study. *What it buys:* no orders can arrive inside the spread, so the price moves *only* when one of the two queues is emptied—the race is the whole story.
3. **Rates do not depend on queue size.** At each best queue, limit orders arrive at rate r_L , market orders at rate r_M and cancellations at rate r_C , all independent Poisson streams of unit-size orders. In contrast to Cont–Stoikov–Talreja, the cancellation rate is a rate for the *whole queue*, not per order: a queue of ten orders loses orders to cancellation no faster than a queue of two. *What it buys:* each queue becomes the simplest random walk there is—up by one at the constant rate r_L , down by one at the constant rate $r_M + r_C$ (in the bucket picture, $r_{\text{in}} = r_L$ and $r_{\text{out}} = r_M + r_C$). The time for such a walk to reach zero has a known closed-form distribution, in terms of Bessel functions, instead of a continued fraction. This is the key simplification. Its cost is the mechanism that, in Cont–Stoikov–Talreja, keeps queues from growing without bound: here long queues are not cancelled faster.
4. **When a queue empties, the book is redrawn.** The price moves one tick, and the sizes of the two new best queues are drawn afresh from a fixed distribution estimated from data, independently of the past. *What it buys:* the deeper levels never have to be modelled—they are replaced by one distribution of “queue sizes just after a price change”—and every price change starts a fresh, identical race, so the price becomes a sequence of independent races.
5. **The two sides are symmetric and independent.** The same rates apply at the bid and the ask, and the two queues move independently. *What it buys:* the race is between two independent copies of the same random walk.

Two further simplifications are made for particular results. The cleanest formula for the probability of an up move assumes the *balanced* case, $r_L = r_M + r_C$, in which the rates cancel out; and the arctan form of that probability, and the volatility result, come from *limits*—large queues observed over long horizons—in which the random walks are replaced by Brownian motions. Each simplification is a modelling choice, not an empirical finding, and each is relaxed by a later model: queue-size-dependent rates in the queue-reactive model (Section 6.3), self-excitation in Hawkes models (Section 7).

Empirically the rates nearly balance: r_L is slightly below $r_M + r_C$ (for example a ratio of about 0.95 for Citigroup in their data). Three results follow.

Time to the next price change. The waiting time has a closed-form distribution. For one queue of x orders, the probability that it has not emptied by time u is an integral of a Bessel function, and for the two independent queues the probability that no price change has occurred is the product of the two (Proposition 1 of the paper). Figure 6 evaluates it. Its tail is the important part. When inflow and outflow balance exactly, the probability that the price has not yet moved decays only like $1/u$, so the *expected* waiting time is infinite: very long quiet periods are not rare accidents but a built-in feature. When inflow is somewhat smaller than outflow, the curve follows the balanced one for a while and then falls away exponentially, after a time of order $1/(\sqrt{r_M + r_C} - \sqrt{r_L})^2$ —the closer the rates are to balance, the later the cut-off, so nearly balanced queues still produce long waits. Long quiet periods punctuated by bursts are thus a direct consequence of (nearly) balanced

queues, before any self-excitation is added. Cont and de Larrard (2013) state that for $r_L < r_M + r_C$ the tail decays like $1/u^2$; evaluating their exact expression, and checking it by simulation, shows the exponential cut-off instead, which is what one expects for a random walk drifting towards zero. The $1/u^2$ statement should be read as describing an intermediate range of times, not the ultimate tail.

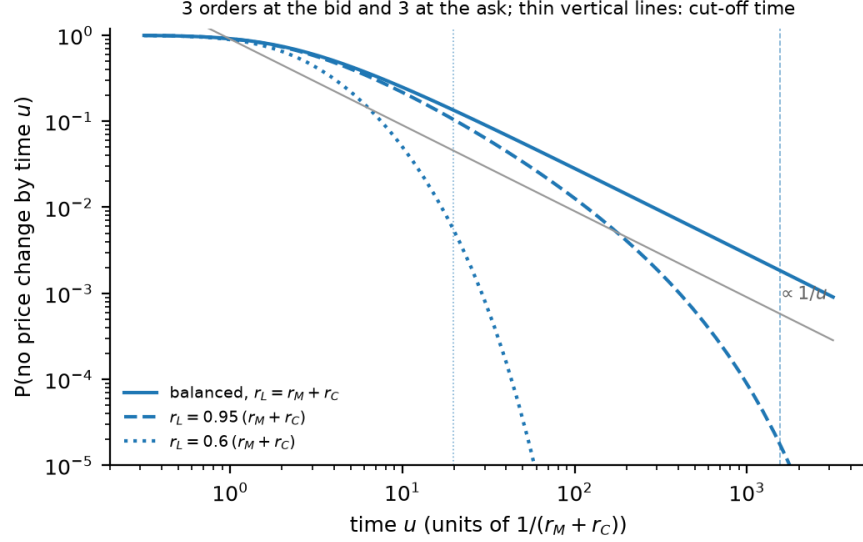


Figure 6: Cont–de Larrard model: probability that the price has not changed by time u , starting from three orders at each best quote, computed from the closed-form expression, on logarithmic axes. Time is in units of the mean time between departures from a queue. Balanced rates (solid) give a $1/u$ tail and an infinite mean waiting time; with inflow below outflow (dashed, dotted) the curve breaks away exponentially, later the closer the rates are to balance. Stylised rates.

Probability that the next move is up. In the balanced case the probability depends only on the two queue sizes, not on the rates—it is the probability that a symmetric two-dimensional random walk started at the queue sizes hits the ask axis first. In the companion heavy-traffic analysis (Cont and de Larrard, 2012), with uncorrelated queue changes of equal size and intensity on the two sides, it takes the simple form

$$\mathbb{P}(\text{next mid move is up} \mid v_{1,t}^b, v_{1,t}^a) = \frac{2}{\pi} \arctan\left(\frac{v_{1,t}^b}{v_{1,t}^a}\right). \quad (21)$$

In words: the probability rises from 0 to 1 as the bid queue grows relative to the ask queue, and equals $1/2$ when they are equal. It is the “thin side breaks first” rule of Section 4, made exact for this model. Figure 7 draws it. Lines of equal probability fan out from the origin: what matters is the *ratio* of the two queues, not their difference—ten orders against five behaves like two against one. In Figure 1, with 29 lots bid and 14 offered, it gives $(2/\pi) \arctan(29/14) \approx 0.71$.

(The general formula in the paper allows for correlated queue changes; its printed zero-correlation special case has the two queues interchanged, and the form above is the one implied by the paper’s general expression.)

Figure 8 compares this formula with the Cont–Stoikov–Talreja model and the data of Table 9. The two models agree closely when the ask queue is short; when it is long, the arctan formula lies further below the data than the CST model does. The comparison is only indicative, since the formula is derived for a balanced, heavy-traffic limit and is not fitted to this stock.

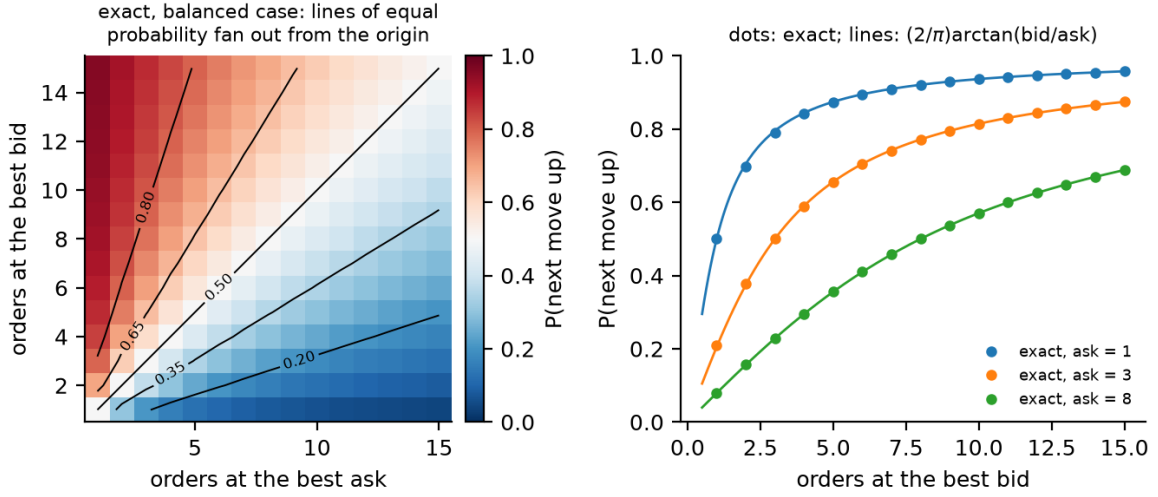


Figure 7: Cont-de Larrard model, balanced case: probability that the next mid move is up. Left: the exact expression of the paper’s Proposition 2 over a grid of queue sizes; the contours of equal probability are nearly straight lines through the origin, so the probability depends mainly on the ratio of bid to ask. Right: the exact values (dots) against the simple form $(2/\pi)\arctan(v^b/v^a)$ (lines), which is already very close for small queues. Neither depends on any rate: in the balanced case only the queue sizes matter.



Figure 8: Probability that the next mid move is up for three sizes of the ask queue: the Cont-de Larrard formula $(2/\pi)\arctan(v^b/v^a)$ (dashed), the Cont-Stoikov-Talreja model (solid) and the empirical frequencies (dots) of Table 9. The arctan formula uses no fitted parameters.

From queues to volatility. Over long horizons the price behaves like a random walk whose volatility, in the balanced case, is proportional to $\delta\sqrt{r_L/\Theta}$, where Θ is the average product of the bid and ask queue sizes just after a price change. Volatility rises with order activity and falls with the depth that reappears after each move—a microstructural explanation of why thin, busy markets are volatile. Across Dow Jones stocks the authors find ten-minute volatility roughly proportional to this quantity. Figure 9 shows the shape of the relation: volatility grows with the square root of order activity, so four times the activity doubles it, and falls with the square root of the depth that reappears after each move. It is a formula for volatility in which the only inputs are counts and sizes from the order flow.

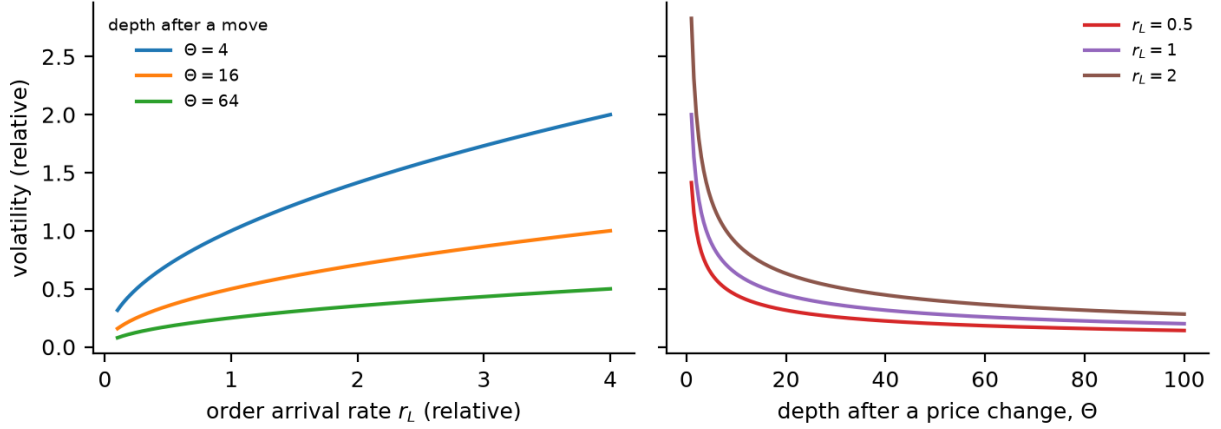


Figure 9: Cont-de Larrard model: volatility $\propto \delta\sqrt{r_L/\Theta}$, in relative units, against the order arrival rate for three depths (left) and against the depth after a price change for three arrival rates (right). Busier markets are more volatile; markets that refill deeply after each move are calmer.

Example 6.2 (Why the rates must depend on the state). A best-ask queue holds 200 lots. Averaged over the day, net depletion (executions plus cancellations minus additions) runs at 40 lots per second, so the first estimate gives $200/40 = 5$ seconds. Now condition the rate on OFI. When buying pressure is in its top decile, the net drain is 90 lots per second and $\mathbb{E}[\tau^a] \approx 200/90 \approx 2.2$ s; in the bottom decile it is 15 lots per second and $\mathbb{E}[\tau^a] \approx 13$ s. The same queue has expected lifetimes six times apart, depending on what the rest of the market is doing. A model that uses one daily average rate would be wrong in both directions.

Depletion and adverse selection. Queue depletion is also where adverse selection comes from. Consider a resting buy order at the back of the best-bid queue. It is filled only when everything ahead of it has been traded or cancelled. Because it is last in line, in practice it is filled mostly by a burst of selling large enough to empty the whole level. And an emptied bid level is, by point 1 above, a downward price move. The order is therefore filled at the very moment the price is moving against it: the buyer acquires the lots just as they become cheaper. The fill and the bad price move are not two unlucky events that happen to coincide; they are the same event.

An order near the *front* of the queue is in a different position. It can be filled by a small sell order that takes a few lots and leaves the rest of the queue, and the price, where they were. Such fills carry little information, so front-of-queue orders are less adversely selected. This is one source of the front-back value difference in Moallemi and Yuan (2017), whose model splits the value of an order into a static component—the spread earned minus adverse selection, which increases with queue position—and a dynamic component, the option value of holding the queue position.

Example 6.3 (Filled by the move). In Figure 1 we join the best bid at 5000.00 with one lot, behind 29 lots. Suppose a 30-lot sell market order arrives. It takes all 29 lots ahead of us and our lot, and the 5000.00 level is gone: the best bid is now 4999.75 and the mid has fallen from 5000.125 to 5000.00 or below. We bought at 5000.00 and the market immediately values the contract lower. Had we been first in the queue, a 2-lot sell would have filled us and left 28 lots and the price untouched.

Figure 10 makes the point quantitative inside the Cont–Stoikov–Talreja model. An order is placed in a best-bid queue of six orders, at each possible position, facing three orders at the ask, and the races are simulated with the rates of Table 8. The closer to the front, the more often the order is filled before the price moves; and among the orders that *are* filled, the probability that the next price move is down rises steadily from front to back, always above the probability of a down move from the same book without conditioning on the fill. The fill itself is bad news, and it is worse news the further back the order sat.

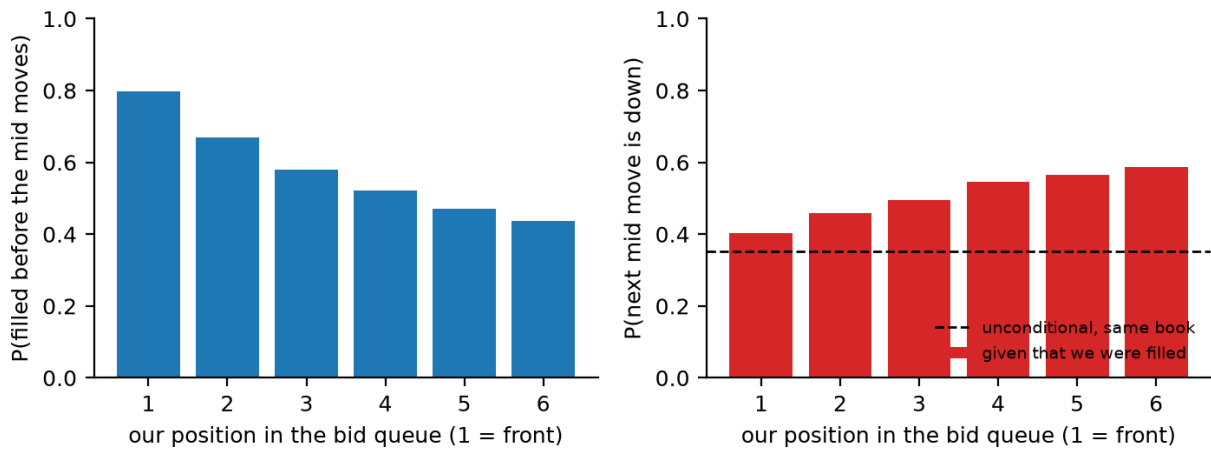


Figure 10: Adverse selection by queue position, Cont–Stoikov–Talreja model with the rates of Table 8: six orders at the best bid (ours at the position shown), three at the ask. Left: probability that our order is filled before the mid moves. Right: probability that the next mid move is down, among the paths on which our order was filled (bars), against the probability of a down move from the same book with no conditioning (dashed). Stylised simulation for illustration; not data.

Model card: Cont–de Larrard

What you calibrate. Three rates per best queue, r_L , r_M , r_C , and the distribution of the new best-queue sizes after a price change (which gives Θ).

How. Counting events at the best quotes over a sample, and the empirical distribution of queue sizes just after each price change.

What you do with it. The probability that the next move is up, the distribution of time to the next price change, and a volatility estimate $\propto \delta \sqrt{r_L / \Theta}$ from order-flow rates alone.

How to picture it. A drunkard’s walk in two dimensions, starting at the point (bid size, ask size): whichever axis the walk hits first decides the price move.

6.3 The queue-reactive model

The queue-reactive model as a market simulator. The queue-reactive model is, first of all, a *simulator*: a program that generates synthetic order-book data. Given the rates fitted to a stock, it produces, event by event, a stream of limit orders, cancellations and market orders at the first

few price levels, the resulting queue sizes, and the price moves they cause—an artificial trading day that looks statistically like the real ones. It is a *generative* simulator in the sense of Section 1.5: it invents new market paths from a model, rather than replaying a recorded one as the Kaspar system of Section 12.3 does. Two features make it useful. The paths it generates reproduce the observed behaviour of queues; and because every rate depends on the current state of the book, a trader’s own simulated order changes what the simulated market does next, so tactics can be tested against a market that reacts. The book it generates is coarse: queues are counts of orders of average size, there are no individual order identities, and only the first few levels exist. And, like any generative model, what it generates is not evidence about the real market (Principle 1): it is a tool for asking “what if”, whose answers are only as good as the model. Apart from one formula, for the long-run size of a queue, all of its results are obtained by running it.

The simulation loop. Figure 11 shows one step of the loop. The state is the list of queue sizes at the first few levels on each side, plus the reference price. From the current sizes, the fitted tables give the rate of every possible event at every queue—an arrival, a cancellation, or (at the best quotes) a market order. The total of all these rates sets how long until the next event: the waiting time is drawn from an exponential distribution with that total rate. Which event it is, and at which queue, is drawn with probability proportional to its rate, so frequent events are picked more often. The queue is updated. If a best queue has just been emptied, the price may move, and if it moves the book is either shifted one level or reset. Then the loop repeats from the new state. Because the rates are recomputed from the new queue sizes at every step, the simulated market reacts to everything that has happened in it, including any orders the trader inserts.

Significance of the model. The two previous models fix the behaviour of traders in advance: Cont–Stoikov–Talreja assume constant arrival and trading rates and cancellations proportional to queue size; Cont–de Larrard assume constant rates throughout. Huang et al. (2015) drop these assumptions and let the data say how traders behave. Their model is known for four things.

1. **Behaviour is measured, not assumed.** Each event rate is estimated separately for every queue size, by counting, with no formula imposed. The measured rates have shapes neither earlier model allows: market orders are much more frequent when a queue is short—traders hit thin queues—and cancellations rise with queue size only up to a point and then level off.
2. **It reproduces the queues it is fitted to.** The long-run distribution of queue sizes implied by the fitted rates matches the observed one closely, and better than a model with constant arrival rates and proportional cancellation.
3. **It reacts to the trader’s own orders.** Because every rate depends on the current queue sizes, an order placed in the simulated book changes how the book behaves afterwards. That makes the model a realistic simulator for testing execution tactics, which is how the authors use it.
4. **It measures how much of the price movement the queues can explain.** The model has a switch: price changes can come purely from queues emptying, or some of them can come with a “reset” of the whole book, standing for news. With the resets switched off, the queues alone produce at most about a third of the observed volatility, so most price variation must come from information arriving from outside the book. The paragraph “Prices” below explains how this works.

Later models build on it: queue-reactive Hawkes models add self-excitation (Section 7.18), and a deep version learns the rate functions with neural networks (Bodor and Carlier, 2025).

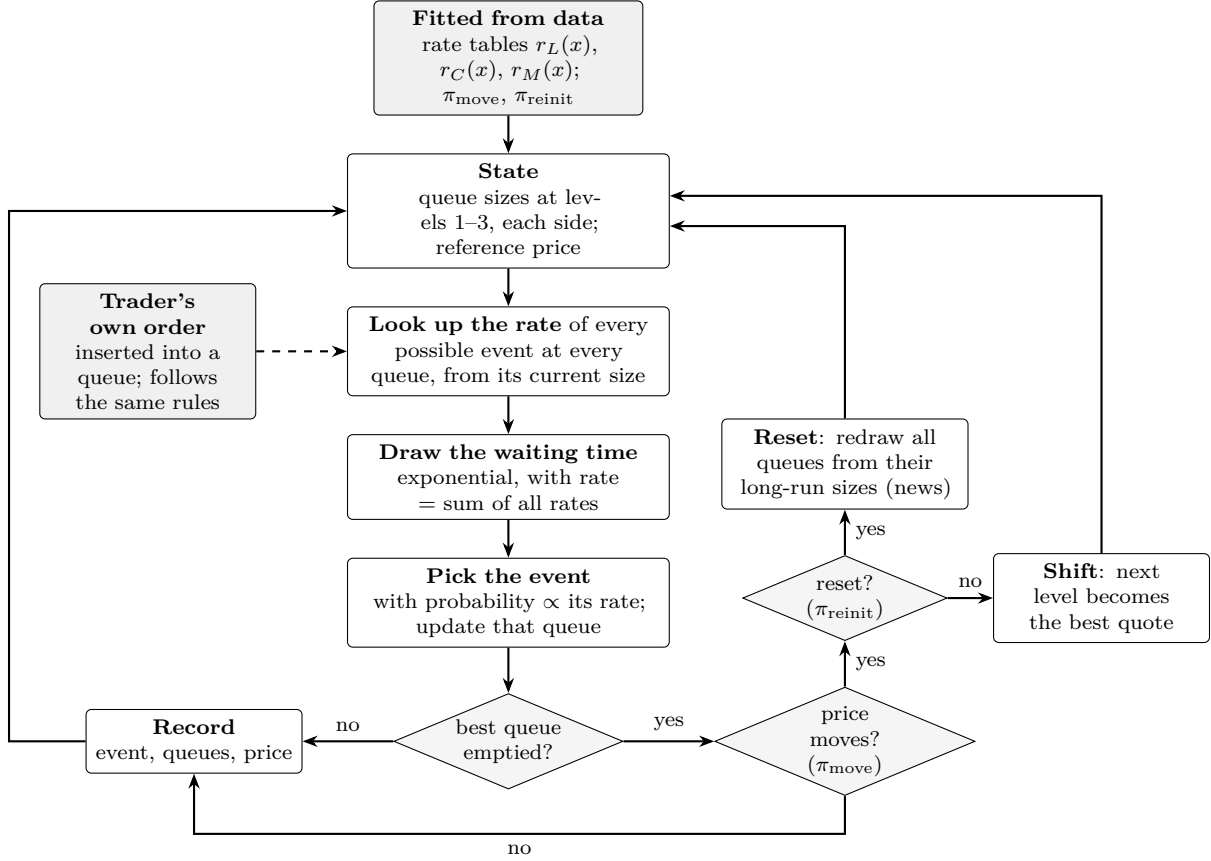


Figure 11: One step of the queue-reactive simulator. The fitted rate tables are the only input from data. At every step the rates are looked up from the current queue sizes, so the simulated book reacts to its own history and to any order the trader inserts. Price moves come from emptied best queues; most of them, in the calibration of [Huang et al. \(2015\)](#), come with a reset of the whole book that stands for news.

Comparison with the two previous models. The three queue models of this section answer different questions in different ways (Table 11). Cont–Stoikov–Talreja compute the probabilities of the next price move exactly from the current queues, by a numerical transform inversion. Cont–de Larrard simplify until those answers become formulas. The queue-reactive model goes the other way: it makes the rates realistic, and pays for it by giving up most formulas—which is why it is used as a simulator.

Table 11: The three queue models of this section: what each assumes, what it answers, and how.

	Cont–Stoikov–Talreja	Cont–de Larrard	Queue-reactive
How traders behave	constant arrival and trading rates; cancellations proportional to queue size	constant rates for everything	every rate an arbitrary function of queue size, measured from data
What is tracked	the whole book	the two best queues; the book is redrawn after each price move	the first few levels; the book may be redrawn after a price move
Main questions	next move up or down; fill before the move; making the spread—from the current queues	the same, plus waiting time to the next move and long-run volatility	typical queue sizes; how prices and volatility arise; fill chances and placement tactics for a real order
How answered	exact, by numerical inversion of transforms; simulation for long-run features	closed-form formulas	one formula (long-run queue sizes); simulation for everything else
Strength	exact answers for the next move	intuition: how answers depend on rates	realism; reacts to one’s own orders

The idea. Cont–Stoikov–Talreja make cancellations proportional to queue size and keep the other rates fixed. Huang et al. (2015) ask the data instead: *how do traders actually behave when a queue is short, and when it is long?* Their answer is the queue-reactive model, in which the rate of every kind of event at a queue is an arbitrary function of the current size of that queue (and, in richer versions, of the neighbouring queues), estimated without assuming a functional form.

State and rates. The book is described by the queues at the first few price levels on each side of a reference price p_{ref} (the mid when the spread is an odd number of ticks; three levels in the paper, because levels four and five behave like level three). Queue sizes are counted in units of the average size of events at that level, so a queue of size x holds about x typical orders. In the basic version each queue is a birth–death process like those of Section 6.1, but with size-dependent rates: limit orders arrive at rate $r_L(x)$, and cancellations and market orders remove units at rates $r_C(x)$ and $r_M(x)$, where x is the current queue size.

In words: the model writes down a separate arrival, cancellation and trading rate for each possible queue size, rather than one rate for all sizes. The data decide how the rates change with size.

Calibration by counting. For every event at a queue, record the time since the previous event at that queue, the event type, and the queue size just before the event. The estimator of the paper is

$$\hat{r}_L(x) = \frac{\#\{\text{limit orders that arrived while the queue held } x\}}{\text{total time the queue spent at size } x},$$

and likewise for cancellations and market orders. (The paper writes it as the inverse of the mean waiting time at size x , times the fraction of events at size x that were limit orders; the two forms

are algebraically equal.) Bid and ask are pooled by symmetry, and recording restarts whenever p_{ref} moves.

In words: to learn how fast orders arrive when a queue holds x units, count the arrivals that happened while it held x units and divide by how long it held x units in total.

A family of four models. Huang et al. (2015) build the model in steps, each adding one kind of dependence.

Model I: independent queues

Each queue’s rates depend only on its own size. Each queue is then a separate birth–death process, which is what makes the closed-form result below possible.

Model IIa: the second level watches the first

The rates at the second level depend also on whether the best queue is empty, and market orders hit the second level when the first is gone.

Model IIb: the two sides watch each other

The rates at the best bid depend also on the size of the best ask (bucketed as empty, small, usual or large), and vice versa.

Model III: prices

The reference price moves when a best queue empties, with the occasional reset of the whole book described below.

The one closed-form result: how long queues typically are. In Model I a queue of size x grows at rate $r_L(x)$ and shrinks at rate $r_C(x) + r_M(x)$. In the long run, the fraction of time it spends at each size, $\pi(x)$, settles down, and it satisfies a simple balance: the queue must move up from x to $x + 1$ as often as it moves down from $x + 1$ to x . That gives

$$\frac{\pi(x+1)}{\pi(x)} = \frac{r_L(x)}{r_C(x+1) + r_M(x+1)}, \quad \text{so} \quad \pi(x) = \pi(0) \prod_{y=0}^{x-1} \frac{r_L(y)}{r_C(y+1) + r_M(y+1)}, \quad (22)$$

with $\pi(0)$ chosen so that the probabilities add up to one.

In words: the chance of finding the queue one order longer is the chance of finding it at its current size, times the ratio of “joining” to “leaving” at that size. Where joining is faster than leaving the distribution climbs; where leaving is faster it falls; the most likely size is where the two rates cross.

Feeding in the measured rate functions gives a predicted distribution of queue sizes, which the authors compare with the sizes observed every 30 seconds: the agreement is close, and better than with constant rates and proportional cancellation. Figure 12 shows, with stylised rates of the shapes reported, how the realistic shapes change the distribution.

Estimated rates on Euronext Paris data. On France Telecom and Alcatel-Lucent (Euronext Paris, 2010–2012, five levels), the rates are far from those of the simpler models. At the best level, the limit-order rate is roughly constant in queue size (lower when the queue is empty); the cancellation rate increases concavely up to about 25 units and then flattens, rather than growing linearly as Cont–Stoikov–Talreja assume; and the market-order rate *decreases* exponentially with the available volume—traders hit small queues. The long-run distribution of queue sizes implied by the fitted rates matches the empirical one closely, and better than a Poisson model with linear cancellation. Richer versions let the best-level rates depend on the opposite queue: insertion falls and market orders rise as the opposite queue grows.

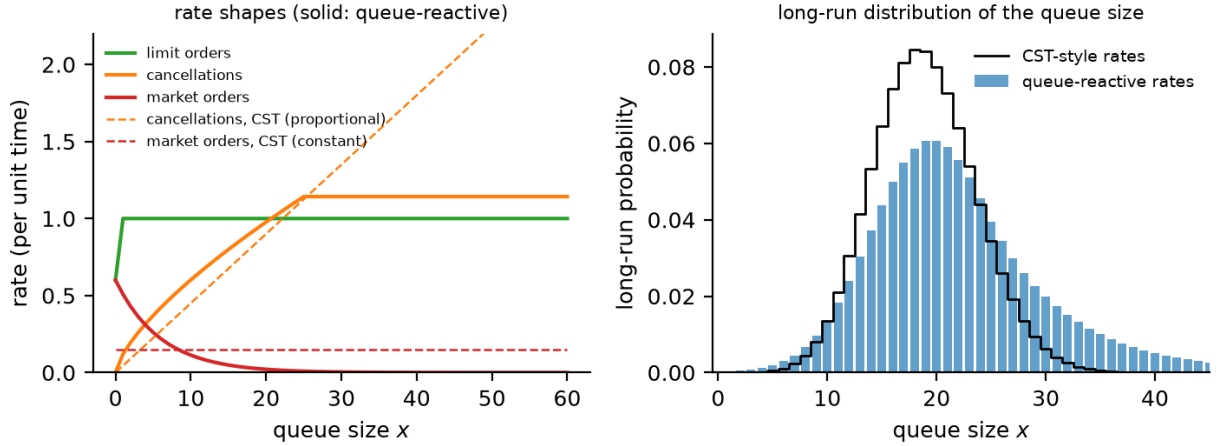


Figure 12: Queue-reactive rates and the long-run distribution of the queue size. Left: stylised rate functions with the shapes reported by Huang et al. (2015)—roughly constant arrivals, cancellations that rise and then flatten, market orders that fall off as the queue grows (solid)—against Cont–Stoikov–Talreja style rates with proportional cancellation and constant market orders (dashed). Right: the long-run distributions implied by each, from the balance formula. Because cancellations stop growing for long queues, long queues are much more likely under the queue-reactive shapes. Stylised rates, not fitted values.

Prices: two kinds of price change. The full queue-reactive model adds a price, and it lets the price change in two different ways.

A mechanical move

A best queue empties and the reference price moves one tick (in the model, with probability π_{move} when this happens). The rest of the book stays as it was: the queues simply shift one place—the old second level becomes the new best quote. The book keeps its memory, so what happens next is still shaped by the queues that were already there.

A move with a reset

With probability π_{reinit} , the price move also wipes the book: all the queues are replaced by a fresh draw from their typical long-run sizes around the new price. This stands for news. When new information arrives, traders reprice and the old queues stop mattering; the book after the move has nothing to do with the book before it. ($\pi_{\text{reinit}} = 1$, a reset at every move, is the Cont–de Larrard assumption of Section 6.2.)

The two probabilities are calibrated so that the simulated price matches two features of the real one: its ten-minute volatility, and how often a price move is followed by another in the same direction rather than a reversal. For France Telecom $\pi_{\text{move}} = 0.7$ and $\pi_{\text{reinit}} = 0.85$: most price changes come with a reset.

Figure 13 shows a stretch of simulated trading: the best queues wander, a price move happens when one of them reaches zero, and most moves come with a reset of the book.

Experiment without queue resets. The authors then switch the resets off ($\pi_{\text{reinit}} = 0$) and let every emptied queue move the price ($\pi_{\text{move}} = 1$), which gives the queues the largest possible influence. The result is a price driven only by order-book mechanics. Its volatility is at most about 5 basis points, against 14 observed—about a third. And it reverses far too often. The reason is visible in the mechanics: right after a mechanical move, the book is typically lopsided the other

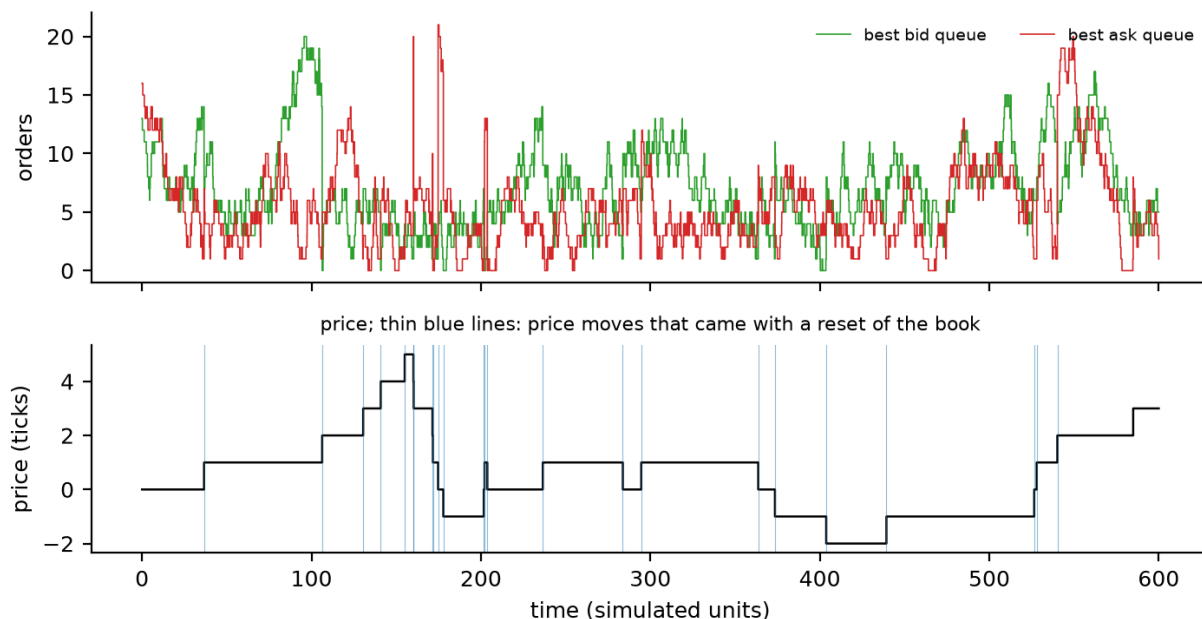


Figure 13: A run of a queue-reactive simulator with two levels per side, rate functions of the shapes in Figure 12 scaled to short queues, $\pi_{\text{move}} = 0.7$ and $\pi_{\text{reinit}} = 0.85$. Top: sizes of the best bid and best ask queues. Bottom: the price, in ticks; thin vertical lines mark price moves that came with a reset of the book. Stylised simulation for illustration; not data.

way, so the next move tends to undo the last one, and the price bounces back and forth instead of wandering. Real prices wander much more. The conclusion is that the queues explain the small, back-and-forth part of price movement, and that most of the variance comes from information arriving from outside the book—which, in the model, is the job of the resets. One practical reading of this result is as a statement about what an order-book model can and cannot be expected to forecast: the mechanics of the next tick or two, not the larger moves driven by news.

Uses: the questions answered by simulation. With the rates fitted, the model is run forward to answer questions that have no formula.

Will my order be filled?

A trader’s order is placed in the simulated queue and followed until it is filled or the price moves away. Because the rates react to the queue, the order’s own presence, and the orders that join behind it, change the outcome. The authors find that the different versions of the model give similar execution probabilities, while a model with constant (Poisson) rates overstates them.

How should a large order be worked?

A large order is split into slices over time, and two tactics are compared in simulation: placing each slice and leaving it (“fire and forget”), or keeping it repriced at the best quote (“pegging”), under different schedules for the slice sizes.

Where does volatility come from?

As described above, switching the resets on and off measures how much of the price movement the queues alone can produce.

Its neural extension learns the dependence of the rates on the book with networks (Bodor and Carlier, 2025), and queue-reactive Hawkes models add self-excitation to the size-dependent rates

(Section 7.18). Its neural extension learns the dependence of the rates on the book with networks (Bodor and Carlier, 2025), and queue-reactive Hawkes models add self-excitation to the size-dependent rates (Section 7.18).

Using it in practice: what a built simulator is for. Suppose the simulator has been built and fitted to a contract one trades. What can be done with it? The honest answer has two halves. It is a realistic laboratory for questions about *the mechanics of execution*—how orders join, wait in and leave queues, and how one’s own orders change that. It is not a source of evidence about *whether a trading idea makes money*, because the only information it contains about future prices is what was put into it, and in particular its news-driven price moves are random by construction. Table 12 lists the main uses with that line drawn. The published applications are fill probabilities and the comparison of placement tactics (Huang et al., 2015); for its neural extension, reproducing the square-root law of market impact, order-size distributions and correlations between queues on Bund futures (Bodor and Carlier, 2025); and, for related reactive simulators built on neural Hawkes processes, serving as the training environment for a reinforcement-learning market maker (Lalor and Swishchuk, 2025). The other rows are uses that follow from what the model is, not reported results.

Table 12: What a fitted queue-reactive simulator can and cannot be used for.

Use	What you do	What to trust, and what not
Fill probabilities and times	Insert an order at each queue position and queue size; run many paths; tabulate the chance of a fill within a horizon and the time it takes. The tables can feed an execution engine.	The mechanics of the queue, which the model is fitted to. Not the adverse-selection part: which fills are followed by bad price moves depends on information the model does not have.
Tuning execution tactics	Run an execution algorithm against the simulator with different settings—how long to rest before repricing, whether to peg to the best quote, how to split a large order—and compare cost and completion.	Rankings of tactics whose differences come from queue mechanics. Treat absolute costs with caution, and expect to overfit the simulator if many settings are tried.
Impact of one’s own orders	Insert one’s orders and measure how the simulated book and price respond.	The mechanical part of impact, how queues are consumed and refill. Not the impact that comes from other traders inferring information from one’s trading.
Stress testing	Change the rates—thinner queues, more aggressive flow, slower refill—and check that an algorithm’s limits, cancellations and safeguards behave.	Behaviour under stylised extreme conditions. Not the probability of such conditions.
Training environment	Use the simulator as the environment for a learning agent (execution or market making), because, unlike replay, it reacts to the agent’s orders.	A starting point for the agent. Its policy must then be evaluated on recorded data.
Testing research code	Generate data with known rates and check that estimators, feature pipelines and book reconstruction recover them.	Correctness of the code. Nothing about the market.
Intuition and teaching	Switch features on and off—resets, state dependence—and watch the effect.	Qualitative understanding.
<i>Not:</i> estimating trading profit	Backtesting a directional strategy on simulated prices.	Any profit is an artefact: the simulated price moves contain no information beyond the fitted rates.

Place among the evaluation tools. The simulator and historical replay (Section 12.1) are complementary. Replay uses the real market but cannot react: a simulated order sits in the recorded queue, and the market behaves exactly as it did, whatever the order does. A queue-reactive simulator reacts, but its market is synthetic. A sensible workflow uses each for what it does well: explore and narrow down execution settings in the reactive simulator, where trying many variants is cheap; evaluate the survivors in queue-exact replay on recorded data, counting every variant tried (White, 2000; Harvey et al., 2016); and only then trade them on paper and live. The simulator’s results are never the final evidence.

Ingredients for building a queue-reactive simulator. The ingredients follow the steps of this subsection:

1. data for the first few price levels with each change classified as a limit order, a cancellation or a trade—market-by-order data make this direct; market-by-price data require inferring it from size changes, as in Section 6.1;
2. the rate tables, by counting events and time spent at each queue size, separately by level (and by the opposite queue’s size, for the richer versions);
3. the two price probabilities, chosen so that the simulated price matches the real ten-minute volatility and the ratio of continuations to reversals;
4. the simulation loop of Figure 11, a few dozen lines of code;
5. validation before use: the simulated queue-size distributions, times between events and price statistics should match the real ones, on data not used for fitting.

Model card: Queue-reactive model (a market simulator)

What it is. A generative simulator of the first few levels of the book: from fitted rates it produces synthetic streams of limit orders, cancellations and market orders, the resulting queue sizes and the price, and it reacts to orders the trader inserts. One closed-form result (the long-run distribution of queue sizes); everything else by running it.

What you calibrate. For each of the first few levels, a table of rates by queue size: limit orders $r_L(x)$, cancellations $r_C(x)$ and market orders $r_M(x)$ (in richer versions, also by the size of the neighbouring or opposite queue); and two probabilities for prices, π_{move} (a price move when a best queue empties) and π_{reinit} (a reset of the book with that move).

How. Rates: count the events of each type that happened while the queue held x orders, and divide by the total time it held x orders. Price probabilities: choose them so that the simulated price matches the real ten-minute volatility and the real ratio of continuations to reversals. Validate the fitted simulator on held-out data before use.

What you do with it. Estimate fill probabilities and times by queue position; compare and tune execution tactics (resting time, pegging, slicing a large order) in a market that reacts; measure the mechanical impact of one’s own orders; stress-test algorithms; train learning agents. Narrow down settings here, then evaluate the survivors in queue-exact replay. Do not use it to estimate trading profit: its price moves carry no information beyond the fitted rates.

How to picture it. A toll booth whose drivers and attendant react to the line: when it is long, more drivers give up and leave; when it is short, cars are served faster. The simulator is that booth running on its own, with the trader able to join the line and watch what happens.

6.4 Summary: what the queue models tell a trader

The common idea. All the models of this section start from the same picture. Each best queue is a bucket that fills with limit orders and drains through trades and cancellations (Section 6). The price moves when a best queue empties, so the direction of the next move is a race between the two best queues, and the wait for a resting order to be filled is a race between the orders ahead of it and the opposite queue. What the models disagree on is how traders behave, and therefore how the buckets fill and drain—and that choice decides what each model can deliver.

The three models in one line each. Table 11 compares them in full.

Cont–Stoikov–Talreja (Section 6.1)

constant arrival and trading rates, cancellations proportional to queue size; exact answers, from the current queues, to the three questions about the next price move—by numerical transform inversion.

Cont–de Larrard (Section 6.2)

rates independent of queue size and a book redrawn after every move; the same questions answered by formulas that show how the answers depend on queues and rates, plus the waiting time to the next move and volatility.

Queue-reactive (Section 6.3)

every rate an arbitrary function of queue size, measured from data; a realistic simulator of the book that reacts to one’s own orders, with one closed-form result.

Five lessons common to the three models. Five lessons recur across the three models, and each is visible in a figure of this section.

1. **The thin side breaks first, and the ratio of the queues is what matters.** The probability of an up move rises with the bid queue and falls with the ask queue (Figure 3); in the balanced case it depends only on their ratio (Figure 7).
2. **A fill and an adverse move are often the same event.** The books in which a resting bid is most likely to be filled are those in which the price is most likely to fall next (Figure 5), and among filled orders, those further back in the queue are followed by worse moves (Figure 10).
3. **Queue position is worth money.** Every order ahead lowers the chance of a fill before the price moves (Figure 4), and the difference in value between the front and the back of the queue can be comparable to the spread (Moallemi and Yuan, 2017).
4. **Long quiet periods and volatility follow from the order flow.** Nearly balanced inflow and outflow make very long waits between price moves likely (Figure 6), and volatility grows with order activity and falls with depth (Figure 9).
5. **Queues explain the mechanics, not the news.** Book mechanics alone produce at most about a third of observed volatility, and far too many reversals; most price variance comes from outside the book (Section 6.3).

From the models to decisions. These results translate directly into execution choices.

Limit or market

If the expected time for a passive order to reach the front and be filled is longer than the horizon

over which the trader’s signal is valid, waiting forfeits the signal; crossing the spread is then the better choice despite its cost.

When to cancel

If the estimated depletion time of the side on which an order rests collapses—below the trader’s own latency, for instance—or the race turns against the order, it is about to be filled just before an adverse move, and should be cancelled or repriced.

Where to queue

Moallemi and Yuan (2017) split the value of a resting order into a *static* part—the spread earned at execution minus the adverse-selection cost, which grows with queue position—and a *dynamic* part—the option value of holding a place in the queue that advances over time. They find that the front-back difference can be comparable to the bid-ask spread for some large-tick US stocks (for example BAC and CSCO), and smaller for others.

How to estimate fill chances from data

The time to fill is a waiting time that, for orders cancelled first, is never observed; survival models (Cox, 1972), which handle such *censored* observations, are the natural statistical tool (Section 10.14).

How to test tactics

A reactive simulator such as the queue-reactive model is the place to compare execution tactics cheaply; queue-exact replay on recorded data is the place to evaluate the survivors (Section 12.3).

Queue models thus serve execution in two ways: they estimate *when* a resting order will be filled, and, because a depletion is a price move, *how informative* the fill is likely to be (Section 11.3). They say which way the price goes next only through the race between the two sides.

Omissions common to the three models. Orders arrive as Poisson streams, or at rates that depend only on the current queues: an event never makes further events more likely, so the bursts and clustering of real order flow (Section 3) are missing. Orders have a single size. Nothing in the models carries information about where the price is going beyond the queues themselves, so they describe the mechanics of the next move, not the reasons for larger ones. And their evidence comes mostly from large-tick equities. The first of these gaps—events that trigger events—is the subject of the next section.

7 Hawkes processes

Limits of the Poisson queue models. The queue models of Section 6 assume that orders arrive as independent streams at rates fixed over the sample. In a calm book that is a reasonable approximation: the Cont–Stoikov–Talreja up-move probabilities of Table 9 track the empirical frequencies closely. It breaks down exactly when it matters most. After a large trade or a news release, rates jump to several times their average for a few seconds, and a model that uses the day’s average rate underestimates how fast queues empty (Example 6.2). The missing ingredient is that events cause events. This section introduces the standard model of that mechanism.

Organisation of this section. The section goes from the simplest model to the most elaborate. The first subsections work with a single stream of events: why the Poisson process fails for order flow (Section 7.1), the four ideas the section is built on, in plain language and pictures (Section 7.2), the language of point processes—counts and the conditional intensity (Section 7.3), the two simpler

models a Hawkes process is compared with, the Poisson and renewal processes (Section 7.4), the simplest Hawkes model, with one event type (Section 7.5), its branching ratio (Section 7.6), what Hawkes processes look like (Section 7.7), and the statistics that tell the three apart on real data (Section 7.8). The section then moves to several interacting event types (Sections 7.9–7.10), connects events to prices (Section 7.11), explains how the models are fitted and what fitted models find (Sections 7.12–7.14), and then adds one ingredient at a time: sizes and labels on events, dependence on the state of the book, parameters that change over time, other kernels and link functions, and finally neural networks (Sections 7.15–7.19). It closes with what Hawkes models are used for (Section 7.20). The material of Sections 7.1 and 7.3–7.8 is adapted from Appendix A of [Maciejewski \(2026a\)](#), which also supplies the empirical figures quoted for CME NQ market data.

7.1 Limitations of the Poisson process

The object being modelled. Watch packets arrive from a market-data feed and write down the time of each arrival:

```
09:30:00.213    <- 1st arrival
09:30:00.451    <- 2nd
09:30:00.802    <- 3rd
09:30:01.014    <- 4th
09:30:01.317    <- 5th
...
```

That list of timestamps is what a point-process model describes—nothing else, no prices or sides, only *when*. The same applies to order-book events of any kind.

The Poisson process. The simplest model of events arriving at random times is the *Poisson process* ([Daley and Vere-Jones, 2003](#)). Divide time into tiny slices and, independently in each slice, let an event happen with a small fixed probability; in the limit, events arrive at a constant average rate μ per second. The number N_w of events in any window of length w has the distribution

$$\mathbb{P}(N_w = k) = e^{-\mu w} \frac{(\mu w)^k}{k!}, \quad k = 0, 1, 2, \dots \quad (23)$$

In words: the chance of seeing exactly k events in w seconds depends only on the average rate and the length of the interval—not on when the interval starts, and not on anything that happened before it. The process has no memory: an event now makes the next one neither more nor less likely. The waiting times between events are independent and exponentially distributed, and the count in an interval has variance equal to its mean.

Three consequences of this independence are worth spelling out, because each can be checked against data.

(1) The gap between consecutive arrivals is exponentially distributed with mean $1/\mu$. At $\mu = 300$ per second the mean gap is

$$\text{mean gap} = \frac{1}{\mu} = \frac{1}{300/\text{s}} \approx 3.3 \text{ ms.}$$

The gap distribution is *memoryless*: however long one has been waiting, the expected time to the next arrival is still 3.3 ms. The probability that a gap exceeds a length u is

$$\mathbb{P}(\text{gap} > u) = e^{-\mu u},$$

so at this rate

$$\mathbb{P}(\text{gap} > 3.3 \text{ ms}) \approx 36.8\%,$$

$$\mathbb{P}(\text{gap} > 6.6 \text{ ms}) \approx 13.5\%$$

and

$$\mathbb{P}(\text{gap} > 10 \text{ ms}) \approx 5.0\%.$$

The probability that a gap is shorter than a tenth of the mean is

$$\mathbb{P}(\text{gap} < \frac{1}{10} \text{ mean}) = 1 - e^{-0.1} \approx 9.52\%,$$

whatever the rate. On CME NQ packet data the share is 73% (Maciejewski, 2026a).

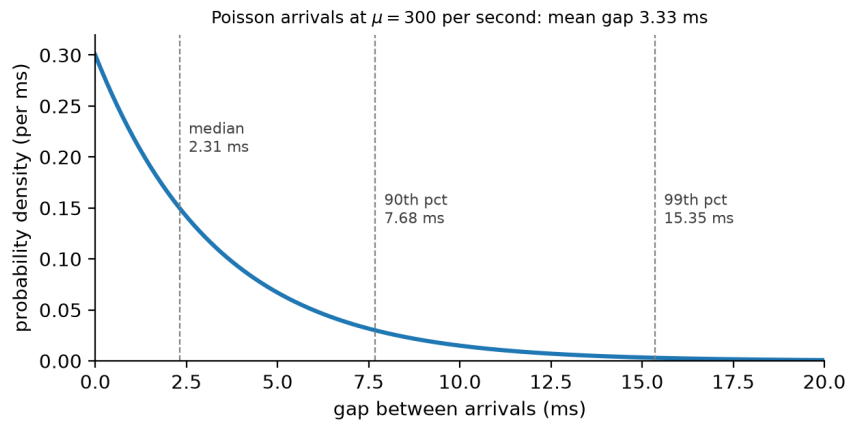


Figure 14: The exponential density of the gaps between arrivals of a Poisson process at 300 arrivals per second. Dashed vertical lines mark the median, the 90th and the 99th percentile. Half the gaps are shorter than the median of 2.3 ms and 10% are longer than 7.7 ms. Real market-data streams have far more mass near zero and a much heavier right tail. Redrawn from Maciejewski (2026a).

(2) The count in any window of length w is Poisson-distributed, as above, with

$$\mathbb{E}[N_w] = \mu w, \quad \text{Var}[N_w] = \mu w,$$

so the ratio of variance to mean is 1 for every w . This ratio is the *Fano factor* (Section 7.8); a value other than 1 rejects the Poisson model.

(3) Counts in non-overlapping windows are independent. The number of arrivals in the first second says nothing about the number in the next.

Poisson as the null hypothesis. The Poisson process is the stream with no memory, no clustering and no regularity: arrivals are as uncoordinated as possible. Any departure from it is structure in the arrival stream.

For order-book events these properties fail, in three ways that can be seen in any day of data.

1. **Events cluster.** Activity comes in bursts. A large trade is followed by a flurry of new orders, cancellations and further trades as other participants react; a news release does the same on a larger scale. Between bursts the market can be quiet for long stretches. In a Poisson process an event says nothing about the next one, so bursts can only arise by chance, and large ones are vanishingly rare.
2. **Counts vary far more than Poisson allows.** Because of clustering, the number of events per second has a variance much larger than its mean, whereas a Poisson count has variance equal to its mean (Bacry et al., 2015).
3. **Event types trigger each other.** A buy market order makes further buy market orders more likely, because large orders are split into many small ones and because other traders follow; a trade that empties a level triggers cancellations and new limit orders around it. A set of independent Poisson processes, one per event type, cannot represent any of this.

A fourth, milder departure is that average activity changes predictably through the day, typically highest near the open and the close. A Poisson process with a time-varying rate (an *inhomogeneous* Poisson process) can absorb that pattern, but it still cannot make events cause other events, which is the defining feature of order flow.

Bunching and memory measured on a feed. Two of them can be measured directly on a feed. The first is *bunching*: many gaps are far shorter than the exponential law predicts (73% against 9.52% above). The second is *memory in the rate*: after a burst starts, more arrivals are likely to follow. Modelling either needs a process whose rate can change with its own history—a *conditional-intensity point process*.

Example 7.1 (How rare is a burst under Poisson?). Suppose events arrive at an average rate of $\mu = 10$ per second. Under a Poisson model the probability of 30 or more events in a single second is about 2.5×10^{-7} . Over a 6.5-hour trading day of 23 400 seconds, the expected number of such seconds is about 0.006—roughly one every 170 trading days. In real order flow, seconds with several times the average activity occur routinely around large trades and announcements. A model that calls them once-a-year events will badly underestimate how quickly queues can empty (Section 6).

A compact way to remember the difference: a Poisson process is *drizzle*—drops fall independently at a steady rate, and one drop says nothing about the next. Order flow is a *storm*—long dry spells broken by downpours, in which each burst of rain makes more rain likely for a while.

The Hawkes process keeps what is convenient about the Poisson process—events at random times, described by a rate—but lets the rate *rise after each event* and then decay back. It is the smallest change to the Poisson model that makes events cause events. The next subsections give the language needed to state it precisely.

7.2 The four ideas in pictures

Before the formal definitions, this subsection gives the four ideas the rest of the section is built on—*intensity*, *self-excitation*, *cross-excitation* and *branching*—in plain language, with pictures. Each is defined precisely later; the numbers here are illustrative.

1. Intensity: the speedometer of events. The intensity $\lambda(t)$ is how fast events are arriving *right now*, in events per second. It is not a count of what has happened but a rate: like a car's speedometer, it can read 300 at one instant and 20 a second later. It translates into a chance:

if the intensity is 300 events per second, the chance of an event in the next millisecond is about $300 \times 0.001 = 0.3$. In a Poisson process the needle never moves. In a Hawkes process it jumps up every time an event happens and then drifts back down towards a resting value, the *background rate*. The lower panel of Figure 15 shows two such needles.

2. Self-excitation: events of a kind make more of the same kind. After a buy market order, more buy market orders become likely for a while: the buy intensity jumps and then decays. One common reason is a large order being executed in pieces, so that one child order is followed by the next; another is other traders reacting to the same news or to the trade itself. In Figure 15, each buy makes a large jump in the blue curve.

3. Cross-excitation: events of one kind make events of another kind more likely. The same buy can also raise the intensity of a *different* event type: sell market orders, say, or cancellations at the best ask. A buy that takes liquidity at the ask may draw sellers who see a better price, or prompt sellers resting at the ask to cancel before the price moves through them. In Figure 15, each buy also makes a smaller jump in the orange curve. Self-excitation tends to produce runs of the same event; cross-excitation links the two sides and the different event types of the book.

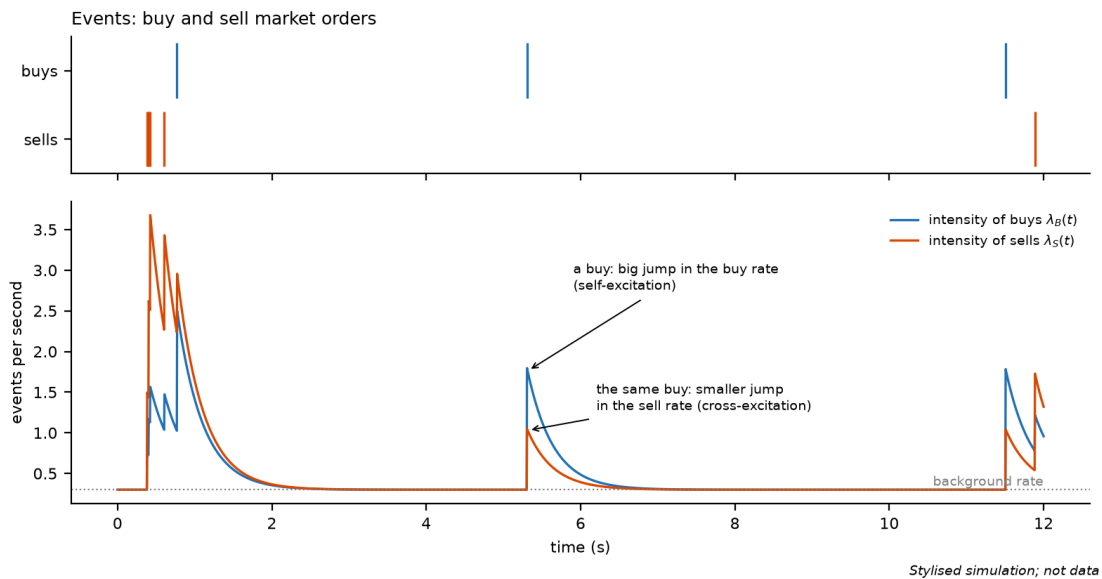


Figure 15: Intensity, self- and cross-excitation. Top: a simulated stream of buy and sell market orders. Bottom: the intensity of each. Each event makes both curves jump, by more for its own type (self-excitation) than for the other (cross-excitation); between events both decay towards the background rate (dotted). Where events come close together, as near the start, the jumps pile up and the rates climb. Two-type Hawkes process with exponential kernels; stylised simulation, not data.

4. Branching: family trees of events. The same process can be read in a second way, as families. Some events are *immigrants*: they arrive on their own, from outside, at the background rate—news, a trader’s decision unrelated to recent activity. Every event, immigrant or not, then has a random number of *children*: events it directly triggers. Children have children of their own, and so on (Figure 16). The **branching ratio** Γ is the average number of children per event. Its value decides everything:

- If $\Gamma = 0$, nobody has children: every event is an immigrant, and the stream is Poisson.
- If $\Gamma = 0.8$, each event has on average 0.8 children, 0.64 grandchildren, and so on, and one immigrant's family has on average $1 + 0.8 + 0.64 + \dots = 1/(1 - 0.8) = 5$ events. Four out of every five events were triggered by an earlier one; only one in five came from outside.
- As Γ approaches 1, families become very large and long-lived. At $\Gamma \geq 1$ a single immigrant can start a cascade that never dies out, and the process is unstable.

With several event types, the branching ratio becomes a table, the *branching matrix*: the entry in the sells row and buys column, Γ_{SB} in Section 7.10, is the average number of sells triggered directly by one buy. Its diagonal entries measure self-excitation, its off-diagonal entries cross-excitation.

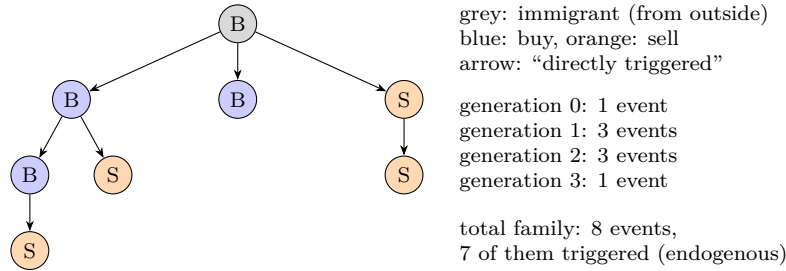


Figure 16: One family in a two-type Hawkes process. An immigrant buy triggers two buys and a sell (self- and cross-excitation), which trigger further events. The branching ratio is the average number of arrows leaving an event; for the process to be stable it must be below one, so that families die out. One illustrative family, drawn by hand; a branching ratio below one does not stop an individual family from being larger.

Intensity as speed, branching as acceleration: an analogy and its limits. It is tempting to say that the intensity is like speed and the branching ratio like acceleration. The first half holds: the intensity is the speed at which the count of events grows, it changes from moment to moment, and it is what a speedometer would show. The second half does not. The branching ratio is a fixed property of the model, while acceleration changes from moment to moment; a high branching ratio does not mean the rate is rising now—in a quiet spell the intensity drifts down whatever Γ is; and Γ is not a rate of change at all but a count, the number of extra events one event eventually produces. What plays the part of acceleration is the kernel: the kick each event gives and how quickly it wears off. Table 13 completes the analogy.

Table 13: The driving analogy for a Hawkes process with an exponential kernel.

Car	Hawkes process
speed	intensity $\lambda(t)$
cruising speed	background rate μ
a tap on the accelerator: the speed jumps	an event: the intensity jumps by α
engine braking back towards cruising speed	decay at rate ω towards μ
the extra distance one tap ends up producing	the branching ratio $\Gamma = \alpha/\omega$: the extra events one event ends up triggering

The twist that makes it a Hawkes process is that, in this car, the extra distance itself causes more taps on the pedal. If each tap produces on average fewer than one further tap ($\Gamma < 1$), the car always settles back to cruising speed. If it produces one or more ($\Gamma \geq 1$), taps breed taps faster than braking can cancel them and the speed runs away. In short: the intensity is the speedometer,

the kernel is the kick each event gives the pedal and how fast it wears off, and the branching ratio is the gain of the loop—the same role as the reproduction number of an epidemic, the average number of people each case infects, with the same threshold at one.

Relation between the intensity view and the branching view. The two views describe the same process. The jump in the intensity after an event is that event’s chance of having children in the following instants; the area under the jump—the total extra intensity it ever adds—is the expected number of its children, the branching ratio. Self- and cross-excitation are the two kinds of arrow in the family tree. Sections 7.3 to 7.10 make each of these ideas precise: the intensity in Section 7.3, the one-type model in Section 7.5, the branching ratio in Section 7.6, and self- and cross-excitation with the branching matrix in Sections 7.9 and 7.10. On CME NQ data, four of every five events are triggered by earlier ones (Section 7.6).

7.3 The language of point processes

Points, processes and marks.

- A **point** is a single instant at which something happens, such as the timestamp of one packet or one order-book event.
- A **process** is a random object that unfolds in time; here, a random list of arrival times $t_1 < t_2 < t_3 < \dots$.
- A **point process** is a random sequence of points on the time axis, specified by *when* events happen and nothing else.

If each event carries additional information (side, price, size, message type), the process is a **marked point process**: the timestamps are the points and the extra information is the *mark*. Four order-book messages in the first second of a session might read

```
09:30:00.001234567    Add,    bid 21432.50, size 3
09:30:00.001234571    Add,    ask 21432.75, size 5
09:30:00.017891234    Cancel, order X, bid 21432.25
09:30:00.238491702    Trade,  ask 21432.75, size 2
```

The point process is the four timestamps; message type, side, price and size are marks. The multivariate Hawkes process of Section 7.9 is a marked point process whose mark is the event type i .

The probability space. An arrival stream is modelled as a **stochastic process**, a family of random variables indexed by time, on a probability space $(\Omega, \mathcal{A}, \mathbb{P})$:

- Ω , the *sample space*, is the set of all possible outcomes; here an outcome is one complete realisation of a session’s arrivals.
- $\mathcal{A}$, the σ -*algebra*, is the collection of yes-or-no questions about the outcome to which a probability can be assigned, such as “did at least one packet arrive between 09:30:00 and 09:30:01?”. It is closed under negation and countable union.
- $\mathbb{P} : \mathcal{A} \rightarrow [0, 1]$ assigns a probability to each such event, with $\mathbb{P}(\emptyset) = 0$, $\mathbb{P}(\Omega) = 1$, and probabilities of disjoint events adding up.

History: information accumulated over time. The information available at time t grows as time passes. Formally it is a **filtration**, an increasing family of σ -algebras

$$\{\mathcal{I}_t\}_{t \geq 0}, \quad \mathcal{I}_{t'} \subseteq \mathcal{I}_t \text{ for } t' \leq t.$$

$\mathcal{I}_t$ is everything observable up to and including time t , and $\mathcal{I}_{t-}$ everything observable strictly before t . “Conditional on $\mathcal{I}_{t-}$ ” means “given everything seen strictly before t ”. Throughout this survey, conditioning on the observable information X_t has this meaning: a model may use only what is in the history.

The counting process. The observable object is the random step function

$$N(t) = \#\{n : t_n \leq t\},$$

the number of arrivals up to time t . It starts at $N(0) = 0$, never decreases, and jumps by one at each arrival. The number of arrivals in the interval $(t, t + \Delta t]$ is $N(t + \Delta t) - N(t)$.

Conditional intensity. At time t , ask for the probability that at least one arrival occurs in the next short interval of length Δt , given everything seen so far. The **conditional intensity** is that probability divided by Δt , in the limit of a very short interval:

$$\lambda(t \mid \mathcal{I}_{t-}) = \lim_{\Delta t \downarrow 0} \frac{1}{\Delta t} \mathbb{P}(N(t + \Delta t) - N(t) \geq 1 \mid \mathcal{I}_{t-}).$$

It is the instantaneous arrival rate given the history, in arrivals per second; in the rest of the paper it is written simply $\lambda(t)$. At an intensity of 300 per second the probability of an arrival in the next millisecond is about $\lambda(t) \Delta t = 300 \times 0.001 = 0.3$. For a short interval, and for the expected count over a longer one,

$$\mathbb{P}(\text{arrival in } (t, t + \Delta t]) \approx \lambda(t) \Delta t, \quad \mathbb{E}[N(t + w) - N(t)] = \mathbb{E} \int_t^{t+w} \lambda(t') dt'.$$

Specifying λ as a function of the history specifies the point process completely. For a Poisson process $\lambda(t) = \mu$ and the history carries no information. For a Hawkes process the intensity is μ plus a decaying contribution from each past arrival.

7.4 Two simpler models: Poisson and renewal processes

A Hawkes process is best understood next to the two simpler models it is usually compared with. In this subsection and the next there is a single event type; arrivals are indexed by n and counting windows have length w , as elsewhere in the survey.

(a) Poisson. $\lambda(t) = \mu > 0$. Gaps are independent and exponential with mean $1/\mu$; the count in a window of length w is Poisson with mean μw . Every summary statistic takes its reference value: the Fano factor is 1 at every w , the Hurst exponent is 0.5, and the gaps are uncorrelated.

(b) Renewal process. The gaps $\Delta t_n = t_n - t_{n-1}$ are independent draws from a common distribution, not necessarily exponential. The intensity depends only on the time since the last arrival. A renewal process is Poisson only when the gaps are exponential. Gaps with a heavy-tailed but finite-variance distribution produce many short gaps and a Fano factor well above 1, but the

Fano factor levels off as w grows instead of continuing to rise, and the gaps are uncorrelated. (If the gaps have infinite variance, the Fano factor of a renewal process can also keep rising, so a rising Fano factor alone does not prove self-excitation.) Randomly shuffling the order of the gaps leaves a renewal process statistically unchanged; comparing a statistic on a real stream with the same statistic on its shuffled gaps therefore separates what the *order* of the gaps carries from what their *lengths* carry.

7.5 The simplest Hawkes model: one event type

In a **Hawkes (self-exciting) process** each past arrival raises the intensity for a while:

$$\lambda(t) = \mu + \sum_{n: t_n < t} \phi(t - t_n).$$

$\mu \geq 0$ is the **background rate**, the intensity with no prior arrivals. ϕ is the **kernel**: an arrival u seconds ago still contributes $\phi(u)$ to the intensity now. The kernel is the shape of one event's influence on the future. Dropping several stones into a pond is the usual picture: the disturbance at any moment is the sum of the ripples still spreading, each faded by the time since its stone fell. For the self-exciting specification used here the kernel is non-negative, so a past event can only raise future intensity (inhibitory extensions are discussed in Section 7.18), and its total area must be finite, so that the past does not accumulate without bound. Section 7.9 extends the model to several event types that excite each other.

Kernel families. The *exponential kernel* $\phi(u) = \alpha e^{-\omega u}$ has two parameters: $\alpha \geq 0$, the jump in intensity immediately after an arrival, and $\omega > 0$, the decay rate, whose inverse $1/\omega$ is the kernel's decay time. It is Markovian: the current intensity summarises everything needed from the past. Right after an arrival the intensity rises by α ; between arrivals it relaxes toward μ : if $\lambda(t_n)$ is the intensity just before the arrival at t_n , then u seconds after it the intensity is $\mu + (\lambda(t_n) + \alpha - \mu) e^{-\omega u}$ (until the next arrival); so it can be updated in constant time per arrival—a useful property for a low-latency system. The *power-law kernel* $\phi(u) = \alpha (1 + u/u_0)^{-(1+\zeta)}$, with the same jump α , a time scale $u_0 > 0$ and an exponent $\zeta > 0$, decays more slowly; Hardiman et al. (2013) find power-law kernels on E-mini S&P mid-price changes over lags from seconds to days.

7.6 The branching ratio

$$\Gamma = \int_0^\infty \phi(u) du, \quad \Gamma = \alpha/\omega \text{ for the exponential kernel.}$$

Γ is the mean number of arrivals that one arrival triggers *directly*. The Hawkes process is equivalently a **branching process** (Hawkes and Oakes, 1974): every arrival is either an *immigrant*, generated by the background rate μ , or the *child* of an earlier arrival, and each arrival has on average Γ children. The expected size of the family descended from one immigrant is the geometric series

$$1 + \Gamma + \Gamma^2 + \Gamma^3 + \dots = \frac{1}{1 - \Gamma} \quad (\Gamma < 1),$$

a series that Section 7.9 generalises to several event types. For $\Gamma < 1$ the process is *subcritical*: families are finite and the long-run rate is $\bar{\lambda} = \mu/(1 - \Gamma)$. As $\Gamma \rightarrow 1$ it becomes *near-critical*: family sizes and durations grow without bound and long-range dependence appears. At $\Gamma = 1$ the process is *critical*: each family still dies out with probability one, but its expected size is infinite, and with

a positive background rate there is no stationary version. For $\Gamma > 1$ it is *supercritical*: with positive probability one arrival starts a cascade that never ends, and the intensity grows without bound.

On CME NQ market data, an exponential-kernel fit gives $\Gamma \approx 0.80$ on both packets and matching-engine transactions, with a kernel decay time $1/\omega$ of about $167 \mu\text{s}$: four of every five arrivals are triggered by earlier ones (Maciejewski, 2026a). Estimates for other markets, and the debate about how close to one the branching ratio really is, are reviewed in Section 7.13.

The critical boundary in practice. At $\Gamma = 1$ the stationary rate $\mu/(1 - \Gamma)$ is infinite. On a finite window the process still fires finitely often, but the variance of the count grows faster than linearly with the window. A maximum-likelihood fit that lands at $\Gamma = 1$ on real data may signal non-stationarity, a mis-specified kernel or a numerical problem rather than a genuinely critical market; whether markets are critical or merely close to it is itself debated (Hardiman et al., 2013; Filimonov and Sornette, 2015).

A note on notation. Much of the literature writes the exponential kernel as $\alpha e^{-\beta u}$ and the (scalar) branching ratio as n . In this survey β is reserved for price impact (Section 4) and n for event indices, so the decay rate is written ω and the branching ratio Γ ; Section 7.9 generalises it to a matrix, also written Γ . With $\phi(u) = \alpha e^{-\omega u}$, the branching ratio is $\Gamma = \alpha/\omega$. Hawkes models have been applied extensively to order flow and price formation (Bacry et al., 2015).

7.7 Simulated Hawkes paths

The long-run rate $\bar{\lambda} = \mu/(1 - \Gamma)$ and the branching ratio Γ look different in an event raster. Figure 17 simulates an exponential-kernel Hawkes process over 15 seconds at nine combinations, $\bar{\lambda} \in \{2, 10, 50\}$ events per second and $\Gamma \in \{0.30, 0.60, 0.90\}$, with a kernel decay time of one second. Read across a row: the long-run rate is the same in every cell, but as Γ grows a larger share of events is triggered by earlier ones, so arrivals bunch, the count climbs in steps rather than a straight line, and the intensity wanders further from its mean and for longer. At $\Gamma = 0.9$ and ten events per second, for example, the intensity drifts between roughly 6 and 27 per second within fifteen seconds. Read down a column: a higher rate makes the raster denser and the count smoother, but does not remove the slow swings of the intensity that the branching ratio produces.

7.8 Telling Poisson, renewal and Hawkes processes apart

Three signatures separate the three models on real data.

(i) *An excess of short gaps.* On CME NQ packet data 73% of gaps are shorter than a tenth of the mean gap; any exponential gives 9.52%. A Hawkes process produces such an excess, but so does a renewal process with heavy-tailed gaps, so this signature rejects Poisson without identifying self-excitation.

(ii) *Correlated gaps.* Under self-excitation, short gaps tend to follow short gaps and long gaps follow long ones, so successive gaps are positively correlated. A renewal process has independent gaps. A rate that varies slowly over the observation window also produces positive gap correlation, so this signature has to be checked against that alternative.

(iii) *Burstiness that grows with the time scale.* The Fano factor rises with the window length on a Hawkes process and levels off on a Poisson process or a renewal process with finite-variance gaps. Clustering is then not a feature of one time scale but looks similar over decades of window lengths. A slowly varying rate can also make the Fano factor rise, which is the effect Filimonov and Sornette (2015) warn about.

Exponential-kernel Hawkes processes: long-run rate $\bar{\lambda}$ (rows), branching ratio Γ (columns)
each cell: events (top), count $N(t)$ (middle), intensity $\lambda(t)$ (bottom); decay rate $\omega = 1/s$

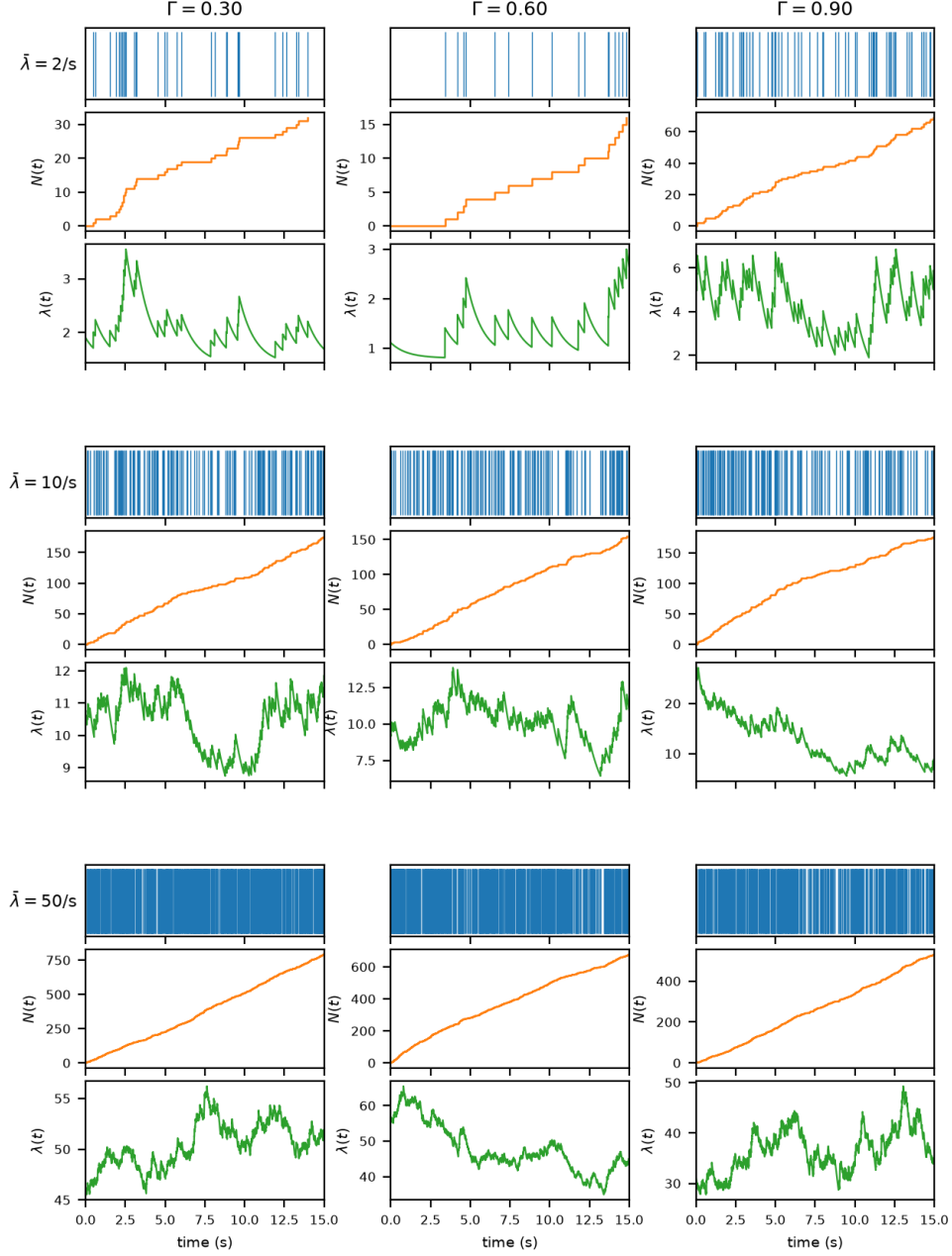


Figure 17: Simulated exponential-kernel Hawkes processes. Each cell stacks the event raster (top), the cumulative count $N(t)$ (middle) and the intensity $\lambda(t)$ (bottom). Rows: long-run rate $\bar{\lambda} \in \{2, 10, 50\}$ events per second. Columns: branching ratio $\Gamma \in \{0.30, 0.60, 0.90\}$. Fifteen seconds, decay rate $\omega = 1/s$. The time scale is chosen for legibility; the kernel fitted on CME data decays about four orders of magnitude faster. Each process is simulated by thinning after a burn-in period, so it starts in its long-run regime. Redrawn after [Maciejewski \(2026a\)](#); stylised simulation.

The Fano factor. For a window length w , cut the observation period into Υ consecutive windows and let

$$N_{w,v} = N(vw) - N((v-1)w)$$

be the count in window v . The Fano factor, also called the *index of dispersion*, is

$$D(w) = \frac{\text{Var}[N_{w,v}]}{\mathbb{E}[N_{w,v}]},$$

the variance of the window counts divided by their mean. For a Poisson process $D(w) = 1$ at every w . For a renewal process with finite-variance gaps $D(w)$ tends to a constant as w grows. For a clustered process $D(w) > 1$. Under self-excitation it grows with w over the time scales on which events trigger one another; with an exponential kernel it eventually levels off at $1/(1-\Gamma)^2$, while a slowly decaying (power-law) kernel or a slowly varying background rate keeps it growing. The measure is named after U. Fano, who introduced it for fluctuations in counts of ions. On CME NQ packet data $D(w)$ grows from about 52 at $w = 0.1$ s to about a thousand at $w = 60$ s, while the same stream with its gaps shuffled stays flat at about 17 (Maciejewski, 2026a).

The Hurst exponent, from the scaling of the Fano factor. When the Fano factor grows as a power of the window length,

$$D(w) \propto w^{2\mathcal{H}-1},$$

the exponent $\mathcal{H}$ is the *Hurst exponent*. $\mathcal{H} = 0.5$ means no long-range dependence (Poisson, and renewal processes with finite-variance gaps); $0.5 < \mathcal{H} < 1$ means long-range dependence, with clustering that looks similar across time scales; $\mathcal{H}$ near 1 means very long memory. It is estimated by a straight-line fit on a log-log plot,

$$\ln D(w) = (2\mathcal{H} - 1) \ln w + \text{constant},$$

so $\mathcal{H} = (\text{slope} + 1)/2$. On CME NQ packet data $\mathcal{H} \approx 0.74$; shuffling the gaps, which keeps their lengths and destroys their order, takes it to 0.50, so the long-range dependence lives in the order of the gaps (Maciejewski, 2026a).

7.9 The multivariate model

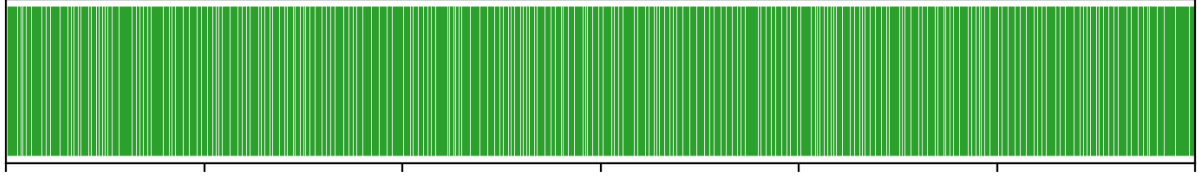
So far there has been a single stream of events. Order flow has several interacting event types—buy and sell market orders, limit orders and cancellations on each side. Let $N_i(t)$ count events of type $i \in \{1, \dots, K\}$. A linear multivariate Hawkes process (Hawkes, 1971) has conditional intensity

$$\lambda_i(t) = \mu_i + \sum_{j=1}^K \int_0^t \phi_{ij}(t-t') dN_j(t'), \quad (24)$$

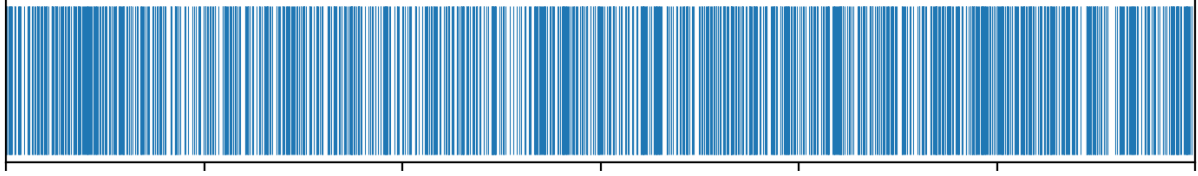
with baseline rates μ_i and non-negative kernels ϕ_{ij} ; it is the one-type model of Section 7.5 with one intensity per event type, and when every kernel is zero it reduces to independent Poisson processes with rates μ_i . Here $\lambda_i(t)$ is the *intensity*: the expected number of type- i events per second at time t . The term $dN_j(t')$ is nonzero only at the instants t' when a type- j event occurred, so the integral is simply a sum over past events, and the formula can be read as

$$\lambda_i(t) = \mu_i + \sum_j \sum_{\text{past type-}j \text{ events at } t'} \phi_{ij}(t-t').$$

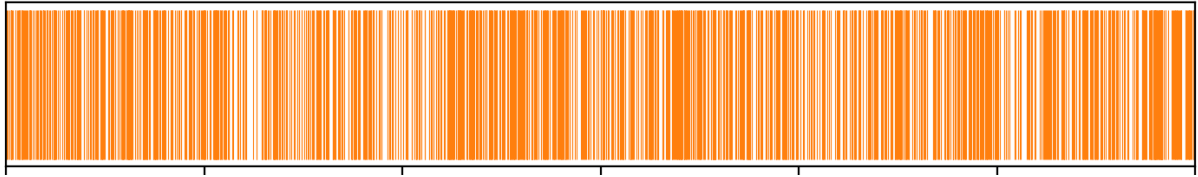
regular (renewal): $D(1\text{ s}) = 0.05$, mean count per second = 19.7



Poisson: $D(1\text{ s}) = 1.06$, mean count per second = 19.9



Hawkes, $\Gamma = 0.6$: $D(1\text{ s}) = 1.56$, mean count per second = 21.7



Hawkes, $\Gamma = 0.9$: $D(1\text{ s}) = 2.69$, mean count per second = 14.0

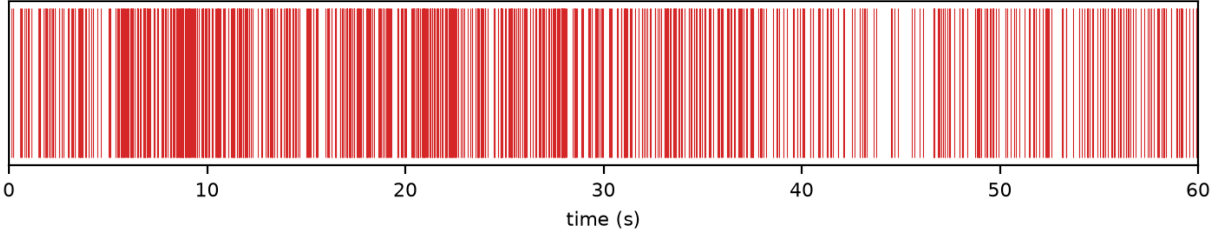


Figure 18: Four simulated processes with a mean rate of about 20 per second, over 60 seconds. Top: a renewal process with very regular gaps (gamma-distributed with shape 25), $D(1\text{ s}) \approx 0.05$: nearly equal counts per window. Second: Poisson, $D \approx 1$, the reference. Third: Hawkes with $\Gamma = 0.6$, $D(1\text{ s}) \approx 1.6$: visible clumping. Bottom: Hawkes with $\Gamma = 0.9$, $D(1\text{ s}) \approx 2.7$: pronounced bursts and long quiet stretches. Redrawn after [Maciejewski \(2026a\)](#); stylised simulation.

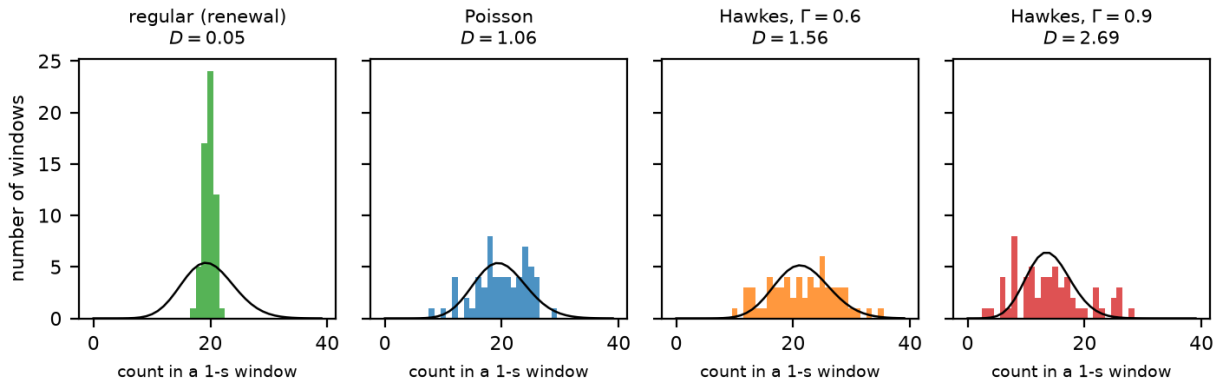


Figure 19: Distribution of arrivals per one-second window (bars) for the four processes of Figure 18, with the Poisson distribution of the same mean overlaid (curve). The regular process is far narrower than Poisson; the Poisson process matches; the Hawkes processes are wider, with extra mass in both tails. The CME NQ packet stream has $D(1\text{ s}) = 108$, far wider than any panel shown ([Maciejewski, 2026a](#)). Stylised simulation.

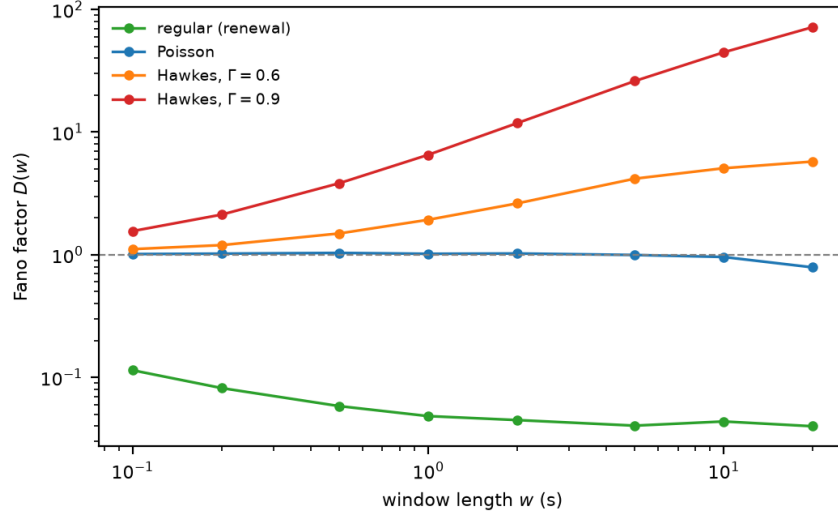


Figure 20: The Fano factor $D(w)$ against the window length w on logarithmic axes, for the four processes of Figure 18 simulated over 3,000 seconds. Regular renewal process: below one, settling near 0.04. Poisson: flat at one. Hawkes with $\Gamma = 0.6$: rising, then levelling off as the window becomes long compared with the one-second kernel. Hawkes with $\Gamma = 0.9$: a steeper rise, with a slope of about 0.7 over windows from 0.1 to 2 seconds, which the Hurst formula would read as $\mathcal{H} \approx 0.84$; with an exponential kernel this growth also levels off eventually, at $D = 1/(1 - \Gamma)^2 = 100$. Redrawn after [Maciejewski \(2026a\)](#); stylised simulation.

In words: the current rate of type- i events is a constant background rate μ_i plus a “bump” for every past event; each bump fades as that event recedes into the past, with a shape set by the kernel ϕ_{ij} . Events make further events more likely for a while, the way an earthquake raises the rate of aftershocks or a viral post triggers reposts.

The branching matrix Γ has entries

$$\Gamma_{ij} = \int_0^\infty \phi_{ij}(u) du,$$

the total area under each kernel; Γ_{ij} is the expected number of type- i events directly triggered by one type- j event. The process is stationary when the spectral radius $\rho(\Gamma) < 1$, the largest absolute eigenvalue of Γ . Then, with $\mu = (\mu_1, \dots, \mu_K)^\top$ and I the $K \times K$ identity matrix, the stationary mean intensity vector $\bar{\lambda}$, with one entry $\bar{\lambda}_i$ per event type, is

$$\bar{\lambda} = (I - \Gamma)^{-1} \mu. \quad (25)$$

In words: expand the inverse as a geometric series,

$$(I - \Gamma)^{-1} \mu = \mu + \Gamma \mu + \Gamma^2 \mu + \dots$$

The long-run event rate is the background events, plus the events they trigger, plus the events *those* trigger, and so on through every generation. The series converges—the cascade dies out—exactly when $\rho(\Gamma) < 1$. Readers who know discrete-time linear systems will recognise the condition: it is the stability condition for $\psi_{k+1} = \Gamma \psi_k + \mu$, a feedback loop whose gain must be below one.

Example 7.2 (Endogeneity in two dimensions). Take buy and sell market orders with $\mu = (1, 1)$ events per second and

$$\Gamma = \begin{pmatrix} 0.6 & 0.2 \\ 0.2 & 0.6 \end{pmatrix}.$$

The eigenvalues are 0.8 and 0.4, so $\rho(\Gamma) = 0.8 < 1$. Then

$$I - \Gamma = \begin{pmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{pmatrix}$$

with determinant 0.12, and

$$\bar{\lambda} = \frac{1}{0.12} \begin{pmatrix} 0.4 & 0.2 \\ 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

Only one event in five is exogenous. The model says a great deal about clustering and nothing, by itself, about direction: the two sides are symmetric.

7.10 Reading the branching matrix

The branching matrix is the economically interesting output of a fitted Hawkes model. Take two event types, buy and sell market orders:

$$\Gamma = \begin{pmatrix} \Gamma_{BB} & \Gamma_{BS} \\ \Gamma_{SB} & \Gamma_{SS} \end{pmatrix}.$$

Γ_{BB} is the average number of further buy market orders directly triggered by one buy market order: buys beget buys, for example because a large order is being executed in pieces. Γ_{SB} is the number of sell orders triggered by a buy: a cross effect, for example traders leaning against a move. A large diagonal relative to the off-diagonal means trends in order flow; a large off-diagonal means reversal. Asymmetry ($\Gamma_{BS} \neq \Gamma_{SB}$) means one side reacts more strongly to the other than vice versa. Linear Hawkes models allow only excitation; inhibition (an event making others *less* likely) requires non-linear extensions. The shape of each kernel ϕ_{ij} says how long an influence lasts: an exponential kernel forgets quickly, a power-law kernel remembers for a long time.

An analogy: acoustic feedback. A microphone near a loudspeaker picks up each sound and plays it again, slightly quieter. If the loop gain is below one, every echo is fainter than the last and a clap dies away; the closer the gain is to one, the longer the ringing lasts. At a gain of one or more the echoes grow and the system howls. The branching ratio is that loop gain for order flow: the average number of events that one event triggers. At 0.8, a single exogenous event produces on average $1/(1 - 0.8) = 5$ events in total, as in Example 7.2; at 0.95 it produces 20.

Example 7.3 (A chain reaction in the book). A stylised sequence at the best ask, all within a few milliseconds:

time	event
0.0 ms	a buy market order takes 10 of the 14 lots at the best ask (exogenous)
0.3 ms	two sellers at the best ask cancel, fearing a move up (triggered)
0.6 ms	a buyer joins the best bid, leaning with the flow (triggered)
0.9 ms	a second buy market order takes the remaining lots; the best ask ticks up (triggered)
1.2 ms	new sellers post at the new best ask; buyers follow at the new bid (triggered)

One outside event produced four more and a price move. In a Hawkes model each row raises the intensity of the event types in later rows—entries Γ_{ij} for cancellation after trade, trade after trade, limit order after cancellation—and the fitted kernels say how quickly that influence fades.

7.11 From events to prices

The simplest bridge from event counts to prices treats each mid move as an event. Let $N^+(t)$ and $N^-(t)$ count the upward and downward mid moves up to time t , each of size δ_m . Then

$$m_t = m_0 + \delta_m (N^+(t) - N^-(t)). \quad (26)$$

In words: the price is a running tally of up-ticks minus down-ticks; modelling the two counting processes models the price.

Write $\lambda^+(t)$ and $\lambda^-(t)$ for the intensities of N^+ and N^- : the expected numbers of up and down moves per second. Bacry et al. (2013) make the two counts *mutually* exciting: an up move raises the intensity of down moves and vice versa,

$$\lambda^+(t) = \mu_0 + \int_0^t \alpha e^{-\omega(t-t')} dN^-(t'), \quad \lambda^-(t) = \mu_0 + \int_0^t \alpha e^{-\omega(t-t')} dN^+(t'), \quad (27)$$

In words: each down move adds a bump of height α to the up-move rate, and vice versa; the bump decays at rate ω . The background rate on each side is μ_0 .

The cross-kernel has total weight $\Gamma_\times = \alpha/\omega < 1$. This one mechanism produces the familiar microstructure effects. Measured over short windows, prices look very volatile because every up-tick tends to be followed by a down-tick (bid–ask bounce); over long windows that back-and-forth cancels. Writing $\sigma^2(w)$ for the variance of price changes per unit time measured over windows of length w , in units of ticks squared,

$$\sigma^2(w) = \Lambda \left(\frac{1}{(1 + \Gamma_\times)^2} + \left(1 - \frac{1}{(1 + \Gamma_\times)^2} \right) \frac{1 - e^{-(\alpha + \omega)w}}{(\alpha + \omega)w} \right), \quad \Lambda = \frac{2\mu_0}{1 - \Gamma_\times}. \quad (28)$$

In words: Λ is the total rate of price moves. For very short windows the measured variance is Λ —every move counts. For long windows it falls to

$$\Lambda/(1 + \Gamma_\times)^2 = 2\mu_0/((1 - \Gamma_\times)(1 + \Gamma_\times)^2),$$

because moves that are immediately reversed contribute nothing to the long-run change. The curve between the two is the “signature plot” that practitioners use to choose a sampling frequency for volatility estimates.

Here cross-excitation *damps* long-run variance. Self-excitation, by contrast, inflates the variance of event *counts* relative to their mean rate (Bacry and Muzy, 2014); which effect dominates for prices depends on the structure of the kernel matrix, not on a single number. The two-dimensional version of the model also reproduces the Epps effect, the drop in measured correlation between two assets when it is computed over very short windows (Bacry et al., 2013).

Bacry and Muzy (2014) extend the construction to four kernels linking trades and price changes—trades exciting trades, prices reverting, trades moving prices, and prices triggering trades—and recover the concave, relaxing impact of large orders executed in pieces. The same machinery was used early to study book resilience (Large, 2007) and to make a zero-intelligence order-book simulator produce realistic spreads (Muni Toke, 2011).

Price moves of random size: compound Hawkes models. The tally above makes two simplifications: every mid move is exactly one tick, and its direction is decided by which of two event streams fired. A *compound* Hawkes model (Swishchuk et al., 2019; Swishchuk and Huffman, 2020) keeps a single Hawkes process $N(t)$ for the *times* of mid moves, with background rate μ and branching ratio Γ as in Sections 7.5–7.6, and lets each move carry a size and sign of its own:

$$m_t = m_0 + \sum_{k=1}^{N(t)} J_k. \quad (29)$$

In words: the price is still a running sum over mid moves, but the k -th move is J_k ticks rather than ± 1 ; the Hawkes process decides *when* the price moves, and J_k decides by *how much* and in which direction.

The moves J_k take one of a few values—for example the average down move and the average up move, or finer size classes—and they form a Markov chain: the probability that move $k + 1$ takes a given value depends on the value of move k . This lets the model say, for instance, that an up move is more likely to be followed by another up move. The chain is assumed independent of the event times.

Implications for volatility. Over long horizons the price behaves like a Brownian motion with drift. For an exponential kernel, after removing the drift—the average move $\bar{J}$ times the number of moves—the price change over a window of length w has standard deviation

$$\text{sd}\left(m_{t+w} - m_t - \bar{J}(N(t+w) - N(t))\right) \approx \sigma_J \sqrt{\bar{\lambda} w}, \quad \bar{\lambda} = \frac{\mu}{1 - \Gamma}, \quad (30)$$

In words: the spread of price changes grows with the square root of the window, as for any diffusion, and its size is the product of two numbers: $\sqrt{\bar{\lambda}}$, from how many moves there are per second, and σ_J , from how big the moves are and how they follow each other. The long-run rate $\bar{\lambda}$ is the background rate amplified by self-excitation, so the closer Γ is to one, the more volatile the price.

The constant σ_J depends on the move sizes and on the transition probabilities of the chain. In the simplest case—moves of $\pm\delta$ ticks, with the same probability p that a move repeats the direction of the previous one, whichever it was—the formula of Swishchuk and Huffman (2020) reduces to

$$\sigma_J^2 = \delta^2 \frac{p}{1 - p}. \quad (31)$$

In words: with independent directions ($p = 1/2$) each move adds δ^2 to the variance; if moves tend to repeat their direction ($p > 1/2$) they pile up and the variance is larger; if they tend to reverse ($p < 1/2$) they cancel and it is smaller.

Figure 21 checks Equation (30) by simulation for three rules for the moves, all driven by the same kind of Hawkes event stream.

Effects of order-flow structure on volatility. Set beside the model of Bacry et al. (2013) above, this gives two ways in which the structure of order flow changes volatility. There, an up move makes a down move more likely, so moves reverse and long-run variance is *damped*—the bid–ask bounce. Here, self-excitation raises the number of moves per second and so *raises* volatility, and the Markov chain of moves raises or lowers it further according to whether directions tend to repeat or reverse.

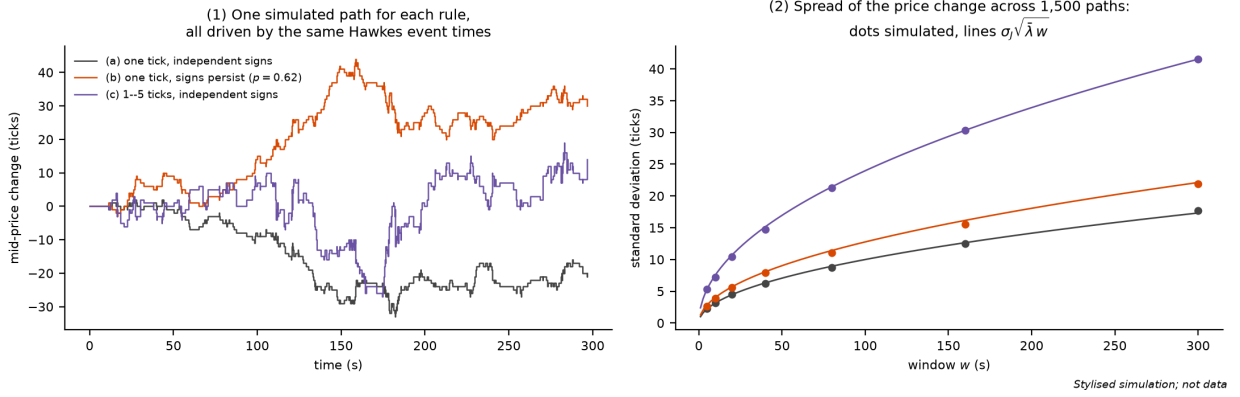


Figure 21: Compound Hawkes prices. Event times come from a Hawkes process with an exponential kernel, background rate 0.5 per second and branching ratio 0.5, so $\bar{\lambda} = 1$ move per second. (a) Moves of one tick with independent directions; (b) moves of one tick that repeat the previous direction with probability $p = 0.62$; (c) moves of 1 to 5 ticks with independent directions. (1) One path for each rule. (2) Standard deviation of the price change over a window, across 1,500 simulated paths (dots), against $\sigma_J \sqrt{\lambda w}$ (lines), with $\sigma_J^2 = 1$, $p/(1-p)$ and the mean squared move size respectively; the simulated values lie within about 7% of the lines. Stylised simulation; not data.

Fit to five Nasdaq stocks. Swishchuk and Huffman (2020) fit the model to the level-1 book of five Nasdaq stocks on one day, 21 June 2012 (Table 14), and compare the standard deviation of price changes over windows from 10 seconds to 20 minutes with Equation (30).

- **Moves of a single size work only where moves really are a single size.** With every mid move set to half a cent, the model tracked the data for MSFT and INTC, where every mid change was half a cent, but “severely” underestimated the variability for AAPL, AMZN and GOOG, where 61%, 53% and 71% of mid changes were larger.
- **Adding sizes fixes most of it.** Giving the chain two values, the average down and the average up move, improved the fit for AAPL and GOOG but not for AMZN. For AMZN a chain of 12 size classes cut the mean squared error of the fit from 0.0208 to 0.0125, and 24 classes only to 0.0123.
- **Final agreement.** Comparing the coefficient of $\sqrt{w}$ predicted by the model with the one fitted directly to the data, the error was 1.4% for AAPL, 4.6% for GOOG, 3.4% for INTC, 6.4% for MSFT and 20.8% for AMZN, whose extra volatility the authors leave unexplained.

These are fits and checks on the same single day, not out-of-sample tests. An earlier version of the model was calibrated on one stock, CISCO, over five days of November 2014 (Swishchuk et al., 2019).

On this one day, the two large-tick stocks, MSFT and INTC, had the highest branching ratios and consecutive mid moves that tended to repeat their direction, while for the other three a move was about as likely to be followed by one in either direction. Lalor and Swishchuk later drive the same kind of price—a direction set by the event type, a size drawn from the data—with a neural Hawkes process (Section 9.5).

7.12 Fitting Hawkes models in practice

One event type, exponential kernel, in detail. The input is a sorted list of arrival times $t_1 < \dots < t_{N_{\text{end}}}$ in a window $[0, t_{\text{end}}]$, where $N_{\text{end}} = N(t_{\text{end}})$ is the number of arrivals, and there are

Table 14: Compound Hawkes fits of [Swishchuk and Huffman \(2020\)](#), 21 June 2012. The branching ratio is our computation from their fitted kernel parameters ($\Gamma = \alpha/\omega$ in this survey’s notation); the repeat probabilities are their estimates of $P(\text{down} \mid \text{down})$ and $P(\text{up} \mid \text{up})$ for consecutive mid moves.

Stock	Branching ratio Γ	$P(\text{down} \mid \text{down})$	$P(\text{up} \mid \text{up})$	Mid changes larger than half a cent
AAPL	0.41	0.50	0.49	61%
AMZN	0.42	0.46	0.46	53%
GOOG	0.44	0.48	0.45	71%
MSFT	0.53	0.63	0.58	none
INTC	0.53	0.61	0.56	none

three parameters:

- μ , the **background rate**: arrivals per second with no recent history;
- α , the **jump**: the rise in intensity, in arrivals per second, at each arrival;
- ω , the **decay rate**: each jump fades as $e^{-\omega u}$, with decay time $1/\omega$.

Fitting means choosing the parameters under which the observed timestamps are most probable, that is, maximising the log-likelihood ([Ozaki, 1979](#))

$$\ln \text{Lik}(\mu, \alpha, \omega) = \sum_{n=1}^{N_{\text{end}}} \ln \lambda(t_n) - \int_0^{t_{\text{end}}} \lambda(t') dt'. \quad (32)$$

In words: the first term rewards the model for predicting a high rate at the moments arrivals actually happened; the second penalises it for predicting more arrivals in total than occurred. If α is too small the model cannot explain the bursts; if α approaches ω it predicts more clustering than there is; a wrong μ misexplains the quiet periods.

For the exponential kernel both terms can be computed in one pass over the data. With

$$\Psi_n = e^{-\omega(t_n - t_{n-1})} (1 + \Psi_{n-1}), \quad \Psi_1 = 0,$$

the intensity at each arrival is $\lambda(t_n) = \mu + \alpha\Psi_n$, and the integral is

$$\int_0^{t_{\text{end}}} \lambda(t') dt' = \mu t_{\text{end}} + \frac{\alpha}{\omega} \sum_{n=1}^{N_{\text{end}}} (1 - e^{-\omega(t_{\text{end}} - t_n)}).$$

Timestamps are shifted to the start of the window before fitting, so that the exponentials do not underflow. The log-likelihood is then maximised numerically over (μ, α, ω) , for example with the Nelder–Mead method, and the optimum checked with a second optimiser. From the fitted parameters follow the branching ratio $\Gamma = \alpha/\omega$, the kernel decay time $1/\omega$, and the long-run rate $\bar{\lambda} = \mu/(1 - \Gamma)$: the background rate amplified by $1/(1 - \Gamma)$ through self-excitation.

Maximum likelihood with several event types. The same method extends to many types. For a K -type process observed on $[0, t_{\text{end}}]$ the log-likelihood is a sum over types,

$$\ln \text{Lik} = \sum_{i=1}^K \left(\sum_{n: \text{event } n \text{ of type } i} \ln \lambda_i(t_n) - \int_0^{t_{\text{end}}} \lambda_i(t') dt' \right), \quad (33)$$

In words: for every type, reward the model for a high intensity at the moments type- i events happened, and penalise it for the total number of type- i events it expected.

Because λ_i depends only on μ_i and on the kernels $\phi_{i1}, \dots, \phi_{iK}$ in row i of the kernel matrix, the i -th term involves only those parameters. As long as no parameter is shared across rows, the fit splits into K independent problems, one per target type, each with $1 + K$ kernels' worth of parameters (Wu et al., 2019; Bacry et al., 2015). For exponential kernels each term is computed in one pass with a running sum (Section 7.12), so the cost grows only linearly with the number of events (Bacry et al., 2015).

Three practical rules follow. First, parameters have natural bounds— $\mu_i > 0$, jumps $\alpha_{ij} \geq 0$, decay $\omega > 0$ —and the optimiser should enforce them. Second, nothing in the likelihood forces the fitted Γ to satisfy $\rho(\Gamma) < 1$; it must be checked after fitting, and a fit at or above one read as a sign of non-stationarity or a wrong kernel rather than as a fact about the market (Section 7.6). Third, the likelihood is not convex in the decay rates, but if the decays are fixed in advance—for example a few exponentials with chosen time scales—it becomes convex in the background rates and jumps (Wu et al., 2019), so the remaining fit has a single optimum.

Expectation–maximisation. Direct maximisation can be unstable because the likelihood is flat in some directions. The branching structure suggests a different route (Veen and Schoenberg, 2008): if we knew which earlier event triggered each event, or that it was a background event, the parameters would be simple averages. The EM algorithm alternates between guessing those parents probabilistically and re-estimating. For one type with kernel $\alpha e^{-\omega u}$ (Lewis and Mohler, 2011):

- *E step:* the probability that event n was triggered by earlier event n' is $\alpha e^{-\omega(t_n - t_{n'})} / \lambda(t_n)$, and the probability that it was a background event is $\mu / \lambda(t_n)$;
- *M step:* μ is the expected number of background events divided by t_{end} ; Γ is the expected number of triggered events divided by the number of events; and $1/\omega$ is the probability-weighted average of the parent–child delays $t_n - t_{n'}$; then $\alpha = \Gamma\omega$.

In words: give each event a fractional family tree, then count: how many events had no parent, how many children an event has on average, and how long children take to arrive.

Moments and cumulants. A second family avoids the likelihood altogether and matches statistics of the counts. The Wiener–Hopf method estimates kernel shapes from the empirical second-order statistics of the event streams (Bacry and Muzy, 2016) (it underlies the non-parametric results of Bacry et al. (2016)); the cumulant method NPHC estimates only the integrated kernels—the branching matrix itself—from second- and third-order integrated cumulants, with no assumption on kernel shape (Achab et al., 2018); and method-of-moments calibration with closed-form moments can be rolled forward through time cheaply (Da Fonseca and Zaatour, 2014).

The branching ratio from count dispersion. The simplest moment estimator needs only counts. Split the data into windows of length w , and compute the mean and variance of the number of events per window; their ratio is the Fano factor $D(w)$ of Section 7.8. For a one-type Hawkes process whose correlations die out on time scales much shorter than w , Hardiman and Bouchaud (2014) show that

$$\Gamma \approx 1 - \sqrt{1/D(w)}. \quad (34)$$

In words: self-excitation makes counts burstier than Poisson; the more bursty they are, the closer the branching ratio is to one. A Poisson stream ($D = 1$) gives $\Gamma = 0$.

The estimate is biased low for finite windows and exact only as w grows without bound, and because the sample variance can be zero the authors report medians. Applied to the Fano factors

of CME NQ packet data reported in Section 7.8—about 52 at $w = 0.1$ s and about a thousand at $w = 60$ s—the formula gives $\Gamma \approx 0.86$ and $\Gamma \approx 0.97$, against 0.80 from the exponential-kernel likelihood fit on the same kind of data. Two cautions make this a teaching example rather than a measurement. The estimate keeps rising with w , the same symptom that a slowly varying background rate produces (Filimonov and Sornette, 2015). And the same stream with its gaps shuffled—which destroys all self-excitation but keeps the gap distribution—has $D \approx 17$, for which the formula would still report $\Gamma \approx 0.76$: dispersion alone does not prove excitation, because a renewal process with very irregular gaps is also over-dispersed.

Non-stationarity. A single fit over a long window assumes a constant background rate. Because activity follows intraday patterns and changes across regimes, such fits overstate endogeneity (Filimonov and Sornette, 2015). In practice models are fitted over short windows, with a time-varying background, or with the background modelled explicitly. The effect can be large. On EBS foreign-exchange data for 2012, a model without news gives branching ratios of 0.84 to 0.95; adding a kernel for scheduled macroeconomic announcements lowers them (Rambaldi et al., 2015). With a flexible time-varying background rate, the branching ratio estimated for Nikkei 225 mini futures is 0.41 (Omi et al., 2017). Bursts at unknown times can also be detected and separated from the background (Rambaldi et al., 2018).

Online use. With exponential kernels the intensity is updated in constant time per event, which makes Hawkes intensities usable inside a low-latency trading loop. The parameters change more slowly. One workable scheme, which we describe rather than cite, holds the decay rates fixed during a session and re-fits the background rates and jumps on a rolling window, starting each fit from the previous solution; with fixed decays each re-fit is a convex problem. Fully online estimators also exist: for example, Yang et al. (2017) update non-parametric kernels with projected gradient steps on a discretised, regularised likelihood, with regret growing only logarithmically in time. Whatever the scheme, the gradient of the log-likelihood contains the compensator term $-\int \lambda_i$ as well as the event term, so an update that looks only at the events (“the rate was high when events happened”) and ignores how many events the model expected will push the intensities up without limit.

Goodness of fit: time rescaling. A fitted point process can be checked with a classical transformation. Replace each event time t_n by the expected number of events up to it under the fitted model,

$$t_n \mapsto \int_0^{t_n} \hat{\lambda}(t') dt'.$$

If the model is right, the transformed times form a Poisson process with rate one, so the gaps between them are independent and exponential with mean one (Ogata, 1988; Brown et al., 2002). A quantile–quantile plot of those gaps against the exponential distribution, or a Kolmogorov–Smirnov test, shows where the model fails—for example, too few very short gaps means the model underestimates clustering.

Simulation. Hawkes paths are generated by *thinning* (Ogata, 1981): propose a candidate event using an upper bound on the current intensity, which is valid because exponential intensities only decay between events, and accept it with probability equal to the true intensity divided by the bound. Simulated paths are how a fitted model answers questions that have no closed form, such as the probability that a queue empties within a given latency.

Software. The open-source library tick (Bacry et al., 2018) implements maximum-likelihood fits with exponential and sum-of-exponential kernels, EM, Wiener–Hopf and cumulant estimators, and simulation.

Model card: Linear (multivariate) Hawkes

What you calibrate. Background rates μ_i ; the kernels ϕ_{ij} —for exponential kernels, the jumps α_{ij} and decay ω —and hence the branching matrix Γ .

How. Maximum likelihood, split into one problem per target type, with bounded parameters and a one-pass running sum for exponential kernels; or EM, Wiener–Hopf, cumulant and dispersion estimators. Fit on short or rolling windows, check $\rho(\Gamma) < 1$ afterwards, and test the fit by time rescaling.

What you do with it. Forecast event intensities and, from them, drift, expected OFI and depletion times; simulate order flow by thinning; monitor how much activity is endogenous; raise a cancellation alarm when the aggressive intensity against a resting order spikes.

How to picture it. Aftershocks after an earthquake, or acoustic feedback: each event raises the rate of further events, which fades; the branching ratio is the loop gain.

7.13 Empirical findings from fitted Hawkes models

Branching ratios. Estimates of the branching ratio for financial event streams have been high since the mid-2000s, but the exact value is contested. On E-mini S&P 500 futures, Filimonov and Sornette (2012) find it rising from about 0.3 around 1998–2000 to 0.7–0.8 after 2007, with spikes to 0.9. Hardiman et al. (2013), using power-law kernels, find the kernel integrating to approximately one—a market at criticality. Filimonov and Sornette (2015) argue that estimates near one are biased upward by outliers, kernel regularisation, edge effects and regime shifts—fitting a constant background rate to a period in which the true background varies; their re-estimation gives values that later fluctuate between about 0.8 and 1.1. On CME NQ market data, an exponential-kernel fit to packet arrivals gives about 0.80 (Section 7.6). The defensible summary is that a large majority of events are endogenous, and that the precise number depends on the kernel family and on how non-stationarity is handled.

Kernel shape and long memory. Non-parametric estimation (Bacry et al., 2016) finds kernels that decay roughly as power laws with exponent slightly above one (in simulation the method recovers power laws over six decades of time). In an eight-dimensional model of DAX and Bund futures (price moves, trades, limit orders and cancellations, each split by side), trades, limit orders and cancellations mainly excite events of the same type and side, while price moves on one side mainly excite price moves on the other—mean reversion. The persistence of order flow is a long-documented fact: the signs of market orders form a long-memory process with a Hurst exponent of about 0.7 (Lillo and Farmer, 2004), explained mainly by the splitting of large orders into many small ones (Lillo et al., 2005; Bouchaud et al., 2009). Power-law Hawkes kernels are the point-process expression of the same fact.

7.14 Near-criticality

Behaviour of a Hawkes process as the branching ratio approaches one. A branching ratio of 0.99 is not “a bit more than 0.9”. Three things change sharply as Γ approaches one, and together they are what *near-critical* means (Figure 22).

- **Shocks are amplified enormously.** One outside event produces on average $1/(1 - \Gamma)$ events in total: 10 at $\Gamma = 0.9$, 100 at 0.99. Most activity is then reaction rather than news.

- **The market takes much longer to calm down.** With an exponential kernel of decay rate ω , the expected extra activity after a shock fades at rate $\omega(1 - \Gamma)$, not ω . The reason fits in two lines: between events the extra intensity decays at rate ω , but while it is above zero it keeps producing events, each of which adds α back; on average these two effects combine to

$$\frac{d}{dt} \mathbb{E}[\lambda(t) - \mu] = -\omega \mathbb{E}[\lambda(t) - \mu] + \alpha \mathbb{E}[\lambda(t)], \quad (35)$$

and after a single outside event the extra activity therefore decays as $e^{-(\omega-\alpha)t} = e^{-\omega(1-\Gamma)t}$.

In words: the rate falls because old excitation wears off, and rises because the events it produces add new excitation; the net speed of calming down is the difference, which shrinks to nothing as $\Gamma \rightarrow 1$.

With ω equal to one per second, the market calms down in about 2 seconds at $\Gamma = 0.5$, 10 seconds at 0.9 and 100 seconds at 0.99. Physicists call this *critical slowing down*.

- **Bursts become wildly uneven.** The number of events in the family started by one outside event is random. Because each event's number of children is Poisson with mean Γ , the family size follows a known distribution (the Borel distribution), shown in Figure 22a. At $\Gamma = 0.99$ the average family has 100 events, yet more than a third of families (37%) consist of the outside event alone and half have at most two; the average is carried by rare enormous ones—1.7% of families exceed 1,000 events. Near one, the typical burst is small and the occasional burst is huge.

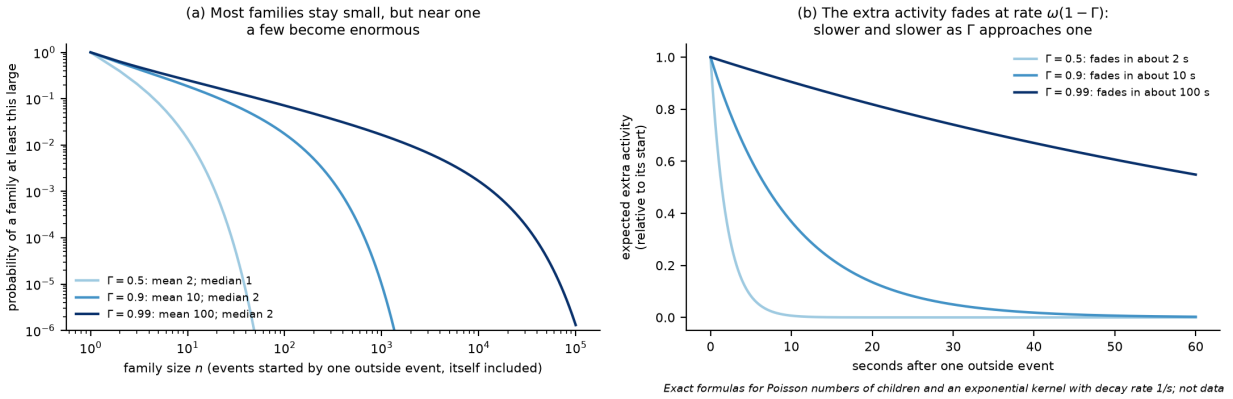


Figure 22: Near-criticality. (a) Probability that the family of events started by one outside event has at least n events, for branching ratios 0.5, 0.9 and 0.99 (log scales). The mean family size is $1/(1 - \Gamma)$, but the median stays at one or two; near one the mean is carried by rare, very large families. (b) Expected extra activity after one outside event, for an exponential kernel with decay rate one per second: it fades at rate $\omega(1 - \Gamma)$, so the time to calm down is $1/(\omega(1 - \Gamma))$. Exact formulas, not data.

Consequences of near-criticality for markets. These three effects describe a market that is very sensitive: small outside news can set off long, large bursts of activity, most activity is self-generated, and the size of the next burst is hard to predict. That is why whether markets sit close to one is debated (Section 7.13), and why a changing background rate, which can make a market look closer to one than it is (Section 7.20.1), matters. It also explains the mathematical results below: long, slowly fading bursts of order flow, added up over time, become a volatility that is itself random and persistent.

The long-run limit. When the branching ratio approaches one, appropriately rescaled Hawkes processes converge to continuous limits. With light-tailed kernels the limit is an integrated Cox–Ingersoll–Ross process, and the price model of Bacry et al. (2013) converges to a Heston stochastic-volatility model (Jaisson and Rosenbaum, 2015). With heavy-tailed kernels—power laws with tail $\propto u^{-(1+\zeta)}$ with $\zeta \in (1/2, 1)$ —the limit is a *rough* fractional process with Hurst parameter $\zeta - 1/2$ (Jaisson and Rosenbaum, 2016), which the authors offer as a microstructural foundation for rough volatility. Empirically, log-volatility does behave like a fractional process with a Hurst exponent of order 0.1—for example 0.125 for DAX and 0.082 for Bund futures (Gatheral et al., 2018). The same limits can also produce the *leverage effect*—volatility rising when prices fall: if liquidity is asymmetric between bid and ask, so that selling pressure is more self-exciting than buying pressure, the long-run limit is a Heston model with leverage for light-tailed kernels and a rough Heston model for heavy-tailed ones (El Euch et al., 2018, 2019). Rough models are hard to compute with because they are not Markovian; Abi Jaber and El Euch (2019) approximate them by multifactor stochastic-volatility models that are Markovian, and therefore tractable. The lesson for LOB modelling is qualitative: strong endogenous excitation, together with long memory, generates volatility that is itself random and persistent. It is not that markets literally sit at a singularity, nor that a single scalar multiplies volatility.

7.15 Marks: events with sizes and labels

Events of different size and type. So far every event of a given type has counted the same: a 1-lot trade and a 100-lot trade each leave the same bump in the intensity. It seems obvious that a large trade should excite more. Whatever extra information an event carries—its size, side, price level, how aggressive it was, how far it moved the price—is called its *mark*, and a Hawkes process in which marks affect the intensity is a *marked* Hawkes process. This subsection explains the three ways marks can enter, and what the evidence on order-book data shows.

The formal set-up. When each event n carries a mark, the process is a marked point process. Its intensity can be split into two parts: the rate at which events of any kind arrive, called the *ground intensity*, and the probability distribution of the mark given that an event occurs (Rasmussen, 2018). With that split, the log-likelihood is a sum of a term for the event times and a term for the marks, so the two can be fitted separately when they share no parameters.

Three ways a mark can enter.

1. **As an event type.** Cut the mark into a few classes—small, medium and large trades; aggressive or not; at the best price or one tick away—and treat each class as a separate type of event in a multivariate Hawkes model (Section 7.9). This is the most common route by far, and it is not an approximation in disguise: a mark cut into M classes is formally the same as an unmarked Hawkes process with M times as many event types (Bacry and Muzy, 2016). Its cost is the number of kernels, which grows with the square of the number of types.
2. **As a multiplier.** Keep one kernel and multiply it by a function of the mark, so that event n adds $\phi(t - t_n)g(\text{mark}_n)$ to the intensity—the “typical choice” in the words of Bacry et al. (2015). A larger trade then excites more, but with the same time profile (Figure 23a). The function g is often a power of the size or a linear function of it.
3. **As part of the kernel.** Let the whole bump depend on the mark, $\phi(t - t_n, \text{mark}_n)$, so that a larger trade can excite for longer as well as more (Figure 23b). This is the most flexible

and the hardest to estimate; it is usually done by cutting the mark into classes (route 1) or non-parametrically.

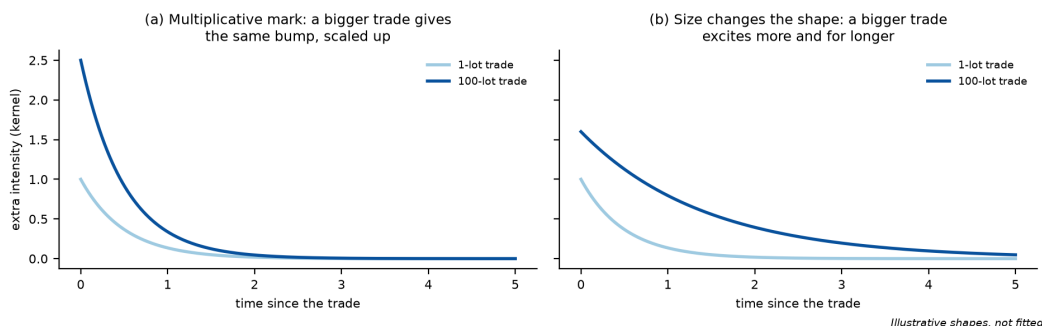


Figure 23: Two ways a trade’s size can change its effect on later events. (a) Multiplicative: the bump keeps its shape and is scaled up. (b) Non-separable: a larger trade’s bump is higher and decays more slowly. On Bund and DAX futures the data favour (b) (Rambaldi et al., 2017). Illustrative shapes, not fitted.

Marks as event types versus marks that change excitation. The routes above differ in one essential way: whether the mark decides *which kind* of event it is, or *how big an effect* it has.

- **The mark as a type (route 1).** The size—or side, or price level—is used only to sort events into a few classes. Inside a class every event counts exactly the same. Lu and Abergel (2018), for example, model the 30 DAX stocks with twelve types: limit orders, cancellations and market orders, on each side, each split by whether it moves the mid-price. A market buy is classed by comparing its size with the queue at the best ask: smaller than the queue, it leaves the price unchanged and is “non-aggressive”; at least as large, it empties the level and is “aggressive”. After that, the model never sees the size again.
- **The mark in the excitation (routes 2 and 3).** The actual value of the size goes into the formula for the bump, so that a bigger trade leaves a bigger, or longer, bump—continuously, not in steps.

Table 15 shows the difference on three market buys hitting a best ask of 100 lots.

Table 15: Three market buys against a best-ask queue of 100 lots, under the two uses of size. The square-root multiplier is an illustrative choice.

Trade	Size used as a type	Size used in the excitation (bump $\propto \sqrt{\text{size}}$)
5 lots	non-aggressive: bump B_0	bump ≈ 2.2 units
90 lots	non-aggressive: the <i>same</i> bump B_0	bump ≈ 9.5 units
500 lots	aggressive: bump B_1	bump ≈ 22.4 units

With size as a type, size matters only at the cut-off, as a step; used in the excitation, it matters everywhere, as a smooth curve.

Frequency of each use in order-book modelling. The first is standard: almost every Hawkes model of the book splits events by side, kind of order, aggressiveness or price level, because that needs nothing new—it is a multivariate Hawkes model with more event types, fitted in the usual way. The second is rare. A 2025 study states that the unmarked Hawkes process “remains the most

widely used in financial market research” and lists seven studies that include marks (Richards et al., 2025). Two influential studies leave sizes out on purpose: Bacry and Muzy (2014) because marks would make estimation much more difficult, and Lu and Abergel (2018) because the average price jump in their data was 1.08 ticks, close enough to one.

Published evidence on excitation marks. Table 16 collects the verified studies in which a size or similar mark changes the excitation.

Table 16: Marked Hawkes models fitted to high-frequency financial data. “Simpler form tested”: whether the study checks its marked model against a simpler one.

Study	Mark	Data	Finding	Simpler form tested
Rambaldi et al. (2017)	trade volume, 6 classes	Bund and DAX futures	large trades excite more and for longer; the multiplicative form is rejected	the multiplicative form, rejected
Fabre and Muni Toke (2025)	trade volume, non-parametric	Coinbase BTC-USD, Dec. 2023	excitation rises with volume, more slowly for larger trades, and levels off above about 50,000 USD	no
Richards et al. (2025)	21 candidate marks	silver futures, July 2015	the best marks—all volume-based, such as order imbalance—are significant in about half of 1,000-event segments	no mark, by a score test
Lee and Seo (2017)	size of the price jump	NYSE stocks, 2008–2011	linear effect, slope about 0.2	no
Deschatre and Gruet (2022)	size of the price jump	German intraday power, 2017	excitation linear in jump size	no
Joseph and Jain (2024)	trade volume, neural kernel	Binance BTC and ETH, one half-hour	a volume effect in two of four event streams	informally, by Q–Q plots
Chevalier et al. (2023)	volume, power law	BNP Paribas trades-through, 2022	average branching ratio 0.83; exponent shown only as a box plot	no
Jain et al. (2024a)	order size	Nasdaq	weak evidence that past sizes affect arrival rates	—

Three conclusions follow. Size does matter for at least some kinds of event: larger trades excite more, on several markets. The simple multiplicative form is not enough where it has been tested: on Bund and DAX futures the shape of the response changes with size, not only its height. And the evidence is thin: few studies, short samples, and no study we found that compares a marked with an unmarked model out of sample. Fauth and Tudor (2012) specify an excitation proportional to a power of the order size but report no estimate of the power; we found no study that estimates such a power and relates it to the square-root law of market impact.

Conditions for using marks. For a first model, split events into a few types by the marks that obviously matter—side, kind of order, whether the event moves the price—which is route 1 and standard practice. Add a size mark when large orders are a large share of what matters for the decision at hand, and test it: the score test of Richards et al. (2025) checks whether a mark matters using only the unmarked fit, before a marked model is built.

7.16 State-dependent Hawkes models

Coupling event rates and book state. The Hawkes models so far let the rate of events depend on past events but not on the book: a cancellation at the best ask raises the rate of further

cancellations by the same amount whether the spread is one tick or five. The queue models of Section 6 do the opposite: in the queue-reactive model the rates depend on the state of the book, but an event has no effect on later rates except through the state it leaves behind. [Morariu-Patrichi and Pakkanen \(2022\)](#) point out that each approach has what the other lacks, and join them in a *state-dependent Hawkes process*.

The model. The book is summarised by a state $X(t)$ that takes one of a few values—for example “spread of one tick” and “spread of two ticks or more”. The model has two parts. First, the intensities are those of a multivariate Hawkes process (Section 7.9), except that the bump left by each past event depends on the state of the book *at the time that event happened*:

$$\lambda_i(t) = \mu_i + \sum_{j=1}^K \sum_{\text{past type-}j \text{ events at } t'} \phi_{ij}(t - t', X(t')). \quad (36)$$

In words: as before, each past event adds a decaying bump to the rate of type- i events; but now the height and duration of the bump also depend on what the book looked like when the event occurred. A trade when the spread is wide can trigger a different reaction from a trade when it is tight.

Second, the state changes only at events, and how it changes depends on the type of the event:

$$\mathbb{P}(\text{state after the event} = x' \mid \text{event of type } j, \text{ state before} = x) = \pi_j(x' \mid x). \quad (37)$$

In words: after each event, the state of the book moves to a new value at random, with probabilities that depend on what kind of event it was: a sell order is likely to lower the queue imbalance, a buy order to raise it.

The two parts make a feedback loop (Figure 24): events change the state, and the state changes how strongly events excite further events. With a single state the model is an ordinary Hawkes process.

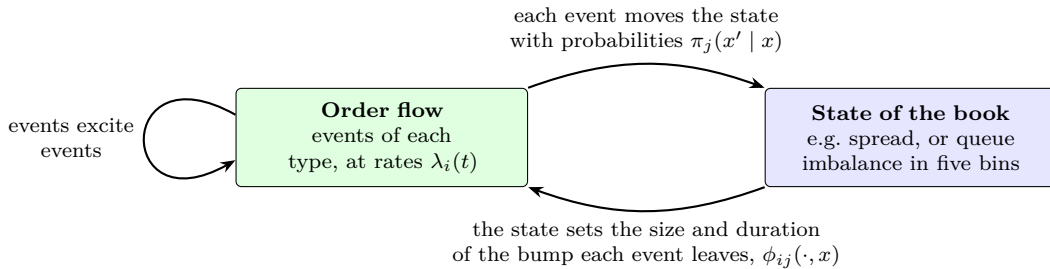


Figure 24: The feedback loop of a state-dependent Hawkes process ([Morariu-Patrichi and Pakkanen, 2022](#)). An ordinary Hawkes process has only the loop on the left; a queue-reactive model has a state, but no excitation between events. Schematic.

Estimation of the state-dependent model. One might expect the coupling to make the model hard to estimate. It does not: the likelihood splits into a part for the transition probabilities and a part for the intensities, so the probabilities $\pi_j(x' \mid x)$ can be estimated on their own—by counting, after each type of event, how often each state was followed by each other state—and the intensities are then fitted by maximum likelihood exactly as for an ordinary multivariate Hawkes process. Simulation is the thinning algorithm of Section 7.12 with one extra step: draw the time of the next event, draw its type in proportion to the intensities, then draw the new state from π_j . The authors provide a Python library, `mpoints`, for both.

Results on a year of Intel level-1 data. Morariu-Patrichi and Pakkanen (2022) fit two versions to a year of Nasdaq level-1 data for Intel (INTC, June 2017 to May 2018, 250 trading days, 12:00 to 14:30 each day, where activity is roughly constant). Events are of two types: *bid events*—buy market orders, buy limit orders at the best bid and cancellations of sell orders at the best ask, all of which push the price up—and *ask events*, the mirror image. The difference between the two counts is a proxy for order-flow imbalance (Section 4). The two versions differ in the state (Table 17).

Table 17: The two state-dependent Hawkes models fitted by Morariu-Patrichi and Pakkanen (2022) to Intel. Kernels are exponential; parameters are estimated separately for each day.

	Spread model	Queue-imbalance model
Event types	bid events, ask events	bid events, ask events
State	spread of one tick, or of two ticks or more	queue imbalance in five equal bins: below -0.6 , -0.6 to -0.2 , -0.2 to 0.2 , 0.2 to 0.6 , above 0.6
Parameters	26	92

- **Excitation is fast and mostly self-excitation.** In every state, bid events excite bid events and ask events excite ask events more than either excites the other, on time scales from 0.1 to 100 milliseconds.
- **The state changes the excitation.** When the spread widens to two ticks or more, self-excitation roughly doubles in size, and cross-excitation lasts much longer. When the queue imbalance shows heavy selling pressure (below -0.6), the self-excitation of ask events rises; under mild buying pressure it nearly halves.
- **The state pulls back towards normal.** The spread stays where it is after 98% of events when it is one tick and after 93% when it is wider; it is six times as likely to narrow from wide to one tick (6%) as to widen from one tick (1%). Transitions of the queue imbalance are also more likely towards the balanced state than away from it, and those after bid events mirror those after ask events.
- **How self-exciting the market is depends on the state.** The spectral radius of the branching matrix (Section 7.9), computed state by state, is higher in the unbalanced states than in the balanced ones. With a wide spread it is systematically above one; with a strongly unbalanced queue it is above 0.9 on half of the days and occasionally above one. The authors conclude that the near-critical behaviour discussed in Section 7.14 is largely a feature of particular states of the book, not of the market at all times.
- **The excitation shifted in February 2018.** In the queue-imbalance model the size of excitation changed around 5 February 2018, the day the VIX volatility index more than doubled.

Testing whether the state dependence improves the fit. Because the state-dependent model contains the ordinary one, it must fit better on the data used to fit it. The authors therefore also check it on the half-hour after the fitting window (14:30 to 15:00) each day: it still fits better than an ordinary Hawkes process, but the improvement is smaller, and small overall, because the two models behave alike in the common states and differ mainly in the rare ones—a wide spread, a strongly unbalanced queue. They also compare it with a plain Hawkes process that simply treats each (event, state) pair as its own event type, which ignores the rule that the next state follows from the current state and the event; with 210 parameters against 92 it fitted worse. The paper

reports results for Intel only; the authors say four other stocks (AMD, MU, SNAP, TWTR) gave largely similar results, which are not shown.

Extensions. The authors list three: power-law or sum-of-exponential kernels in place of exponential ones; background rates μ_i that also depend on the state, which would make both the ordinary Hawkes process and the queue-reactive model special cases; and kernels that can be negative, for inhibition, with a non-linear link to keep intensities positive (Section 7.18). The neural model of Shi and Cartlidge (2022) reviewed in Section 9.4 keeps the state-transition part and replaces the kernels by a network.

Model card: State-dependent Hawkes

What you calibrate. The transition probabilities $\pi_j(x' | x)$ of the state after each event type; the background rates; and, for each pair of event types and each state, the height and decay of the kernel.

How. Count state transitions to estimate π_j ; maximise the Hawkes likelihood for the rest, as for an ordinary multivariate model. Check fit on data outside the fitting window, since the model always fits better in sample than the ordinary Hawkes process it contains.

What you do with it. Intensity forecasts and simulations in which the reaction of order flow depends on the spread or the imbalance; measuring how self-exciting the market is in each state of the book.

How to picture it. The aftershock model with a dial set by the state of the book—and every aftershock can turn the dial.

7.17 Regimes: when the parameters change over time

Parameters that vary during the day. A Hawkes model fitted to a day of data assumes that one background rate and one set of kernels hold all day. Markets do not work like that: activity follows the clock—busy at the open and close, quiet at lunch—jumps around news, and has calm and agitated periods. There are three ways to let the parameters change, from the simplest to the most ambitious (Figure 25):

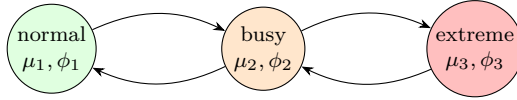
1. **A background rate that varies with time.** Keep the kernels fixed, but let the background rate follow the time of day, or add a term for scheduled news. This is the most common fix.
2. **Kernels that vary slowly.** Let the strength of excitation itself change through the day: *locally stationary* Hawkes processes.
3. **Hidden regimes.** Suppose the market switches at random between a few regimes—say normal, busy and extreme—each with its own background rate and kernels, and that the regime is not observed but inferred from the events: a *Markov-modulated* Hawkes process, fitted by the expectation–maximisation algorithm. It was developed for earthquake sequences, with regimes for main shocks, aftershocks and quiet periods (Wang et al., 2012).

These differ from the state-dependent models of Section 7.16, where the state is observed in the book, such as the spread; here the change is driven by the clock, by news or by a hidden regime.

Effect of regimes on the estimated branching ratio. The main reason to care is the trap of Section 7.20.1: if the background rate changes and the model assumes it constant, the model mistakes busy periods for excitation and overstates the branching ratio. The verified evidence:

- On Nikkei 225 mini futures, a background rate allowed to vary smoothly through the day gives a branching ratio of 0.41, and the authors report that a constant background clearly overestimates it (Omi et al., 2017).

Hidden regimes



a hidden regime switches at random;
in each regime events follow a Hawkes
process with its own background rate and kernels

Figure 25: A Markov-modulated Hawkes process with three regimes. The regime is not observed; it is inferred from the pattern of events. Schematic.

- On foreign-exchange quotes, adding scheduled news lowers the estimated branching ratio from 0.84–0.95, and the authors warn that a model ignoring news “could overestimate” it (Rambaldi et al., 2015).
- On EUR/USD and E-mini data, a constant background biases the branching ratio upward by as much as 0.3 for estimation windows spanning days, and only slightly for windows up to about two hours; with a flexible background the authors reject criticality (abstract only) (Wehrli et al., 2021; Wheatley et al., 2019).
- On BNP Paribas trades-through, letting the background rate vary piecewise linearly through the day raised the share of days on which the model passed all four goodness-of-fit tests from 71–76% to 86–87% (Muni Toke and Pomponio, 2012).

Regime-dependent kernels. Letting only the background vary is not always enough. On Apple trades in 2015, a model with all three parameters of an exponential Hawkes process varying through the day finds estimates that move far more than any fixed daily pattern would explain (Clinet and Potiron, 2018). On Paris stocks, the shape of the spectrum of trade arrivals changes along the day (Roueff and von Sachs, 2019). On the German intraday power market, a test rejects a constant strength of excitation in 20 of 21 sessions, with endogeneity peaking a few hours before delivery (Deschatre et al., 2025). On Australian stock trades, a Hawkes model with a smooth background rate fits better than a standard duration model and gives branching ratios from 0.10 to 0.66 across days (Chen and Hall, 2013).

Hidden regimes on market data. Hidden-regime Hawkes models are hard to estimate, and their authors note that they have “received much less attention” for that reason (Fabre and Muni Toke, 2026). The one verified application to high-frequency market data is theirs: trades in a cryptocurrency pair on Coinbase (SEI-USD, December 2023 to March 2024), with up to four hidden regimes in which both the background rate and the kernel switch. It beats a hidden-regime model without excitation (a Markov-modulated Poisson process) on a model-selection criterion and on out-of-sample goodness-of-fit checks. With three regimes, the market spent 90.3% of the time in a normal regime, 9.6% in a busy one and 0.14% in an extreme-burst regime, which nevertheless carried about a fifth to a quarter of the volume. Fitting one day took from about fifteen minutes to about ten hours, depending on the resolution of the kernel. Swishchuk et al. (2019) give limit theorems for a regime-switching compound Hawkes price, with a two-state illustration on one stock; Vinkovskaya (2014) lets order-flow regimes follow the observed spread on US stocks (abstract only).

Gaps in the literature. We found no study that compares a hidden-regime Hawkes model with a single-regime one out of sample on order-book data. In practice, the verified lesson is the simpler one: let the background rate follow the time of day and the news before reading anything into the branching ratio.

7.18 Variants of the Hawkes model

The linear model with exponential kernels is a starting point. Each variant below relaxes one of its assumptions; Table 18 gives the model card for each.

Sums of exponentials. Power-law kernels capture long memory but lose the constant-time update. A power law can be approximated over many decades by a small number of exponentials with geometrically spaced decay rates—for an exponent of two, about 1.7 exponentials per decade (Bochud and Challet, 2007)—and each exponential keeps its own running sum, so the intensity is still updated in constant time per event. With the decay rates fixed, the likelihood is convex in the remaining parameters (Wu et al., 2019).

Nonlinear and inhibitory kernels. Linear Hawkes kernels can only excite. A nonlinear Hawkes process passes the sum through a function that keeps the intensity non-negative,

$$\lambda_i(t) = h\left(\mu_i + \sum_j \int \phi_{ij}(t - t') dN_j(t')\right),$$

which allows negative kernels, that is, events that make others *less* likely; general conditions for a stationary version were given by Brémaud and Massoulié (1996). In limit-order-book data, Lu and Abergel (2018) observe inhibition effects that lead them to nonlinear Hawkes models, and Rambaldi et al. (2017) find that limit orders (and cancellations) on one side inhibit the same type on the other side, which they read as a fair-price effect. The link functions used in finance so far are simple: a floor at zero, which lets the sum be negative but cuts the intensity off there (Jain et al., 2024a), and an exponential function of the order-book state that multiplies an otherwise linear Hawkes intensity (Sfendourakis and Muni Toke, 2020; Mucciante and Sancetta, 2024), as in the state-dependent form of Section 7. We found no finance application of the saturating (sigmoid) links used elsewhere with reported results.

Marks and regimes. Two variants have their own subsections: marks, the size or other attributes of each event (Section 7.15), and regimes, parameters that change over time (Section 7.17).

Quadratic Hawkes and the Zumbach effect. Volatility has a time asymmetry: past volatility measured over long horizons predicts future short-horizon volatility better than the reverse—the *Zumbach effect* (Zumbach, 2009). A Hawkes intensity that depends only on past event counts cannot produce it. The quadratic Hawkes model of Blanc et al. (2017) adds a term in the *square* of a moving average of past signed price changes, so that a sequence of moves in the same direction raises activity more than the same number of moves that cancel out. Calibrated on five-minute bins of 133 NYSE stocks over 2000–2009, it reproduces the time asymmetry, and it generates long memory in volatility without requiring the process to be critical. The model also allows a term *linear* in past price changes, which would capture the leverage effect; intraday the authors find its kernel “hardly distinct from zero”, unlike in daily models where it is significantly negative, and set it to zero.

Queue-reactive Hawkes. Wu et al. (2019) combine the two ideas of this part: the background rates depend on the current queue size, as in Section 6.3, and events excite each other through sum-of-exponential kernels. On Bund and DAX futures the Hawkes term greatly improves the pure queue-reactive model, both in the distribution of times between events and in the distribution of queue sizes; market-order and price-change rates are mainly functions of the volume imbalance, while limit and cancellation rates are not.

Cross-asset Hawkes. Events in one instrument can excite events in another; this is the point-process form of lead–lag, discussed in Section 4.6. The two-asset model of Bacry et al. (2013) reproduces the Epps effect on Euro-Bund and Euro-Bobl futures, and lead–lag can be read off its second-order properties; Da Fonseca and Zaatour (2017) fit a multi-asset Hawkes model whose diffusive limit links high- and low-frequency correlation and which captures lead–lag on Eurex assets. For interest rates, the verified Hawkes studies use European government-bond futures—Bund and Bobl (Bacry et al., 2013, 2016; Rambaldi et al., 2017). We found no Hawkes study of US Treasury futures, of the BrokerTec cash market, or of lead–lag between Treasury cash and futures; the main microstructure study of BrokerTec we found uses a vector autoregression (Fleming et al., 2018). At a much lower frequency, Hainaut (2016) drive a short-rate model with a bivariate mutually exciting process standing for supply and demand in fixed-income markets, fitted to one-year swap rates from 2004 to 2014.

Table 18: Model cards for the Hawkes variants.

Variant	What you calibrate	How	What you do with it	How to picture it
Sum of exponentials	jumps per (pair of types, decay); decays fixed on a geometric grid	convex likelihood once decays are fixed	long memory with constant-time updates, in software or hardware	a power law drawn with a few rulers of different lengths
Nonlinear / inhibitory	signed kernels; the link function	maximum likelihood (poorly convex)	represent events that discourage others	a thermostat as well as an amplifier
Marked (volume classes)	one set of kernels per size class	as for a multivariate model, one type per class	let large trades matter more, with their own time profile	a big stone makes bigger, longer ripples
Markov-modulated	per-regime background and decay; switching rates	EM over hidden regimes	separate quiet and busy regimes	weather that switches between drizzle and storm
Quadratic (Zumbach)	self-excitation kernel; return-feedback kernel	on binned returns (GMM, then likelihood)	volatility forecasts with the right time asymmetry	a trend, not just activity, wakes the market up
Queue-reactive Hawkes	size-dependent background; excitation kernels	maximum likelihood with fixed decays	simulate queues with realistic clustering	the toll booth of Section 6.3, with rush hours

7.19 Neural point processes

The common structure of the models so far. Everything so far in this section has the same structure: the rate of type- i events is a background rate plus a sum of bumps, one per past event, each with a fixed shape ϕ_{ij} . The variants of Section 7.18 change one ingredient at a time—the shape of the bump, its sign, its dependence on size or on the queue. A neural point process changes all of them at once: it keeps the idea of an intensity, but lets a neural network compute it from the history. This subsection explains what that means, step by step.

The part of the Hawkes model replaced by the network. A Hawkes model can be taken apart into three pieces (Figure 26, top):

1. a **background rate** μ_i for each event type;
2. a **memory** of the past: for each pair of event types, a running total of decaying bumps—every past type- j event adds $\phi_{ij}(t - t_{n'})$ to the memory for type i , and the bump shrinks at a fixed speed as time passes;

3. a **readout** that turns the memory into a rate: add the bumps to the background rate.

A neural point process keeps the outside of this machine—a rate for each event type, which can be turned into a likelihood, a forecast of the next event and a simulator—and replaces the inside: pieces 2 and 3. The memory becomes a vector of numbers updated by a network as each event arrives, and the readout becomes a learned weighted score of that vector, made positive (Figure 26, bottom). Table 19 lists the correspondence part by part.

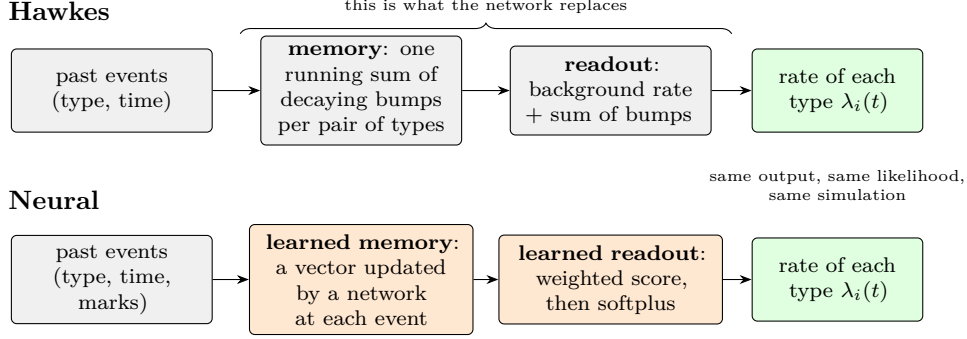


Figure 26: What a neural point process replaces. Grey: kept from the Hawkes model. Orange: replaced by learned parts. Green: the output—a rate for each event type—is the same kind of object in both, so both are fitted by the same likelihood and simulated in the same way. Schematic.

Table 19: The parts of a Hawkes model and their neural counterparts.

Part	Hawkes	Neural point process
What an event contributes	a bump of fixed height $\phi_{ij}(0)$	a change to the memory vector, computed from the event <i>and</i> the current memory
How the contribution fades	at a fixed speed, the same after every event	at speeds the network sets after each event; can fade, grow or reverse
How contributions combine	by adding	any way the network learns: a pattern can count for more than its parts
Turning memory into a rate	add to the background rate	weighted score, passed through softplus so the rate is positive
Kept unchanged	a rate for each event type; the likelihood of Section 7.12; simulation by thinning	

The Hawkes memory as leaky buckets. Picture the Hawkes memory as a row of leaky buckets, one per event type. Every event pours a fixed amount of water into some of the buckets; every bucket leaks at its own fixed speed; the rate of type- i events is the background rate plus the water in bucket i . Water can only be added, and it can only leak away. In a neural point process a controller watches the events as they arrive and decides, each time, how much to pour into which bucket, how fast each one should now leak, and even whether to drain a bucket or let it *fill* slowly over the next moments. The buckets are the network’s memory; the controller is what is learned.

One update step: classical Hawkes and continuous-time LSTM. The clearest comparison is between the classical Hawkes model with exponential kernels and the neural Hawkes process of Mei and Eisner (2017), because both keep a memory that jumps at each event and then relaxes exponentially (Figure 27). In the classical model the memory for type i is the excess of its rate over the background rate. At a type- j event it jumps by a fixed amount, the kernel height for that pair of types; between events it decays towards zero at a fixed speed ω ; the rate is the background rate

plus the memory. In the neural Hawkes process, at each event a network reads the event and the current memory and sets three things for every memory cell: where it starts, the *target* it relaxes towards, and how fast—Equation (39). The rate is a learned score of the memory, passed through softplus. Read this way, the classical update has the same shape as the neural one with the jump fixed in advance, the target fixed at zero and the speed fixed at ω ; the neural model learns all three, and lets them depend on everything that has happened so far.

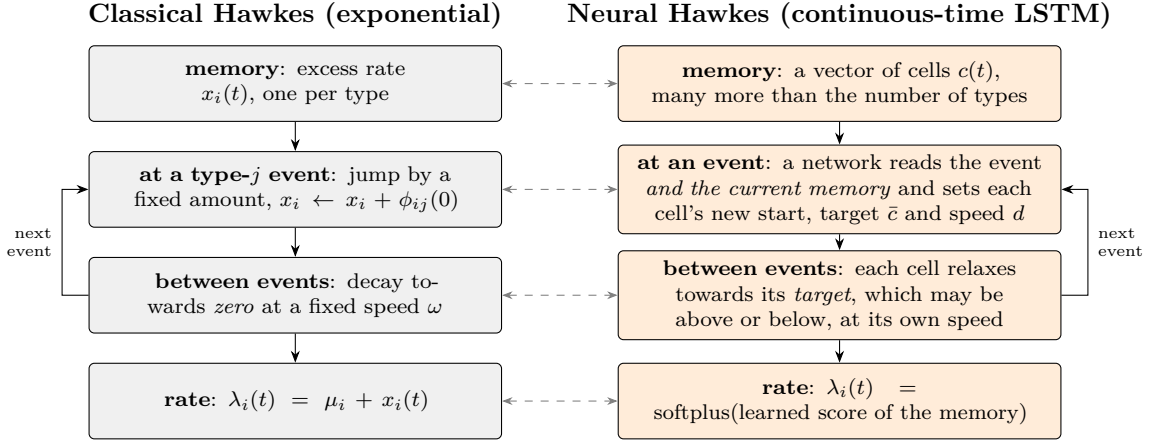


Figure 27: One step of the classical exponential Hawkes model (left) and of the neural Hawkes process (Mei and Eisner, 2017) (right). Dashed arrows join corresponding parts. The classical model has the same shape as the neural one with the jump fixed by the kernel, the target fixed at zero and the speed fixed at ω ; the neural model computes all three from the history at every event. Schematic.

Three limitations of the linear Hawkes model. Three limitations of the linear Hawkes model motivate the step.

1. **Effects only add.** Each past event contributes its own bump, independently of the others. In the chain reaction of Example 7.3, a trade followed by two cancellations at the same level is far more alarming than either alone: together they say the level is about to go. A sum of separate bumps cannot express “A *and then* B matters more than A plus B”.
2. **The shape is fixed in advance.** The analyst chooses exponential, power law, or a sum of exponentials, and the data only set the parameters.
3. **The rate can only jump up and then decay.** Between events, a linear Hawkes intensity with positive kernels can only fall. Real flow can do the opposite: after a sweep, new limit orders may arrive slowly at first and then faster as participants re-quote.

Figure 28 draws the three limitations as intensity curves, next to shapes that a neural model can produce.

The three parts of a neural point process. Every neural point process has three parts.

1. Describe each event as a vector

Event n is turned into a vector of numbers (an embedding) built from its type, the time since the previous event $t_n - t_{n-1}$, and any marks such as size or side.

2. Summarise the history

A sequence model reads the embeddings of events $1, \dots, n$ in order and produces a *history vector*

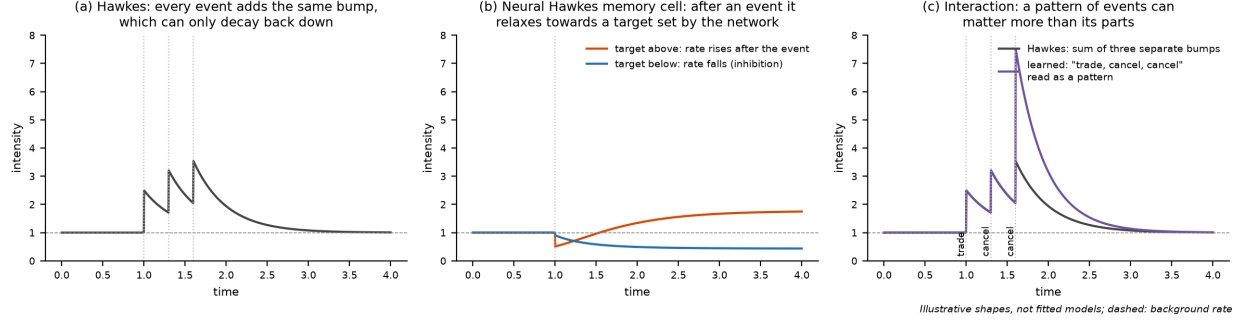


Figure 28: What a fixed sum of bumps cannot draw. (a) A linear Hawkes intensity after three events: each adds the same bump, and between events the rate can only decay. (b) One memory cell of a continuous-time LSTM (Equation (39), Section 7.19.1): after an event the cell relaxes towards a target chosen by the network, so the rate can rise for a while after an event, or fall below the background rate (inhibition). (c) The same three events read as a pattern: a model that has learned that “trade, cancel, cancel” usually precedes a sweep raises the rate after the third event by more than the sum of three bumps. Hand-specified illustrative shapes, not fitted models.

$\mathbf{z}^{\text{hist}}(t)$ that summarises everything that happened before time t . This is the role played in a Hawkes model by the running sums of decaying bumps.

3. Turn the summary into intensities

For each event type i , the intensity is

$$\lambda_i(t) = \text{softplus}(\langle \mathbf{o}_i, \mathbf{z}^{\text{hist}}(t) \rangle), \quad \text{softplus}(y) = \ln(1 + e^y), \quad (38)$$

where $\mathbf{o}_i$ is a vector of learned output weights for type i and y stands for any real number.

In words: score the current history with a weighted sum, one set of weights per event type, and pass the score through the softplus, a smooth function that is always positive and close to the score itself when the score is large. Rates must be positive; softplus guarantees it, as the exponential did in the state-dependent model.

A linear Hawkes model is the special case in which the “summary” is the vector of running sums $\sum \phi_{ij}(t - t_{n'})$ and the output step simply adds them to μ_i . A neural model replaces the hand-written summary with a learned one.

Training: the same likelihood as Hawkes. The parameters—the network weights and the output weights $\mathbf{o}_i$ —are fitted by maximising exactly the log-likelihood of Section 7.12: reward high intensity at the times events happened, penalise the total expected number of events $\int \lambda_i$. Gradient descent replaces the specialised optimisers, because the network has many parameters. One new difficulty appears. For an exponential kernel the integral $\int \lambda_i$ has a closed form; for a neural intensity it usually does not, and is estimated numerically—commonly by evaluating the intensity at randomly sampled times between events and averaging (Monte Carlo integration). Two lines of work remove this approximation: Omi et al. (2019) model the *integral* of the intensity with a network constrained to be increasing, and obtain the intensity as its derivative, so the likelihood is exact; Shchur et al. (2020) skip the intensity altogether and model the probability distribution of the time to the next event directly, for example as a mixture of simple distributions, which also makes sampling the next event exact.

Three designs for the history encoder. The models in the literature differ mainly in step 2: how the history is turned into a memory. There are three families (Figure 29). A *recurrent* model

reads the events one at a time and carries a memory forward; an *attention* model lets the newest event look back at every earlier event directly and decide how much each matters; a *state-space* model carries a memory forward like a recurrent model, but one that fades by an amount set by the actual time between events. The next three subsections take them in turn; Table 20 compares them.

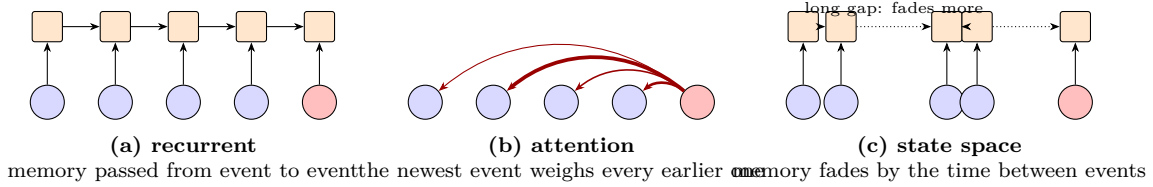


Figure 29: Three ways a neural point process builds its memory of the past. Blue: earlier events; red: the newest event; orange: memory. (a) A recurrent network (RMTPP, neural Hawkes) updates its memory event by event. (b) An attention model (Transformer Hawkes, self-attentive Hawkes) compares the newest event with every earlier event and weights them, the thickness of the arrows. (c) A state-space model (Mamba Hawkes) updates its memory event by event, letting it fade more across a longer gap in time. Schematic.

7.19.1 Recurrent memory: RMTPP and the neural Hawkes process

An early and widely cited model, RMTPP (Du et al., 2016), updates a recurrent network’s state at each event and lets the intensity change between events through a fixed function of the elapsed time. The *neural Hawkes process* (Mei and Eisner, 2017) makes the recurrent state itself evolve in continuous time, as in Figure 27: each memory cell of an LSTM decays exponentially between events towards a target value set at the last event,

$$c(t) = \bar{c} + (c(t_n) - \bar{c}) e^{-d(t-t_n)}, \quad t_n \leq t < t_{n+1}, \quad (39)$$

In words: the cell relaxes from where the last event left it, $c(t_n)$, towards a target $\bar{c}$, at a speed d ; all three are computed by the network at each event from the event and the memory.

Because the target can be above or below the current value, the intensity can rise or fall between events, and one event can make another *less* likely—the inhibition that linear Hawkes cannot represent. The weakness the later families address is that the memory is a single vector passed from event to event: what happened long ago reaches the present only through every intermediate update, and the events must be processed one after another, which is slow to train on long sequences (Zuo et al., 2020).

7.19.2 Attention: the Transformer and self-attentive Hawkes processes

Attention over past events. Instead of passing a memory along, an attention model lets the newest event look back at all earlier events at once and weigh them (Section 10.7 explains attention with an order-book example). Each earlier event receives a weight according to how well its content matches the present; the weighted combination of the earlier events is the memory. Two papers of 2020 built Hawkes-type models this way, with the same idea but different ways of turning the memory into an intensity.

The Transformer Hawkes process. In the model of Zuo et al. (2020), each event is represented by the sum of a learned vector for its type and a vector of sines and cosines of its time stamp, so

that the network knows when it happened. Several layers of masked attention—each event may look only at earlier events—produce a hidden vector $\mathbf{h}(t_n)$ for every event. Between event n and the next, the intensity of type i is

$$\lambda_i(t) = \text{softplus}_i\left(a_i \frac{t - t_n}{t_n} + \langle \mathbf{w}_i, \mathbf{h}(t_n) \rangle + b_i\right), \quad t_n \leq t < t_{n+1}, \quad (40)$$

In words: three terms inside a softplus: a term that changes in proportion to the time since the last event, a learned score of the memory, and a constant. The memory sets the level of the rate; between events the rate drifts steadily up or down, depending on the sign of the learned coefficient a_i .

The integral of the intensity in the likelihood has no closed form and is approximated by Monte Carlo sampling or the trapezoidal rule. For the predictions reported, the authors add two output layers on top of $\mathbf{h}(t_n)$ —a classifier for the next type and a regression for the next time—trained jointly with the likelihood. Because attention compares every pair of events, all events of a sequence can be processed in parallel, and many layers can be stacked, which the authors give as its advantages over recurrent models.

The self-attentive Hawkes process. Zhang et al. (2020) keep closer to the classical form. After each event, attention over the history produces, for each type i , three numbers: a starting level η_i , a long-run level μ_i and a decay rate γ_i . The intensity then relaxes exponentially from one to the other,

$$\lambda_i(t) = \text{softplus}\left(\mu_i + (\eta_i - \mu_i) e^{-\gamma_i(t-t_n)}\right), \quad t_n < t \leq t_{n+1}, \quad (41)$$

In words: exactly the shape of an exponential Hawkes intensity after an event—a jump and a decay towards a background level—except that the start, the end and the speed are computed by attention from the whole history, and the start may be below the end, which gives inhibition.

Time enters as a phase shift in the sine and cosine encodings of each event’s position. The authors argue that the attention weights give an interpretable measure of how much each type of event influences each other type.

Benefits and costs of attention. Attention can reach an event far in the past directly, without passing through every intermediate update, and the weights it assigns can be inspected. Its cost grows with the square of the number of events attended to, which limits how much history it can use; the Mamba Hawkes process was motivated in part by that limit (Gao et al., 2024).

7.19.3 State-space memory: the Mamba Hawkes process

A state-space recurrence over events. A state-space model (Section 10) is a recurrence: a memory vector $\mathbf{z}$ is multiplied by a matrix and the new input is added. A *selective* state-space model, Mamba, lets the step size of the recurrence vary from input to input, so that the model decides how much to forget at each step. Gao et al. (2024) use this to build the Mamba Hawkes process, with one change: the step size is the actual time between events, $\Delta_n = t_n - t_{n-1}$,

$$\mathbf{z}_n = e^{\Delta_n \mathbf{A}} \mathbf{z}_{n-1} + \mathbf{B}_n \mathbf{x}_n, \quad (42)$$

In words: at each event, the old memory $\mathbf{z}_{n-1}$ is shrunk by an amount that depends on how long ago the previous event was—more after a long gap—and the new event $\mathbf{x}_n$ is written in, with a weight $\mathbf{B}_n$ that depends on the event.

Here $\mathbf{A}$ is a learned matrix, $\mathbf{x}_n$ the embedding of event n , and $\mathbf{B}_n$ is computed from it (the paper uses a slightly more elaborate weight, which follows from turning the continuous-time recurrence into a discrete one). The intensity is then computed from the memory exactly as in the Transformer Hawkes process, Equation (40). The authors show that in the simplest case—one memory number, $\mathbf{A} = -1$ —the recurrence becomes an exponentially weighted average in which the weight on the past is $e^{-(t_n - t_{n-1})}$: the memory of an exponential Hawkes process, in which old events fade with elapsed time. A variant, MHP-E, passes the output of Mamba layers through Transformer layers. The model processes a sequence in time proportional to its length, where attention needs time proportional to its square.

Table 20: The three families of neural point process.

	Recurrent (neural Hawkes)	Attention (THP, SAHP)	State space (Mamba Hawkes)
Memory	one vector, updated event by event	a weighted combination of all earlier events	one vector, updated event by event
How time enters	each cell decays at a learned speed between events	encoded in each event’s representation	the memory fades by the time between events
Reaching far back	only through every intermediate update	directly	through the recurrence, with input-dependent forgetting
Cost per sequence of L events	proportional to L , but sequential	proportional to L^2 , in parallel	proportional to L
Closest to classical Hawkes	a learned version of the exponential decay	SAHP: exponential relaxation with learned start, end and speed	simplest case: exponentially weighted memory

Shchur et al. (2021) survey the family and its design choices.

7.19.4 Learning only the kernel

A smaller step keeps the Hawkes sum of bumps and learns only the shape of each bump. Shallow Neural Hawkes (Joseph et al., 2022) represents the kernel by a network with one hidden layer, giving a non-parametric estimate of the kernel. On 120,000 Bitcoin market orders from Binance on one day in May 2020 it reports a better in-sample likelihood than the expectation–maximisation and Wiener–Hopf estimators of Section 7.12. Unlike the models above it keeps the additive Hawkes form: each past event adds its own bump, whatever the order in which the events arrived.

7.19.5 Applications to financial events and order books

The shared financial benchmark. The general-purpose models above were all tested on the same financial data set, introduced with RMTTP (Du et al., 2016): one day of transactions in one stock, with two event types, buy and sell, 414,800 events in all, cut into sequences of about 2,074 events (Zuo et al., 2020). Table 21 collects the results reported on it. (The self-attentive Hawkes process was not tested on it.)

Two things stand out. First, the likelihood and the timing error improve a great deal from one family to the next, while the accuracy of predicting buy or sell barely moves: all five models lie between 61.95% and 62.7%. The newer models describe *when* events happen much better, but are no better at saying *which side* comes next. Second, the evidence is thin: one stock, one day, two event types, no classical Hawkes model in the comparison, no spread across repeated runs in the tables, and an accuracy for the Transformer Hawkes process that differs between the paper that introduced it (62.64%) and the paper that compares with it (62.23%). The Mamba Hawkes paper is a preprint.

Table 21: Neural point processes on the financial-transactions data set (one stock, one day, buy and sell events), as reported by [Zuo et al. \(2020\)](#) and [Gao et al. \(2024\)](#). Log-likelihood per event: higher is better. Accuracy: predicting whether the next event is a buy or a sell. RMSE: error in predicting the time of the next event.

Model	Log-likelihood per event	Type accuracy (%)	Time RMSE
RMTPP (recurrent)	−3.89	61.95	1.56
Neural Hawkes (recurrent)	−3.60	62.20	1.56
Transformer Hawkes (attention)	−1.11	62.64 (62.23 in Gao et al., 2024)	0.93
Mamba Hawkes (state space)	0.974	62.5	0.592
Mamba + Transformer (MHP-E)	0.966	62.7	0.556

Models built for the order book. The order-book applications use richer event types and compare with a classical model:

- **Parallel neural Hawkes** ([Shi and Cartlidge, 2022](#)), reviewed in Section 9.4: one continuous-time LSTM unit per event type (bid and ask submissions and cancellations), with the state-dependent transitions of Section 7.16. On five days each of INTC, MSFT and JPM it beats a classical state-dependent Hawkes model on every measure—for example 49.7% against 42.7% accuracy for the next event’s type on MSFT—and a self-attentive Hawkes model and a single neural Hawkes unit as well. Its margin over a plain LSTM classifier is 2 to 3 percentage points.
- **An independent re-run** ([Zheng et al., 2024b](#)) of the same models on six data sets—two days of MSFT and four from Chinese exchanges—found the parallel model ahead of the plain LSTM on five of the six and behind on one, and a timing error worse than the classical Hawkes model on one of the six.
- **A neural Hawkes market** ([Lalor and Swishchuk, 2025](#)), reviewed in Section 9.5: twelve event types, one day of five stocks, used as the environment for a market-making agent. It matches the volatility of the real mid-price but not its mean reversion, and no classical Hawkes model is fitted for comparison.
- **Known only from abstracts or titles:** a Transformer Hawkes process whose attention is biased by a term that decays exponentially with elapsed time, tested on financial event prediction ([Hwang et al., 2025](#)); a neural marked Hawkes process for the order book ([Chung et al., 2024](#)); and a deep Hawkes model used as the environment for high-frequency market making ([Kumar, 2024](#)). We could not verify their data or results.

Gaps in the literature. We found no study that compares the three families—recurrent, attention and state-space—on order-book data, and none that compares any of them out of sample with a multivariate Hawkes model using sum-of-exponential kernels on many instruments. The verified order-book evidence comes from one recurrent design, on a handful of stocks over a few days.

7.19.6 Benefits and costs

Example 7.4 (What a learned summary can add). Take the chain reaction of Example 7.3. A two-type Hawkes model of trades and ask cancellations gives the intensity of a further buy market order as background plus one bump for the trade and one for each cancellation, whatever their order. A neural model reads the sequence “buy trade, cancel, cancel” as a whole. If, in the training data, that particular pattern was usually followed within a millisecond by a sweep of the level, the model can raise the intensity of buy market orders far more than the sum of the separate bumps

would, and leave it unchanged after the same three events in a different order. Whether it learns this depends entirely on whether the pattern is frequent enough in the data; the flexibility is useless without the examples.

Situations where a neural point process helps. The extra flexibility pays only in some situations. It helps when there is a great deal of data—[Shi and Cartlidge \(2022\)](#) train on about four million windows of events—so that patterns of events recur often enough to be learned; when interactions between events matter, as in the chain reaction above; when there are many event types, since a multivariate Hawkes model needs a kernel for every ordered pair of types (144 for twelve types) while a network shares its weights across them; and when the task is to predict the next event or to simulate order flow. On order-book data the gains are measurable but moderate: on MSFT, a single neural Hawkes unit predicted the type of the next event correctly 46.8% of the time against 42.7% for a classical state-dependent Hawkes model, and the parallel version reached 49.7%, though the single unit was slightly behind a plain LSTM classifier ([Shi and Cartlidge, 2022](#)) (Section 9.4). A classical Hawkes model remains the better choice when the branching ratio or matrix is itself the object of interest, when the intensity must be updated in a few operations per event inside a trading system, or when data are scarce.

Costs of a neural point process. The flexibility has three costs, and each matters in this survey’s setting.

- **Interpretability.** A linear Hawkes fit returns a branching matrix whose entries have a meaning (Section 7) and a branching ratio that can be monitored. A neural model returns weights; quantities such as the fraction of endogenous events have no direct equivalent.
- **Speed.** An exponential Hawkes intensity is updated with a few multiplications per event. A neural intensity needs a forward pass of a network per event, and more if the intensity is needed between events—a cost that matters inside a trading loop (Section 13).
- **Guarantees.** The stability condition $\rho(\Gamma) < 1$ has no simple analogue, so a simulated neural model can drift into regimes never seen in training.

The evaluation follows Principle 2. The baseline is a multivariate Hawkes model, ideally with sum-of-exponential kernels, fitted to the same events; the comparison is the held-out log-likelihood per event, the time-rescaling test of Section 7.12 (which applies to any model that outputs an intensity), and, for a trading use, the economic value of the forecasts. A neural point process earns its place when it improves these by more than its cost in speed and interpretability.

Model card: Neural point processes

What you calibrate. The embedding of events, the weights of the sequence model that summarises the history (recurrent, continuous-time recurrent, attention or state-space), and the output weights $\mathbf{o}_i$ that turn the summary into an intensity for each event type.

How. Gradient descent on the point-process log-likelihood, with the integral of the intensity estimated by Monte Carlo, or made exact by modelling the integral or the next-gap distribution directly. Compare on held-out likelihood and by time rescaling against a multivariate Hawkes baseline.

What you do with it. Predict the type and time of the next event; simulate order flow, including as a training environment for execution or market-making agents; capture interactions between events that additive kernels miss.

How to picture it. A Hawkes model whose bumps are drawn by a network that has read the whole history in order, instead of by a fixed formula: it can notice that “trade, cancel, cancel” means more than

the three events separately.

7.20 Uses of Hawkes models

Capability beyond a Poisson model. Everything in this section comes down to one capability. A Poisson model says that the next second will look like any other second. A Hawkes model says that the next second depends on what just happened: right after a burst, the next second will be busier, and the model says how much busier, for which kinds of event, and for how long. Whenever a decision depends on *how busy the next moments will be*, and the answer changes after bursts, a Hawkes model is the natural tool. Anyone who has sized a buffer, a thread pool or a rate limiter has met the same problem: traffic that arrives in bursts, where the average rate is a poor guide to the peak. On CME NQ market data, the arrival of packets and of matching-engine transactions follows such a process, with a branching ratio of about 0.8 and excitation that fades within about 167 microseconds (Maciejewski, 2026a) (Section 7.6).

Table 22 lists the questions a Hawkes model answers and where each is treated. The rest of this subsection takes them one at a time, with numbers.

Table 22: Questions a Hawkes model answers.

Question	What the model provides	Where
How many events of each type in the next few milliseconds?	the current intensities $\lambda_i(t)$	Use 1
What would a realistic stream of orders look like, bursts included?	simulation by thinning	Use 2
How much of the activity is a reaction to earlier activity? Who triggers whom?	the branching ratio and matrix	Use 3, §7.20.1
Which way is the price drifting? How soon will this queue be empty?	drift, expected order-flow imbalance, depletion time	Use 4, §7.20.2
How should I execute an order, or quote, when flow comes in bursts?	a Markov state for control problems	Use 5, §7.20.3

Use 1: forecasting short-horizon activity. The fitted intensities are a forecast: at an intensity of 300 events per second, about 0.3 events are expected in the next millisecond. Because each intensity jumps after events and then decays, the forecast is high right after a burst and falls back to the background rate in quiet times, which a Poisson model, with one rate for the whole day, cannot do. The forecast is per event type: a burst of buy market orders raises the forecast for more buys (self-excitation) and for cancellations at the ask (cross-excitation) by different amounts.

Use 2: simulating realistic order flow. A fitted model can be run forward to generate artificial streams of events with the same clustering as the real ones (Section 7.12). Such streams can drive a simulated book, test how a strategy or a piece of software behaves in a burst, or serve as the environment for a learning agent (Section 9.5). Adding prices gives the models of Section 7.11. A Poisson simulator produces the right average number of events but not the bursts, and the bursts are when queues empty and fills go wrong.

Use 3: measuring the market. The fitted parameters are themselves informative. The branching ratio says what share of activity is a reaction to earlier activity rather than new information from

outside—four in five events on CME NQ. The branching matrix says which event types trigger which (Section 7.10): whether buys mostly trigger buys (runs of trades in one direction) or sells (reversal), and whether trades trigger cancellations. Fitted state by state, it shows when the market becomes more reactive, for example when the spread widens (Section 7.16). Section 7.20.1 explains how to read and use the branching ratio, and how it can mislead.

Use 4 and 5: decisions. The remaining two uses turn the intensities into trading decisions, and are described in the last two subsections, after a closer look at the branching ratio: quantities a trader can read off directly, and control problems that can be solved because an exponential Hawkes model has a small Markov state.

Questions a Hawkes model does not answer. A Hawkes model is a model of *timing*. It can reproduce the clustering of order flow very well while saying little about the sign of the next price move, because a burst of buying and a burst of selling are both bursts. On a standard financial data set, neural Hawkes models that described the timing of events far better than earlier ones predicted whether the next trade was a buy or a sell no better, all at about 62% (Section 7.19.5). Hawkes models should therefore be judged separately on each task of Table 1: forecasting intensities, simulating, predicting the price, and supporting execution.

Cases where a Poisson model is sufficient. A Poisson model is enough when activity is steady over the horizon that matters, and a queue model of Section 6 when what matters is the current size of the queues rather than the recent history of events. A neural point process (Section 7.19) is worth its cost only with a great deal of data and when patterns of events, not just their count, carry information.

7.20.1 Using the branching ratio

Interpretation of the branching ratio. The branching ratio Γ is the average number of events that one event directly triggers (Section 7.6). Three readings of it are useful in practice (Table 23):

- **The share of activity that is reaction.** In a stationary Hawkes process a fraction Γ of all events were triggered by earlier events and a fraction $1 - \Gamma$ came from outside. At $\Gamma = 0.8$, four events in five are reactions to earlier events.
- **The amplification of a shock.** One event from outside—a news item, one large trader’s decision— produces on average $1/(1 - \Gamma)$ events in total, itself included. Ten outside events become 20 at $\Gamma = 0.5$, 50 at 0.8 and 200 at 0.95.
- **How fragile the amplification is.** Near one, small changes in Γ change the amplification a great deal: from 0.9 to 0.95 it doubles, from 10 to 20. A market whose branching ratio drifts towards one becomes more and more sensitive to outside events, and cascades last longer.

Published uses of the branching ratio. Three kinds of question have been asked of it.

- **How reactive a market is, and whether that has changed.** On E-mini S&P 500 futures, [Filimonov and Sornette \(2012\)](#) found the branching ratio rising from about 0.3 around 1998–2000 to 0.7–0.8 after 2007, with spikes to 0.9; the size of such numbers, and whether markets sit at one, is debated (Section 7.13).

Table 23: Reading a branching ratio. One outside event produces $1/(1 - \Gamma)$ events in total.

Branching ratio Γ	Share of events triggered by earlier events	Events per outside event	10 outside events become
0	0%	1	10
0.5	50%	2	20
0.8	80%	5	50
0.9	90%	10	100
0.95	95%	20	200

- **When a market is most reactive.** Fitted separately for each state of the book, the measure is higher when the book is unbalanced than when it is balanced; for Intel it was above one with a wide spread (Section 7.16).
- **How much activity is news and how much is reaction.** Adding the times of scheduled macroeconomic announcements to a Hawkes model of foreign-exchange quotes lowers the estimated branching ratio, from 0.84–0.95 without them (Rambaldi et al., 2015); a related method finds bursts of activity of unknown timing, and reports that only a small fraction coincide with news (Rambaldi et al., 2018).

Bias from a time-varying background rate. A fitted branching ratio is only as good as the model’s background rate. If the true background rate varies—busy and quiet periods, the open and close, news—and the model assumes it constant, the model can explain the busy periods only by making events trigger events, and the estimated Γ rises. Figure 30 shows how strong the effect can be: a stream with no self-excitation at all, whose background rate switches between 1 and 5 events per second, is fitted with a branching ratio of 0.62 to 0.92 depending on how often the rate switches. Real studies find the same in a milder form: on Nikkei 225 mini futures, a model whose background rate is allowed to vary over the day gives a branching ratio of 0.41, and the authors report that a constant background clearly overestimates it (Omi et al., 2017); Filimonov and Sornette (2015) list regime shifts among the reasons why estimates near one are biased upward.

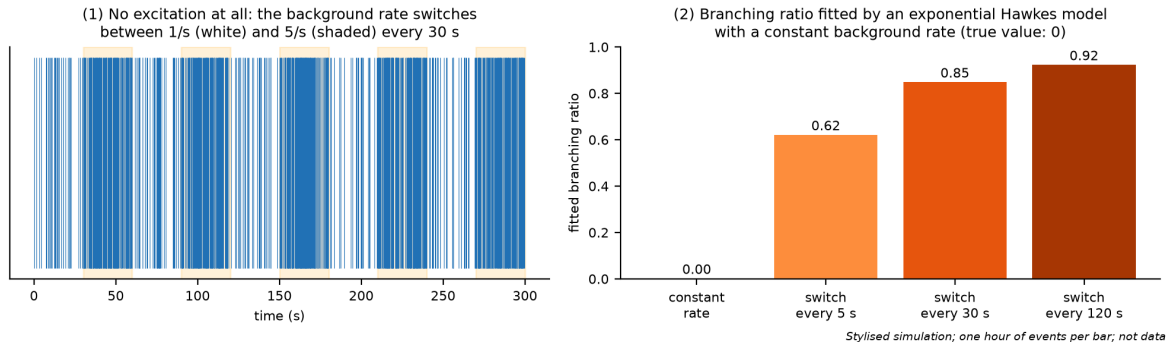


Figure 30: A branching ratio from no excitation. (1) A stream of events with no self-excitation; only the background rate changes, between 1 and 5 per second. (2) The branching ratio estimated by maximum likelihood with an exponential Hawkes model that assumes a constant background rate, for one hour of events: zero when the rate is truly constant, 0.62 to 0.92 when it switches every 5, 30 or 120 seconds. Stylised simulation; not data.

Dependence on estimator, window and kernel. The estimate depends on the method and window: with a simple estimator based on the variance of event counts in windows of fixed length,

the E-mini estimate rises over time, but it stays roughly constant if the window shrinks as trading speeds up (Hardiman and Bouchaud, 2014). And it depends on the shape of kernel assumed: power-law fits gave about one on the same market on which exponential-type fits gave 0.7–0.8 (Section 7.13). A branching ratio is therefore comparable only with other branching ratios estimated the same way.

A branching-ratio monitor. With those cautions, a running estimate can serve as a gauge of how self-generated current activity is. The following design is our suggestion, not a tested recipe: refit on a rolling window of fixed length, always with the same kernel family and estimator; let the background rate vary with the time of day, or model scheduled news, so that predictable changes in activity are not counted as excitation; compare each value with the same instrument’s own history rather than with a universal threshold, since published values differ widely by instrument, period and method; and, if the book state matters, estimate it state by state. Read a high value as a sign that activity is feeding on itself and that bursts will be larger and longer—information for risk limits, quote sizes and capacity—not as a forecast of the direction of the price.

7.20.2 From intensities to decisions

A fitted Hawkes model gives, at every instant, the expected rate of each kind of event. Four quantities a trader can use follow directly from these rates. Each is a short-horizon quantity: it assumes the rates stay roughly where they are now, which is true only for a short time, because they decay between events.

Drift. In the price model of Section 7.11, up and down moves of size δ_m arrive at rates $\lambda^+(t)$ and $\lambda^-(t)$, so over a short horizon H the expected mid change is approximately

$$\mathbb{E}[m_{t+H} - m_t] \approx \delta_m (\lambda^+(t) - \lambda^-(t)) H.$$

In words: if up moves are currently arriving faster than down moves, the price is expected to drift up, by one tick per surplus up move.

For example, if up moves are currently arriving at 3 per second and down moves at 1 per second, then over the next half second the mid is expected to rise by $(3 - 1) \times 0.5 = 1$ tick.

Expected order-flow imbalance. Model six event types at the best quotes: limit orders, cancellations and market orders on each side, with intensities $\lambda^{L,b}, \lambda^{C,b}, \lambda^{M,b}$ (bid queue added to; bid queue cancelled; market *buy*, which takes from the ask) and $\lambda^{L,a}, \lambda^{C,a}, \lambda^{M,a}$ (ask queue added to; ask queue cancelled; market *sell*, which takes from the bid). With $\bar{v}$ the average event size and the best prices unchanged, the expected OFI per second is

$$\bar{v} \left(\underbrace{\lambda^{L,b} - \lambda^{C,b} - \lambda^{M,a}}_{\text{net change of the bid queue}} - \underbrace{(\lambda^{L,a} - \lambda^{C,a} - \lambda^{M,b})}_{\text{net change of the ask queue}} \right).$$

In words: in Section 4, OFI was computed *after the fact*, by going through the events that had happened and adding up how much the bid queue grew minus how much the ask queue grew. Here the same sum is computed *in advance*: instead of counting events that happened, it uses the Hawkes intensities to say how many of each event are expected in the next second, and adds up the OFI those expected events would produce. The signs are the same as before: anything that adds to the

bid queue (a buy limit order) or takes from the ask queue (a market buy, an ask cancellation) is buying pressure; the reverse is selling pressure.

Example (illustrative numbers). With an average event size of 5 lots, suppose the intensities are, per second, 10 buy limit orders, 6 bid cancellations and 2 market sells at the bid, and 8 sell limit orders, 9 ask cancellations and 5 market buys at the ask. The bid queue is expected to change by $5 \times (10 - 6 - 2) = +10$ lots per second, the ask queue by $5 \times (8 - 9 - 5) = -30$; the expected OFI is $10 - (-30) = +40$ lots per second of buying pressure. The regression of Section 4 turns pressure into price: with $\beta = 0.01$ points per lot, it predicts a rise of about 0.4 points over the next second. So the Hawkes model supplies a *forecast* of OFI, and the OFI regression turns that into a forecast of the price change—valid, like the other quantities here, only while the intensities stay near their current values.

Depletion time. The bucket picture of Section 6 needs an inflow and an outflow. For the best ask, $r_{\text{in}} = \bar{v} \lambda^{L,a}$ and

$$r_{\text{out}} = \bar{v} (\lambda^{M,b} + \lambda^{C,a}),$$

so with the current intensities

$$\mathbb{E}[\tau^a] \approx \frac{v_{1,t}^a}{r_{\text{out}} - r_{\text{in}}}, \quad r_{\text{out}} > r_{\text{in}}.$$

When this falls below the trader’s own latency T_{lat} , a sell order resting in that ask queue is in danger: if the trader decides to cancel, the level may be swept—filling the order just before the price moves up—before the cancellation arrives. The full probability $\mathbb{P}(\tau^a \leq T_{\text{lat}})$ has no closed form under a Hawkes model, but can be estimated by simulating from the current state by thinning.

Example: depletion time during a burst. Take a best ask of 200 lots, an average event size of 5 lots, and, in normal times, 10 limit orders, 8 cancellations and 4 market buys per second at the ask. Inflow is $5 \times 10 = 50$ lots per second, outflow $5 \times (8 + 4) = 60$, so the queue shrinks by 10 lots per second and the expected depletion time is $200/10 = 20$ seconds. Now a burst of buying hits. Excitation raises market buys to 20 and cancellations at the ask to 14 per second: outflow becomes $5 \times 34 = 170$ lots per second, the queue shrinks by 120, and the expected depletion time falls to $200/120 \approx 1.7$ seconds. A Poisson model, using the day’s averages, still says 20 seconds. Figure 31 follows the estimate as the excitation decays. The estimate uses the current intensities as if they stayed fixed, so it is rough when they are changing quickly; the exact probability needs simulation.

7.20.3 Execution and market making

Exponential kernels and the Markov property. With an exponential kernel, the current intensity summarises everything needed from the past: the pair (count of events, current intensity) evolves as a Markov process. A sum of several exponentials needs one extra state variable per exponential (Bacry et al., 2015). That is what lets execution and market-making problems, normally solved by dynamic programming over a Markov state, include self-exciting order flow. Power-law kernels are handled by approximating them with sums of exponentials, as in Section 7.18.

Optimal execution. Alfonsi and Blanc (2016) solve the problem of executing a large order explicitly in a price model with both permanent and transient impact, in which order flow follows a Hawkes process. A key result concerns *price manipulation*—strategies that profit merely by pushing the price and trading back: with Poisson order flow it always exists in their model, while with

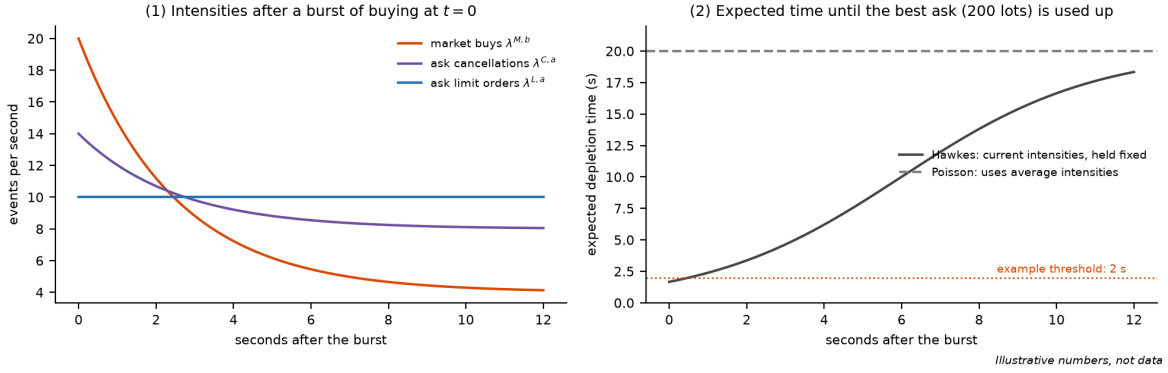


Figure 31: Depletion time after a burst. (1) Intensities at the best ask after a burst of buying at time zero, decaying back to normal. (2) Expected time until the 200-lot best ask is used up, computed from the current intensities held fixed (solid), against the Poisson answer from average intensities (dashed). For a few seconds after the burst the Hawkes estimate is a fraction of the Poisson one; a trader whose cancellation takes longer than the dotted threshold is at risk in that window. Illustrative numbers, not data.

Hawkes order flow it disappears when the decay of the Hawkes kernel matches the speed at which the price recovers and the kernel’s total weight matches the transient share of impact.

Market making. Cartea et al. (2014) model buy and sell market orders as mutually exciting processes with exponential kernels and solve for the quotes of a market maker, with the intensities as state variables, so that the quotes respond to the current level of excitation; the model requires the decay of excitation to be fast enough relative to its strength for the intensities to stay stable. The paper was republished in *SIAM Review* as an outstanding contribution (Cartea et al., 2018). Further work treats market making with Hawkes order flow and non-Markovian kernels (Jusselin, 2021).

8 Meta-models: learning on top of the classical models

The classical models of Sections 4–7 each compress the book into a few numbers with a clear meaning: the order-flow imbalance, the queue imbalance and the micro-price, the intensity of each kind of event, the probability that the ask queue empties first. Each is a partial view. A *meta-model* takes these outputs—not the raw book—as its inputs, and lets a learning algorithm combine them into a forecast: the probability of an up move, the expected price change, or the probability that a resting order is filled. The classical models act as feature generators; the learner estimates how much each matters and, if it is flexible enough, how they interact. In machine-learning terms this is *stacking*: the outputs of several first-stage models become the inputs of a second-stage model.

Meta-models are the first of three ways of adding learning, in decreasing order of imposed structure: meta-models (this section), structural hybrids (Section 9) and deep networks (Section 10). A meta-model differs from each of its neighbours in one respect. The classical models fix in advance how the signals combine; a meta-model learns it. A deep network is asked to rediscover order-flow imbalance or self-excitation from raw snapshots; a meta-model is given them ready-made, and the evidence of Section 8.6 is that engineered inputs of this kind outperform raw book inputs when the learner is held fixed. A structural hybrid builds classical structure *inside* a network—a Hawkes kernel in an attention layer, a point process with neural intensities; a meta-model keeps the classical models intact and learns only how to combine what they say. Section 9.1 draws this line in detail.

The price of the extra inputs is a larger space of choices—which sub-models, which windows, which learner—and so a greater risk of fitting noise (Section 8.7).

8.1 The architecture

Figure 32 shows the structure. Each classical model runs on the same event stream and is updated as the book changes. Their outputs are collected, at each decision time, into one feature vector. The learner maps that vector to a forecast, which is calibrated so that it can be read as a probability (Section 10.5), and the forecast feeds the execution decision of Part III.

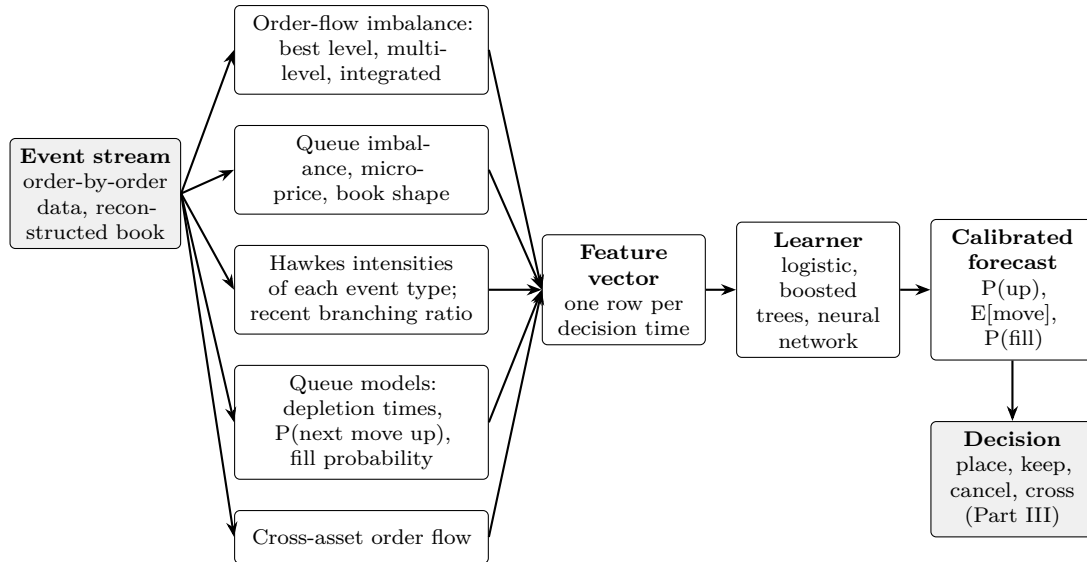


Figure 32: A meta-model. Classical models of the book (left) each turn the event stream into a few interpretable numbers; a learner (right) combines them into a calibrated forecast for the execution decision. The classical models are fitted first and kept fixed while the learner is trained on their out-of-sample outputs (Section 8.7).

8.2 Inputs supplied by the classical models

Table 24 lists the inputs, what each carries, and what it costs to keep up to date, as described in the sections that define them. Most are running sums, or table look-ups precomputed offline, updated once per event.

8.3 Learners used in the literature

Table 25 lists the learners that published studies have placed on top of engineered or classical-model inputs, and what each study found. They range from penalised linear models to random forests, boosted trees and feed-forward neural networks with one or two hidden layers; the detailed results are in Section 8.6.

Results of learner comparisons. Where studies compared learners on the same inputs, the differences were small. In the two largest studies a penalised linear or logistic model was as good as, or slightly better than, random forests, boosted trees and neural networks (Aït-Sahalia et al., 2026; Easley et al., 2021); the only learner that failed was unpenalised least squares, through overfitting. On raw book inputs, Briola et al. (2020) found a multilayer perceptron matching

Table 24: Inputs a meta-model can take from the classical models of this part.

Input	What it carries	Cost per event	Section
OFI at several windows; multi-level and integrated OFI	net buying pressure, recent and deeper in the book	running sums	4
Queue imbalance; micro-price adjustment $m_t^{\text{micro}} - m_t$	which side is likely to break; expected drift from the top of the book	a division; a table look-up	4, 4.5
Book-shape features	depth beyond the touch, slope, resilience	sums over levels	4.4
Event-level features	whether size left a queue by trade or by cancellation; message mix	counters	5
Hawkes intensities $\lambda_i(t)$; rolling branching ratio	how active each event type is now; how self-generated the activity is	constant time (exponential kernels)	7
Queue-model outputs: depletion times, $\mathbb{P}(\text{up})$, fill probability	the race between the two best queues; the fate of a resting order	table look-up or formula	6
Order’s own queue position Q_t^{ahead}	how far the order is from being filled	book state	11.2
Cross-asset OFI	pressure in related instruments	running sums	4.6

Table 25: Learners applied to engineered or classical-model inputs in the studies surveyed.

Study	Inputs	Learners	Finding
Kercheval and Zhang (2015)	prices, volumes and derived attributes of each book entry	multi-class support vector machines	selected features effective for short-term moves (abstract)
Ntakaris et al. (2020)	273 hand-crafted book, technical and quantitative features, including a logistic-regression output	least-squares, linear discriminant, radial-basis network	a few features can reach the best performance
Nousi et al. (2019)	hand-crafted FI-2010 features vs learned representations	linear SVM, one-layer network, MLP	hand-crafted beat autoencoder features
Ntakaris et al. (2019)	hand-crafted econometric and technical sets vs LSTM-autoencoder features	MLPs, CNNs, LSTMs	hand-crafted better; shallow MLPs suffice
Zheng et al. (2013)	lagged volumes, price gaps, order signs and counts	logistic and lasso-logistic regression	target: price jumps between trades; evaluation split not verified
Easley et al. (2021)	six classical measures (Roll, Kyle’s lambda, Amihud, VPIN, ...)	random forest; logistic regression	logistic slightly more accurate
Aït-Sahalia et al. (2026)	117 trade- and quote-based predictors	lasso, ridge, random forest, boosted trees, neural networks, least squares	all similar except unpenalised least squares
Muni Toke and Yoshida (2020)	book state and Hawkes-based history	logit-type ratio of intensities	combination beats either alone
Guo et al. (2018)	autoregressive volatility and order-book features	learned gate mixing the two	lowest error in 10 of 12 months
Prata et al. (2024)	probability outputs of 15 deep models	two-layer MLP	never beat the best single model

a CNN–LSTM while logistic regression trailed. We found no verified study in which a flexible learner on classical inputs clearly outperformed a penalised linear model on the same inputs. A penalised linear model—lasso (Tibshirani, 1996), which also sets the weights of unhelpful inputs to zero—is therefore the natural first meta-model and the baseline any more flexible learner must beat. Whatever the learner, its output must be calibrated, because the execution decision uses it as a probability.

The lasso. The lasso (Tibshirani, 1996) is least-squares regression with a limit on the total size of the coefficients. Write $f_{n,k}$ for input k at observation n —for example the OFI over a one-second window, or a Hawkes intensity— Y_n for the target, and β_k for the weight on input k (a regression coefficient, as β is in Section 4). The lasso chooses the weights to

$$\text{minimise} \quad \sum_n \left(Y_n - \beta_0 - \sum_k \beta_k f_{n,k} \right)^2 + \varrho \sum_k |\beta_k|, \quad (43)$$

where $\varrho \geq 0$ sets how strongly large weights are penalised.

In words: fit the data as well as possible, but pay a price proportional to the sum of the absolute sizes of the weights. With $\varrho = 0$ this is ordinary least squares; as ϱ grows, the weights shrink towards zero.

Tibshirani’s original statement is the equivalent constrained form: minimise the squared error subject to $\sum_k |\beta_k|$ being at most a chosen constant. The important property is that, because the penalty uses absolute values, it does not merely shrink weights—it sets many of them *exactly* to zero, so the lasso selects inputs as well as fitting them. The reason is geometric. With two inputs, the set of weights allowed by the constraint is a diamond with corners on the axes; the best fit within it is usually found at a corner, where one weight is zero. A penalty on the *squares* of the weights (ridge regression) gives a disc instead, which has no corners, so weights become small but rarely zero. For a classification target the same penalty is added to the log-likelihood of a logistic regression. Two practical points follow from the form of the penalty. Inputs must be put on a common scale first, since the penalty treats a unit of every weight alike; and ϱ is chosen by validation—for market data, time-ordered validation (Section 10.5)—usually as the value giving the best out-of-sample error. For a meta-model, the lasso therefore answers directly which classical inputs earn a place: in the study of Ait-Sahalia et al. (2026), the inputs it selected most often were signed transaction volume and past returns, ahead of book imbalance.

8.4 State models and event models measure different things

Stock and flow. The classical models of this part fall into two groups that look at the book in different ways (Sections 4 and 5). *State* models read a snapshot: queue imbalance, the micro-price and the shape of the book describe how liquidity is arranged *now*—which side is thin, how deep the book is. *Event* models read the stream of orders: order-flow imbalance, Hawkes intensities and message-level models describe what is *happening*—how much pressure is arriving, how busy each kind of event is, and whether size is leaving a queue because it is traded or because it is cancelled. In the language of economics, state models measure a *stock* and event models a *flow*. Because they measure different things, their forecasts should carry partly different information, and their errors need not move together. That matters for combining, because a combination gains most when its components are dissimilar (Breiman, 1996).

Partial overlap between state and event information. The two are not independent. The state at the end of a window is the state at its start plus the events in between, so a model that sees a full history of snapshots can reconstruct the *net* flow: order-flow imbalance at the best level is computed from changes in the best queues (Cont et al., 2014). What snapshots cannot recover is the *cause* of each change. A queue that shrinks by 300 lots looks the same whether a buyer traded against it or sellers cancelled (Example 4.3), and the timing and clustering of events—what a Hawkes intensity measures—are lost when snapshots are sampled. Conversely, an event model that ignores the current state does not know which side is close to breaking. Table 26 summarises what each group sees and misses.

Table 26: What state models and event models of the book see, and what each misses.

	Examples	What it measures	What it cannot see
State (stock)	queue imbalance, micro-price, book shape	how liquidity is arranged now: which side is thin, how deep the book is	why the book reached this state; how fast it is changing
Event (flow)	OFI, Hawkes intensities, message-level models	how much pressure is arriving, how busy each event type is, trades versus cancellations	where the book stands relative to breaking, unless the state is also supplied

Published evidence. Three verified results point the same way, though none tests the question directly for the classical models.

- Predicting the sign of the next trade, a model that combines the book’s state with Hawkes-based event history beat both the state-only and the history-only models, by a little over 2.5% on all trades and by more than 13% on the trades where the sign changed (Muni Toke and Yoshida, 2020).
- Deep models trained on individual messages and deep models trained on book snapshots produced less correlated forecasts, and averaging them improved on DeepLOB by about half to three quarters of an F1 point, where averaging snapshot models alone did not do so consistently (Zhang et al., 2021).
- Within a single group the overlap can be large: adding the imbalance of trades to order-flow imbalance raised the explained share of price changes only from about 65% to 67% (Cont et al., 2014).

A proposed meta-model design. The argument suggests a specific design for a meta-model: pair at least one state model with at least one event model, rather than several models of the same kind. It also suggests how to check it before building anything elaborate. Measure, out of sample, the correlation between the state models’ and the event models’ forecast errors; if it is low, a combination has room to help, and the combination should then beat each group alone and the nulls of Section 8.5. We found no study that reports this error correlation for the classical order-book models; it is an open question (Section 14).

8.5 Combining forecasts: methods borrowed from the general forecasting literature

Scope of this subsection. This subsection is not about order-book models. It summarises a general literature—on how to combine the forecasts of several models into one—that was developed

and tested almost entirely *outside* order books: on quarterly macroeconomic data (output growth and inflation), on daily stock-index returns, on monthly stock returns, and on standard machine-learning benchmark data sets. We include it because its methods apply directly to one case that arises in a meta-model. Some classical outputs are features—OFI over a window, a Hawkes intensity—and are best passed to a learner (Section 8.3). Others are already forecasts of the same quantity: the up-move probability of a queue race, the drift implied by the micro-price, a logistic regression on queue imbalance. Combining such forecasts is exactly the problem this literature studies.

Each paragraph below states the data its evidence comes from. Only a handful of results use order-book or intraday data, and they are marked explicitly. Whether the general findings carry over to order-book forecasts made every few milliseconds has not been tested; none of the methods below has been applied to a combination of the classical order-book models of this part (see the last paragraph).

Six ways to combine, at a glance. Figure 33 sketches the six approaches discussed below. In every panel, three component models (for example a queue-race probability, a micro-price drift and an OFI regression) each produce a forecast, shown as $\hat{y}_1, \hat{y}_2, \hat{y}_3$, and the approaches differ only in how those forecasts become one.

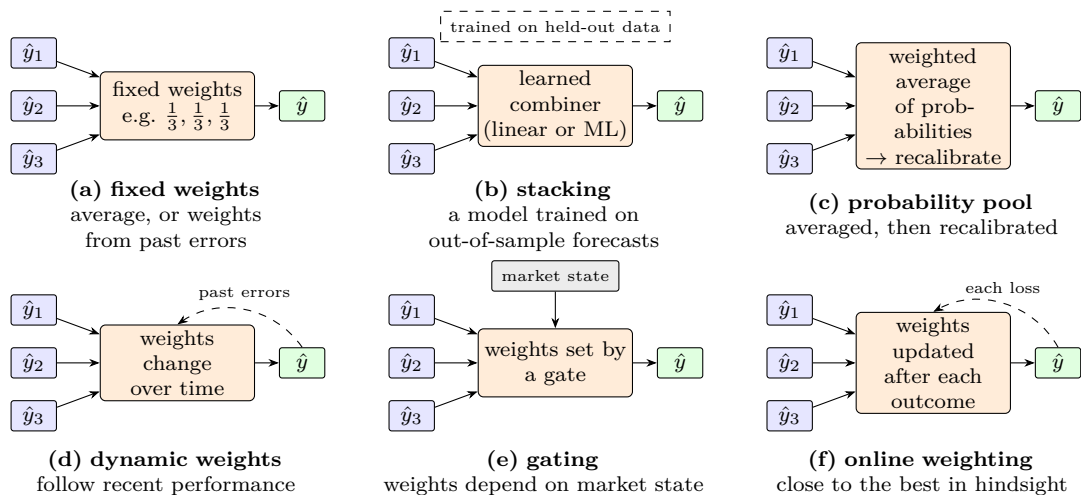


Figure 33: Six ways to combine the forecasts of several models into one. Schematic.

The equal-weight average as baseline. Any combination—and any machine-learned combiner especially—has to be judged against what could be had without it. Four benchmarks follow from the evidence in this subsection and in Section 10, and a combination earns its place only if it beats all four out of sample.

1. **The simple average of the same component forecasts.** This is the null of the forecast-combination literature: estimated weights frequently fail to beat it (the puzzle below), so a combiner that cannot beat equal weights has learned nothing usable.
2. **The best single component, chosen in advance.** The model that did best on past data, selected before the test period—not the best one in hindsight, which no one could have traded.
3. **A naive forecast of the target.** For direction forecasts, the base rate of each class, and, when the label is built from overlapping averages, the forecast that repeats its last observable

value: on smoothed labels this naive forecast came within about one point of DeepLOB (Lee, 2026).

4. **The economic null.** For a trading use, the same execution policy run without the combined forecast—for instance the model-free benchmark of Section 12.3.4—under the same execution model and costs.

The comparison should use a proper score for the forecast and net profit for the decision, on data used for nothing else, and the winner should be reported with a model confidence set (Hansen et al., 2011) rather than as the single best score, counting every combination tried (White, 2000; Harvey et al., 2016).

Learned combiners. Yes: that is approach (b), stacking, with the combiner itself a learned model—a regression, a logistic model, boosted trees or a neural network—trained on the component forecasts, and possibly on other inputs too, in which case it becomes the meta-model of Section 8.3. The published evidence says three things about it. It needs the component forecasts to be out of sample, or it learns their overfitting (the medical example below). It gains most when the components are dissimilar and the data are plentiful (Breiman, 1996; Ting and Witten, 1999). And it does not automatically beat simpler combinations: estimated weights often lose to a plain average (the puzzle below), and on order-book data a neural-network combiner of fifteen deep models never beat the best of them (Prata et al., 2024). A learned combiner is therefore a candidate that must beat the simple average out of sample, not a default.

Neural-network combiners: what has been tried. Neural networks have been used as combiners since the mid-1990s, in four forms; Table 27 lists the main studies.

A small network on the forecasts alone

Donaldson and Kamstra (1996) feed two daily stock-index volatility forecasts (a moving-average variance and a GARCH model) into a network with one hidden layer of one to three units, which outputs the combined forecast; Harrauld and Kamstra (1997) evolve such networks by evolutionary search. On squared error the network did not beat an ordinary least-squares combination: it tied on the S&P 500 and Toronto indices and was slightly worse on the Nikkei and FTSE (Donaldson and Kamstra, 1996), and in the evolutionary study the GARCH forecast alone had the lowest out-of-sample error—with the combiner trained on in-sample component forecasts (Harrauld and Kamstra, 1997).

A feed-forward stacker

A multilayer perceptron takes the component forecasts, sometimes with extra inputs, and outputs the forecast. On limit-order-book data this is the LOBCAST stack of fifteen deep models, which never beat the best single model (F1 55.9 against 61.2 and 53.2 against 59.5 on two years of Nasdaq data) though it did beat a weighted majority vote on FI-2010 (Prata et al., 2024). Recent stacking studies of volatility and stock direction (Ramos-Pérez et al., 2019; Barnwal et al., 2019) report no simple-average or linear-stacking baseline, so they cannot answer the question.

A network that reads the forecasts as a sequence or image

For2For passes the matrix of base forecasts through a convolutional network with a linear shortcut, or an LSTM, and outputs the combined forecast (Zhao and Feng, 2020).

A network that outputs weights from the raw series

Convolutional networks read the history of the series itself and output a weight for each base

forecast through a softmax (Ma and Fildes, 2021; Felici and Sudoso, 2026). This is the neural counterpart of FFORMA, which learns combination weights from features of the series with gradient-boosted trees rather than a network (Montero-Manso et al., 2020).

Gated mixtures of experts (Jacobs et al., 1991; Guo et al., 2018) are a fifth, related form, in which the gate and the experts are trained together rather than stacked on separately fitted forecasts. Attention-based combiners, a sixth form, are described in Section 8.5.1.

Neural combiners against the equal-weight average. It depends on how much data the combiner is trained on. On a few financial series, the 1990s networks did not reliably beat a linear combination. Trained across thousands of series, they do beat the simple average, but by little or nothing over a boosted-tree combiner: on retail sales the convolutional combiner reached a relative error of 0.968, against 0.986 for the simple average, 0.974 for FFORMA and 1.000 for the best single model (Ma and Fildes, 2021); on the M4 competition series, a weight-learning network scored 0.794 to 0.846 on the competition’s error measure against 0.911 to 0.949 for the simple average (Felici and Sudoso, 2026), while For2For scored 0.8345 against 0.838 for FFORMA (Zhao and Feng, 2020). For2For lost to a simple linear model on the smaller weekly and hourly subsets (359 and 414 series). The studies agree on three conditions: the combiner is trained on forecasts the components had not seen, the components are dissimilar (mixed pools beat single-type pools, and a diversity term helped (Ma and Fildes, 2021; Felici and Sudoso, 2026)), and weighting the forecasts works better than selecting one of them (relative error 0.968 against 1.050 (Ma and Fildes, 2021)). None of these large-sample results comes from intraday or order-book data; there, the one verified neural combiner did not beat the best single model.

Table 27: Neural networks used to combine forecasts. “Beats average” compares with the simple average of the same component forecasts where the study reports it.

Study	Combiner	Data	Result
Donaldson and Kamstra (1996)	one-hidden-layer network on two forecasts	daily volatility, four indices	tied or slightly worse than least-squares combination
Harrald and Kamstra (1997)	evolved networks	daily volatility	best single model (GARCH) lowest error
Prata et al. (2024)	two-layer MLP on 15 models’ probabilities	limit-order books	never beat the best single model
Zhao and Feng (2020)	CNN with linear shortcut, or LSTM	M4 series	slightly better than FFORMA overall; worse than linear on small subsets
Ma and Fildes (2021)	CNN outputting weights from the raw series	retail sales	beats average (0.968 vs 0.986) and FFORMA (0.974)
Felici and Sudoso (2026)	CNN outputting weights, with diversity term	M4 series	beats average (0.79–0.85 vs 0.91–0.95)
Montero-Manso et al. (2020)	boosted trees (not a network), for contrast	M4 series	learns weights from features of each series; selection 10% worse than weighting

Fixed weights, and the forecast-combination puzzle. The oldest approach averages the forecasts with fixed weights: equal weights, weights inversely proportional to each model’s past squared error (Bates and Granger, 1969), or weights estimated by regressing the outcome on the forecasts (Granger and Ramanathan, 1984). A long-standing finding is that simple averages are hard to beat. On quarterly output-growth forecasts for seven countries (macroeconomic data), Stock and Watson (2004) found the simple mean had the lowest average loss of all the combination methods

they tried (0.648, against 1.048 for following the recently best model and up to 0.976 for weights that varied over time), and that the methods with the least adaptive weights did best and were the most stable. Claeskens et al. (2016) explain this “forecast combination puzzle”: weights estimated from data make the combination biased and add variance, with no guarantee of beating equal weights. Equal weights fail when one model is much better than the others: in one classification problem a majority vote had a 13.0% error rate against 2.6% for the best single model (Ting and Witten, 1999).

Stacking. Stacking learns the combination from cross-validated predictions of the component models (Wolpert, 1992; Breiman, 1996). Breiman (1996) fits non-negative weights by least squares on such predictions and finds that the non-negativity constraint matters more than forcing the weights to sum to one, and that stacking gains most when the component models are dissimilar. On ten standard machine-learning benchmark data sets, Ting and Witten (1999) find that combining predicted class *probabilities* works better than combining predicted labels: a linear combination of probabilities beat the best single model in nine of them and tied in the tenth, with significant gains mainly on the largest. On *order-book* data the results are weaker: a learned combination of fifteen deep models never beat the best one (Prata et al., 2024), and an equal-weight average of three book-based models was not consistently better than DeepLOB alone, while adding models built on individual messages improved F1 by about half a point (Zhang et al., 2021).

The cost of fitting the combination in sample. The one measured comparison we found of a second-stage model trained on in-sample versus out-of-sample first-stage predictions comes from medicine, not finance. Tibshirani and Efron (2002) combine a learned gene-expression classifier with six clinical predictors in a logistic regression on 78 patients. With the classifier’s in-sample predictions its odds ratio is 60.1; with predictions made out of sample it is 4.7. Its apparent reduction in error, 0.269 to 0.141, shrinks under full cross-validation to 0.295 to 0.282, a difference of 0.013 with a standard error of 0.091. We found no study that measures this effect for stacked financial or order-book models.

Combining probabilities. When the components output probabilities, the natural combination is a weighted average of them—a *linear pool*. Ranjan and Gneiting (2010) prove that a linear pool of calibrated probability forecasts is itself not calibrated: it is under-confident. Recalibrating the pool with a beta-transformed combination improved the Brier score from 0.0799 to 0.0781 on probability-of-precipitation forecasts for 29 US cities, against 0.0816 for the best single forecast. On daily S&P 500 returns, a pool of two models beat the better of them by 37 log-score points over 7,324 days (Geweke and Amisano, 2011), but a single model that combines both components’ structure scored slightly better than pooling them (Gneiting and Ranjan, 2013).

Weights that change over time. Dynamic model averaging lets the weights evolve with recent performance (Raftery et al., 2010). The evidence is mixed. In Raftery et al.’s application the gain came only in an early, unstable period (mean squared error 68.9 against 77.5 for the best single model). For quarterly US inflation, dynamic averaging and selection beat static Bayesian averaging (eight-quarter error 22.09 against 29.99) (Koop and Korobilis, 2012). Heavily adaptive weights lost in Stock and Watson (2004). Elliott and Timmermann (2005) show that weights that switch with a hidden regime help only when the regimes persist; with frequent switching and small samples the simple average was best.

Weights that depend on the state: gating. A *mixture of experts* uses a gating model that looks at the inputs and decides how much to trust each component (Jacobs et al., 1991; Jordan and Jacobs, 1994). In the original study, on a vowel-recognition task, gating sped up training but did not improve accuracy (Jacobs et al., 1991). On *cryptocurrency order-book* data, gated mixtures did better than adding the same order-book features linearly: for hourly Bitcoin volatility (Guo et al., 2018) and for intraday trading volume across two exchanges, where the gated mixture had lower root-mean-squared error than the ARMA-GARCH benchmarks but gradient-boosted trees had lower mean absolute error in several cases (Antulov-Fantulin et al., 2021).

Online weighting. Algorithms from online learning update the weights after every outcome and guarantee that, over time, the combination does nearly as well as the best component or the best fixed combination in hindsight (Littlestone and Warmuth, 1994; Freund and Schapire, 1997; Cesa-Bianchi and Lugosi, 2006). Applied to thirteen machine-learning models of monthly US stock returns, one such algorithm reached a Sharpe ratio of 2.77 against 2.74 for the best single model, with half its worst monthly loss (0.08 against 0.16); in the same study equal weights (2.56) beat fixed weights estimated on validation data (2.28) (Remlinger et al., 2023). That study uses monthly data.

Calibrating the result. On standard machine-learning benchmark data sets, Niculescu-Mizil and Caruana (2005) find that some learners produce distorted probabilities: support-vector machines and boosted trees push them towards the middle, naive Bayes towards 0 and 1, while neural networks, bagged trees and logistic regression are close to calibrated. A sigmoid correction (Platt, 2000) or isotonic regression (Zadrozny and Elkan, 2002) fitted on data not used for training repairs this; the sigmoid works better with fewer than a few hundred to a thousand calibration cases (Niculescu-Mizil and Caruana, 2005).

Implications for order-book meta-models. Read across from these other settings, the evidence points to an order of attack rather than a single method: start from an equal-weight or non-negatively constrained combination of the classical forecasts; move to stacking or a learner only with enough data and with out-of-sample first-stage predictions; recalibrate any pooled or learned probability before using it; and treat time-varying or state-dependent weights as hypotheses that must beat the simple average out of sample. No published study we found applies these methods to a combination of the classical order-book models of this part.

8.5.1 Attention as a combiner

A Transformer’s attention layer (Section 10) computes a weighted average of a set of items, with weights decided by comparing a *query* with a *key* for each item. That is exactly the shape of a forecast combination: the items are the component models, the values being averaged are their forecasts, and the weights say how much to trust each model. What changes from one design to another is what plays the query and what plays the keys—in other words, what the combiner looks at when it decides whom to trust. Figure 34 shows the structure. With M component models, model m ’s forecast $\hat{Y}_t^{(m)}$, and a weight $a_{m,t}$ for each,

$$\hat{Y}_t = \sum_{m=1}^M a_{m,t} \hat{Y}_t^{(m)}, \quad a_{m,t} = \frac{\exp(\text{score}(\text{query}_t, \text{key}_{m,t}))}{\sum_{m'=1}^M \exp(\text{score}(\text{query}_t, \text{key}_{m',t}))}, \quad (44)$$

In words: each model gets a score for how well its key matches the current query; the scores are turned into weights that are positive and sum to one; the combined forecast is the weighted average of the models’ forecasts. Because the query changes with t , so do the weights.

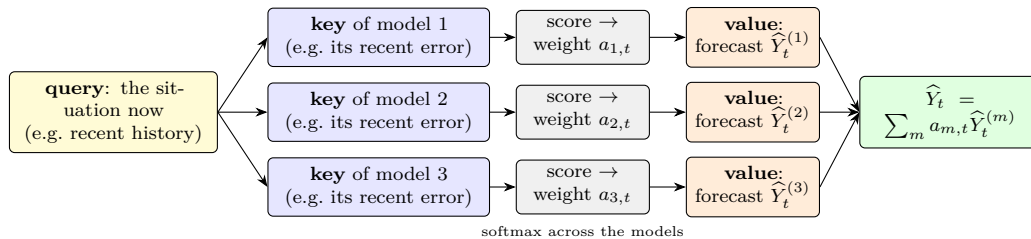


Figure 34: Attention used to combine forecasts. The query describes the current situation, each model has a key, the match between them sets the model’s weight, and the combined forecast is the weighted average of the models’ forecasts (the values). Schematic.

Attention targets in published combiners. We found three that we could read in full, all preprints; Table 28 compares them.

- **Which model suits this series now.** In Bourgin et al. (2021) the query is a recurrent network’s summary of the series so far, and each model’s key is a learned vector that comes to represent the situations in which that model is most useful. The weights therefore change from series to series and from step to step. The component models are trained together with the combiner, as in a mixture of experts, rather than fitted first. The data are synthetic; the combiner was more robust to a changing data-generating process than the simple average and linear stacking, but the results are given only as figures.
- **Recent errors as evidence of who will be right next.** In Patel and Wikner (2023) the component models are fixed in advance. The query is the most recent observed values, each model’s key is its most recent forecast errors, and the values are the models’ current forecasts; the weights are trained to make the next combined forecast as accurate as possible. On a chaotic test system (Lorenz-63) whose parameters drift, it kept forecasts valid for a median of 2.60 time units against 0.20 for a linear-regression combination, but only when the query covered the last 5 steps rather than 1 (0.30). On weekly COVID-19 deaths it did not consistently beat the existing ensemble of the COVID-19 Forecast Hub, which pools many more models, and linear regression was best in one of the four test periods.
- **Which sub-network to use.** In Chen et al. (2025) the “models” are sub-networks of one Transformer that read the same input at different resolutions, and self-attention runs across their forecasts, blended with weights updated online from recent errors. On cloud-workload data it reports 13.9% lower squared error than the best competing method, but no comparison isolates what the attention adds.

Weights from past performance versus weights from the current state. A combiner can give a model a high weight because it *has been* right lately, or because it is expected to *be* right next. The two coincide when accuracy persists and diverge when it does not—when the model that did best in one regime is the wrong choice in the next. The distinction is subtle for learned combiners, because every one of them is trained to make the *next* combined forecast as accurate as possible:

Table 28: Attention-based forecast combiners read in full. None uses financial or order-book data.

Study	Query	Keys	Components	Result
Bourgin et al. (2021)	summary of the series so far	one learned vector per model	5 LSTMs, trained jointly with the combiner	synthetic data; beat average and linear stacking (figures only)
Patel and Wikner (2023)	recent observed values	each model's recent error	fixed in advance	beat linear regression on Lorenz-63 with a 5-step query; not the Hub ensemble on COVID-19
Chen et al. (2025)	self-attention across the forecasts	same	sub-networks of one Transformer	13.9% lower error than best baseline; attention not isolated

Table 29: What combination weights are based on. Rule-based weights assume that past accuracy carries forward; learned weights are trained to predict the next outcome from whatever evidence they are given.

Method	Evidence the weights use	What a high weight means
Inverse past error (Bates and Granger, 1969)	realised past errors, through a fixed rule	has been right
Dynamic model averaging; online weighting	realised recent errors, through an update rule	has been right lately
Gating, mixture of experts	the current inputs	expected to be right in this situation
Bourgin et al. (2021)	a summary of the series so far; no information on past accuracy	expected to suit this situation
Patel and Wikner (2023)	recent true values and each model's recent errors	expected to be right next, judged partly from recent errors
Chen et al. (2025)	attention over the forecasts, blended with weights updated from recent errors	both

its weights are always, by construction, a forecast of who will be right next. What differs is the evidence the weights are computed from (Table 29).

In [Patel and Wikner \(2023\)](#), recent errors are inputs, not a rule: the network learns, by minimising next-step error, how to turn recent states and errors into weights, and could in principle learn that a recently accurate model is about to fail. On the Lorenz system there is some sign of forward-looking behaviour: in forecasts many steps ahead, after the true values stop arriving, the most heavily weighted model is the one whose parameter matches the true, drifting parameter, which the authors describe as anticipating the change. But the authors also state that their method assumes the biases of the component models change little over time, and give the continual updating of the COVID-19 Forecast Hub models as one reason it did not beat that ensemble.

Tests of whether attention weights anticipate or follow performance. None of the three studies includes a case in which recent accuracy is misleading—for example, one in which the recently best model tends to do worse next. That is the experiment that would separate a combiner that anticipates which model will be right from one that simply follows the recent winner, and it has not been done. For order-book data, where a model’s accuracy in one regime may say little about the next, it is the test that matters.

Known only from abstracts. One journal paper, [Yao and Koprinska \(2027\)](#), describes attention that computes ensemble weights from each member’s past and predicted future performance, on seven time-series data sets; [Chen et al. \(2024\)](#) combine an LSTM and a Prophet model with additive attention for electricity load and report beating Ridge and neural-network combiners. We could not read their full texts, so their designs and numbers are not verified here.

Models mistaken for attention combiners. Three groups of models are easily mistaken for one. In weather post-processing, attention runs across the members of a single model’s ensemble of forecasts, to correct them or produce a distribution, not across different models ([Finn, 2021](#); [Ben Bouallègue et al., 2024](#)). In mixture-of-experts forecasters such as Time-MoE ([Shi et al., 2025](#)), a router chooses internal layers of one network, not other models’ forecasts. And several ensembles use attention *inside* their component models while combining them with a fully connected layer or boosted trees ([Bui, 2025](#)).

Implications for order-book meta-models. Attention adds one idea to the combiners above: the weights can depend on the current situation, described however the designer chooses—the recent history, or each model’s recent errors. For order-book meta-models this is the gating idea of the previous paragraphs in another form. The evidence for it is thin: three preprints, none on financial data, none compared with FFORMA, and the one comparison against a strong existing ensemble did not show a consistent gain. Like any learned combiner, it is a candidate that must beat the simple average out of sample.

8.6 Published evidence

The published work does not use the word “meta-model”, but a sizeable body of it feeds engineered or classical-model inputs to a learner. Five findings stand out, each resting on a small number of studies, most of them on a few stocks over short periods.

1. Engineered inputs beat raw inputs when the learner is held fixed. Every head-to-head comparison we could verify points the same way. Holding the network architecture fixed and changing only the input, [Lucchese et al. \(2024\)](#) find that order-flow inputs place a stock-specific model among the statistically best models in 89% of stock-horizon cases, against 11% for the raw ten-level book (model confidence sets at the 99% level, ten Nasdaq stocks). [Tsantekidis et al. \(2020\)](#) replace raw prices by stationary features—prices relative to the mid, recent mid changes, cumulative depth—and every network improves: an LSTM’s F1 score at a 100-event horizon rises from 0.34 to 0.44, an MLP’s from 0.26 to 0.39 (five Finnish stocks, ten days). The Tampere group’s studies of hand-crafted features reach the same conclusion against learned representations: a multilayer perceptron on windows of the hand-crafted FI-2010 features scores an F-measure of 56 against 30 on autoencoder features ([Nousi et al., 2019](#)); hand-crafted econometric, technical and quantitative feature sets beat features learned by an LSTM autoencoder, even with shallow networks ([Ntakaris et al., 2019](#)); and, from a pool of 273 hand-crafted features, the authors report that a combination of only a few can reach the best performance ([Ntakaris et al., 2020](#)). That last study is also an early instance of stacking in the strict sense: one of its features is itself the output of an online logistic regression predicting a change in the best quote, and it is the feature most often selected first. These results come from short samples, and the learned representations they lose to are autoencoders rather than end-to-end networks such as DeepLOB; the direction of the finding is consistent, its size is not established. Two studies we know only from their abstracts point the same way: [Tashiro et al. \(2019\)](#) feed a convolutional network encodings of individual orders rather than book snapshots and report that smoothing filters on those encodings improve accuracy; [Palguna and Pollak \(2016\)](#) build mid-price predictors from features of the current and recent book and, inside an order-execution task on five months of Nasdaq data, report a trading-cost improvement “on the order of one basis point” for the two best features.

2. With good inputs, the choice of learner matters little. Two large studies that use classical microstructure measures as inputs find almost no difference between learners. [Äit-Sahalia et al. \(2026\)](#), on 101 S&P 100 stocks over 2019–2020 with 117 inputs built from trades and quotes—book imbalance, signed volume, past returns, Kyle-type price-impact ratios and durations—find that lasso, ridge, random forests, gradient-boosted trees and feed-forward neural networks with one or two hidden layers “have very similar performance”, with only unpenalised least squares failing through overfitting. They report a median out-of-sample R^2 of about 10% for five-second returns and a directional accuracy of about 64%, most of it from the last ten milliseconds or ten trades. Notably, inputs derived from transactions were more useful than inputs derived from the state of the book. [Easley et al. \(2021\)](#) feed six classical measures—the Roll measure and Roll impact, Kyle’s lambda, the Amihud ratio, a volatility index and VPIN—for 87 futures contracts to a random forest, and predict the direction of changes in spreads, volatility and higher moments a week ahead. Out-of-sample accuracy reaches 0.54–0.61; logistic regression on the same inputs is “typically . . . slightly more accurate” than the forest; and some measures with high in-sample importance have little out-of-sample predictive power, and the reverse. Both studies use time-aware protocols: rolling five-day training windows with a separate tuning block in the first, purged cross-validation in the second. A recent preprint finds the same inputs ranked the same way across assets: [Bieganowski and Ślepaczuk \(2026\)](#) fit gradient-boosted trees to engineered top-of-book, imbalance and VWAP-to-mid features of five cryptocurrency perpetual futures, one second apart, from 2022 to 2025, predicting the three-second return. The ranking of input importance, measured by SHAP values, was stable across the five assets, and the importance of the imbalance grew with the relative tick size.

3. Hawkes-process outputs as inputs. The cleanest evidence is [Muni Toke and Yoshida \(2020\)](#), who predict the sign of the next market order on 36 Paris-listed stocks. Their model combines the state of the book (imbalance, spread, last trade sign) with exponentially weighted counts of past market orders computed with the parameters of a fitted Hawkes model. The combination beats both the state-only and the Hawkes-only predictors: on average by a little over 2.5% on all trades, and by more than 13% over the best Hawkes model on the trades where the sign changes. The authors call it a theoretical exercise, since it ignores latency. Other studies use Hawkes models as one stage of a pipeline—predicting the time of the next event, which a second model then uses ([Cestari et al., 2025](#); [Raffaelli et al., 2026](#))—or as input features for a neural network ([Fabre and Challet, 2025](#)); we could not extract an ablation that isolates the value of the Hawkes inputs from them. A 2026 conference paper uses bursts in Hawkes intensity to decide when order-book predictions should not be acted on ([Yadav et al., 2026](#)); we could read only its title and publication details.

4. Mixing a classical model with a learned one. [Guo et al. \(2018\)](#) forecast hourly Bitcoin volatility with a learned gate that mixes an autoregressive volatility model with a model of order-book features. It had the lowest error in ten of twelve monthly test periods, against baselines including GARCH, random forests, gradient-boosted trees and LSTMs; adding the same order-book features linearly to the autoregressive model instead made it worse in all twelve. How the classical and learned parts are combined mattered as much as which inputs were used. For daily realised volatility, [Rahimikia and Poon \(2020\)](#) (a working paper; abstract only) fit machine-learning models to the variables of the HAR model—the standard linear model of realised volatility—together with order-book and news data. They report that high-dimensional models beat HAR in 90% of the out-of-sample period except in periods of extreme volatility, and that putting the machine-learning models into an ensemble with HAR improves it, while using them as direct substitutes calls for caution. The published companion paper ([Rahimikia and Poon, 2026](#)) keeps the model linear, adding order-book and news variables to HAR-type models, and reports that depth measures help more than slope measures and that the gains are largest on high-volatility days.

5. Stacking deep models on top of each other adds little. A stack of fifteen published deep LOB models, combined by a two-layer multilayer perceptron trained on their out-of-sample probability forecasts, never beat the best single model, and lost 5 to 6 F1 points to it on new Nasdaq data; the authors attribute this to the base models agreeing too often ([Prata et al., 2024](#)). Averaging models built on genuinely different inputs—individual messages and book snapshots—gained only half to three quarters of an F1 point over DeepLOB ([Zhang et al., 2021](#)). A two-stage design on Brazilian (B3) data, in which a convolutional network extracts features from the book and gradient-boosted trees combine them with trade records that include broker identities, reports accuracy 8% higher than the convolutional network alone, 5 points of it from the trees and 3 from the added trade information ([Bileki et al., 2022](#)); we read only the abstract, which does not say whether the 8% is absolute or relative. This is what the original stacking literature predicts: combining gains most when the combined models are dissimilar, and combination weights must be fitted on cross-validated rather than in-sample predictions, or they overfit ([Breiman, 1996](#)).

Gaps in the literature. We found no published study that feeds the outputs of the queue models of Section 6—depletion times, the up-move probability of the race, fill probabilities—into a learner, and no study that adds one family of classical inputs at a time and measures the economic value of each. Several papers also show the protocol problems that a meta-model invites: test metrics averaged over the last training epochs with no separate validation set ([Tsantekidis et al., 2020](#)), and

input smoothing written as a centred, and therefore forward-looking, filter (Wang, 2025). The case for meta-models is therefore plausible and partly supported—good inputs matter more than clever learners—but the specific design of Figure 32 remains to be tested.

8.7 Training protocols in the literature

A meta-model is fitted in two stages, and the published work agrees on one rule for the second stage: the learner must be trained on predictions of the first-stage models that were made out of sample. In Breiman’s words, if the combination is fitted on in-sample predictions of the first-stage models it “will overfit the data”, and it must be fitted on their cross-validated predictions instead (Breiman, 1996); the one measured example we found of what happens otherwise is described in Section 8.5. The studies of Section 8.6 implement this, and the time-ordering of market data, in different ways:

- Prata et al. (2024) train their stacked model on the base models’ predictions over the test period, so that its training data are out of sample for the base models, and evaluate it on a held-out part of that period;
- Easley et al. (2021) use ten-fold cross-validation with purging, removing about a week of observations around each test fold so that no overlapping information leaks into training;
- Aït-Sahalia et al. (2026) refit on a rolling window of the previous five days and predict the next day, re-tuning hyperparameters every twenty days on a separate block;
- Muni Toke and Yoshida (2020) calibrate on the previous trading day and predict the next.
- Bieganowski and Ślepaczuk (2026) use walk-forward validation with a gap between training and test data, and tune hyperparameters by an inner time-series cross-validation.

These are instances of the general requirements of Section 10.5—chronological splits, purging and embargo (López de Prado, 2018). The statistics literature on cross-validation for dependent data reaches the same place by another route. *h-block* cross-validation removes the *h* observations before and after each test observation from the training data (Burman et al., 1994), and Racine (2000) proposes *h_v-block* cross-validation; see also Bergmeir and Benítez (2012) on cross-validation for evaluating time-series predictors. On 62 real and 3 synthetic series, Cerqueira et al. (2020) find that cross-validation works for stationary series, but that when the series is non-stationary the most accurate estimates of performance come from out-of-sample methods that keep the observations in time order. A meta-model adds one more set of choices to count: every combination of inputs, windows and learners tried is a trial that must be counted (White, 2000; Harvey et al., 2016), and the final choice should be assessed out of sample. The question of which inputs earn their place is Principle 2 applied input by input; as noted in Section 8.6, no study we found reports that ablation in economic terms.

8.8 Summary

- **Definition of a meta-model.** A learner whose inputs are the outputs of the classical models—order-flow imbalance, queue imbalance and the micro-price, Hawkes intensities, queue-model probabilities—rather than the raw book: stacking, in machine-learning terms (Figure 32).
- **Inputs matter more than learners.** Engineered and order-flow inputs beat raw book inputs when the learner is held fixed, and with good inputs the choice of learner matters little: in the two largest studies a penalised linear or logistic model was as good as, or slightly better than,

forests, boosted trees and neural networks (Aït-Sahalia et al., 2026; Easley et al., 2021). The lasso is the natural first meta-model, because it also selects which inputs earn a place.

- **Combine dissimilar models.** State models measure how liquidity is arranged now; event models measure what is happening and why. Their information overlaps only partly, the little direct evidence suggests combining them helps (Muni Toke and Yoshida, 2020; Zhang et al., 2021), and the correlation of their errors can be measured before anything elaborate is built.
- **Combining forecasts: simple averages are hard to beat.** The general forecasting literature, mostly from macroeconomic and lower-frequency financial data, finds that simple averages are hard to beat, that learned combinations need out-of-sample component forecasts and plentiful data, that pooled probabilities must be recalibrated, and that time-varying or state-dependent weights help only under specific conditions. Neural-network combiners beat the simple average when trained across thousands of series, but not reliably on a few series, and the one tested on order-book data did not beat the best single model. A machine-learned combiner is a candidate, not a default.
- **Baselines.** The simple average of the same forecasts, the best single model chosen in advance, a naive forecast of the target, and the same execution policy without the forecast—all out of sample, scored properly, with every configuration counted.
- **Train the second stage on out-of-sample first-stage outputs.** Otherwise the combination learns the first stage’s overfitting (Breiman, 1996); the one measured example, from medicine, shows the apparent gain largely disappearing under proper validation (Tibshirani and Efron, 2002).
- **Gaps in the literature.** No published study feeds queue-model outputs into a learner, reports an input-by-input economic ablation, or applies these combination methods to the classical order-book models; the design of Figure 32 remains to be tested.

Model card: Meta-model (stacked model)

What you calibrate. First, each classical model (OFI slopes, micro-price tables, Hawkes parameters, queue-model rates); then the learner that combines their outputs, and a calibration map for its forecasts.

How. Train the learner only on out-of-sample predictions of the classical models, as stacking requires (Breiman, 1996); use chronological splits with purging (Section 10.5); count every input set and learner tried.

What you do with it. Produce a calibrated forecast—P(up), expected move, P(fill)—from many partial views of the book at once, for the execution decisions of Part III; measure, by ablation, which classical models add value.

How to picture it. A panel of specialists, each reading the book in its own way, and a chair who has learned from experience how much to trust each of them, and when.

9 Structural hybrids

Sections 8 and 9 both join classical models with machine learning, and it is easy to confuse them. This section starts by drawing the line precisely.

9.1 Structural hybrids versus meta-models

Meaning of “structural”. Every classical model in Sections 4–7 makes a specific claim about *how* the book works, written as a formula with a few parameters: a Hawkes model says that each event raises the rate of later events by an amount that decays with time; a queue-reactive model says that the rate of orders at a level depends on the sizes of the queues; the OFI regression says that the mid-price moves in proportion to net order flow. We call that claim—the form of the formula, as opposed to the values of its parameters—the model’s *structure*. A *structural hybrid* is a network whose architecture is forced to follow such a structure, with some part of it (a rate function, a decay kernel) learned from data instead of fixed in advance. The label is this survey’s; it is not a standard term. It should not be confused with “structural model” in economics, which means a model derived from the decisions of market participants.

Classical model outside or inside the learner. In a *meta-model* the classical models stay outside the learner: they are fitted first, kept fixed, and the learner sees only the numbers they produce. In a *structural hybrid* the classical model’s structure is built *into* the learner itself: the network is shaped like a Hawkes process or a queue model, it reads the raw events, and the whole thing is trained as one model.

Put differently, a meta-model asks “given what each classical model says, what should we forecast?”, while a structural hybrid asks “what if a network had the form of a classical model, but with some of its fixed formulas replaced by learned functions?”. Figure 35 shows the two side by side, and Table 30 compares them point by point.

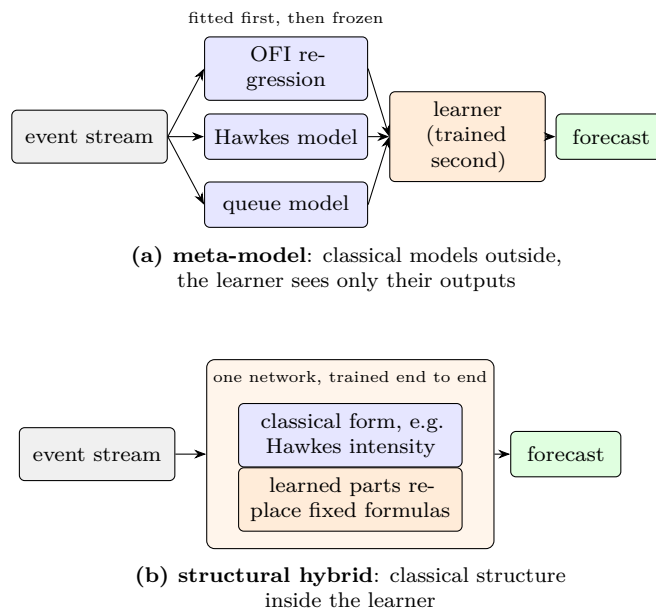


Figure 35: The difference between a meta-model and a structural hybrid. (a) Classical models are fitted separately and frozen; a second-stage learner combines their outputs. (b) A single network reads the events and has the form of a classical model, with some of its fixed formulas replaced by learned functions; everything is trained together. Schematic.

Designs that sit between the two sections. Two designs are neither clearly a meta-model nor clearly a structural hybrid; which one they are depends on what they are given and how

Table 30: Meta-models (Section 8) and structural hybrids (this section) compared.

	Meta-model	Structural hybrid
Where classical knowledge enters	as separate models whose outputs are inputs	as the shape of the network itself
What the learner reads	the classical models’ outputs (and possibly market state)	the raw events or book
How it is trained	in two stages: classical models first, then the learner	as one model, end to end
What is learned	how to weight and combine the classical outputs	the parts of the classical model that were fixed formulas, such as a kernel or a rate function
What stays interpretable	each classical model, unchanged	the overall form, e.g. “events excite events”
Main risk	leakage between stages; too many inputs	a flexible network inside the structure may drift from it
Examples	lasso or boosted trees over OFI, imbalance and Hawkes intensities (Section 8)	neural Hawkes processes, deep queue-reactive model, Hawkes-biased attention

they are trained. A network that reads the raw book *and* the classical outputs as extra inputs (“concatenation”, below) is half meta-model, half deep model: the classical models are still outside, but the network also learns from the raw data. And a *residual* or *gated* combination of a classical forecast and a network forecast (below) is a combination in the sense of Section 8.5 when the two are trained separately, and a hybrid when they are trained together so that the network learns only what the classical model misses. We describe both here because they mix classical and learned parts in one forecast, but the test for each is the same as in Section 8.5: does it beat the classical model and the network on their own?

Motivation for building structure in. The motivation is that a purely learned model must rediscover from noisy data facts that are already known: that events excite further events, that queues deplete and prices move when they do, that price impact is roughly linear in order flow over short windows. Supplying this structure should make a model more sample-efficient, more stable outside its training regime, and easier to interpret. These are hypotheses; the published evidence for them in LOB data is still thin, and each hybrid should be tested against its parts.

9.2 Two ways to build classical structure in

1. A classical skeleton with neural parts. Keep the classical model’s form and replace one of its fixed formulas by a network. A Hawkes process says that the rate of each event type depends on past events; a *neural* Hawkes process keeps that form but lets a recurrent network compute the rates from the history, instead of a sum of fixed kernels (Section 7.19). A queue-reactive model says that event rates depend on the state of the queues; a *deep* queue-reactive model keeps that form but computes the rates with neural networks (Bodor and Carlier, 2025). The output is still a point process—with intensities, a likelihood and a simulator—but more flexible than the classical one.

2. A neural architecture with a classical prior. Keep the network and bias it towards classical behaviour: a Hawkes kernel added to the attention scores, so that influence decays with elapsed time as excitation does; or a state-space model whose memory shortens when event intensity is high. The output is still a network forecast, but one that cannot ignore what is known about how

order flow behaves. The next three subsections each take apart a published model of the first kind; Section 9.6 then describes smaller designs, including the second kind.

9.3 A structural hybrid in detail: the deep queue-reactive model

This subsection takes one published structural hybrid apart step by step: the deep queue-reactive model of Bodor and Carlier (2025), built by two authors at BNP Paribas and Université Paris 1. We chose it because it starts from a classical model already covered in this survey—the queue-reactive simulator of Section 6.3—so each step shows exactly what is kept from the classical model, what is replaced by a network, and what that replacement buys. All numbers below are those reported in the paper.

Step 0: the classical queue-reactive model. In the queue-reactive model, a queue of size x receives limit orders, cancellations and market orders at rates $r_L(x)$, $r_C(x)$, $r_M(x)$ that depend *only* on x . The waiting time to the next event is exponential with the total rate, and the type of the event is drawn in proportion to its rate (Figure 11). For a recorded sequence of events $k = 1, \dots, N$, with type a_k , queue size x_k just before the event and waiting time Δt_k since the previous event, the likelihood of the rates is

$$\mathcal{L}_{\text{QR}} = \prod_{k=1}^N e^{-R(x_k) \Delta t_k} r_{a_k}(x_k), \quad R(x) = r_L(x) + r_C(x) + r_M(x). \quad (45)$$

In words: each factor is the probability that nothing happened for Δt_k seconds (the exponential term) and that what then happened was an event of type a_k (its rate). The best rates make the observed sequence as likely as possible.

Because the rates depend only on x , the best value of each can be found by counting:

$$\hat{r}_a(n) = \frac{\text{number of type-}a \text{ events seen at queue size } n}{\text{total time spent at queue size } n}. \quad (46)$$

In words: the rate of an event at queue size n is how often it happened there, per second spent there.

For example (illustrative numbers), if a queue spent 2.0 seconds in total at size $n = 3$, and during that time received 6 limit orders, 3 cancellations and 1 market order, then $\hat{r}_L(3) = 3$, $\hat{r}_C(3) = 1.5$ and $\hat{r}_M(3) = 0.5$ events per second. The whole fitted model is a table: one row per queue size, one column per event type, filled by counting.

This table is the formula that the hybrid will replace. It has two limits that Bodor and Carlier (2025) set out to remove. First, a rate can depend on nothing but the size of its own queue: not on the last event, not on the time of day, not on the opposite side of the book. Second, every queue is modelled separately from every other.

Step 1: keep the likelihood, replace the table by a network. The *deep* queue-reactive model keeps Equation (45) unchanged, but replaces the queue size x_k by a vector $\mathbf{y}_k$ of market features, and the table by a neural network with weights $\mathbf{W}$:

$$r_a(x_k) \longrightarrow r_a^{\mathbf{W}}(\mathbf{y}_k), \quad \text{loss}(\mathbf{W}) = \sum_{k=1}^N \left[R^{\mathbf{W}}(\mathbf{y}_k) \Delta t_k - \log r_{a_k}^{\mathbf{W}}(\mathbf{y}_k) \right]. \quad (47)$$

In words: the loss is minus the logarithm of the likelihood in Equation (45): it penalises predicting a high total rate during a long quiet spell (first term), and predicting a low rate for the event that actually happened (second term). Minimising it is maximum likelihood, exactly as before; only the thing being fitted has changed, from a table of numbers to the weights of a network.

Counting no longer works, because no two states $\mathbf{y}_k$ are exactly alike; the network is fitted by gradient descent instead. The network is a multi-layer perceptron with two hidden layers of 128 and 32 units and three outputs, one rate per event type; the outputs pass through a ReLU so that rates cannot be negative. With $\mathbf{y}_k = x_k$ alone, the deep model is a smoothed version of the classical table. The point is what *else* can now go into $\mathbf{y}_k$. The authors test two additions, one at a time:

- **The type of the previous event.** In the data, an event tends to be followed by another of the same type: after a cancellation, the next event is a cancellation 73% of the time; after a limit order, a limit order 76% of the time. The classical model and its size-aware extension cannot represent this, since their rates ignore history: their rows of transition probabilities are identical, with a cancellation following a cancellation 44% of the time. With the previous type added to $\mathbf{y}_k$, the deep model gives 66% and 74% (Figure 37a).
- **The hour of the day.** With the hour added, the fitted market-order rate varies through the trading day, as real activity does; the classical rates are the same at every hour.

Adding either feature raised the log-likelihood on validation data and the accuracy of predicting the next event’s type, and adding both gave the best values.

This is the structural part. The network does not decide *how* events are generated: that is still the queue-reactive mechanism—exponential waiting times, an event type drawn in proportion to its rate, queues updated by a matching engine. The network decides only *how fast* each event type arrives in a given state.

Step 2: one model for the whole book, with order sizes. The final model, which the authors call *multidimensional* (MDQR), makes two further changes. First, instead of one network per queue, a single network outputs the rate of every event type at every level at once: 3 types $\times$ 10 levels (five per side) = 30 rates, all computed from one shared state $\mathbf{y}_k$ that describes the whole book. An event is now a type a_k and a level ℓ_k , and the likelihood becomes

$$\prod_{k=1}^N e^{-R^{\mathbf{W}}(\mathbf{y}_k) \Delta t_k} r_{a_k, \ell_k}^{\mathbf{W}}(\mathbf{y}_k), \quad R^{\mathbf{W}}(\mathbf{y}) = \sum_{a \in \{L, C, M\}} \sum_{\ell} r_{a, \ell}^{\mathbf{W}}(\mathbf{y}). \quad (48)$$

In words: the same form as Equation (45), but the clock now runs for the whole book: the waiting time depends on the sum of all 30 rates, and the event picked is a (type, level) pair. Because all rates share one input, activity at the best bid can change the rates at the best ask.

Second, the size of each order is modelled too. Over 99.9% of order sizes in the data are below 200 lots, so the size is treated as one of 200 classes, and a second network with the same hidden layers outputs a probability $p^{\mathbf{W}}(v \mid a_k, \ell_k, \mathbf{y}_k)$ for each size $v = 1, \dots, 200$. It is trained by cross-entropy, the standard loss for classification:

$$\text{loss}_{\text{size}}(\mathbf{W}) = - \sum_{k=1}^N \log p^{\mathbf{W}}(v_k \mid a_k, \ell_k, \mathbf{y}_k), \quad (49)$$

In words: make the size that was actually observed, v_k , as probable as possible, given the type, level and state.

Because the likelihood splits into a timing-and-type part and a size part, the two networks are trained separately. Table 31 lists what goes into the state and how the networks are built.

Table 31: The state and the two networks of the multidimensional deep queue-reactive model, as reported by Bodor and Carlier (2025).

<i>State $\mathbf{y}_k$</i>	
Queue sizes at 5 levels per side	log-transformed
Spread	in ticks
Trade imbalance	(bid-side traded volume – ask-side) / (sum), over the last 20 s, 1 min, 5 min and 15 min
Type of the last event at each level	limit, cancel or market; learned 2-number embedding
Hour of the day	9 values; learned 2-number embedding
<i>Networks</i>	
Rate network	25 inputs; hidden layers of 256 and 64 units (tanh); 30 outputs (ReLU); loss Eq. (48)
Size network	27 inputs; hidden layers of 256 and 64 units (tanh); 200 outputs (softmax); loss Eq. (49)
Training	Adam optimiser, learning rate cycled between 10^{-5} and 10^{-3} ; stop after 10 epochs without validation improvement
<i>Data</i>	
Euro-Bund futures	June 2022 contract, March–June 2022, 9:00–18:00; 5 levels per side

Market generation by the fitted model. Figure 36 shows one simulation step. It is the loop of Figure 11 with the rate table replaced by the rate network and with a size drawn for each event. The grey boxes are the classical machinery, kept as it was; the orange boxes are learned.

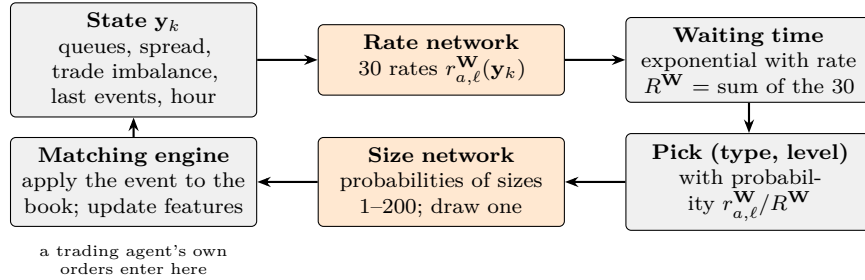


Figure 36: One step of the multidimensional deep queue-reactive simulator. Grey: the classical queue-reactive mechanism, unchanged (Figure 11). Orange: the two parts learned by networks. The paper reports 0.037 ms per generated event for the networks, excluding the matching engine.

Comparison of four models on Bund data. The authors compare four models on the Bund data: the classical queue-reactive model (QR), their earlier size-aware version (SAQR), and the deep versions. Figure 37 re-plots four of the comparisons.

- **Links between queues.** In the data, the volumes at the best bid and best ask have correlation -0.54 , and queues on the same side have correlations of about 0.3 to 0.5 . The classical model, whose queues are independent, gives -0.22 and near-zero same-side correlations; the multidimensional model gives -0.54 and same-side values close to the data (panel b).
- **Queue sizes.** Fitting a gamma distribution to the best-ask volume gives a shape of 1.35 in the data, 3.08 for QR, 1.91 for SAQR and 1.30 for MDQR (panel c).
- **Price impact.** A simulated agent buys the average five-minute volume over five minutes in 10 equal child orders. With the trade-imbalance features in the state, the simulated price rises in a

concave curve and then partly relaxes, the shape reported in empirical studies of large orders; without them, it rises linearly and stays flat. Varying the size of the order, the peak impact grows as the size to the power 0.55 ($R^2 = 0.89$), close to the square-root law.

- **Variability.** The classical models reproduce the average numbers of events per five minutes, but their spread from one window to the next is much smaller than in the data; MDQR’s is closer.
- **Direction of the mid-price.** Starting from each observed book, the simulators generate 500 events, and the direction of the simulated mid is compared with the real one (three classes: up, down, flat). Balanced accuracy is 0.54 for DeepLOB trained directly on the task, 0.56 for QR, 0.58 for SAQR and 0.63 for MDQR (panel d), against 1/3 for guessing. Prediction is not what these simulators were built for.

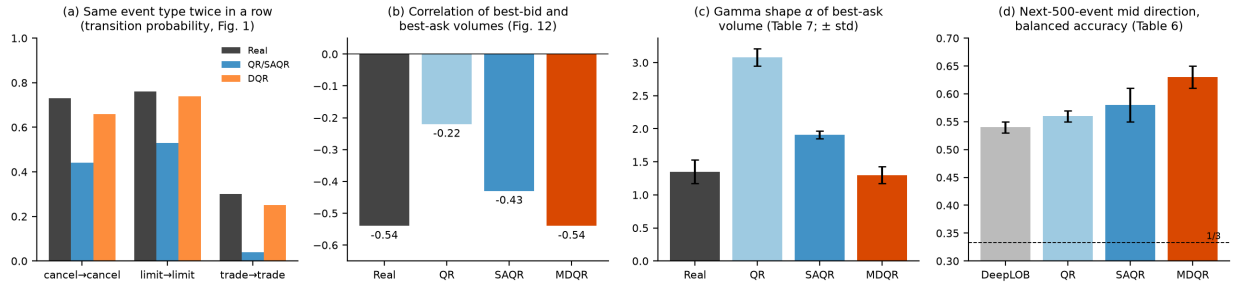


Figure 37: Results reported by Bodor and Carlier (2025) for Euro-Bund futures, re-plotted from the paper’s figures and tables; not our data. (a) Probability that an event is followed by one of the same type: the classical models cannot represent this. (b) Correlation of best-bid and best-ask volumes. (c) Shape of a gamma distribution fitted to the best-ask volume. (d) Balanced accuracy of the direction of the mid over the next 500 events; dashed line: three-class guessing.

Classification as a structural hybrid. Run through Table 30. The classical knowledge enters as the shape of the model—exponential waiting times, rates that depend on the book, events applied to queues by a matching engine—not as the output of a separately fitted QR model. The network reads the book’s state, not the forecasts of other models. The rate network is fitted by the same likelihood as the classical model, in one stage. What it learns is exactly the part that was a fixed table: the rates. And what stays interpretable is the mechanism: one can still ask the fitted model “how fast do cancellations arrive at the best ask when the bid queue is short?” and read off the answer. A meta-model built from the same ingredients would instead fit QR, Hawkes and OFI models separately and train a learner on their predictions—and it would forecast, but it could not be run forward as a simulator.

Limitations. The authors state that the model has been tested on one market, the Euro-Bund future, a liquid large-tick contract, and not yet on small-tick assets. The state is designed by hand, and they report that it took several attempts before the trade-imbalance features that produce the realistic impact curve were added; the network learns the rates, not which inputs matter. The impact relaxation they obtain cannot be checked against real large orders, which they did not have. Two reporting details: the validation and test periods are described for DeepLOB but not separately for the simulators in the prediction comparison; and the time to simulate one trading day is given as 0.92 minutes in the paper’s table and as 1.95 minutes in its text (the table’s 1.95 belongs to another model).

9.4 A prediction example: the state-dependent parallel neural Hawkes process

The deep queue-reactive model was built to simulate. The model of [Shi and Cartlidge \(2022\)](#), from the University of Bristol, is the published structural hybrid we found that is evaluated most fully as a *predictor*: given the recent events in the book, it forecasts what the next event will be and when it will happen, and it is compared both with the classical model it is built from and with a plain network. Its target is the next *event*, not the next price move; we found no published structural hybrid evaluated on price direction that we could read in full. The paper also uses the model as a simulator, which we do not cover here.

Step 0: the classical model. Every event is put into one of four types: an order submitted on the bid side, a cancellation on the bid side, a submission on the ask side, a cancellation on the ask side. The book’s state is summarised by one number, the queue imbalance at the best quotes, $QI = (v^b - v^a)/(v^b + v^a)$ (Section 4), cut into three levels: bid-heavy, balanced, ask-heavy. The classical model is the state-dependent Hawkes process of [Morariu-Patrichi and Pakkanen \(2022\)](#) (Section 7.16): each type has an intensity that jumps after events and decays exponentially,

$$\lambda_k(t) = \mu_k + \sum_{t_h < t} \alpha_{k,h} e^{-\beta_k(t-t_h)}, \quad (50)$$

In words: a background rate plus a decaying bump for each past event; the bump sizes depend on the types of the events involved and on the state of the book.

and after each event the state moves to a new level with probabilities that depend on the event’s type.

Step 1: keep the intensities, compute them with a network. The hybrid replaces Equation (50) by the neural Hawkes process of [Mei and Eisner \(2017\)](#) (Section 7.19), and changes it in two ways. Recall how a neural Hawkes process works. At each event, a continuous-time LSTM reads the event and sets, for each of its memory cells, a starting value $c(t_n)$, a target $\bar{c}$ and a decay rate d . Between events each cell decays from its starting value towards its target,

$$c(t) = \bar{c} + (c(t_n) - \bar{c}) e^{-d(t-t_n)}, \quad t_n \leq t < t_{n+1}, \quad (51)$$

In words: the memory relaxes from where the last event left it towards a target, at a speed that the network also chooses at each event. This plays the role of a Hawkes bump decaying, except that the start, the end point and the speed are all computed by the network from the history.

The cells produce an output $h(t)$, and each intensity is $\lambda_k(t) = \text{softplus}$ of a linear function of $h(t)$, so it is always positive. The exponential decay of Hawkes is still there, but it acts on the network’s memory rather than directly on the intensity.

The two changes are:

- **Parallel units.** Instead of one LSTM for all four types, there are four, one per event type, each computing its own intensity. A shared linear layer mixes their states at each event, so the types can still influence each other. The authors report that this shared layer added about 2 percentage points of accuracy.
- **State dependence.** The imbalance level is embedded and fed in with each event, and, as in the classical model, a learned matrix gives the probability of each new state after each event type.

Step 2: turn intensities into predictions. From the four intensities, with total $\Lambda(t) = \sum_k \lambda_k(t)$, the probability density of the next event happening at time t is

$$p(t) = \Lambda(t) \exp\left(-\int_{t_n}^t \Lambda(s) ds\right), \quad (52)$$

In words: the chance that the next event happens at t is the rate at t times the chance that nothing has happened before t .

The predicted time is the average of this density. The predicted type is the type with the largest share $\lambda_k(t)/\Lambda(t)$ at the time the event actually happens, if that time is known; or the type with the largest share averaged over the density (52), if it is not.

Training. The network reads windows of 49 events and their states and is trained to make the 50th as likely as possible: the loss is the point-process likelihood of Section 7.12 plus the likelihood of the observed state changes. A cross-entropy term on the next type can be added; for this model the authors found it did not help, and the headline results are without it. Every layer is small: two layers of 16 units in each LSTM’s internal layers, two layers of 16 for the embeddings, one of 16 for the intensities. Training uses five days of LOBSTER data for each of three stocks (INTC, MSFT, JPM), about half a million events a day, giving about 4 million windows. Figure 38 shows the architecture.

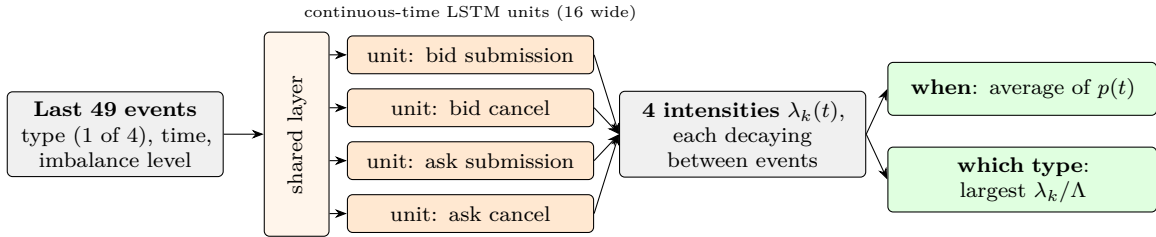


Figure 38: The state-dependent parallel neural Hawkes process of Shi and Cartlidge (2022), as used for prediction. Grey: inputs and the Hawkes-type intensities. Orange: the learned parts. Green: the two predictions. Schematic; the learned matrix of state transitions is not drawn.

The model ladder and its results. The paper compares a ladder of models in which one ingredient changes at a time (Table 32, Figure 39). All models receive the same events and states.

Table 32: The comparison of Shi and Cartlidge (2022), read as a ladder: what each model has, and its accuracy in predicting the type of the next event (% , time of the event given). Guessing among four types: 25%.

Model	What it has	MSFT	INTC	JPM
Hawkes	classical state-dependent Hawkes, exponential kernels; no network	42.67	40.10	48.20
LSTM	plain network; no intensities, so no time prediction or likelihood	47.64	46.04	58.05
CT-LSTM	neural Hawkes, one unit for all types	46.81	44.22	59.62
PCT-LSTM, no state	one unit per type; state not used	48.79	48.16	60.43
PCT-LSTM	one unit per type, with state	49.66	48.38	60.97

- **Against the classical model.** The full hybrid beats the state-dependent Hawkes model on every measure and every stock: for example 49.7% against 42.7% accuracy on MSFT, and a lower negative log-likelihood per event (-0.87 against 0.67) and smaller timing error.
- **Against a plain network.** It beats the plain LSTM, which has the same layer sizes, by 2.0 to 2.9 percentage points of accuracy. The LSTM cannot be compared on timing or likelihood, because it outputs only a type.
- **One unit or four.** A single neural Hawkes unit is *less* accurate than the plain LSTM on MSFT and INTC; splitting it into one unit per type moves the hybrid ahead. The Hawkes-style structure alone beat the plain network on only one of the three stocks; the per-type design beat it on all three.
- **The state.** Removing the imbalance state costs 0.2 to 0.9 points of accuracy.
- **What the model learned about the state.** The fitted transition matrix says that the imbalance level stays the same after more than 90% of events, and that its changes depend on the event’s side: after a bid submission, an ask-heavy book becomes balanced with probability 0.07 and a bid-heavy book with probability 0.02; after an ask submission, the other way round.

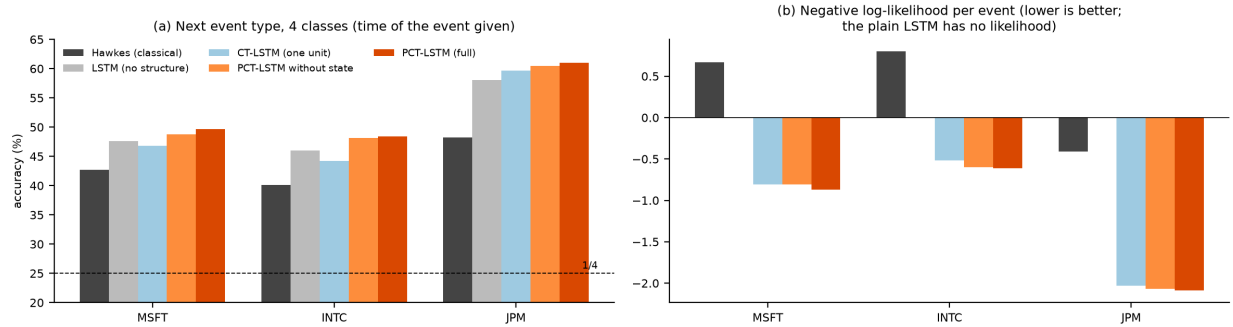


Figure 39: Results reported by Shi and Cartlidge (2022), Table 1, re-plotted; not our data. (a) Accuracy of predicting the type of the next event, with the time of the event given; dashed line: guessing among four types. (b) Negative log-likelihood per event; the plain LSTM has no likelihood.

An independent re-run. Zheng et al. (2024b) re-ran these models with the same layer sizes, five runs each, on six data sets: two days of MSFT from LOBSTER and four from the Shenzhen and Shanghai exchanges. The parallel neural Hawkes model beat the plain LSTM in type accuracy on five of the six (for example 0.47 against 0.37 on one MSFT day) and lost on one (0.38 against 0.41); it had a smaller timing error than the classical Hawkes model on five of the six and a larger one on the sixth (2.59 against 2.10).

Classification as a structural hybrid. It is one model, trained on the Hawkes likelihood, whose intensities follow the Hawkes form—jumps at events, exponential decay between them—with the fixed kernels replaced by a network. Because the output is a set of intensities, it predicts *when* as well as *what*, and it can be run forward as a simulator; the plain LSTM can do neither.

Limitations. The target is the type of the next event among four—submission or cancellation, bid or ask—not the direction of the price; market orders are not a separate type. The paper does not give the dates of the data, how they were split into training and test sets, the imbalance cut-off

for the three states, the number of parameters of each model, or any spread across repeated runs, so the 2–3 point margin over the plain LSTM cannot be judged for significance from the paper alone; the independent re-run above shows the ordering is not fixed. The paper reports two accuracies per model without stating which is which; the order in the text suggests that the first is with the time of the event given, and we use that reading. In the printed likelihood, the log-intensity term is indexed by the state where the event type is meant.

9.5 A third example: a neural Hawkes simulator for market making

The third published hybrid we take apart is the model of [Lalor and Swishchuk \(2025\)](#), from the University of Calgary. Its skeleton is a multivariate Hawkes process (Section 7); a recurrent network replaces the Hawkes kernels; and the fitted model is then used as a market in which a reinforcement-learning agent learns to make markets. Like the deep queue-reactive model, it is built mainly to *simulate*. All numbers below are those reported in the paper unless marked as our computation.

Step 0: the classical skeleton. Every event in the book is put into one of twelve types, according to what it is (limit order, market order, cancellation), its side, and whether it moves the mid-price (Table 33). Six types move the mid; six do not. In the data, the six that move the mid are rare: for the five stocks studied, they make up between 0.5% and 16% of all events.

Table 33: The twelve event types of [Lalor and Swishchuk \(2025\)](#). The superscript is the effect on the mid-price.

	moves the mid (aggressive)	leaves the mid unchanged
limit buy	LB ⁺ : posts above the best bid	LB ⁰
limit sell	LS [−] : posts below the best ask	LS ⁰
market buy	MB ⁺ : empties at least one ask level	MB ⁰
market sell	MS [−] : empties at least one bid level	MS ⁰
cancel buy	BC [−] : empties the best bid	BC ⁰
cancel sell	SC ⁺ : empties the best ask	SC ⁰

The classical model for these twelve event streams is a nonlinear multivariate Hawkes process: the rate of type- i events is

$$\lambda_i(t) = \varphi_i\left(\bar{\lambda}_i + \sum_{j=1}^{12} \sum_{t_{n'} < t, \text{ type } j} \alpha_{ij} e^{-\beta_{ij}(t-t_{n'})}\right), \quad (53)$$

In words: a background rate $\bar{\lambda}_i$ plus one decaying bump for every earlier event, whose height α_{ij} and decay speed β_{ij} depend on the pair of types; the function φ_i bends the total, for example to keep it positive.

The mid-price changes only when an aggressive event happens: up after LB⁺, MB⁺ or SC⁺, down after LS[−], MS[−] or BC[−], by a number of ticks drawn from the sizes of mid changes observed in the data for that stock. Equation (53) has 12×12 pairs of kernel parameters, each with a fixed exponential shape.

Step 1: replace the kernels by a continuous-time LSTM. The hybrid keeps the idea that each type has an intensity $\lambda_i(t)$ that reacts to past events, but replaces the sum of exponential bumps by the neural Hawkes process of [Mei and Eisner \(2017\)](#) (Section 7.19), in the version of [Shi and Cartlidge \(2022\)](#) that gives each event type its own recurrent unit (Section 9.4). At each event

the network reads two inputs: the type of the event, and a three-valued market state from the queue imbalance at the best quotes (bid-heavy, balanced, ask-heavy, with cut-offs at ± 0.4). Each memory cell of the network decays between events towards a target, as in Equation (39). The output of the unit for type i , $h_i(t)$, is turned into an intensity by the softplus:

$$\lambda_i(t) = \ln(1 + e^{h_i(t)}). \quad (54)$$

In words: whatever the network computes, the rate is positive. Because h_i can go down as well as up after an event, one event can make another type *less* likely—the inhibition that Equation (53) with positive α_{ij} cannot express.

The network is fitted to one day of free sample data from LOBSTER (21 June 2012; AAPL, AMZN, GOOG, INTC, MSFT), split 60/20/20 into training, validation and test, for 20 epochs on windows of 100 events, by maximising the point-process likelihood of Section 7.12.

Step 2: generate a market. The fitted intensities drive the same simulation loop as a classical Hawkes model (Section 7.12). The waiting time to the next event is exponential with rate $\Lambda(t) = \sum_i \lambda_i(t)$; the type is drawn with probability $\lambda_i(t)/\Lambda(t)$; if the type is aggressive, the mid moves:

$$m_{\text{after}} = m_{\text{before}} + \text{sgn}_i \cdot J^{\text{mid}}, \quad \text{sgn}_i = \begin{cases} +1 & i \in \{\text{LB}^+, \text{MB}^+, \text{SC}^+\} \\ -1 & i \in \{\text{LS}^-, \text{MS}^-, \text{BC}^-\} \\ 0 & \text{otherwise,} \end{cases} \quad (55)$$

where J^{mid} is a jump size drawn from the empirical distribution of mid changes for that stock.

In words: the network decides what happens next and when; the event type alone decides the direction of the price move; the data decide its size. The network has no output for price.

The new event is fed back to the network, which updates its memory, and the loop repeats.

Step 3: a market-making agent trained in the simulator. The simulated event stream becomes the environment of a reinforcement-learning agent. The agent’s state is the mid-price and its inventory Q_t ; at each step it chooses an action from $\{-1, 0, 1\}$, which the paper describes as choosing whether to post limit orders at the best bid and ask, with inventory capped at ± 5 . Its quotes can be filled only when a market order arrives in the event stream, under two rules:

- an aggressive market order (MB^+ , MS^-) always fills the agent’s quote on that side. The price then moves against the agent, so the paper calls these *adverse* fills;
- a non-aggressive market order (MB^0 , MS^0) fills it with probability 0.2, a value the authors had measured earlier on CME futures. It stands in for queue position: the agent is filled only if the market order reaches it in the queue.

The agent maximises final wealth minus a penalty on inventory,

$$\mathbb{E} \left[Q_T m_T + \text{Cash}_T - \varkappa \int_0^T |Q_t| dt \right], \quad \varkappa = 0.001, \quad (56)$$

In words: make money, but pay a small cost for every unit of inventory held for every unit of time, so that the agent prefers to stay close to flat.

It is trained with the Soft Actor-Critic algorithm on the first 5,000 events and tested on the next 2,500, once in the simulated market and once on the real event stream, for each stock. Figure 40 puts the three steps together.

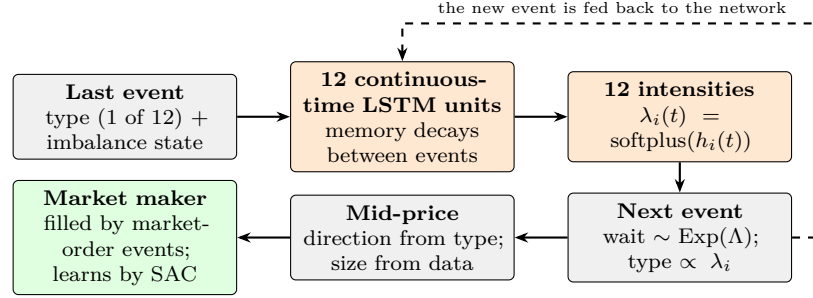


Figure 40: The pipeline of Lalor and Swishchuk (2025). Grey: the Hawkes skeleton (event types, thinning, mid moves by event type). Orange: the learned part, which replaces the sum of exponential kernels. Green: the trading agent. The simulated event stream is generated first; the agent’s quotes are filled by its market-order events but are not part of it.

Measured results. Figure 41 re-plots three comparisons.

- **Predicting the next event.** On the test data, the type of the next event is predicted correctly between 32% (GOOG) and 51% (MSFT) of the time, against 1/12 for guessing. As a rough reference we computed from the paper’s event counts how often always guessing the most frequent type would be right over the whole day: 21–26%. The paper does not fit a classical Hawkes model to compare with; it cites Shi and Cartlidge (2022) for neural Hawkes models beating classical ones (panel b).
- **The simulated price.** The volatility and excess kurtosis of simulated mid log-returns are close to the real ones. The Hurst exponent is not: the real values, 0.19–0.39, indicate a mean-reverting mid; the simulated values are higher, 0.33–0.65, and above 0.5 for four of the five stocks; skewness also differs (panel c). The authors conclude that the model matches the broad characteristics but not finer details such as trending versus mean-reverting behaviour.
- **The mix of events.** The authors note that the simulation has a higher proportion of aggressive events than the data. Computed from their tables, aggressive events are 10–16% of real events for AAPL, AMZN and GOOG and 20–32% of simulated ones; for INTC and MSFT, 0.5–0.6% of real events and 22–27% of simulated ones (panel a).
- **The market maker.** In both the simulated and the real tests, final rewards are mostly negative, most fills come from aggressive market orders, and the ratio of adverse to non-adverse fills is similar: for example 3.21 real and 3.09 simulated for AAPL, 1.95 and 1.61 for MSFT. The authors write that a strategy that only chooses when to quote at the best prices is unlikely to be profitable on its own, and that the point of the exercise is to show how the event-based model can be used.

Classification as a structural hybrid. The Hawkes skeleton is kept: twelve intensities, a likelihood fitted to event times, simulation by drawing the next event in proportion to the intensities, and a price that moves only through events of particular types. The learned part replaces exactly one formula: the sum of exponential kernels in Equation (53). There is no separately fitted Hawkes model whose output is fed to a learner, as there would be in a meta-model.

Limitations. The model is fitted and tested on a single day of data from 2012. No classical Hawkes model is fitted to the same data, so the paper does not show what the network adds over

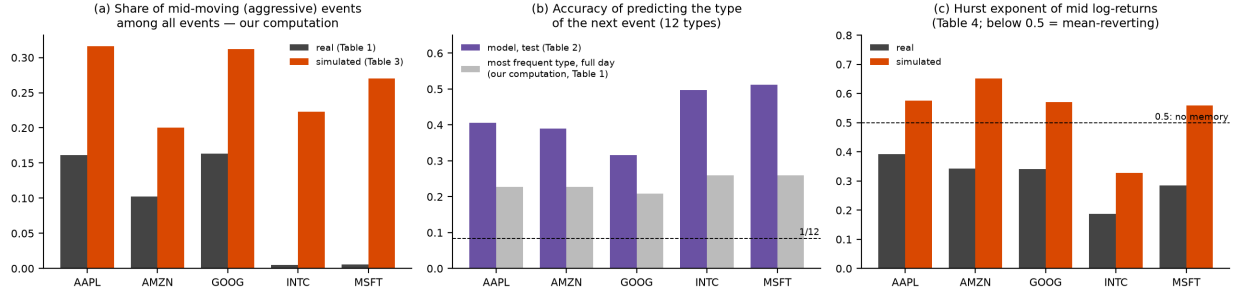


Figure 41: Results of Lalor and Swishchuk (2025), re-plotted; not our data. (a) Share of the six mid-moving event types among all events, real and simulated, which we computed from the paper’s event counts. (b) Test accuracy of predicting the next event’s type, beside the share of the most frequent type over the whole day (our computation; a rough reference, not the paper’s baseline). (c) Hurst exponent of mid log-returns, real and simulated.

Equation (53). The simulated market has many more mid-moving events than the real one, markedly so for INTC and MSFT; adverse fills come only from aggressive market orders, yet the adverse to non-adverse ratios in simulated and real tests are reported as similar, and the paper does not discuss how the two fit together. The non-adverse fill probability of 0.2 was measured on futures and is applied to stocks. The market maker’s results are reported as histograms, without summary numbers. Two printed equations are evidently typos: the cut-offs for the three imbalance states overlap, and the training loss is written with the sign of a log-likelihood and with the log-intensities of all twelve types instead of only the type that occurred.

The three examples side by side. Table 34 summarises what each of the three hybrids keeps and what it learns.

Table 34: The three structural hybrids reviewed in this section.

	Deep queue-reactive (§9.3)	Parallel neural Hawkes (§9.4)	Neural Hawkes market (§9.5)
Classical skeleton	queue-reactive model (§6.3)	state-dependent Hawkes (§7.16)	multivariate Hawkes (§7)
Fixed formula replaced	table of rates by queue size	sum of exponential kernels	sum of exponential kernels
What the network reads	hand-built state: queues, spread, trade imbalance, last events, hour	last 49 events, with an imbalance level	event history, with an imbalance level
Network	feed-forward, 2 hidden layers	4 continuous-time LSTM units	12 continuous-time LSTM units
Event types	limit, cancel, market $\times$ 10 levels	4: submit or cancel, bid or ask	12: by kind, side and effect on the mid
Data	Euro-Bund futures, 3 months	3 US stocks, 5 days each	5 US stocks, 1 day
Main use	realistic simulation	predicting the next event	environment for a market-making agent
Compared with the classical model	yes	yes	no
Compared with a plain network	on price direction only (DeepLOB)	yes (LSTM)	no

9.6 Smaller designs for combining classical structure with a network

The three models reviewed above are all of the first kind in Section 9.2: a whole classical model is kept, and one of its formulas is replaced by a network. This subsection describes four smaller designs, each of which can be added to an ordinary network rather than built around a classical model. Two of them sit between this section and Section 8: whether they count as a meta-model or a structural hybrid depends on how they are trained. The other two are of the second kind—a classical prior built into a neural architecture—and, as far as we found, none has been published for order-book data; they are included because they show concretely what “building structure in” can mean. Table 35 summarises them; each has a diagram below.

Table 35: The four designs of Section 9.6.

Design	What the classical part does	Status for order-book data
Concatenation	classical outputs (OFI, imbalance, Hawkes intensity) are extra inputs	published examples (Section 9.7); between Sections 8 and 9
Hawkes-biased attention	a Hawkes kernel scales attention weights by elapsed time	our proposal; a related model reported in abstract only
Intensity-conditioned memory	event intensity sets how fast a state-space model forgets	our proposal; none found
Gating and residuals	a classical forecast is mixed with, or corrected by, a network forecast	between forecast combination and a hybrid

Concatenation (between a meta-model and a hybrid). The simplest design appends the classical outputs of Table 24 to the network’s raw input, or projects them into the same embedding space as the learned features. With only those outputs as input it becomes the meta-model of Section 8; with the raw input as well, it tests whether the classical outputs add anything the network could not learn itself (Example 9.1).

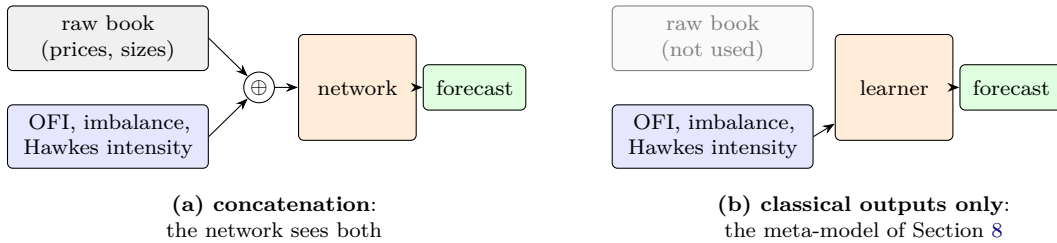


Figure 42: Concatenation. (a) The classical outputs ($\oplus$: placed side by side) are extra inputs next to the raw book. (b) Remove the raw book and the same design is a meta-model. Comparing (a) with a network on the raw book alone is the ablation of Example 9.1. Schematic.

Testing the benefit of concatenation. Adding classical outputs to a network’s input can only help if they tell the network something it could not work out from the raw input itself. A Hawkes intensity, for instance, is computed from the same events the network already reads; a large enough network could in principle compute it, or something as useful, on its own. The test is to train the same network twice, once without and once with the classical outputs, and compare the results.

Example 9.1 (The right ablation). Let S_t be a Hawkes intensity computed from the same events already supplied to a neural model $M_0(X_t)$. Compare $M_0(X_t)$ with $M_1(X_t, S_t)$, using identical data

splits, random seeds and economic evaluation, and report the difference in value ΔV (Section 1.2) with a confidence interval. We expect, but do not test here, that $\Delta V \approx 0$ when S_t is a fixed function of X_t and the network sees enough history to compute it. Concatenation earns its place when S_t brings information the network cannot reconstruct, or when it lets a much smaller network match a larger one.

Hawkes-biased attention. Attention, the building block of Transformer networks (described fully in Section 10), computes, for each event n , a weighted average over earlier events n' , with weights proportional to the exponential of a score. Each event is represented by learned vectors; the score compares a “query” vector of the current event with a “key” vector of each earlier one. A Hawkes kernel can be built into that score:

$$\text{score}(n, n') = \frac{\langle \mathbf{z}_n^{\text{qry}}, \mathbf{z}_{n'}^{\text{key}} \rangle}{\sqrt{\text{dim}_{\text{emb}}}} + \ln \phi(t_n - t_{n'}), \quad t_{n'} \leq t_n, \quad (57)$$

where $\mathbf{z}_n^{\text{qry}}$ and $\mathbf{z}_{n'}^{\text{key}}$ are the query and key vectors of the two events and dim_{emb} their dimension (Section 10 explains them with an order-book example).

In words: because the softmax exponentiates the score, adding $\ln \phi$ *multiplies* each attention weight by the Hawkes kernel: recent events get more weight, and the weight decays with elapsed time exactly as excitation does in a Hawkes process. With an exponential kernel $\phi(u) = \alpha e^{-\omega u}$ the added term is $\ln \alpha - \omega(t_n - t_{n'})$, a penalty linear in elapsed time.

The network learns what to attend to; the kernel imposes how quickly influence fades. Transformer Hawkes processes (Zuo et al., 2020; Zhang et al., 2020) encode time differently, through learned temporal encodings; the explicit kernel bias above is a design we propose for testing rather than a published LOB result. The closest published model we found, by Hwang et al. (2025), adds to the attention scores of a Transformer Hawkes process a bias whose size depends on the event’s type and decays exponentially with elapsed time, and reports results on financial event prediction; we could read only its abstract, so its data and results are not verified here. A minimal implementation:

```
def hawkes_biased_attention(E, t, Wq, Wk, Wv, log_alpha, omega):
    # E: one embedding row per event; t: event times (sorted)
    Q, K, V = E @ Wq, E @ Wk, E @ Wv
    scores = Q @ K.T / sqrt(Q.shape[1])
    dt = t[:, None] - t[None, :]      # time from event n' to event n
    scores = scores + (log_alpha - omega * dt) # log of exp. kernel
    scores[dt < 0] = -inf              # causal: no attention to later events
    return softmax(scores, axis=1) @ V
```

The added cost over standard attention is one subtraction per pair of events.

Figure 43 shows the effect on a single event. In panel (1), standard attention weights earlier events only by how well their content matches the current one, so a strong match long ago can receive the largest weight. Panel (2) is the kernel. In panel (3) the two are combined: old events are discounted, and among recent events the content match still decides which ones count most.

Intensity-conditioned state-space models. Selective state-space models such as Mamba (Gu and Dao, 2023) let the step size of their recurrence depend on the input, so that the model decides how quickly to forget. Making that step depend on the current event intensity or OFI would let the model’s memory shorten when the market is busy and lengthen when it is quiet—the same adaptation that event-time sampling achieves by construction. We know of no published LOB model of this kind.

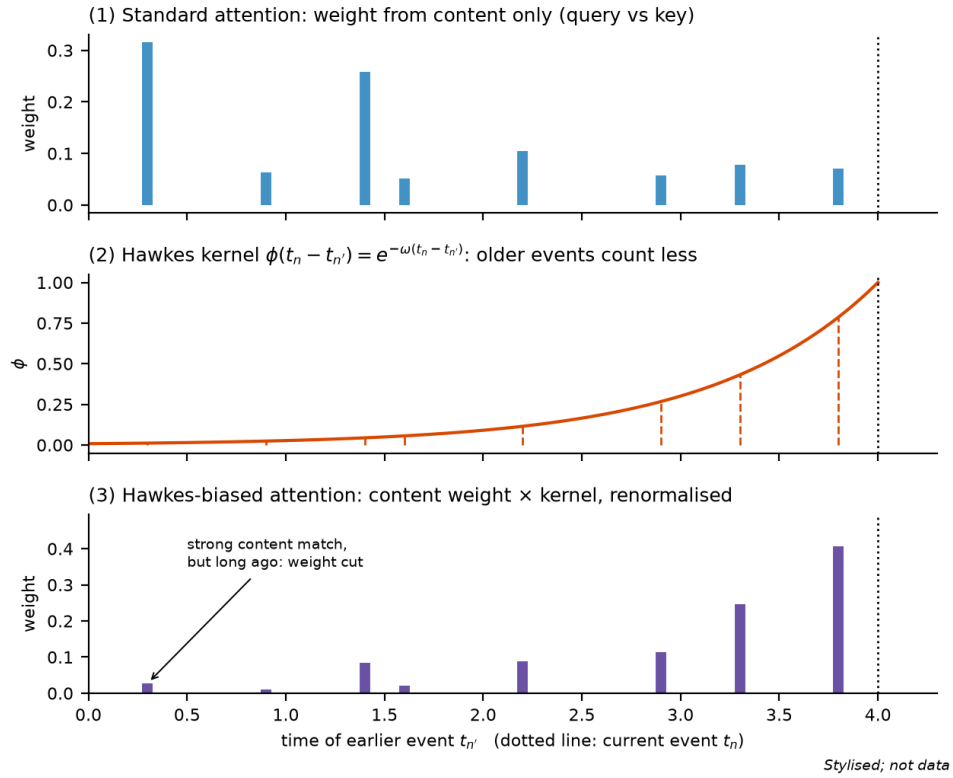


Figure 43: Hawkes-biased attention for one current event (dotted line) and eight earlier events. (1) Weights from content alone. (2) Exponential kernel, $\omega = 1.2$ per second. (3) Weights after the log kernel is added to the scores: each content weight is multiplied by the kernel and the result is renormalised to sum to one. Stylised; the content scores are invented for illustration.

Figure 44 illustrates the idea with memory that fades by a fixed amount per *event* rather than per second. Seen from a moment inside a burst, the model looks back only over the last fraction of a second, because many events have happened there; seen from a quiet moment, it looks back several seconds, but still gives little weight to anything before the burst, because many events separate it from the present. A model with a fixed memory in seconds (dashed) does neither.

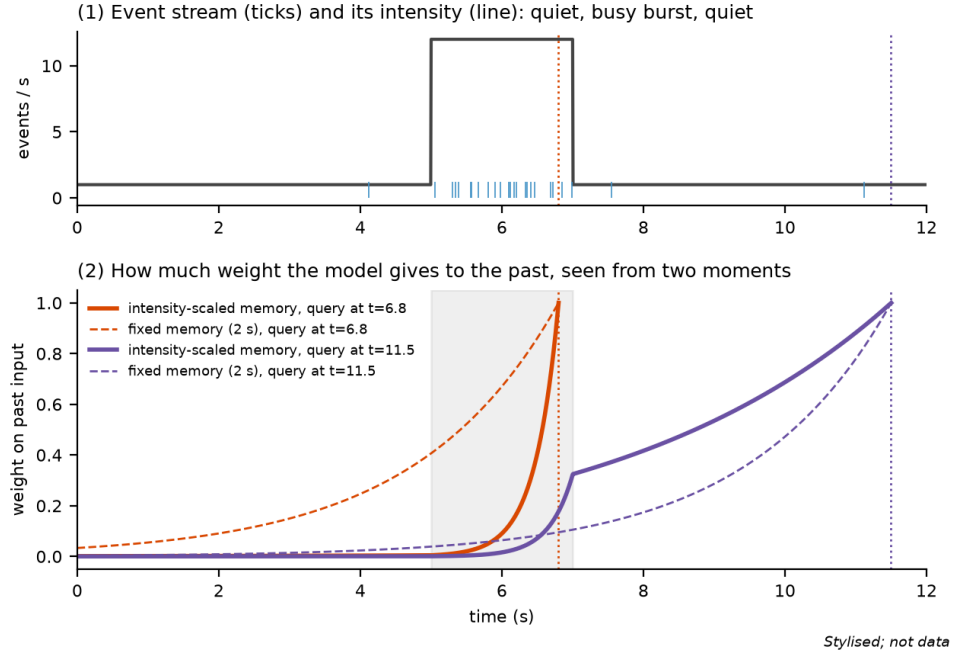


Figure 44: Memory scaled by event intensity versus fixed memory. (1) An event stream with a two-second burst (shaded below). (2) Weight that the model’s state gives to past inputs, seen from inside the burst ($t = 6.8$ s) and from a quiet period ($t = 11.5$ s). Solid: memory fades by a fixed factor per expected event, $e^{-\frac{1}{4} \times (\text{expected events since then})}$. Dashed: memory fades with a fixed time constant of 2 s. Stylised; not data. This is the proposed design, not a published model.

Residual gating (between forecast combination and a hybrid). A statistical model produces a baseline forecast $\hat{Y}_t^{\text{stat}}$, a network produces $\hat{Y}_t^{\text{deep}}$, and a gate $\nu_t \in [0, 1]$ —learned, or driven by volatility or intensity—mixes them:

$$\hat{Y}_t = \nu_t \hat{Y}_t^{\text{stat}} + (1 - \nu_t) \hat{Y}_t^{\text{deep}}. \quad (58)$$

In words: when the gate is near 1 the forecast falls back on the simple, well-understood model; the network contributes only where it has earned trust. Setting ν_t by volatility gives a natural safety layer: in unusual conditions, defer to the model with fewer parameters.

A related form trains the network on the residuals of the statistical model, so that it learns only what the simple model misses.

9.7 Other published hybrids and gaps in the literature

Sections 9.3–9.5 reviewed in full the three structural hybrids for order-book data that we could read and that report results on real data against at least some baselines. This subsection lists the other related work our search turned up, with how much of each we could verify, so that a reader can follow it up.

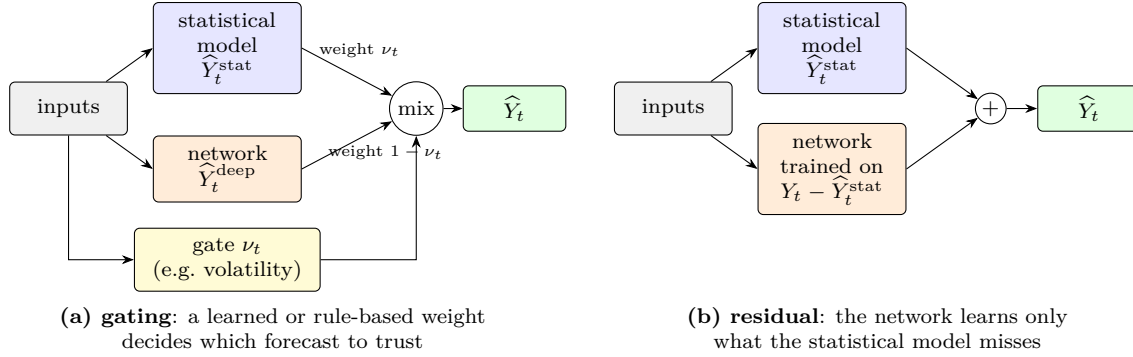


Figure 45: Two designs that sit between forecast combination and a structural hybrid. (a) A gate mixes a statistical forecast and a network forecast. (b) The network is trained on the error of the statistical model, and its output is added back. Trained separately, both are forecast combination (Section 8.5); trained together, they are hybrids. Schematic.

Other hybrids.

- [Gyotoku et al. \(2025\)](#) (preprint, read in full) take a classical model of the *ratios* between event intensities and let deep networks compute its factors, trained by the model’s own likelihood. On simulated data, imposing the model’s multiplicative structure gives better estimates than not imposing it. On real data (TotalEnergies on Euronext Paris, January 2017) the paper shows only how well the fitted model matches the data it was fitted to, with no test set and no baselines.
- [Hwang et al. \(2025\)](#) (abstract only) bias the attention of a Transformer Hawkes process with an exponentially decaying term (Section 9.6).
- [Chung et al. \(2024\)](#) (title and publication details only) propose a neural marked Hawkes process for the order book; we could not read its content.
- [Kumar \(2024\)](#) (abstract only) uses a deep Hawkes model as the environment for a market-making agent, as [Lalor and Swishchuk \(2025\)](#) do.
- UQ-LOB ([Edward Manoharan et al., 2026](#)) is a hybrid in a different sense: it attaches a probabilistic, attention-based neural-process layer to a pretrained encoder.

Close, but not structural hybrids. Three designs come near. [Fabre and Challet \(2025\)](#) compute Hawkes-style features with *fixed* decay rates and feed them to a network—concatenation, in the terms of Section 9.6. [Kolm et al. \(2023\)](#) feed order-flow imbalances to standard networks. [Lee \(2023\)](#) train a recurrent network to output the parameters of a Hawkes model, but for estimating those parameters quickly, not for forecasting.

Designs not found in the literature. We found no published model that builds order-flow imbalance, queue dynamics, the micro-price or a Kyle-type price-impact rule *inside* an end-to-end network to predict the order-book price. Our search covered arXiv, OpenAlex, Crossref and Semantic Scholar, but not Google Scholar or SSRN full texts, and two publishers’ pages could not be read, so this is the result of a limited search, not evidence that no such model exists.

Model card: Structural hybrids

What you calibrate. One model, trained end to end: the classical part’s parameters (for example a Hawkes decay) together with the network that replaces one of its fixed formulas (a rate function, a

kernel), or the network together with the classical prior built into it.

How. Maximum likelihood for hybrids that remain point processes; the usual supervised training for networks with a classical prior. Always alongside the ablation below, and against the classical model and the plain network on their own.

What you do with it. Simulate and forecast order flow with the structure of a classical model and the flexibility of a network—for example a queue-reactive simulator whose rates depend on many state variables.

How to picture it. A classical model with one or two of its fixed formulas swapped for learned ones; the rest of the machine is unchanged.

10 Deep learning for limit order books

Deep learning entered LOB research around 2017 and now dominates the volume of published work. This section first explains what a deep LOB model is (Section 10.1), what it takes in and predicts (Section 10.2), the families of network used and what each assumes (Section 10.3), and how a model is trained and tested, step by step (Section 10.4). It then reviews each family in the literature, what the large benchmark studies found when the models were re-tested, and which design choices matter, and closes with what deep learning adds to the classical models. Short definitions of the terms are in the glossary (Appendix B); textbook treatments are in [Goodfellow et al. \(2016\)](#).

10.1 Definition and design choices of a deep LOB model

Definition. A deep LOB model is a neural network $f_{\mathbf{W}}$, with weights $\mathbf{W}$, that takes a description of the recent book and returns a prediction:

$$\hat{y}_t = f_{\mathbf{W}}(X_t), \quad (59)$$

In words: X_t is what the model is shown at time t —for example the prices and sizes at the first ten levels over the last 100 events—and $\hat{y}_t$ is what it predicts, for example the probabilities that the mid-price—the average of the best bid and the best ask—will be lower, about the same or higher a few events later. The network is a long chain of simple operations (weighted sums followed by simple non-linear functions), and the weights $\mathbf{W}$, often tens of thousands of numbers or more, are what training sets.

The OFI regression of Section 4, $\Delta m = \beta \cdot \text{OFI}$, is also a function from the recent book to a prediction. There the modeller writes down the formula and fitting estimates a single number, β . For a deep model the modeller chooses only the inputs, the network architecture and the loss; training estimates all the weights, and the weights determine the shape of the function (Section 10.16).

Labels. Training needs examples with known answers. For each time t in the historical data, the label y_t is the correct answer for that example, computed from what actually happened after t . For direction prediction it is one of three classes: *down*, *stationary* or *up*. Example: if the mid-price 20 events after t is above the mid-price at t by more than a threshold, $y_t = \text{up}$. Training adjusts $\mathbf{W}$ so that $\hat{y}_t$ agrees with y_t on past data. Labels exist only for historical data; at prediction time the future is unknown. The rule that turns future prices into a label is a choice, and it changes results: Step 3 of Section 10.4 gives the formula, and Section 10.5 shows a label rule that leaks the past into the answer.

Scores. A trained model is scored on test data it has not seen. *Accuracy* is the share of test examples whose predicted class equals the label. Accuracy can mislead when one class is common: if 80% of labels are stationary, always predicting stationary scores 80%. The *F1 score* of one class, say up, combines two shares: of the examples predicted up, the share that were up (precision), and of the examples that were up, the share predicted up (recall). F1 is their harmonic mean, $2PR/(P + R)$, and is high only if both are high. *Macro-F1* is the plain average of the F1 scores of the three classes, so that each class counts equally however rare it is. Always predicting stationary gets macro-F1 of about 0.30 in the example above (F1 of 0.89 for stationary, 0 for up and down). Scores are reported as percentages (70%) or fractions (0.70). Details are in Section 10.5.

Design choices: input, target, network and training. Every model in this section is defined by four choices. Most papers differ from one another in only one or two of them, and the re-tests that taught the field the most changed exactly one while holding the others fixed. Each decision is described below with its main options, the choice made by DeepLOB, the most widely used reference model, and what the evidence says about how much the decision matters.

1. **Input.** The input can be *snapshots* of the book (the price and size at the first few levels on each side, at each of the last few dozen or hundred events), *order flow* (the changes in size at each level between snapshots), or the individual *messages*—each new order, cancellation and trade—as a sequence. A second choice is the window: how many events, or how many seconds, of history. DeepLOB sees the last 100 snapshots of ten levels, a 100×40 matrix (Zhang et al., 2019a). This decision matters a great deal: holding the DeepLOB network fixed and changing only the input, order-flow inputs put the model among the statistically best in 89% of stock-horizon cases, against 11% for the raw snapshots (Lucchese et al., 2024). Section 10.2 compares the three inputs and the classical models that read the same information.
2. **Target.** Most papers predict the *direction* of the mid-price after a horizon of H events—down, unchanged or up—which makes the task a three-class classification. Others predict a *return* (a number rather than a class), the type and time of the *next event*, the *time until a resting order is filled*, or the whole *future book*. Within direction prediction, the exact label matters: how the future price is averaged, from what reference it is measured, and the threshold below which a move counts as “unchanged”. DeepLOB predicts the direction at $H = 20, 50$ and 100 events, comparing an average of future mid-prices with an average of past ones. Changing only the label shows how much this matters: with that label DeepLOB is right 65.9% of the time on a year of Apple data, but with a label measured from the current price it is right 54.6% (Lee, 2026). Section 10.2 lists the targets and Section 10.5 explains the difference.
3. **Network.** The architecture decides which patterns the network finds easy to learn from limited data: a convolutional network looks for the same small pattern everywhere, a recurrent network reads the events in order with a memory, an attention network weighs the past by its content. DeepLOB combines convolutions, which read the book’s structure, with an LSTM, which reads the sequence. Of the four choices this is the one most papers vary, and it has so far mattered least: a plain multilayer perceptron matched a CNN-LSTM on the same inputs (Briola et al., 2020), Transformers have not beaten the DeepLOB family in head-to-head comparisons (Bilokon and Qiu, 2023; Zhong et al., 2025b), and adding attention to the best bilinear network gained 0.4 of a point (Shabani et al., 2022). Section 10.3 describes the families.
4. **Training and testing.** This covers how the inputs are normalised, how the labels are built, how the data are split into training, validation and test periods, which loss is minimised, when

training stops, and how the result is scored. DeepLOB normalises each day with the statistics of the previous five days, trains on six months, tunes on three and tests on the last three, minimises cross-entropy, and stops when validation accuracy stops improving (Zhang et al., 2019a). These choices move results by as much as the network does or more: learned input normalisation added seven to nine points of macro-F1 over plain standardisation (September et al., 2024), and on the FI-2010 data set the choice of split alone separates published scores of about 0.51–0.59 from about 0.76–0.88. Section 10.4 walks through the steps, and Section 10.5 shows what goes wrong when they are done badly.

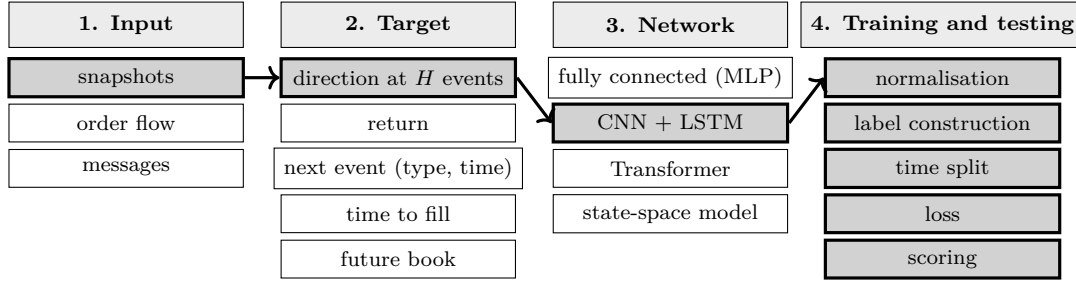


Figure 46: The four design choices of a deep LOB model. Columns 1–3: one option is chosen. Column 4: every step must be specified. Shaded with thick border: DeepLOB (snapshots; direction at 20, 50, 100 events; CNN + LSTM; previous-five-day normalisation, averaged label, 6/3/3-month split, cross-entropy, F1).

Figure 46 shows the options. Table 36 shows how five published models make the four choices. The last two rows are the re-tests mentioned above: each keeps DeepLOB’s network and changes a single decision.

Table 36: Five published models and their four design choices. The last two rows are re-tests that keep DeepLOB’s network and change one decision.

Model	1. Sees	2. Predicts	3. Network	4. Trained and tested
DeepLOB (Zhang et al., 2019a)	100 snapshots, 10 levels	direction at 20, 50, 100 events (averaged label)	convolutions, then LSTM	cross-entropy; 6/3/3 months, London stocks
TransLOB (Wallbridge, 2020)	100 snapshots, 10 levels (FI-2010)	direction (FI-2010 labels)	dilated convolutions, then Transformer	cross-entropy; 7 days train, 3 test
LOBERT (Linna et al., 2025)	512 messages, optionally with snapshots	next message; direction at 10, 50, 100 messages	Transformer encoder	pretrained by filling in hidden messages, then fine-tuned
Lucchese et al. (2024)	varied: snapshots, order flow, volume on a price grid	direction, 3 classes, 10 to 1,000 book updates ahead	DeepLOB, unchanged	10 Nasdaq stocks, 2019; models compared by model confidence sets
Lee (2026)	snapshots	varied: averaged label vs label from the current price	DeepLOB, unchanged	one year of Apple data

10.2 Inputs and outputs of a deep LOB model

This subsection covers what a deep LOB model is given and what it is asked to predict, in five parts: the three input formats (10.2.1); the classical models that use the same information as each format (10.2.2); a measured comparison of the formats (10.2.3); the prediction targets (10.2.4); and the FI-2010 data set on which many results are reported (10.2.5).

10.2.1 The three input formats

Figure 47 shows the three formats side by side.

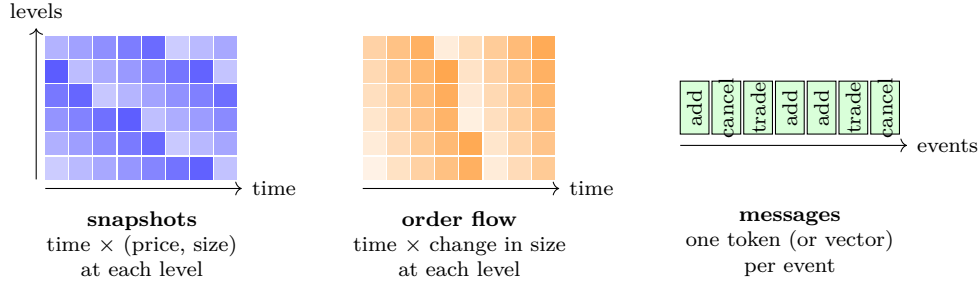


Figure 47: Three ways of presenting the book to a network: a window of snapshots of the first levels; the same window transformed into order flow (changes in size at each level); or the sequence of individual messages. Schematic.

Snapshots. After each event, the prices and sizes at the first ten levels on each side of the book. The model receives the last N snapshots as a grid of time by level; DeepLOB uses $N = 100$ (Figure 48).

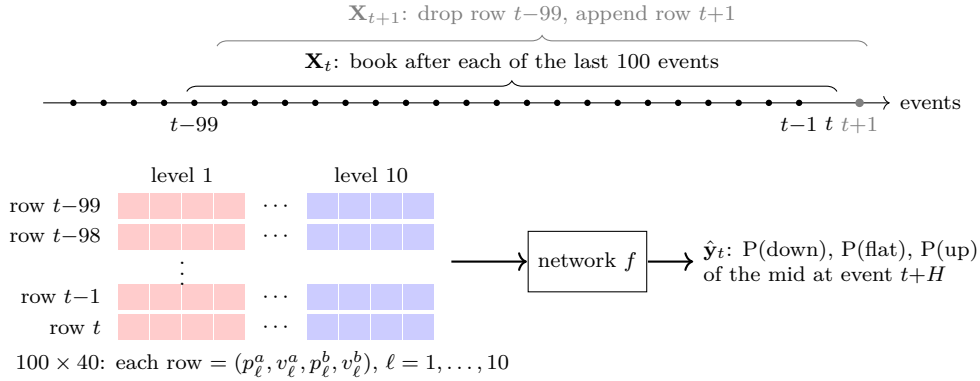


Figure 48: How a snapshot model looks back. After each event the first ten levels of the book are written as one row of 40 numbers. The input $\mathbf{X}_t$ stacks the rows for the last 100 events. One forecast is made per event; at the next event the window moves by one row. Schematic.

Order flow. The change in size at each level between consecutive snapshots, as in multi-level OFI (Section 4). A variant places volumes on a fixed price grid around the mid instead of on levels.

Messages. The individual exchange messages—new orders, cancellations, trades—one per event, as a sequence (Sections 5 and 10.8).

Information lost between formats. All three formats are computed from the same stream of exchange messages (Figure 49). Each step down the list below discards information.

- **Messages:** the raw events. Example: “cancel 5 lots at the best ask, at 09:30:01.000123”.

- **Order flow:** for each level, the net change in size over one step. Example: “best ask -5 ”. Lost: the event type (a 5-lot cancel and a 5-lot trade both give -5 ; Example 4.1) and the time within the step.
- **Snapshots:** the size resting at each level after the step. Example: “best ask 9 lots”. Subtracting two consecutive snapshots gives the order flow back, but not the event types.

Lost information cannot be recovered by going back up the list. In the terms of Section 8.4, a snapshot is a state (a stock); order flow and messages are flows.

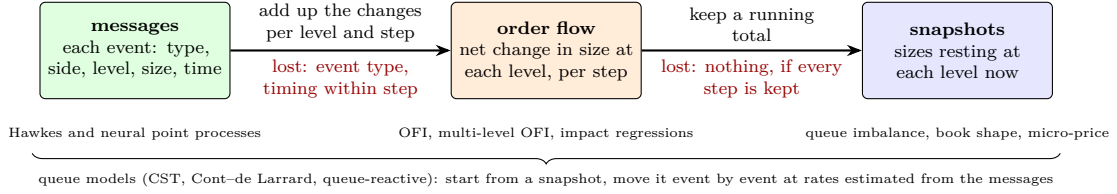


Figure 49: The three inputs of Figure 47 are one event stream summarised to different degrees. Summing messages into order flow loses the event types and their timing, which cannot be recovered. Below each input: the classical models of Sections 4–7 that read the same information. Schematic.

10.2.2 Inputs and the classical models that use the same information

Table 37 lists, for each input, the classical models of this part that use the same information. A network and a classical model that use the same input can be compared directly.

Table 37: Network inputs and the classical models that read the same information.

Input	Classical models	Why they match
Snapshots	queue imbalance (§4), book shape (§4.4), micro-price (§4.5)	each is a function of the sizes resting in the book at one moment
Order flow	OFI and multi-level OFI (§4), regressions of the price change on order flow	OFI <i>is</i> the net change in size at the best quotes; multi-level OFI is exactly this input
Messages	Hawkes and neural point processes (§7)	they model which event comes next and when, which only messages record
Snapshot + messages	queue models (§6)	queue size now, from a snapshot; arrival, cancel and trade rates, estimated from messages

Queue models use two inputs. Example: the expected time until the best ask is emptied needs (1) the number of lots at the best ask now, read from a snapshot, and (2) the rates of limit orders, cancellations and trades at the best ask, estimated from messages.

10.2.3 Measured effect of the input choice

A network given snapshots could in principle compute order flow itself, by subtracting consecutive snapshots. A direct test found that it does not do this well. Lucchese et al. (2024) kept the DeepLOB network fixed and changed only the input. A model counted as “among the best” if a model confidence set test (Section 10.5) could not show it to be worse than the best model. Over all stock–horizon cases, the order-flow input (L2) was among the best in 89% of cases; the raw snapshots (L2) in 11% (Figure 50; see also Section 8.6).

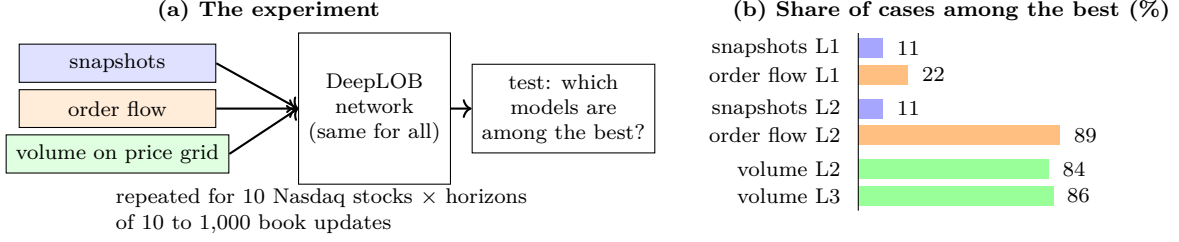


Figure 50: The input experiment of [Lucchese et al. \(2024\)](#). (a) Only the input changes; the network is fixed. “Among the best”: not shown to be worse than the best model by a model confidence set test at the 1% level. (b) Share of (stock, horizon) cases in which each input was among the best, stock-specific models (their Table 5). L1, L2, L3: the paper’s labels for variants of each input. Several inputs can be among the best in the same case, so the shares do not sum to 100%.

Our interpretation, not stated in the cited papers: the input decides which classical quantity the network receives ready-made. Given order flow, the network receives OFI directly. Given snapshots, it must learn to compute OFI from the training data (Figure 51).

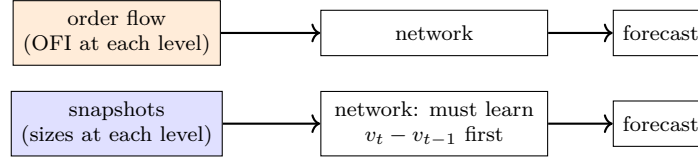


Figure 51: Our interpretation of Figure 50. Given order flow, the network receives OFI ready-made. Given snapshots, it must first learn to subtract consecutive sizes from the training data. Schematic.

10.2.4 Prediction targets

Most deep LOB papers solve one of six problems:

Direction classification

predict whether the mid will be up, down or stationary after a horizon H measured in events or seconds. Labels are usually built by comparing a smoothed future mid with the current one against a threshold (defined precisely in Section 10.4); the choice of smoothing and threshold changes the problem, and a recent paper shows that coupling the smoothing window to the horizon biases the labels ([Berti and Kasneci, 2025](#)).

Multi-horizon forecasting

predict returns (relative price changes), or their direction, over several horizons at once—an “alpha term structure”, the predicted return as a function of the horizon ([Kolm et al., 2023](#); [Zhang and Zohren, 2021](#)).

Next-event prediction

predict the type, side, price and size of the next message—the natural task for message-level and point-process models ([Linna et al., 2025](#)).

Full-book forecasting

predict the whole future book, prices and volumes at several levels ([Jung and Lee, 2025](#)).

Fill-time estimation

predict the distribution of the time until a resting order is filled (Section 10.14).

Generation

produce synthetic message streams or book snapshots (Section 10.15).

10.2.5 The FI-2010 data set

A large share of the literature reports results on FI-2010 (Ntakaris et al., 2018): five stocks from the Helsinki exchange over ten trading days in June 2010, about 4.5 million messages, ten levels, already normalised and downsampled into blocks of ten events. It made comparison possible, and it also made overfitting easy. The DeepLOB authors themselves describe it as “far too short, downsampled and taken from a less liquid market” (Zhang et al., 2019a)—a market with fewer orders and trades, where prices move in larger, rarer steps. Results on it should be read with the comparability fields of Section 2.4 in mind.

10.3 The families of network

Role of the architecture. All the families below are trained in the same way (Section 10.4). They differ in how the weights connect the input to the output. In principle a large enough network of any family can represent any rule from input to output. In practice the data are limited and noisy, and a family learns some rules from fewer examples than others, depending on how its weights are arranged. This built-in preference is called the family’s *inductive bias*. Example: a convolutional network uses the same weights at every price level, so a pattern learned at level 1 is also recognised at level 5 without separate examples; a fully connected network has separate weights for each level and must learn the pattern at each one. Table 38 lists the families used for order books. The figures below show the five main ones on the same small input: a window of 6 snapshots (time, left to right) of 4 numbers each (rows), drawn as a grid; a real model such as DeepLOB uses 100 snapshots of 40 numbers.

Fully connected network (multilayer perceptron, MLP). The grid is unrolled into one long list of $6 \times 4 = 24$ numbers. Each unit of the next layer computes a weighted sum of all 24, with its own 24 weights, followed by a simple non-linear function (Figure 52). Nothing is shared: the weight that links “size at level 1, three events ago” to a unit is unrelated to the weight for “size at level 1, two events ago”. The network has no notion of time or level; it must learn every relation from data.

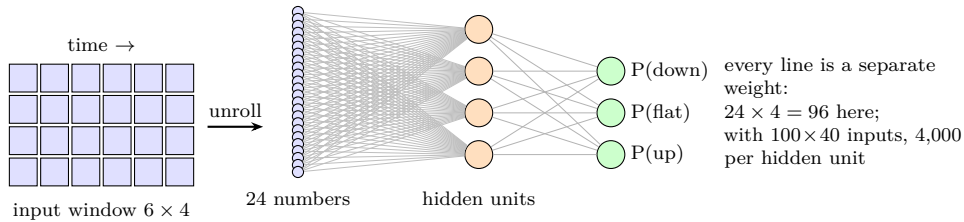


Figure 52: A fully connected network on a window of the book. The window is unrolled into a list, and every input is connected to every hidden unit by its own weight. Schematic.

Convolutional network (CNN). A *filter* is a small grid of weights, here 2×2 : two consecutive events by two adjacent rows. It is placed on the input, the four numbers under it are multiplied by its four weights and added, and the result is one number of the output. The filter is then moved one step and the calculation repeated, until it has visited every position (Figure 53). The same four weights are used everywhere: the network has 4 weights for this filter however large the input is,

Table 38: Families of neural network used for limit order books.

Family	What it assumes about the input	Order-book examples	Section
Multilayer perceptron (MLP)	nothing: the window is flattened into one long vector	baseline in most studies; matches a CNN–LSTM in one (Briola et al., 2020)	10.2, 10.16
Convolutional (CNN)	the same local pattern, in time and across nearby price levels, means the same thing wherever it occurs	Tsantekidis et al. (2017a); DeepLOB (Zhang et al., 2019a)	10.6
Recurrent (RNN, LSTM)	order in time matters; a running memory summarises the past	Tsantekidis et al. (2017b), Dixon (2018a); the last stage of DeepLOB	10.6
Bilinear (TABL, temporal attention-augmented bilinear layer)	the input is a matrix of time by feature; multiply it by one weight matrix on the left, mixing time steps, and one on the right, mixing features, with few weights	Tran et al. (2019)	10.6
Attention, Transformer	what matters is which earlier inputs (times, levels, assets) resemble the present	TransLOB (Wallbridge, 2020), Axial-LOB (Kisiel and Gorse, 2022), TLOB (Berti and Kasneci, 2025), LiT (Xiao et al., 2025)	10.7
State-space (S4, Mamba)	a running memory, updated at each step by learned matrices, that forgets at a learned rate; efficient on long sequences	no peer-reviewed forecaster with baselines found	10.9
Graph networks	levels or assets are nodes whose relations form a graph	HLOB (Briola et al., 2025b)	10.7, 10.11
Neural point processes	events happen at random times, with an intensity	Shi and Cartlidge (2022)	7.19
Encoders pretrained without labels	a network first trained on a task that needs no labels (Section 10.7.1); the internal representation it learns transfers to other tasks	SimLOB (Li et al., 2024); LOBench (Zhong et al., 2025b)	10.10
Generative (GAN, diffusion, autoregressive; Section 10.15)	the goal is to produce realistic order flow, not to predict one label	Coletta et al. (2022), Nagy et al. (2023), MarketGPT (Wheeler and Varner, 2024)	10.15

and a pattern that the filter detects (for example “the size at a level fell”) is detected wherever it occurs. A layer has many filters, each learning a different pattern. DeepLOB’s filters are described in Section 10.6.1.

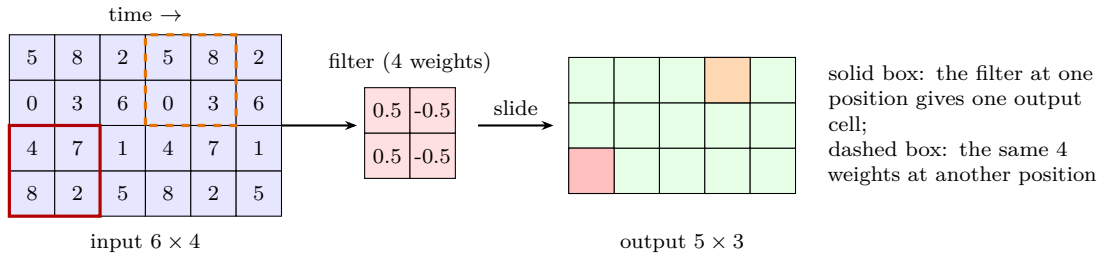


Figure 53: A convolution. One 2×2 filter of weights is applied at every position of the input; each position gives one number of the output. Numbers are illustrative. Schematic.

Recurrent network (RNN, LSTM). The network reads the window one snapshot at a time, in order. It keeps a *memory* vector $\mathbf{h}$. At each step one *cell* takes the new snapshot and the memory from the previous step and computes an updated memory (Figure 54). The cell has one set of weights, used at every step. After the last snapshot, the memory summarises the whole window and a final layer turns it into the prediction. An LSTM is a recurrent cell with gates that decide how much of the memory to keep and how much of the new snapshot to add (Section 10.3, building blocks).

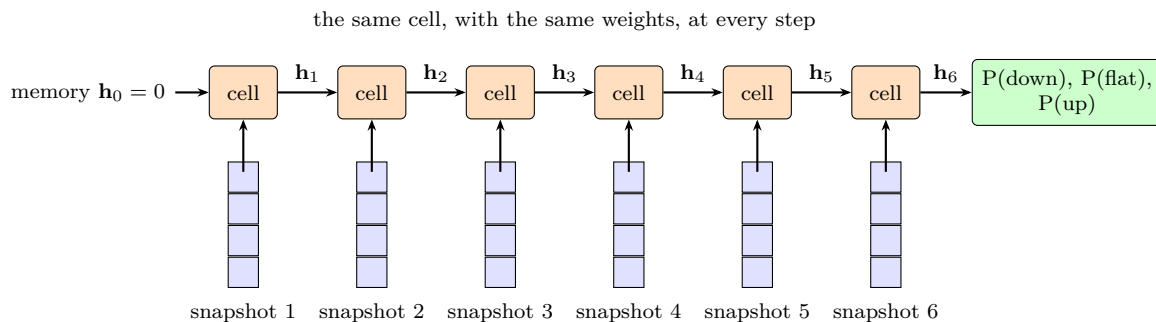


Figure 54: A recurrent network reads the snapshots in order. One cell, with one set of weights, updates a memory vector at each step; the memory after the last snapshot is used for the prediction. Schematic.

Bilinear layer. The window is kept as a matrix $\mathbf{X}$ with one row per snapshot and one column per number. A bilinear layer multiplies it by two weight matrices: $\mathbf{W}_1$ on the left combines snapshots (each output row is a weighted mix of time steps), and $\mathbf{W}_2$ on the right combines numbers within a snapshot (each output column is a weighted mix of features):

$$\mathbf{Y} = \mathbf{W}_1 \mathbf{X} \mathbf{W}_2. \quad (60)$$

In words: first mix over time, then mix over features, with one small matrix for each.

Because time and features are mixed separately, the layer needs far fewer weights than a fully connected layer with the same input and output sizes (Figure 55). TABL (Tran et al., 2019) adds an attention weighting over time steps to this layer.

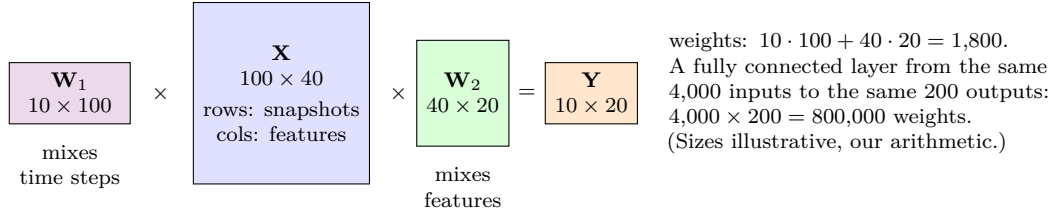


Figure 55: A bilinear layer: the window matrix is multiplied on the left by a matrix that mixes time steps and on the right by a matrix that mixes features. Schematic.

Attention. For the latest snapshot, attention computes a weight for every earlier snapshot: how well the two match, as judged by learned matrices (the queries and keys of Section 10.7). The weights are positive and sum to one. The output is the weighted average of the earlier snapshots’ contents (Figure 56). Unlike the recurrent network, it does not pass a memory along; it looks back at every earlier snapshot directly, and which ones it weights depends on the content of the current one, not on their position. Section 10.7 builds this step by step with numbers.

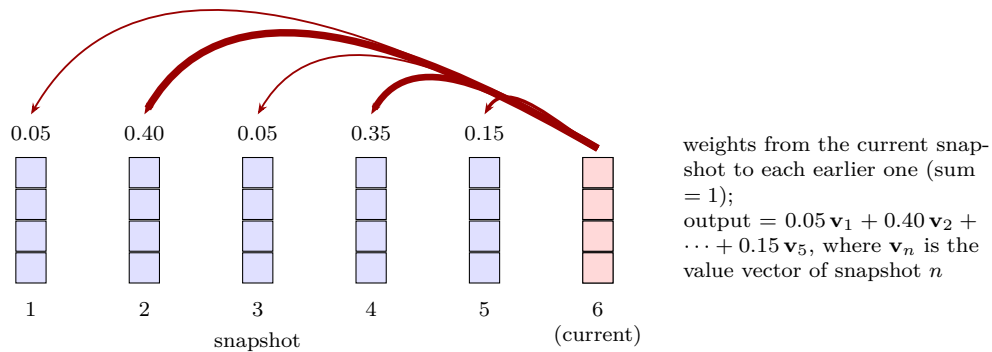


Figure 56: Attention: the current snapshot weights every earlier snapshot by how well they match, and takes the weighted average of their values. Line width shows the weight; numbers are illustrative. Schematic.

The remaining families of Table 38 are variants or combinations of these. A state-space model is a recurrent memory with a particular, efficient update (Section 10.9). A graph network applies the same weights along the links of a graph, as a convolution does along a grid. A neural point process uses a recurrent or attention memory to compute event intensities (Section 7.19).

Building blocks used across families. Most published models combine several families. DeepLOB, the most widely used reference, is a convolutional front end followed by an LSTM (Figure 61). Three blocks recur. A *convolution* is a small set of weights applied at every position of the input—for example a filter that combines the price and size at one level, applied at every level and every time step; because the same weights are reused everywhere, a convolutional network has few weights for the size of its input. An *LSTM* cell keeps a memory vector and, at each time step, decides through learned gates—numbers between 0 and 1, computed from the input, that multiply what passes through—how much of it to keep, how much of the new input to add, and how much to output. *Attention* computes, for the latest input, a weight for every earlier input from how well they match, and takes the weighted average (the queries, keys and values of Section 10.7). An introduction to all three is in Goodfellow et al. (2016).

10.4 Training a deep LOB model: a worked example

Training is where most of the decisions that change results are made, and where most of the problems found in re-tests come from (Section 10.12). This subsection walks through the steps, using the choices of DeepLOB (Zhang et al., 2019a) on its London Stock Exchange data set as a concrete example (Figure 57).

Software. Deep LOB models are written with general-purpose deep-learning libraries. Each library stores the data as arrays (*tensors*), runs the arithmetic on a CPU or a GPU (a graphics processor, built for many arithmetic operations in parallel), and computes the gradients that training needs automatically, so the modeller writes only the forward computation. The main ones are:

- **PyTorch** (Paszke et al., 2019): used by the public code of LOBCAST (Prata et al., 2024), LOBFrame (Briola et al., 2025a) and TLOB (Berti and Kasneci, 2025), each with PyTorch Lightning, a layer that supplies the training loop. The LOBERT paper states that it is implemented in PyTorch (Linna et al., 2025); we found no public code.
- **TensorFlow** with its **Keras** interface (Abadi et al., 2016; Chollet et al., 2015): used by the first public code of DeepLOB (Zhang et al., 2019a) (a PyTorch version was added to the same repository in 2021) and by TransLOB (Wallbridge, 2020).
- **JAX** (Bradbury et al., 2018): used by JAX-LOB (Frey et al., 2023), an order-book simulator that runs on GPUs.
- **scikit-learn** (Pedregosa et al., 2011): not a deep-learning library; used for linear and tree baselines and for computing scores.

Framework use was checked in each project’s public repository in October 2026.

The code in this subsection. Each step below is followed by Python code that implements it: PyTorch for the network and its training, NumPy for the data, scikit-learn for the scores. The files are in `code/deep_lob_example/` with this survey, and the whole example runs on a laptop CPU in minutes. The data are synthetic, so that the example can be run without a data licence; Step 6 uses this to show what the choice of label does when there is nothing to predict.

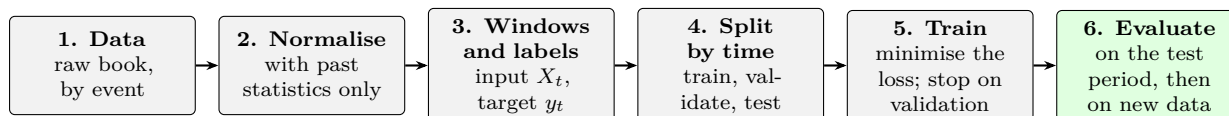


Figure 57: The steps of training and testing a deep LOB model. Schematic.

Step 1: data. DeepLOB uses one year of the book for five London stocks (January to December 2017, 08:30 to 16:00), recorded at every event: about 150,000 events per stock per day, more than 134 million samples in all.

The example replaces this with a synthetic book (Listing 1). The mid-price is a random walk in event time: at each event it moves up one tick with probability 0.1, down one tick with probability 0.1, and otherwise stays. The spread is one tick, and the sizes at every level are independent random numbers. Nothing in this book predicts the next price move. The example uses 20 days of 3,000 events.

Listing 1: Synthetic book: one day of snapshots in DeepLOB’s column order.

```
def make_day(n_events, start_mid, rng):
    steps = rng.choice([-1, 0, 1], size=n_events, p=[0.1, 0.8, 0.1])
    mid = start_mid + TICK * np.cumsum(steps)
    best_ask, best_bid = mid + TICK / 2, mid - TICK / 2
    book = np.empty((n_events, 40))
    for lvl in range(10):
        book[:, 4 * lvl + 0] = best_ask + lvl * TICK          # ask price
        book[:, 4 * lvl + 1] = rng.integers(1, 200, n_events) # ask size
        book[:, 4 * lvl + 2] = best_bid - lvl * TICK          # bid price
        book[:, 4 * lvl + 3] = rng.integers(1, 200, n_events) # bid size
    return book, mid
```

Step 2: normalise. Prices and sizes are on very different scales, and their typical values drift from day to day, so each input is standardised—its mean subtracted, divided by its standard deviation. The statistics must come only from the past: DeepLOB normalises each day with the mean and standard deviation of the previous five days, separately for each stock. Using statistics from the whole sample would leak future information into the inputs. How inputs are normalised turns out to matter a great deal (Section 10.6).

Listing 2: Normalising each day with the previous five days.

```
def normalise_days(days, lookback=5):
    """z-score each day with the mean and std of the previous `lookback`
    days. The first `lookback` days only supply statistics and are
    dropped."""
    out = []
    for d in range(lookback, len(days)):
        past = np.concatenate([days[k][0] for k in range(d - lookback, d)])
        mu, sd = past.mean(axis=0), past.std(axis=0) + 1e-8
        book, mid = days[d]
        out.append(((book - mu) / sd, mid))
    return out
```

Step 3: windows and labels. Each training example pairs an input window with a target. The input X_t is the last 100 snapshots of the book, each with 40 numbers—price and size, bid and ask, at ten levels—so a 100×40 matrix. The target is the direction of the mid-price m_t over the next H events. Because the mid changes in small, noisy steps, the future price is usually averaged: with m_t^+ the average of the next H mid-prices,

$$\text{chg}_t = \frac{m_t^+ - m_t}{m_t}, \quad y_t = \begin{cases} \text{up} & \text{chg}_t > \theta, \\ \text{down} & \text{chg}_t < -\theta, \\ \text{stationary} & \text{otherwise,} \end{cases} \quad (61)$$

In words: compare the average price over the next H events with the price now; call it up or down if the percentage change exceeds a threshold θ , and stationary otherwise.

This label averages only the future; it is measured from the current price. DeepLOB uses it, or a variant that also averages the past, replacing m_t by the average of the previous H mid-prices; this survey calls that variant the *smoothed label*. DeepLOB uses $H = 20, 50$ and 100 events. The threshold sets how many examples fall in each class. Both choices change the problem: averaging

the past as well as the future makes the label partly predictable from the past, which inflates accuracy (Lee, 2026) (Section 10.5).

Listing 3 builds both labels. With `past_average=True` the reference is the average of the last H mid-prices (the smoothed label); with `False` it is the current mid-price m_t (the label of Equation (61), which we call leak-free). The example uses $H = 20$ and sets θ on the training days so that about a third of the labels are stationary.

Listing 3: Labels and input windows.

```
import numpy as np
from synthetic_book import TICK

def smoothed_change(mid, H, past_average):
    """Relative change of the average of the next H mids, measured from
    the current mid (leak-free) or from the average of the last H mids
    (smoothed)."""
    T = len(mid)
    c = np.concatenate([[0.0], np.cumsum(mid)])
    t = np.arange(H - 1, T - H) # need H past and H future mids
    m_plus = (c[t + 1 + H] - c[t + 1]) / H # mean of mid[t+1 .. t+H]
    m_minus = (c[t + 1] - c[t + 1 - H]) / H # mean of mid[t-H+1 .. t]
    ref = m_minus if past_average else mid[t]
    return t, (m_plus - ref) / ref

def to_class(change, theta):
    # 0 down, 1 flat, 2 up
    return np.where(change > theta, 2, np.where(change < -theta, 0, 1))

def windows(book, t, length=100):
    """Input window for each time t: the last `length` snapshots,
    a length 40 matrix."""
    keep = t >= length - 1
    idx = t[keep][:, None] + np.arange(-length + 1, 1)[None, :]
    return book[idx], keep # shape (N, length, 40)

def relative_prices(X, raw_windows):
    """Replace the price columns of each window by (price - mid at the
    window's last event), in ticks. Sizes keep their day-level
    normalisation."""
    X = X.copy()
    last = raw_windows[:, -1, :]
    mid_last = (last[:, 0] + last[:, 2]) / 2 # (best ask + best bid) / 2
    for col in range(0, 40, 2): # columns 0, 2, ... are prices
        X[:, :, col] = (raw_windows[:, :, col] - mid_last[:, None]) / TICK
    return X
```

Step 4: split by time. DeepLOB trains on the first six months, uses the next three to tune and to decide when to stop, and tests on the last three. The split must follow time: shuffling examples before splitting puts almost identical, overlapping windows into both training and test sets, and reports skill the model does not have. Several published studies made this mistake (Section 10.5). The example trains on 9 days, validates on 3 and tests on the last 3, after the first 5 days are used only for the normalisation statistics.

Listing 4: Splitting by time.

```
def split_by_time(items, n_train, n_val):
```

```

    """Train on the earliest days, validate on the next ones, test on the
    rest. No shuffling."""
    n = n_train + n_val
    return items[:n_train], items[n_train:n], items[n:]

```

Step 5: train. The network outputs three probabilities, $\hat{p}_{\text{down}}$, $\hat{p}_{\text{stat}}$ and $\hat{p}_{\text{up}}$. Training adjusts the weights to make the probability of the observed label as high as possible, by minimising the *cross-entropy* loss over the training examples,

$$\mathcal{L}_{\text{CE}}(\mathbf{W}) = -\frac{1}{N} \sum_t \ln \hat{p}_{y_t}(X_t; \mathbf{W}), \quad (62)$$

In words: for each example, take the probability the network gave to the label that actually occurred; the loss is the average of minus its logarithm, so confident correct answers cost little and confident wrong answers cost a lot.

The weights are adjusted by gradient descent on small batches of examples. Adam is a widely used variant of gradient descent; the *learning rate* sets the size of each step; a *batch* is the group of examples whose losses are averaged for one step; an *epoch* is one pass over all the training data. (“Weights” and “parameters” are used interchangeably.) DeepLOB uses Adam with a learning rate of 0.01, batches of 32 examples, and stops when accuracy on the validation months has not improved for 20 epochs (about 40 epochs in all). The trained network has about 60,000 weights and takes about a quarter of a millisecond per prediction; the paper does not state the device or the batch size used for that timing (Section 13.2).

Listing 5 is DeepLOB in PyTorch, following the description of Section 10.6.1. Printing its parameter count gives 60,947: the same as Table 41 stage by stage, except the LSTM, where PyTorch keeps two bias vectors per gate instead of one ($4 \times 64 = 256$ more). Listing 6 is the training loop. It uses Adam with a learning rate of 0.0001 and batches of 128, the values in the authors’ public TensorFlow notebook, which differ from the 0.01 and 32 stated in the paper. To keep the run short, training stops after 5 epochs without improvement instead of 20, and after 30 epochs at most.

Listing 5: DeepLOB in PyTorch.

```

import torch
from torch import nn

def conv(c_in, c_out, k, s=(1, 1), pad="same"):
    if s != (1, 1):
        pad = 0
        # strided convolutions shrink the columns
    return nn.Sequential(nn.Conv2d(c_in, c_out, k, stride=s, padding=pad),
                        nn.LeakyReLU(0.01))

def time_convs(c):
    # two 4 x 1 convolutions over consecutive events
    return nn.Sequential(conv(c, c, (4, 1)), conv(c, c, (4, 1)))

class DeepLOB(nn.Module):
    def __init__(self):
        super().__init__()
        # price with size
        self.stage1 = nn.Sequential(conv(1, 16, (1, 2), (1, 2)),
                                    time_convs(16))

        # ask with bid
        self.stage2 = nn.Sequential(conv(16, 16, (1, 2), (1, 2)),

```

```

        time_convs(16))

    # across the book
    self.stage3 = nn.Sequential(conv(16, 16, (1, 10), pad=0),
                                time_convs(16))

    # Inception module: three branches over different time scales
    self.inc_a = nn.Sequential(conv(16, 32, (1, 1)),
                                conv(32, 32, (3, 1)))
    self.inc_b = nn.Sequential(conv(16, 32, (1, 1)),
                                conv(32, 32, (5, 1)))
    self.inc_c = nn.Sequential(
        nn.MaxPool2d((3, 1), stride=1, padding=(1, 0)),
        conv(16, 32, (1, 1)))
    self.lstm = nn.LSTM(input_size=96, hidden_size=64, batch_first=True)
    self.out = nn.Linear(64, 3)

def forward(self, x):
    # x: batch x 100 x 40
    h = x.unsqueeze(1) # batch x 1 x 100 x 40
    h = self.stage3(self.stage2(self.stage1(h))) # batch x 16 x 100 x 1
    h = torch.cat([self.inc_a(h), self.inc_b(h), self.inc_c(h)],
                  dim=1) # batch x 96 x 100 x 1
    h = h.squeeze(3).transpose(1, 2) # batch x 100 x 96
    _, (h_last, _) = self.lstm(h) # memory after the last event
    # batch x 3 scores; the softmax is inside the loss
    return self.out(h_last[-1])

```

Listing 6: Training with early stopping on the validation days.

```

import copy
import torch
from torch import nn

def train(model, X_tr, y_tr, X_va, y_va, lr=1e-4, batch=128,
          max_epochs=30, patience=5, device="cpu"):
    model.to(device)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    # softmax + minus log-probability of the true class
    loss_fn = nn.CrossEntropyLoss()
    best, best_val, bad = None, float("inf"), 0
    for epoch in range(max_epochs):
        model.train()
        # shuffle batches within the training period only
        order = torch.randperm(len(X_tr))
        for i in range(0, len(order), batch):
            j = order[i:i + batch]
            loss = loss_fn(model(X_tr[j].to(device)), y_tr[j].to(device))
            opt.zero_grad(); loss.backward(); opt.step()
        val = validation_loss(model, X_va, y_va, loss_fn, device)
        print(f"epoch_{epoch+1}: validation_loss_{val:.4f}")
        if val < best_val:
            best, best_val, bad = copy.deepcopy(model.state_dict()), val, 0
        else:
            bad += 1
            if bad >= patience:
                break
    model.load_state_dict(best) # stop: validation has not improved
    return model # keep the best weights seen

@torch.no_grad()
def validation_loss(model, X, y, loss_fn, device, batch=1024):

```

```

    model.eval()
    total = 0.0
    for i in range(0, len(X), batch):
        xb, yb = X[i:i + batch].to(device), y[i:i + batch].to(device)
        total += loss_fn(model(xb), yb).item() * len(xb)
    return total / len(X)

@torch.no_grad()
def predict(model, X, device="cpu", batch=1024):
    model.eval()
    out = [model(X[i:i + batch].to(device)).argmax(1).cpu()
            for i in range(0, len(X), batch)]
    return torch.cat(out)

```

Step 6: evaluate. On the test months DeepLOB reports F1 scores of about 70%, 63% and 61% for the three horizons, and similar numbers on five stocks it had not seen. Three further checks separate a useful model from a good-looking one, and the later subsections show each one failing in published work: compare with simple baselines on the same inputs, including a naive forecast of the label (Section 10.5); re-test on a later period and on other instruments (Section 10.12); and measure what a trader would make after costs, not only classification accuracy (Sections 10.12 and 11.1).

Step 6 in the example: results on a market with nothing to predict. The example trains the network four times: two labels (smoothed, leak-free) times two ways of presenting prices (z-scored by day as in Step 2, or measured in ticks from the mid at the window's last event, the function `relative_prices` in Listing 3). Each run is scored on the 3 test days against three rules that use no network (Listing 7): the past-only rule of Section 10.5 (is the mid now above or below its average over the last 20 events?), repeating the most recent label that is already known at t (the label of $t - 20$), and always predicting the most common class. Table 39 gives the results; `run_output_zscore.txt` and `run_output_relative.txt` hold the full output.

Listing 7: Scoring.

```

from sklearn.metrics import accuracy_score, f1_score

def score(name, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    # F1 averaged over the three classes
    f1 = f1_score(y_true, y_pred, average="macro")
    print(f"{name:<34s}_accuracy_{acc:.3f}_macro-F1_{f1:.3f}")
    return acc, f1

# Two baselines that use no network:
#   past_rule:   classify the change from the average of the last H mids
#               to the current mid
#   last_label:  repeat the most recent label that is fully known at
#               time t (the label of t - H)

```

What the table shows:

- **Leak-free label: nothing works**, as it should on a random walk. Every method is near one in three; DeepLOB predicts the most common class for every example (macro-F1 0.176).
- **Smoothed label, relative prices: 57.6% on an unpredictable market.** The network learns the part of the label that is already known at t (Equation (63)) and matches the past-only rule. Every point above one in three comes from the label, none from the book.

Table 39: The example of this subsection: test-day scores on a synthetic market in which the price is a random walk, so that nothing can be predicted. Prices: z-scored by day, or relative to the mid at the window’s last event. One run per configuration (seed 0); simulation, not market data.

Label	Prices	DeepLOB	Past-only rule	Repeat last label	Most common class
<i>each cell: accuracy / macro-F1</i>					
Smoothed	z-scored	0.389 / 0.187	0.560 / 0.565	0.406 / 0.401	0.389 / 0.187
Smoothed	relative	0.576 / 0.576	0.560 / 0.565	0.406 / 0.401	0.389 / 0.187
Leak-free	z-scored	0.358 / 0.176	0.328 / 0.326	0.337 / 0.335	0.358 / 0.176
Leak-free	relative	0.358 / 0.176	0.328 / 0.326	0.337 / 0.335	0.358 / 0.176

- **Smoothed label, z-scored prices: the network learns nothing.** Z-scored with five days of statistics, the price columns mostly record where the price is relative to the previous days (average size 2.1 units on the 3 test days). The movement within a window is much smaller: the median change over 20 events on those days is 0.039 units. In these runs the network did not pick it up and predicted the most common class. Measuring prices from the current mid makes the movement visible. The input normalisation decided whether this network could use the information at all, a small instance of the finding of Section 10.6 that normalisation matters as much as architecture.

These are single runs on one synthetic data set, meant to show the steps and the label effect, not to rank methods.

10.5 Scoring, label construction, data splits and targets

This subsection defines the scores used in the deep LOB literature, then shows two ways in which a reported score can overstate what the network learned, and why two targets with similar names are not comparable.

Scoring metrics. Direction classifiers are scored by comparing their predicted class with the true one on the test data. *Accuracy* is the share of cases predicted correctly. For one class, say “up”, *precision* is the share of predicted “ups” that were right, and *recall* the share of true “ups” that the model caught; the *F1 score* combines the two into one number (their harmonic mean), high only if both are high. *Macro-F1* averages the F1 score over the three classes, so that a rare class counts as much as a common one. Papers report these as percentages (70%) or as fractions (0.70); they are the same scale. A *model confidence set* is a statistical test that, given several models, keeps those that cannot be shown to be worse than the best; “among the statistically best” means kept by such a test (Hansen et al., 2011). None of these scores says anything about money: a model can classify most windows correctly and still not make a profit once trades are costed (the end of this subsection, and Section 11.1).

Two sources of inflated accuracy: the label and the split. A reported accuracy measures what the network learned only if the label and the data split do not leak information. Two published re-tests show that in this literature they often do:

- **Label** (Check 1). DeepLOB on a year of Apple data: 65.9% with its smoothed label, 54.6% with a label measured from the current price (Lee, 2026).
- **Split** (Check 2). FI-2010: macro-F1 of about 0.76–0.88 under a 7-day/3-day split, about 0.51–0.59 under anchored day-by-day cross-validation.

Check 1: label leakage from smoothed labels. The label used by DeepLOB and much of the literature compares the average of the next H mid-prices, m_t^+ , with the average of the last H , m_t^- (the variant of Equation (61) with m_t^- in place of m_t). Adding and subtracting the current mid-price m_t splits it into two parts:

$$m_t^+ - m_t^- = \underbrace{(m_t^+ - m_t)}_{\text{the future: unknown at } t} + \underbrace{(m_t - m_t^-)}_{\text{the past: known at } t} . \quad (63)$$

In words: the label asks “will the next twenty prices average more than the last twenty?”. Part of the answer is already in: if the price has risen over the last twenty events, it is now above its recent average, and the future average will tend to be above the past average even if the price never moves again.

The second part is known when the prediction is made, so any model that notices the recent trend gets that part of the label for free. Figure 58 shows how much it is worth. On a simulated market in which the price is a pure random walk—nothing about the future can be predicted—a rule that looks only at whether the price is above or below its average over the last twenty events predicts the direction of the smoothed label correctly 74% of the time; the same rule predicts the direction of a label measured from the current price, $m_t^+ - m_t$, exactly as often as a coin, 50%.

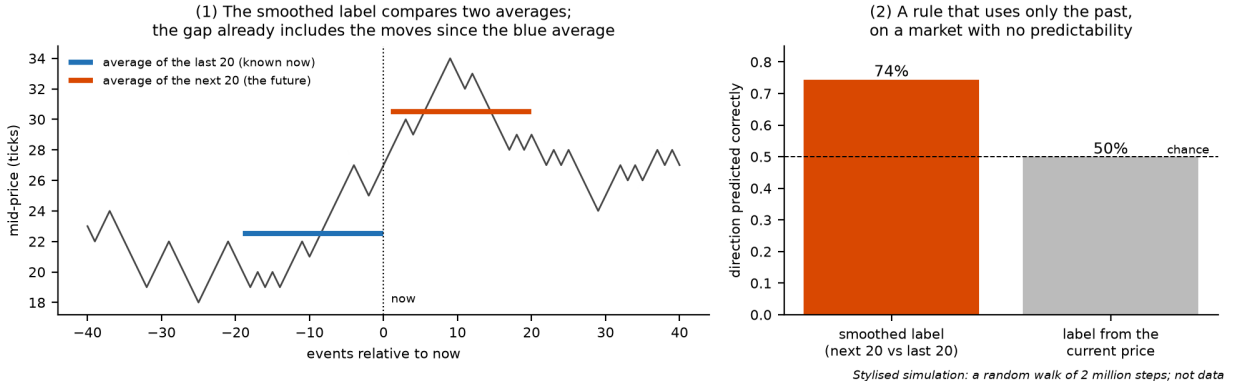


Figure 58: Why a smoothed label inflates accuracy. (1) The label compares the average of the next 20 mid-prices with the average of the last 20; at “now” the price is already above its recent average, so part of the gap is already known. (2) On a pure random walk, where nothing can be predicted, a rule using only the past gets the direction of the smoothed label right 74% of the time, and the direction of a label measured from the current price 50% of the time. Stylised simulation, not data.

This is what Lee (2026) found on real data. Re-running DeepLOB on a full year of Apple order-book data with the smoothed label, the network was right 65.9% of the time—but a naive forecast that simply repeats the most recent value of the same label that could be observed was right 64.8% of the time. With the label measured from the current price, DeepLOB’s accuracy fell to 54.6%, and to 53.6% with only the best level of the book, so the nine deeper levels added about one point. Most of the headline accuracy came from the label, not from the book.

Check 2: the protocol. On the FI-2010 data set, the way the data are split moves scores by more than most changes of architecture. Under anchored day-by-day cross-validation—train on the first days, test on the next, then add that day to the training data and repeat, averaging the scores—macro-F1 scores lie around 0.51–0.59 (September et al., 2024; Passalis et al., 2020b); under the common split of seven days for training and three for testing, published scores are about

0.76–0.88 (Tran et al., 2021a; Zhang et al., 2019a). These come from different papers and models, so part of the gap may be the models; but the gap is larger than the differences between architectures under either protocol. The conclusion: a gain claimed for an architecture on FI-2010 means little unless the split is stated.

Other protocol problems. Several further ways of getting accuracy that is not there appear in published work. Randomly shuffled rather than time-ordered splits (Shi et al., 2021) put overlapping windows—which share almost all their snapshots and have nearly the same label—into both the training and the test sets, so the test measures memory, not prediction. Choosing the model by its score on the training data, with no validation set (Tran et al., 2021b), selects the model that fits noise best. Duplicated positive examples—cases where the predicted event did occur—in an evaluation set (Mäkinen et al., 2019) count the same success several times. Test sets balanced by down-sampling the common class—discarding examples of the frequent class until the classes are equal in size (Zhong et al., 2025b)—report accuracy on a mix of classes the model will never meet in use.

Weak baselines. A model can also look good because its comparison is too weak. Ntakaris and Ibikunle (2024) forecast the *level* of the mid-price for 100 S&P 500 stocks over three months of 2022 with a reinforcement-learning “policy engine”, against a naive forecast, ARIMA (a classical linear time-series model) and several neural networks. For Amazon, the root-mean-square error—the typical size of the forecast error—is 0.602 for the naive forecast, above ARIMA (0.364) and their model (0.0559). For a price level, repeating the last value is usually among the hardest baselines to beat, so a naive forecast that loses by this much calls for checking how it was defined; we did not check it.

Differences between similar-sounding targets. Two models scored on targets that sound alike may be solving different problems, and a model that does well on one can do badly on another. Six targets appear in this literature:

$$\begin{aligned} &\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t), \quad \mathbb{E}[\Delta m_{t,H} \mid X_t], \quad \mathbb{P}(\Delta m_{t,H} > \theta \mid X_t), \\ &\mathbb{P}(T_{\text{move}} \leq H \mid X_t), \quad \mathbb{P}(\text{Fill} \mid X_t), \quad \mathbb{E}[\Pi \mid X_t], \end{aligned} \tag{64}$$

where $\Delta m_{t,H}$ is the change in the mid-price over the horizon H , $\theta > 0$ is a move threshold, Π is the trader’s profit, δ is one tick, and T_{move} is the time until the mid first moves by one tick,

$$T_{\text{move}} = \inf\{u > 0 : |m_{t+u} - m_t| \geq \delta\}.$$

Table 40 spells out the question each one asks.

Table 40: Six targets and the questions they ask.

Target	Question	What it is useful for
$\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$	will the mid be higher after H ?	direction, regardless of size
$\mathbb{E}[\Delta m_{t,H} \mid X_t]$	by how much, on average?	sizing a position
$\mathbb{P}(\Delta m_{t,H} > \theta \mid X_t)$	will it rise by more than θ ?	moves large enough to pay for costs
$\mathbb{P}(T_{\text{move}} \leq H \mid X_t)$	will it move at all within H ?	timing; how long to wait
$\mathbb{P}(\text{Fill} \mid X_t)$	will my resting order be filled?	choosing between passive and aggressive orders
$\mathbb{E}[\Pi \mid X_t]$	what do I expect to earn?	the decision itself

Two published results show that these really are different problems. In Lee (2026), splitting the 53.6% accuracy of DeepLOB’s level-1 variant (best prices and sizes only, label measured from the current price) on Apple into its parts, the network was right 69.4% of the time about *whether* the price would move, and 71.1% of the time about the direction when it correctly predicted a move; with only the best prices as input, without their sizes, its accuracy on direction fell to about 50%. Whether the price moves and which way it moves are different questions, with different answers in the data. And in the re-test of DeepLOB on fifteen Nasdaq stocks by Briola et al. (2025a), the highest F1 score for direction, 0.79 (one large-tick stock, one horizon, counting only predictions made with confidence of at least 0.9), came with a score of 0.00 on a measure of correctly executed round-trip trades—a buy followed by a sell, or the reverse; the measure is described in Section 10.12—at the same setting: a high classification score did not translate into trades that could be executed correctly (Section 10.12). Comparing models scored on different targets in Equation (64)—or reporting one target and acting on another—is a common source of misleading conclusions. Benchmark papers should state the target, the horizon, the clock on which the horizon is measured (events or seconds), and how a prediction would be acted on.

10.6 Convolutional and recurrent models

The fully connected layer. In a standard, *fully connected* layer each output is a weighted sum of *all* the inputs, with its own weight for each input. Applied to a window of the book, the 100×40 grid is first flattened into one list of 4,000 numbers. A single layer of 64 outputs then needs $4,000 \times 64 + 64 = 256,064$ weights. Worse, it treats every position separately: a sudden drop in the best bid’s size at event 10 and the same drop at event 90 are handled by different weights, so the network must learn the same pattern again for every place it can occur—which takes far more data than an order book usually offers.

The convolutional layer. A convolutional layer replaces this with a small *filter*—a few weights covering a small patch of the grid, for example two adjacent numbers, or four consecutive events at one level—that slides across the whole input. At each position, the output is the weighted sum of the inputs under the filter. Two ideas make it work: each output looks only at a small neighbourhood of the input (*local connectivity*), and the same weights are used at every position (*weight sharing*). A pattern is therefore detected wherever it occurs, by the same few weights, and a layer of 16 filters of size 1×2 has only $16 \times 2 + 16 = 48$ weights (Figure 59).

A worked example (illustrative numbers). Take the size of the best bid over six consecutive events: 10, 10, 10, 4, 4, 4. Slide a filter with weights $(-1, +1)$ over pairs of consecutive values: the outputs are $-10 + 10 = 0$, then 0, then $-10 + 4 = -6$, then 0 and 0. The filter is a *change detector*: it is zero while the queue is steady and strongly negative where the queue drops, at whatever event the drop happens. A network does not have to be told to look for drops; if they help it predict, training will set some filter’s weights to something like $(-1, +1)$. Stacking layers lets later filters combine the patterns found by earlier ones—a drop at the bid together with additions at the ask, say—over wider and wider stretches of the window.

Convolution applied to a window of book snapshots. Treating the window as an image, with time along one axis and the book’s columns along the other, a convolutional network assumes that the same local pattern means the same thing wherever it appears in time and at whatever level it appears. For time this is natural: a depleting queue is a depleting queue whether it happened 10 or 90 events ago. For the book’s columns, the filter sizes and steps must be chosen so that a filter

combines numbers that belong together—a price with its own size, an ask with a bid at the same level—which is exactly what DeepLOB’s first layers do (Section 10.6.1).

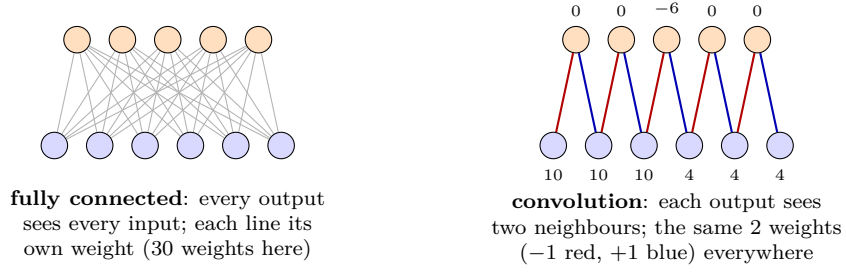


Figure 59: A fully connected layer and a convolution on the same six inputs, the size of the best bid over six events. The convolution with weights (−1, +1) detects the drop from 10 to 4 wherever it occurs. Illustrative numbers. Schematic.

First models. Tsantekidis et al. (2017a) were the first to apply a convolutional network to sequences of book snapshots, treating the window like an image whose two axes are time and price level; DeepLOB’s authors quote its FI-2010 F1 scores as 55–59% across horizons (Zhang et al., 2019a). A companion paper used an LSTM with 40 hidden units on the last 100 snapshots, reporting F1 scores of about 61–66% on FI-2010 against 48–56% for a multilayer perceptron (Tsantekidis et al., 2017b). Dixon (2018a) applied a recurrent network to E-mini S&P 500 futures (a contract on the US stock index), using Level-2 data—prices and sizes at several levels, without individual orders—to classify the next price flip, the next change of the mid-price, up or down.

10.6.1 DeepLOB in detail

Zhang et al. (2019a) combined the convolutional and recurrent ideas into what became the most widely used reference architecture for order books. This subsection takes it apart layer by layer; the training choices were described in Section 10.4.

The input. The model sees the last 100 snapshots of the book, one per event. Each snapshot is 40 numbers—the price and size at each of the ten best levels on each side—arranged in a fixed order:

$$(p_1^a, v_1^a, p_1^b, v_1^b, p_2^a, v_2^a, p_2^b, v_2^b, \dots, p_{10}^a, v_{10}^a, p_{10}^b, v_{10}^b),$$

where p_i^a and v_i^a are the ask price and size at level i , and p_i^b, v_i^b the same on the bid. The input is therefore a 100×40 matrix, like a one-channel image with time down the rows and the book across the columns (Figure 60). The order matters, because the first layers are designed around it.

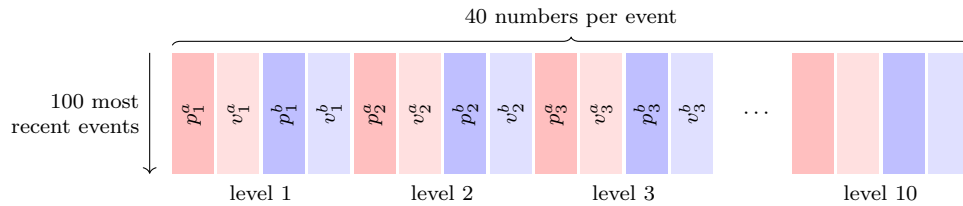


Figure 60: The input of DeepLOB: the last 100 snapshots of the book, each laid out as price and size, ask then bid, level by level. Red: ask side; blue: bid side. Schematic.

Stage 1: price with size, at each level and side. The first convolution has 16 filters of size 1×2 —one event by two adjacent columns—moved two columns at a time (a *stride* of 2). Each filter therefore combines a price with the size next to it, (p_i^a, v_i^a) or (p_i^b, v_i^b) , and the same 16 filters are applied to every level and both sides. The stride is essential: with a step of one column, a filter would also combine v_i^a with p_i^b , mixing a size with an unrelated price. The 40 columns become 20, one per side and level. Two further convolutions of size 4×1 —four events by one column—then combine each of these features over four consecutive events, a short smoothing over time.

Stage 2: ask with bid, at each level. A second 1×2 convolution with stride 2 combines the ask and bid features of the same level, reducing 20 columns to 10, one per level, again followed by two 4×1 convolutions over time. The authors relate the result to the micro-price (Section 4.5): a feature that weighs the bid and ask prices of a level by their sizes, now available at every level rather than only the best (Zhang et al., 2019a). The network is not forced to compute a micro-price; the layer has the right inputs to.

Stage 3: across the book. A convolution of size 1×10 combines all ten levels into a single column, followed again by two 4×1 convolutions. After the three stages each event is described by 16 learned features that summarise the whole visible book: the 100×40 input has become 100×16 .

Stage 4: several time scales at once (the Inception module). Two building blocks are needed first. A 1×1 convolution mixes the 16 features of a single event without looking at its neighbours: a small network applied to each event separately. *Max-pooling* over 3 events replaces each value by the largest of it and its neighbours. A convolution over four events looks at one fixed time scale. The *Inception module* runs three branches side by side on the 100×16 features and joins their outputs (Figure 62): a 1×1 convolution followed by a convolution over 3 events; a 1×1 convolution followed by one over 5 events; and a max-pooling over 3 events followed by a 1×1 convolution. Each branch has 32 filters, so each event now has $3 \times 32 = 96$ features. The authors compare the branches to moving averages of different lengths, as used in chart-based trading rules, except that the weights are learned.

Stage 5: memory over the 100 events (the LSTM). An LSTM with 64 units—its memory vector has 64 numbers—reads the 96 features of the 100 events in order, keeping a memory that it updates at each step through learned gates. Its output after the last event summarises the whole window. The authors chose an LSTM rather than fully connected layers partly because it needs far fewer weights for the same job.

Stage 6: the prediction. A final layer turns the 64 LSTM outputs into three numbers and a softmax turns them into probabilities for down, stationary and up, which sum to one.

Padding, activations and regularisation. Every convolution pads its input with zeros so that the time dimension stays at 100. The non-linearity after each convolution is a leaky ReLU, which passes positive values unchanged and multiplies negative ones by 0.01, a value chosen on the validation set. There is no pooling outside the Inception module: the authors report that pooling made the model under-fit—too constrained to capture the patterns, even on the training data—because, unlike in an image, *where* in time a pattern occurs matters.

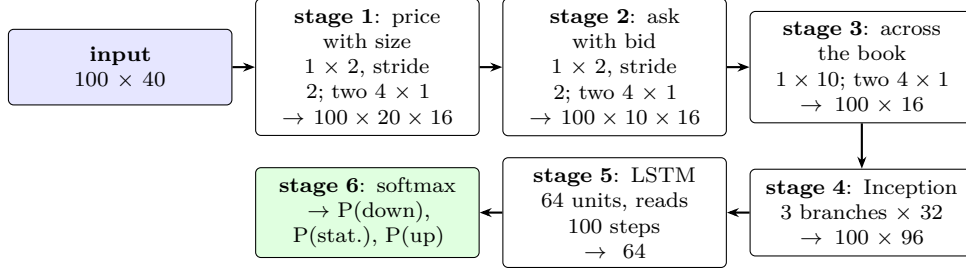


Figure 61: DeepLOB stage by stage, with the shape of the data after each stage (events $\times$ columns $\times$ filters). The shapes follow from the filter sizes and strides given by Zhang et al. (2019a).

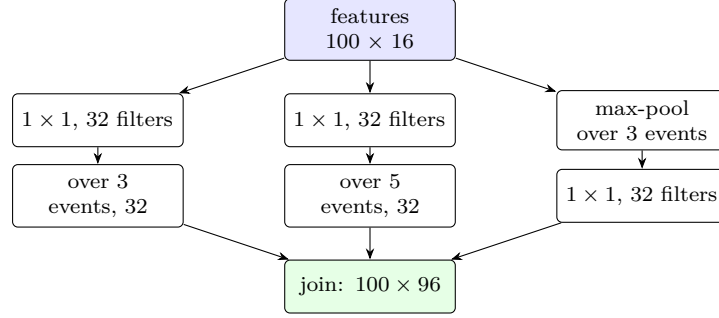


Figure 62: The Inception module of DeepLOB: three branches look at the same features over different time scales, and their outputs are joined. After Zhang et al. (2019a).

Parameter count by stage. Table 41 counts the weights stage by stage. The total, about 60,700, matches the “about 60,000” the authors report. Two-thirds of the weights are in the LSTM; the convolutions that read the book are cheap, because each small filter is reused at every level and every event.

Table 41: Weights in DeepLOB, by stage. Our count from the layer sizes in Zhang et al. (2019a) (filter weights plus one bias per filter; LSTM with four gates).

Stage	Weights	How counted
1: price with size	2,128	$1 \times 2: 2 \cdot 16 + 16$; two $4 \times 1: 2(4 \cdot 16 \cdot 16 + 16)$
2: ask with bid	2,608	1×2 on 16 channels: $2 \cdot 16 \cdot 16 + 16$; two 4×1
3: across the book	4,656	1×10 on 16 channels: $10 \cdot 16 \cdot 16 + 16$; two 4×1
4: Inception	9,888	three 1×1 ($16 \cdot 32 + 32$ each); 3×1 and 5×1 on 32 channels
5: LSTM, 64 units	41,216	$4(64(96 + 64) + 64)$
6: output	195	$64 \cdot 3 + 3$
Total	60,691	

Fixed structure versus learned weights. DeepLOB is not a blank network: the filter sizes and strides of stages 1–3 encode how a book is organised—prices belong with their sizes, asks with bids at the same level, and levels with each other—so that the network cannot mix unrelated numbers and needs few weights to read the book. What it learns is how to weigh these combinations, which time scales matter, and how to accumulate them into a forecast. The authors report a forward pass—one evaluation on one input—of about a quarter of a millisecond, without stating the device or batch size.

Reported results and later re-tests. On a year of London Stock Exchange data for five stocks, it reports F1 scores of about 70%, 63% and 61% at horizons of 20, 50 and 100 events, with similar numbers on five stocks not seen in training—evidence that it learns features that transfer. Its trading simulation, as the authors state, uses mid-prices without transaction costs and is “not . . . a fully developed, stand-alone trading strategy”; trading at the mid ignores the spread that every real trade pays, so such simulations overstate profit. Later re-tests qualify the headline numbers: most of the accuracy on the smoothed label can be matched by a naive forecast, and with a label measured from the current price the accuracy falls to about 55% (Section 10.5); re-implementations score lower on new data (Section 10.12). DeepLOB remains the standard baseline that new models are compared with.

10.6.2 Lighter models, normalisation and variants

Smaller networks. Not every gain needs more capacity. The temporal attention-augmented bilinear network (TABL) multiplies the 100×40 input matrix by one small weight matrix on the left, mixing time steps, and one on the right, mixing features, and adds a simple attention over time (Section 10.7.1). With about 10^4 weights, a sixth of DeepLOB’s, it reaches competitive FI-2010 results (Tran et al., 2019).

Effect of input normalisation. Step 2 of Section 10.4 standardised each input with past means and standard deviations (“z-scoring”). Some models instead *learn* how to normalise: deep adaptive input normalisation (DAIN) learns, for each input feature, how much to shift and rescale it (Passalis et al., 2020a), and bilinear normalisation does so along both the time and the feature axes (Tran et al., 2021a). The gains are large. On FI-2010, DAIN raised the macro-F1 of a simple MLP from about 55% with z-scoring to about 68%; bilinear normalisation raised a TABL network’s F1 from 78.44% to 88.06% at one horizon (Tran et al., 2021a); and an independent group, using the stricter anchored protocol, found that learned normalisations add about seven to nine macro-F1 points over z-scoring (58.89% for bilinear normalisation and 57.76% for DAIN, against 50.52%) (September et al., 2024). By comparison, adding multi-head attention to the best TABL network gained 0.4 of a point (76.01% to 76.42%) (Shabani et al., 2022). The conclusion: how the inputs are scaled can move results by around ten points, the choice of network by much less. Because the statistics of LOB data drift from day to day, normalisation is a modelling decision in its own right.

Variants that output more than a class. Some variants also say how uncertain a prediction is. *Dropout* switches off a random subset of units during training; keeping it on at prediction time and repeating the prediction many times gives a spread of outputs, which serves as an estimate of uncertainty—a cheap form of a *Bayesian* network, whose weights are treated as uncertain rather than fixed. Such a version of DeepLOB reached an F1 score of 0.60 against 0.58 for the ordinary model (Zhang et al., 2018). *Quantile-regression* heads predict, for example, the 10th and 90th percentiles of the return, giving a prediction interval rather than a class (Zhang et al., 2019b); Bayesian bilinear networks do the same for the TABL family (Magris et al., 2023). These add a measure of uncertainty more than they add accuracy.

Predicting events other than the next move. Mäkinen et al. (2019) predict price jumps—unusually large price changes—with a CNN–LSTM with attention and reach an F1 score of 0.72, against 0.69 for an LSTM. But a model that uses only the time of day reaches 0.66, so most of the skill is intraday seasonality: regular patterns by time of day, such as busier opens and closes. Their evaluation sets also contain duplicated positive examples.

Model card: Convolutional and recurrent models (DeepLOB family)

What you calibrate. The network weights (from about 10^4 to 10^5 parameters for TABL and DeepLOB), and before that the input normalisation, the label threshold and the horizon.

How. Supervised training with a cross-entropy loss on labelled windows of snapshots (down/stationary/up), with chronological train, validation and test splits.

What you do with it. Short-horizon direction probabilities, used as a signal or as one input to an execution decision.

How to picture it. The book as an image—time down the rows, the book’s columns across—scanned by small filters that find local patterns, followed by a memory that reads the patterns in order.

10.7 Attention and Transformers for a single asset

10.7.1 Attention and Transformers from the beginning

Selecting relevant past events. A model shown the last 100 events of the book has to decide which of them matter for its prediction. The families met so far answer this in fixed ways. A multilayer perceptron gives each position in the window its own fixed weight: event number 37 always counts the same, whatever it was. A convolution applies the same small filter everywhere, so it sees only short local patterns. A recurrent network reads the events in order and squeezes everything into a memory passed from step to step, so a distant event reaches the present only through every update in between. *Attention* answers the question differently: the weight given to each earlier event is computed afresh, every time, from *what the events are*. If the current event is a large buy, the cancellations at the ask a few milliseconds earlier can receive most of the weight; if it is a quiet limit order, they can receive almost none.

Attention as a weighted look-up. Think of a search: a *query* describes what you are looking for, each item carries a *key* describing what it is, and a *value*, the content you get back. Attention compares the query with every key, turns the match scores into weights that are positive and sum to one, and returns the weighted average of the values—a “soft” look-up that mixes the best-matching items rather than picking one. In a network the query, keys and values are all vectors computed from the events themselves by learned matrices; what is learned is *how to ask and how to describe*, not which position matters.

A worked example. The current event is a buy market order that empties the best ask. Before predicting what happens next, the model asks one question of the recent past: *were sellers leaving the ask?*

In this hand-made example each key has two numbers with a meaning: how much the event is a sign of *sellers leaving* the ask, and how much it is a sign of *buyers arriving*. The query uses the same two numbers to say what the current event is looking for: $(1, 0)$, “sellers leaving, and nothing else”. The value of each event is the number of lots it removed from the ask. (In a trained model, keys, queries and values are learned and their numbers usually have no simple meaning.)

Event	What happened	Key: (sellers leaving, buyers arriving)	Value: lots removed from ask
1	cancellation of 40 lots at the ask	$(2, 0)$	40
2	limit buy of 10 lots at the bid	$(0, 1)$	0
3	buy trade of 20 lots at the ask	$(1, 1)$	20

1. **Score each event** by how well its key matches the query: multiply the two numbers pairwise and add. Event 1: $1 \times 2 + 0 \times 0 = 2$. Event 2: $1 \times 0 + 0 \times 1 = 0$. Event 3: $1 \times 1 + 0 \times 1 = 1$.

2. **Turn the scores into weights** that are positive and sum to one (the softmax): take e^{score} for each event—7.39, 1.00 and 2.72—and divide each by their total, 11.11. The weights are 0.67, 0.09 and 0.24.
3. **Average the values with these weights:** $0.67 \times 40 + 0.09 \times 0 + 0.24 \times 20 \approx 31.5$ lots (computed with unrounded weights).

The result is a weighted average of the lots each event removed, about 31.5 lots per event, in which the events that look like sellers leaving count most. It is not a total: the three events together removed 60 lots, and a weighted average can never exceed the largest single value, 40. The cancellation dominates because its key matches the question best; the limit buy at the bid, which says nothing about sellers, gets almost no weight. A different current event, with a different query, would weight the same three events differently.

Figure 63 draws the worked example. Figure 64 shows what makes attention different from the other families: the same eight earlier events are weighted in two different ways depending on the current event. A buy that sweeps the ask puts most weight on the recent ask cancellations; a quiet limit order puts it on the other limit orders. Figure 65 shows a whole attention layer: every event, as it arrives, spreads its attention over the events before it.

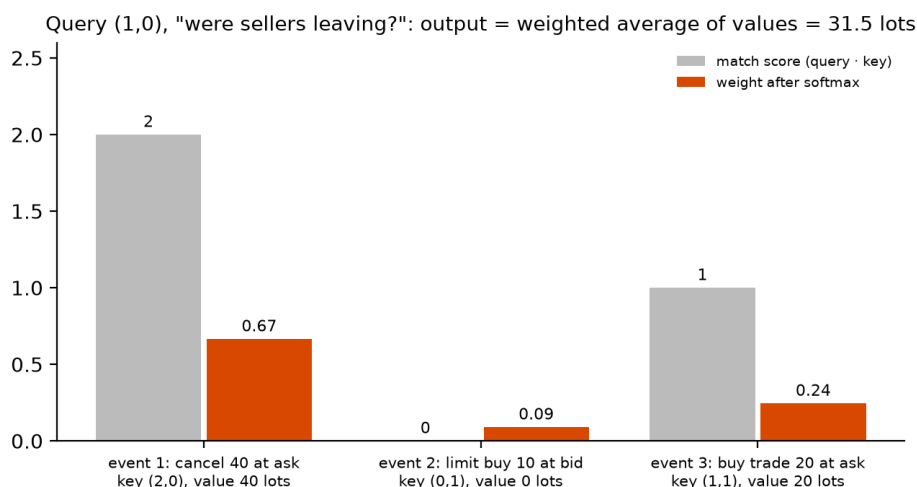


Figure 63: The worked example above. Grey: the score of each earlier event, the match of its key with the query (1,0), “were sellers leaving?”. Orange: the weights the softmax turns the scores into. The output is the weighted average of the values, 31.5 lots. Hand-made numbers.

The Transformer. A Transformer (Vaswani et al., 2017) is a network built from repeated blocks, each containing two steps (Figure 66). First, *self-attention*: every input element—every event, or every snapshot—produces a query, a key and a value, and is replaced by the weighted average it gets from attending to the elements before it. Second, a small multilayer perceptron is applied to each element separately. Each step adds its result to its input rather than replacing it (a *residual* connection) and rescales the result (*layer normalisation*: each element’s vector is shifted and scaled to have mean 0 and variance 1), which makes deep stacks trainable. Three further ingredients complete it. *Several heads*: several attentions run side by side with different learned matrices, so that one can look for, say, recent cancellations while another looks for large trades. A *position or time encoding*: attention on its own ignores order—shuffling the inputs would shuffle the outputs in

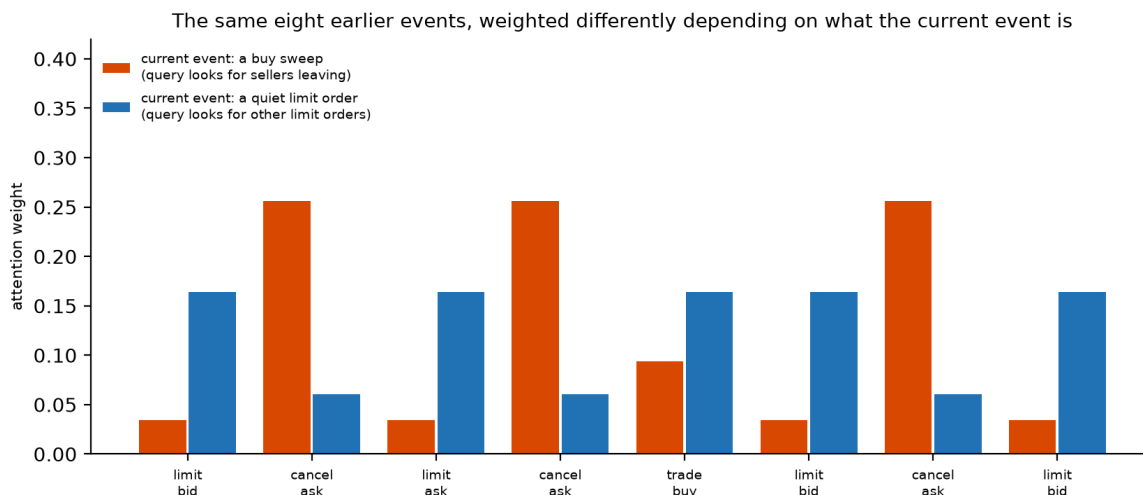


Figure 64: The same eight earlier order-book events, weighted by two different queries. Orange: the current event is a buy sweep, whose query looks for sellers leaving; the ask cancellations receive most of the weight. Blue: the current event is a quiet limit order, whose query looks for other limit orders. Hand-specified vectors, not a trained model.

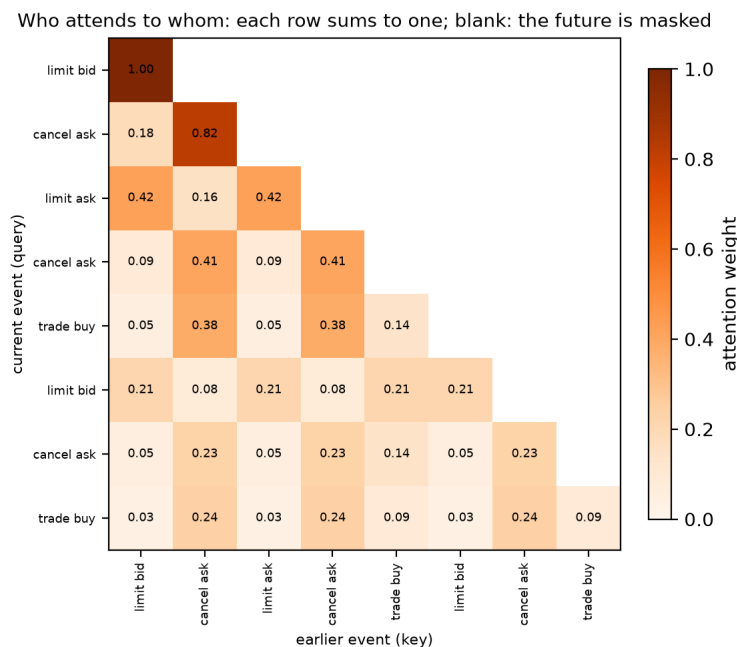


Figure 65: The attention weights of a whole sequence. Row n shows how event n divides its attention among events $1, \dots, n$; each row sums to one. Cells to the right of the diagonal are blank because an event may not attend to later events (masking, explained below). Hand-specified vectors, not a trained model.

the same way—so a vector encoding each element’s position, or its time stamp, is added to it at the start. A *causal mask*: when predicting, an element may attend only to earlier elements, never to later ones. A final layer, the *head*, turns the output into the prediction, for example three class probabilities.

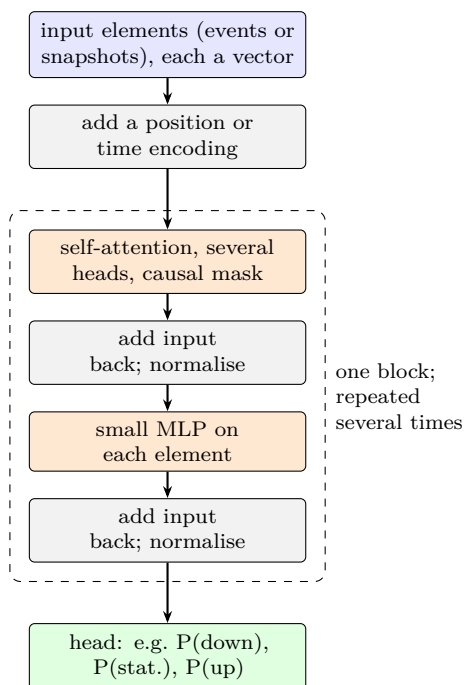


Figure 66: A Transformer. Each block mixes information across elements (self-attention) and then transforms each element on its own (MLP); blocks are stacked. Schematic.

Differences from a standard network. Table 42 sets the families side by side. The essential difference is that in a Transformer the weights that mix different inputs are not fixed parameters: they are computed from the inputs each time. The learned parameters are the matrices that produce queries, keys and values, and the per-element MLP. Two practical consequences follow: all elements can be processed in parallel during training, unlike a recurrent network; and the cost grows with the square of the window length, because every element is compared with every other.

Table 42: How four families weight the past and handle order.

Family	How much each earlier input counts	How order is known	Cost for a window of L
MLP	a fixed learned weight for each position	by the position in the input vector	proportional to L
CNN	a fixed small filter, the same everywhere	by where the filter is	proportional to L
RNN, LSTM	through a memory passed from step to step	by reading in order	proportional to L , one step at a time
Transformer	computed from the content of the inputs, each time	by an added position or time encoding	proportional to L^2 , in parallel

Attention and Hawkes kernels. A Hawkes process (Section 7) also weights past events, and the comparison clarifies both. In a Hawkes model the weight of a past event is set by *how long ago* it happened and by its type: a fixed kernel that decays with elapsed time, the same for every event of that type. In standard attention the weight is set by *what the event is*, through the match of its key with the current query; time enters only through the position or time encoding, and nothing forces old events to count less. The two can be combined. *Hawkes-biased attention*, a design proposed in Section 9.6, adds the logarithm of a Hawkes kernel to the attention scores, so that each weight is the content match multiplied by a decay in elapsed time: relevant events count more, old events count less. The Transformer Hawkes process of Zuo et al. (2020) and the self-attentive Hawkes process of Zhang et al. (2020)—developed concurrently and both presented at ICML 2020, building on the neural Hawkes process of Mei and Eisner (2017)—are a different combination (Section 7.19.2): they use attention to compute the *intensities* of a point process and are trained by its likelihood, so their output is a rate of events rather than a class label. Table 43 summarises the differences.

Table 43: Hawkes kernels, attention, and their combinations.

Model	What sets the weight on a past event	What it outputs
Hawkes kernel (§7)	elapsed time and event type, through a fixed decaying kernel	an intensity
Standard attention (this subsection)	the content of the event, matched with the current query; time only through an encoding	whatever the head predicts, e.g. a class
Hawkes-biased attention (§9.6)	content match multiplied by a decay in elapsed time	whatever the head predicts
Transformer and self-attentive Hawkes processes (Zuo et al., 2020; Zhang et al., 2020) (§7.19.2)	content, with time encoded; in the self-attentive version, attention sets the initial level, the long-run level and the decay rate of each event’s influence	an intensity, trained by likelihood

Transformer inputs on order-book data. The elements a Transformer attends over are called *tokens*, and for the book there are three common choices (Figure 67), corresponding to the inputs of Section 10.2:

- **One token per snapshot.** The window of the last 100 snapshots, each 40 numbers, is a sequence of 100 tokens; attention runs along time, and in some models also across the 40 numbers of each snapshot. TransLOB works this way on FI-2010 windows of 100×40 (Wallbridge, 2020), as do Axial-LOB and TLOB, which alternate attention along time and across features (Kisiel and Gorse, 2022; Berti and Kasneci, 2025).
- **One token per message.** Each order-book message—add, cancel, execute, with its side, price and size—becomes a token, much as words do for a language model. Prices and sizes are put into a few classes so that the number of distinct tokens is manageable: LOBERT has 293 distinct message tokens, built from type, side, distance in ticks from the opposite best price, and size (Linna et al., 2025) (Section 10.8).
- **One token per patch.** A block of the book—for example one side over a short window—becomes one token, as patches of an image do in vision Transformers—Transformers for images, which cut an image into small square tiles and treat each tile as a token (Xiao et al., 2025).

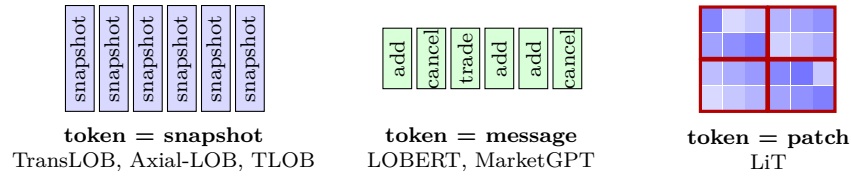


Figure 67: Three ways to cut order-book data into the tokens a Transformer attends over. Schematic.

Training. There are two ways, and they correspond to the two kinds of token.

- **With labels, like DeepLOB.** Snapshot Transformers are trained exactly as in Section 10.4: windows of snapshots as inputs, direction labels as targets, cross-entropy loss, time-ordered splits. TransLOB, for example, was trained for 150 passes over the first seven days of FI-2010 and tested on the last three (Wallbridge, 2020).
- **By predicting the next message, like a language model.** Message Transformers are trained to predict the next message from the ones before it; the target is the data itself, so no labels are needed (*self-supervised* training). LOBERT is first pretrained by filling in hidden messages and then fine-tuned to predict the next message, on Nasdaq messages for four stocks over 80 trading days of 2015 (Linna et al., 2025) (Section 10.10.1); MarketGPT, with about 100 million parameters, is trained on Nasdaq messages to generate realistic message streams (Wheeler and Varner, 2024). A model trained this way can then generate order flow (Section 10.15), or its internal representation can be reused, with a small extra network trained on labels, for a forecasting task (Section 10.10).

Vocabulary used in the rest of this section. A few terms recur from here on.

- An *encoder* turns a sequence into a sequence of vectors, one per element, each summarising that element in its context; during training each element may attend to elements on both sides. A *decoder* sees only earlier elements and is built to generate the next one; GPT is a *decoder-only* Transformer, BERT an *encoder-only* one (Devlin et al., 2019). An encoder followed by a decoder that outputs a different sequence is a *sequence-to-sequence* model.
- A *representation* is the set of vectors a network computes internally from its input; a good one makes later tasks easy.
- *Pretraining* is a first, usually self-supervised training on a large amount of data; *fine-tuning* continues training the pretrained network, usually with a small added output layer (a *head*), on a specific labelled task. A network pretrained once and adapted to many tasks is called a *foundation model*. When the pretrained part is kept fixed during fine-tuning it is *frozen*; when all weights are trained together, training is *end to end*.

The rest of this subsection describes the models that have been built this way, and what they achieve.

Formal treatment. The next subsubsection writes the attention step precisely, with an order-book example worked in numbers.

10.7.2 Attention on the book, in detail

A model that attends along more than one axis of the book is called *spatial-temporal*: “space” is the price ladder (and, in multi-asset models, the set of instruments), “time” is the sequence of snapshots

or events.

Queries, keys and values, step by step. This paragraph builds the attention step from the ground up on order-book events, and then writes it as a formula.

Step 1: each event becomes a vector. A network works with numbers, so each event n is first described by a vector $\mathbf{x}_n$, its *embedding*. It can be built from the event’s attributes—its type and side, its size, its distance from the best price, the time since the previous event—by a learned table or a learned matrix. In the example below the embedding has three numbers: whether the event is a cancellation at the ask, whether it is a limit order at the bid, and its size, scaled to between 0 and 1 (Figure 68).

	ask cancel?	bid limit?	size	
1: ask cancellation, size 0.5	1	0	0.5	$= \mathbf{x}_1$
2: bid limit order, size 0.2	0	1	0.2	$= \mathbf{x}_2$
3: ask cancellation, size 0.8	1	0	0.8	$= \mathbf{x}_3$
4 (current): buy sweep, size 1.0	0.5	0.5	1.0	$= \mathbf{x}_4$

Figure 68: Step 1: each event becomes a vector of three numbers, its embedding $\mathbf{x}_n$. Illustrative numbers, used in the worked example below.

Step 2: each event gets three roles. Three learned matrices, $\mathbf{W}^{\text{qry}}$, $\mathbf{W}^{\text{key}}$ and $\mathbf{W}^{\text{val}}$, turn the embedding into three shorter vectors:

$$\mathbf{z}_n^{\text{qry}} = \mathbf{W}^{\text{qry}} \mathbf{x}_n, \quad \mathbf{z}_n^{\text{key}} = \mathbf{W}^{\text{key}} \mathbf{x}_n, \quad \mathbf{z}_n^{\text{val}} = \mathbf{W}^{\text{val}} \mathbf{x}_n. \quad (65)$$

In words: the same event plays three parts. Its *query* says what it is looking for in the past (“were sellers leaving?”). Its *key* is how it advertises itself to later events (“I am a cancellation at the ask”). Its *value* is the information it hands over when it is chosen (“I was this large”).

Figure 69 shows the multiplication for the example’s matrices.

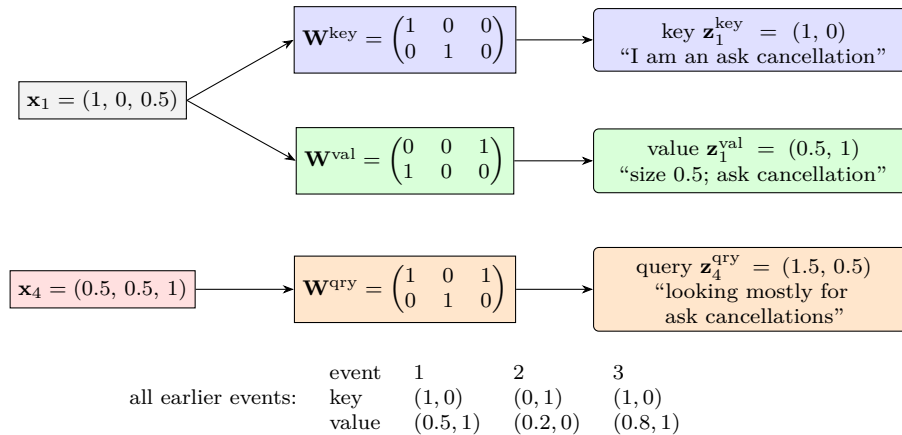


Figure 69: Step 2: each event’s embedding is multiplied by three learned matrices. The earlier events supply keys and values (top, for event 1; the table lists all three). The current event supplies the query (bottom). Example: $\mathbf{W}^{\text{key}} \mathbf{x}_1 = (1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0.5, 0 \cdot 1 + 1 \cdot 0 + 0 \cdot 0.5) = (1, 0)$.

Keeping the key and the value separate lets an event be *found* for one reason and *pass on* something else: a cancellation can be found because it is a cancellation at the ask, and pass on its

size. A library is a useful picture: the query is your search terms, the key is a book’s catalogue entry, and the value is the book.

Step 3: score each earlier event. The current event compares its query with the key of each earlier event n' by a dot product—multiply the two vectors element by element and add—which is large when they point the same way. The score is divided by $\sqrt{\dim_{\text{emb}}}$, where $\dim_{\text{emb}}$ is the length of the query and key vectors (2 in the example below), so that scores do not grow just because the vectors are longer.

Step 4: turn scores into weights. The softmax turns the scores into weights that are positive and sum to one: each weight is the exponential of a score, divided by the sum of the exponentials of all the scores.

Step 5: average the values. The output for the current event is the weighted average of the earlier events’ values. It is a new vector that summarises the past *from the current event’s point of view*. Figure 70 carries the example through steps 3 to 5.

	3. score $\mathbf{z}_4^{\text{qry}} \cdot \mathbf{z}_n^{\text{key}}$	3. $\div \sqrt{2}$	4. soft- max weight	5. weight $\times$ value
event 1: ask cancel	$1.5 \cdot 1 + 0.5 \cdot 0 = 1.5$	1.06	0.40	0.40 (0.5, 1)
event 2: bid limit	$1.5 \cdot 0 + 0.5 \cdot 1 = 0.5$	0.35	0.20	0.20 (0.2, 0)
event 3: ask cancel	$1.5 \cdot 1 + 0.5 \cdot 0 = 1.5$	1.06	0.40	0.40 (0.8, 1)
			total = 1	sum: (0.56, 0.80)

Figure 70: Steps 3 to 5 for the worked example. The query of the buy sweep, (1.5, 0.5), is scored against each key, the scores are scaled and turned into weights, and the values are averaged with those weights. The two ask cancellations receive 0.80 of the weight. Output: average size $0.4 \cdot 0.5 + 0.2 \cdot 0.2 + 0.4 \cdot 0.8 = 0.56$; share of ask cancellations $0.4 + 0 + 0.4 = 0.80$.

Written as one formula, event n attends to earlier event n' with weight

$$\text{att}_{nn'} = \frac{\exp(\langle \mathbf{z}_n^{\text{qry}}, \mathbf{z}_{n'}^{\text{key}} \rangle / \sqrt{\dim_{\text{emb}}})}{\sum_{n'' \leq n} \exp(\langle \mathbf{z}_n^{\text{qry}}, \mathbf{z}_{n''}^{\text{key}} \rangle / \sqrt{\dim_{\text{emb}}})}, \quad \text{output}_n = \sum_{n' \leq n} \text{att}_{nn'} \mathbf{z}_{n'}^{\text{val}}, \quad (66)$$

In words: the numerator is the exponential of event n' ’s score (steps 3 and 4); the denominator adds up the same quantity over all events up to n , so the weights sum to one; the output is the weighted average of the values (step 5).

A worked example with numbers. Figure 71 shows the flow; here are the numbers (illustrative, not from a trained model). Three earlier events have embeddings $\mathbf{x}_1 = (1, 0, 0.5)$ for an ask cancellation of size 0.5, $\mathbf{x}_2 = (0, 1, 0.2)$ for a bid limit order of size 0.2, and $\mathbf{x}_3 = (1, 0, 0.8)$ for a larger ask cancellation. The current event, a buy that sweeps the ask, has $\mathbf{x}_4 = (0.5, 0.5, 1.0)$. Take

$$\mathbf{W}^{\text{qry}} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{W}^{\text{key}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{W}^{\text{val}} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

- **Roles.** The key matrix keeps the event’s type: $\mathbf{z}_1^{\text{key}} = \mathbf{z}_3^{\text{key}} = (1, 0)$ for the ask cancellations, $\mathbf{z}_2^{\text{key}} = (0, 1)$ for the limit order. The value matrix passes on the size and the flag “ask cancellation”: $\mathbf{z}_1^{\text{val}} = (0.5, 1)$, $\mathbf{z}_2^{\text{val}} = (0.2, 0)$, $\mathbf{z}_3^{\text{val}} = (0.8, 1)$. The sweep’s query is $\mathbf{z}_4^{\text{qry}} = (1.5, 0.5)$: it looks mostly for ask cancellations.

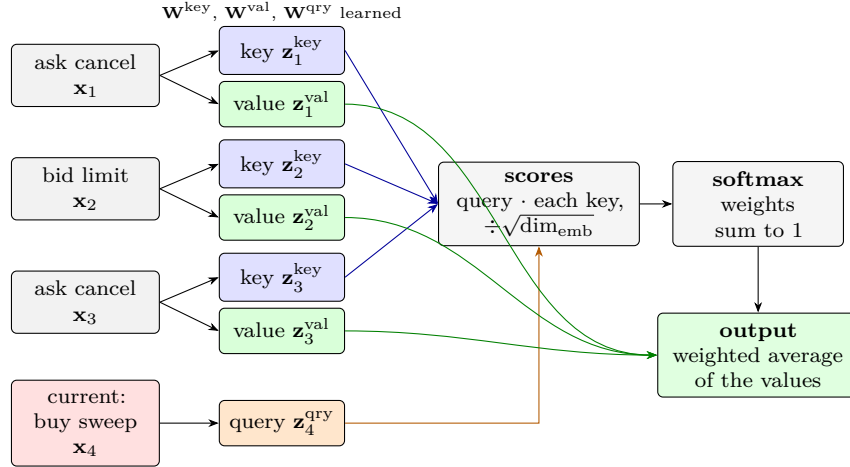


Figure 71: The attention step on order-book events. Each event’s embedding is turned into a key and a value (earlier events) and a query (current event) by three learned matrices; the query is scored against each key, the scores are turned into weights, and the output is the weighted average of the values. Schematic.

- **Scores.** The dot products with the keys are 1.5, 0.5 and 1.5; divided by $\sqrt{2}$, they are 1.06, 0.35 and 1.06.
- **Weights.** The softmax gives 0.40, 0.20 and 0.40: the two ask cancellations receive four-fifths of the attention.
- **Output.** The weighted average of the values is (0.56, 0.80): in words, “80% of the relevant recent activity was sellers cancelling at the ask, and those cancellations averaged size 0.56”. That vector is what the sweep passes on to the next layer.

Here the matrices were chosen by hand to make the example readable. In a real model they are learned: training adjusts W^{qry} , W^{key} and W^{val} , together with the rest of the network, to whatever makes the predictions most accurate. Nobody tells the model to look for cancellations; if they help, it learns to.

Example: attention after a sweep of the best ask. A buy market order empties the best ask—a *sweep*, a market order large enough to take everything resting at the best price. If training has taught the model that sweeps matter when sellers have been leaving, the sweep’s query matches the keys of the ask cancellations in the last few milliseconds, which receive most of the weight; the bid limit orders and the quiet stretch before them receive little. The output is, in effect, a summary “sellers were pulling before this sweep”—the chain reaction of Example 7.3, read off by a learned look-up rather than a fixed kernel.

Applied to a window of snapshots, attention can run along different axes of the grid (Figure 72): along *time*, so that each moment attends to earlier moments; along *levels* or features, so that each level attends to the others at the same moment; or, in multi-asset models (Section 10.11), along *assets*. Axial and dual-attention models alternate between the first two.

TransLOB. Wallbridge (2020) combined five dilated causal convolutions—*causal* because each output uses only the current and earlier events, *dilated* because the filter skips steps (reading every second, fourth, ... event) to see further back with few weights—with two Transformer blocks using masked (causal) self-attention, and reported FI-2010 F1 of 81–92% across horizons. An independent re-implementation obtained 59% on the same data (Prata et al., 2024)—the largest gap between

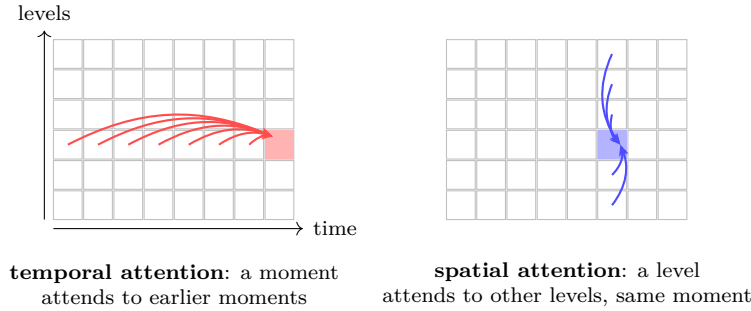


Figure 72: Attention along two axes of a window of snapshots. Temporal attention (left) lets the current moment weight earlier ones; spatial attention (right) lets each price level weight the other levels at the same moment. Schematic.

claimed and reproduced performance among the fifteen models tested there, and a reminder that results on one small data set are fragile.

Axial and dual attention. Axial-LOB (Kisiel and Gorse, 2022) applies gated attention—attention whose output is multiplied by a learned gate between 0 and 1—separately along the time and feature axes, keeping the parameter count below 10^4 . TLOB (Berti and Kasneci, 2025) alternates temporal self-attention (across snapshots) with spatial self-attention (across the features of a snapshot); its companion MLPLOB replaces attention by multilayer perceptrons that mix along the two axes, and matches TLOB on several data sets. Three observations from TLOB are instructive. Performance on Nasdaq and Bitcoin data falls steeply with the horizon—for INTC, F1 drops from about 80% at 10 events to about 50% at 100—whereas on FI-2010 it *rises* with the horizon, a sign of the data set’s peculiarities. Predictability on the same stock declined between 2012 and 2015. And when the label threshold is set to the average spread, so that only moves large enough to pay for crossing the spread—buying at the ask and selling at the bid—count, F1 drops to 31–41% (TSLA, horizons of 50 to 200 events): once a move must cover the cost of trading, the classifier is weak. The authors conclude that the methods “are not sufficiently mature for practical deployment”.

Patches, full books and graphs. Three designs depart from one-token-per-snapshot. LiT (Xiao et al., 2025) cuts the book into patches—one side of the book over a short time window—and applies it to cryptocurrency data from the Binance exchange. Jung and Lee (2025) forecast the entire five-level book with a sequence-to-sequence Transformer and a loss that penalises forecasts violating the ordering of prices (ask prices must rise, and bid prices fall, level by level); on one day of data a linear model is slightly better on the overall forecasting error and on volumes, but worse on the mid-price and much worse at respecting the ordering of prices. HLOB (Briola et al., 2025b) links the volume levels of the book into a graph according to how they move together, and processes it with a convolutional network designed for such graphs.

Multi-horizon models. Zhang and Zohren (2021) attach a sequence-to-sequence decoder with attention to the DeepLOB encoder to produce forecasts at several horizons at once; they also report that training on specialised processors (Graphcore IPUs) took about 15% of the time needed on a GPU, which matters when models are retrained often.

Transformers against simpler models. Head-to-head comparisons on the same data do not show Transformers winning. On twelve days of order-book data for a cryptocurrency pair (ETH–

USDT), five general-purpose time-series Transformers did not beat DeepLOB-family networks at classifying the next move: at a horizon of twenty events the best Transformer was right 68.89% of the time against 70.29% for DeepLOB and 73.10% for an LSTM with a trend decomposition (one that splits the series into a slow trend and fluctuations), and in a simple trading simulation with costs the Transformer-based models did worse (Bilokon and Qiu, 2023); this is a preprint on a single pair. In the LOBench benchmark of five Shenzhen stocks, models trained on one stock and applied to the other four reached mean recalls (Section 10.5) of 0.6814 for DeepLOB, 0.6682 for a general-purpose Transformer (iTransformer) and 0.6568 for TransLOB (Zhong et al., 2025b). A preprint that tracks accuracy against inference compute on FI-2010 reports that an attention-free model that mixes along the two axes reaches TLOB-level scores at lower latency (Hedges, 2026); its tables could not be verified here.

GPT-style models trained on messages. Some studies train decoder-only Transformers—the architecture of GPT—from scratch on sequences of order-book messages. MarketGPT, about 100 million parameters trained on NASDAQ messages with a ticker token—a token naming the stock—so that one model serves several stocks, is used to generate realistic message streams for a simulator rather than to forecast (Wheeler and Varner, 2024). Chen and Glasserman (2026) train such a model on simulated order books and find that, although it produces valid message sequences almost perfectly, its implicit model of the book is wrong, and it “finds predictability where none exists”. We found no study that applies a pretrained text language model to order-book forecasting with reported accuracy.

Model card: Attention and spatial-temporal Transformers

What you calibrate. The query, key and value matrices of every attention layer, the per-element MLP weights, and the choice of axes (time, levels, assets) along which to attend.

How. Supervised training as for the convolutional models; reproduction studies show that the result depends heavily on the data set and labelling, so re-testing on new data is part of calibration.

What you do with it. Direction and multi-horizon forecasts, full-book forecasts, and multi-asset models that choose which peers to read.

How to picture it. Every event asks a question of every earlier event and listens most to the ones whose answers match.

10.8 Tokenisation: how vocabularies of market events are built

Models borrowed from language processing—BERT-style encoders and GPT-style decoders (Section 10.7.1)—read sequences of *tokens* drawn from a fixed vocabulary (Section 5). To apply them to an order book, each market event has to be turned into one or more tokens, and the choice matters for three reasons. The model’s output layer has one entry per token, so a larger vocabulary means more classes to predict and more tokens that occur too rarely to learn. Splitting an event into several tokens makes sequences longer, and the cost of attention grows with the square of their length. And every binning of a continuous price or size throws away detail. The published models make three choices, summarised in Table 44 and illustrated in Figure 73.

1. Token unit: a field or a whole event. Generative models in the line of Nagy et al. (2023) split each message into its fields—type, side, price, size, time since the previous message, and so on—and give each field its own tokens: 22 tokens per message from a vocabulary of 12,011 in their model, 24 per message in MarketGPT (Wheeler and Varner, 2024). A context of 500 messages is then 11,000 tokens long. Most later models instead map each *event* to a single token. LOBERT

does this by listing the combinations of type, side, binned price and binned size that occur in the training data (293 tokens, including special ones) (Linna et al., 2025); UQ-LOB, following it, uses 439 tokens built from five attributes (Edward Manoharan et al., 2026). MarS builds the vocabulary as a Cartesian product, $3 \times 32 \times 32 \times 16 = 49,152$ tokens for order type, price, volume and time interval (Li et al., 2025); TradeFM likewise combines action, side, price depth, volume and time interval into one of $2 \times 2 \times 16 \times 16 \times 16 = 16,384$ tokens (Kawawa-Beaudan et al., 2026). A third option learns the vocabulary: M3 keeps a learned *codebook*, a list of 32,768 prototype vectors, and replaces each event by the nearest prototype (Zhang et al., 2026), as vector-quantised autoencoders do (van den Oord et al., 2017). At the other extreme, some models avoid a hand-designed vocabulary altogether, reading the raw bytes of each message (256 symbols, 32 per event) (Li and Chen, 2025) or applying byte-pair encoding—the method of text language models, which repeatedly merges the most frequent pair of symbols into a new symbol (Sennrich et al., 2016)—to the text of the messages, as the baselines of the LOB-Bench benchmark do (Nagy et al., 2025).

2. Discretisation of continuous fields. Prices and sizes are continuous, so they must be binned, and the reference point matters. LOBERT measures price in ticks from the best quote on the opposite side and uses hand-chosen bins (0, 1, 2, 3, 5 and 10 ticks; sizes 0, 50, 100 and 200 shares with a round-lot flag) (Linna et al., 2025); Nagy et al. (2023) measure ticks from the mid, clipped at ± 999 ; TradeFM uses quantile bins for price—bins chosen so that each holds the same number of observations—and equal-width bins on the logarithm for size and time; M3 measures price from the day’s opening price. General time-series foundation models (Section 10.10), built for single price series rather than books, make similar choices: Chronos scales each series by its mean and uses uniform bins (4,096 tokens) (Ansari et al., 2024); Kronos learns a binary code of 20 bits per candlestick—the open, high, low and close prices of one interval— split into a coarse and a fine token of 1,024 values each (Shi et al., 2026). Others do not discretise at all: TimesFM and Moirai read patches of the series directly (Das et al., 2024; Woo et al., 2024), and LLMTime writes numbers as strings of digits for an unchanged language model (Gruver et al., 2023). Older symbolic methods for time series, such as SAX, cut the range of values at points chosen so that each bin would be equally likely for normally distributed data (Lin et al., 2003, 2007).

3. Whether exact values are kept alongside. Binning loses detail. LOBERT and UQ-LOB therefore feed exact values next to the token: LOBERT keeps price, size and time as continuous inputs (time is not tokenised at all), and UQ-LOB adds seven continuous features, including order-flow and queue-imbalance measures. MarS adds the volumes of the book to the order token’s embedding. TradeFM and M3 keep no continuous channel in the token stream.

Published evidence. Few studies isolate the tokenisation, and several comparisons change more than one thing at once.

- In LOBERT, combining the coarse token with a regression on the exact value reproduced the distribution of prices better than either alone, measured by the Wasserstein distance—how far one distribution must be moved to match another; lower is better: 10.04, against 15.44 for the token alone and 10.85 for the regression alone (Linna et al., 2025).
- In M3, measuring price from the opening price with a learned codebook gave much smaller price errors than mid-relative bins when the mid had to be simulated (the error exceeded in only 1% of cases was 28 ticks, against 764); the comparison changes both the reference point and the discretiser (Zhang et al., 2026).

- In Kronos, predicting the coarse token and then the fine one beat predicting both at once, measured by the information coefficient—the correlation between forecasts and outcomes; higher is better: 0.0431 against 0.0345, and both beat direct regression on continuous values (0.0212); splitting the 2^{20} -entry vocabulary into two halves cut the model from 1,842 million to 102 million parameters, at twice the decoding steps (Shi et al., 2026).
- Outside finance, discrete learned tokens beat patches with the same architecture in the TOTEM study (Talukder et al., 2024).
- The byte-pair-encoded baseline of LOB-Bench reproduced market impact far worse than the field-token model of Nagy et al. (2023), but it also differs in model, size and inputs, so this is not a clean test of tokenisation (Nagy et al., 2025).

We found no study that compares field tokens, composite event tokens and learned codebooks on the same order-book data with everything else held fixed.

Table 44: How published models turn order-book events into tokens.

Model	Tokens per event	Vocabulary	Fields and discretisation	Exact values kept
Nagy et al. (2023)	22	12,011	every field; price in ticks from the mid (clipped); time in digit groups	no
MarketGPT (Wheeler and Varner, 2024)	24	about 12,100	as above, plus instrument	no
LOBERT (Linna et al., 2025)	1	293	type, side, price and size bins (combinations observed)	price, size, time
UQ-LOB (Edward Manoharan et al., 2026)	1	439	type, side, size, price distance, simultaneity flag	7 features
MarS (Li et al., 2025)	1	49,152	type $\times$ price $\times$ volume $\times$ interval	book volumes
TradeFM (Kawawa-Beaudan et al., 2026)	1	16,384	action $\times$ side $\times$ price depth $\times$ log size $\times$ log interval	no
M3 (Zhang et al., 2026)	1	32,768	learned codebook; price from the open	book embedding
ByteGen (Li and Chen, 2025)	32 bytes	256	raw binary fields	no

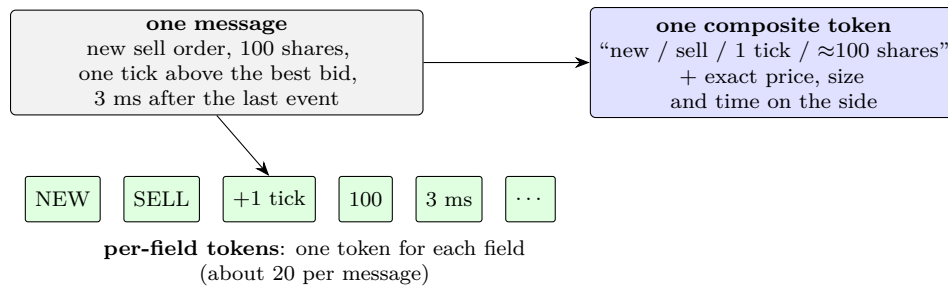


Figure 73: Two ways of tokenising the same message. Field-level schemes (Nagy et al., 2023) emit one token per field; event-level schemes (Linna et al., 2025) map the whole event to a single token from a vocabulary of binned combinations, and may pass the exact values alongside. Schematic.

10.9 State-space models

Structured state-space sequence models such as S4 (Gu et al., 2022) and Mamba (Gu and Dao, 2023) keep a state vector $\mathbf{s}$ and update it at each step with the new input $\mathbf{x}$ by a learned linear rule, $\mathbf{s} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{B}\mathbf{x}$, reading predictions off the state. This is the same mathematical object as the classical state-space model of control theory (Kalman, 1960), and it can be computed in parallel during training. Mamba lets $\mathbf{A}$ and $\mathbf{B}$ depend on the input, so the model decides at each step how much to forget. Such models handle long sequences at a cost proportional to their length, unlike attention, which makes them attractive for event streams, which are long and irregular (Section 7.19.3 describes a Hawkes version). We found no peer-reviewed S4- or Mamba-based order-book forecaster with comparisons against baselines. The closest is a design tuned for fast hardware, applied to cryptocurrency tick data (recorded at every trade or price change), whose abstract reports 62.7% next-tick directional accuracy without a baseline for comparison (Popov and Huber, 2026). State-space models for order books remain an open direction rather than an established result (Section 14).

Model card: State-space sequence models

What you calibrate. The matrices of a learned linear recurrence and, for selective models such as Mamba, the functions that set its step size from the input.

How. Supervised training by gradient descent, with the recurrence computed in parallel during training.

What you do with it. Long-context forecasts on long, irregular event streams; for LOB data this is an open direction rather than an established result.

How to picture it. A Kalman-style running state, updated at every event, whose rate of forgetting is learned.

10.10 Pretrained encoders, foundation models and uncertainty

A *foundation model* is pretrained once on a large, generic corpus and then adapted to many specific tasks. Two recent contributions from the same group illustrate a useful layering for LOB data.

10.10.1 LOBERT in detail

LOBERT (Linna et al., 2025) is described by its authors as a “general-purpose encoder-only foundation model” for order-book messages. Each word in that description has a specific meaning, and together they describe a way of building models that differs from everything earlier in this section.

Terminology.

- **Message-level.** It reads the stream of individual order-book messages—every new order, change, cancellation and execution, as in Level 3 data—rather than windows of book snapshots.
- **Encoder.** It is a Transformer (Section 10.7.1) that turns a sequence of messages into a sequence of vectors, one per message, each summarising that message in its context. “Encoder-only”, as in the BERT language model (Devlin et al., 2019), means that during its first training every message may attend to messages both before and after it; a “decoder”, as in GPT, sees only the past and is built to generate the next token.
- **Pretrained.** It is first trained on a generic task that needs no labels—filling in hidden messages—on a large amount of data, so that it learns how message streams are structured.

- **Fine-tunable, a foundation model.** The pretrained network is then adapted to specific tasks, such as predicting the next message or the direction of the mid-price, by adding a small output layer (a *head*) and training briefly on that task. One expensive pretraining serves many cheaper tasks.

Step 1: one token per message. A language model turns text into tokens drawn from a fixed vocabulary. LOBERT does the same for messages. Each message is described by five fields, and each field is put into a few classes:

- *side*: buy or sell;
- *type*: new order, edit, delete, execution, or hidden order;
- *price*: the distance in ticks from the best price on the opposite side—the best bid for a sell, the best ask for a buy—put into the classes 0, 1, 2, 3, 5 and 10 ticks;
- *size*: put into the classes 0, 50, 100 and 200 shares, chosen because sizes cluster at round numbers (over 60% of sizes in the data are exactly 100);
- *round*: whether the size is exactly on one of those values or between them.

The five classes are joined into one token, so that, for example, a new sell order two ticks from the best bid for exactly 100 shares becomes a single token. Only the combinations that occur in the training data are kept; with three special tokens (for padding, hidden positions and unknown messages) the vocabulary has 293 tokens. Executions always occur at the best price, so their price class is set to zero.

Step 2: continuous embeddings of prices and sizes. Putting prices and sizes into classes loses detail, and rare values would need many extra tokens. LOBERT therefore keeps three exact numbers alongside each token: the price distance, the size, and the time since the previous message, each rescaled to lie between 0 and 1 (linearly up to a point, then more and more compressed, so that extreme values cannot dominate). Time is kept only as a number, because the authors found no pattern in it worth turning into classes.

Step 3: add the book. Optionally, each message is accompanied by the snapshot of the book just after it: the ten best prices and sizes on each side, 40 numbers, with prices measured in ticks from the opposite best price and sizes compressed to between 0 and 1.

Step 4: combine into one vector per message. The token is looked up in a learned table of vectors; the three exact numbers and the snapshot are turned into vectors by learned layers; and the vectors are added up, the snapshot through a learned gate that decides how much of it to use. Each message is now one vector.

Step 5: a Transformer encoder with time encoding. The vectors pass through a stack of Transformer blocks (Figure 66). Order is encoded twice: by the message’s position in the sequence, and by a rotary encoding of the cumulative time between messages, so that attention can tell two messages one millisecond apart from two messages a second apart. Sequences are 512 messages long.

Step 6: the heads. The output vector at a position feeds four heads: one predicts the token (a choice among the 293), trained by cross-entropy; three predict the exact price distance, size and time, trained by squared error. The three number heads also see the token head’s output, so that the predicted number can be kept within the range of the predicted token—the “combined” mode the authors found to give the most realistic values. Figure 74 draws the whole pipeline.

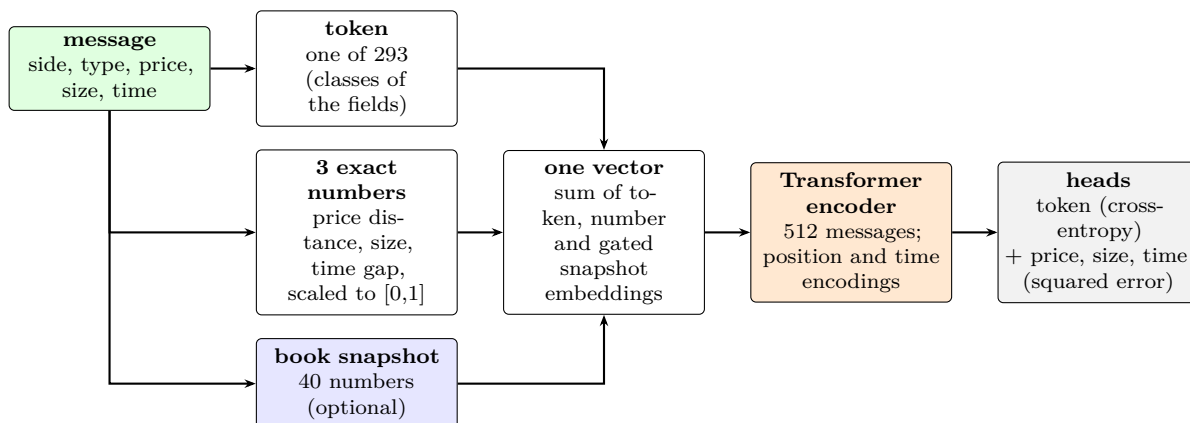


Figure 74: LOBERT’s pipeline: each message becomes one token plus three exact numbers, optionally with the book snapshot after it; these are combined into one vector per message, passed through a Transformer encoder that is aware of the time between messages, and decoded by a token head and three number heads. After Linna et al. (2025).

Two-stage training.

1. **Pretraining by masked message modelling.** Random messages in each sequence of 512 are hidden, and the model is trained to reconstruct them—token and exact numbers—from the messages on *both* sides, as BERT is trained to fill in hidden words in a sentence. No labels are needed: the answer is the message that was hidden. One subtlety: the snapshot after a message reveals what the message did to the book, so the snapshots are hidden too, at 90% of positions, to stop the model reading the answer off them.
2. **Fine-tuning for a task.** The pretrained weights are the starting point for each task. For *next-message prediction*, a causal mask lets each position see only earlier messages, and the model learns to predict the next one. For *mid-price direction*, a classification head is added on top and the model is trained with labels: the direction of the average mid-price over the next $h = 10, 50$ or 100 messages relative to the current one, with a threshold that grows with the horizon (zero at 10 messages, 0.232 at 50 and 0.332 at 100, in ticks, by our calculation from the paper’s formula). Because the label starts from the current price, it does not have the leakage of Section 10.5.

The data are Nasdaq ITCH messages for four stocks (AAPL, INTC, MSFT, FB): 80 trading days from May to September 2015 for training, the next 10 days for validation and the last 10 of September for testing—about 470 million messages, cut into 919,000 sequences of 512. Training uses the AdamW optimiser with a learning rate of 5×10^{-5} , for ten passes over the data, with batches of 32.

Reported results. On next-message prediction, compared with a state-space model of similar size (S5 (Nagy et al., 2023), re-implemented with 1.2 million parameters, against LOBERT’s 1.1

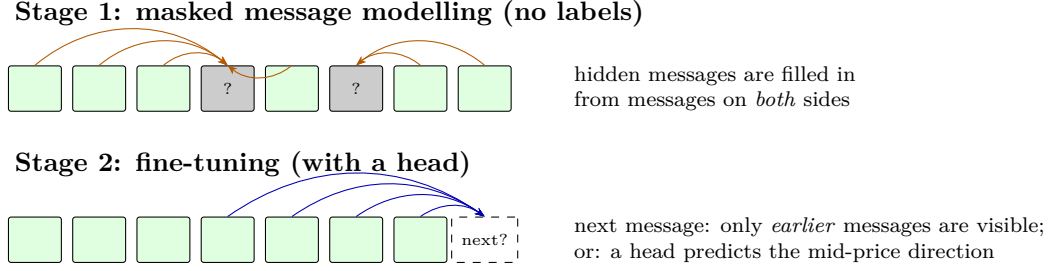


Figure 75: LOBERT’s two training stages. Stage 1 hides random messages (grey) and learns to reconstruct them from both sides; stage 2 starts from those weights and trains for a task, with a causal mask for next-message prediction or a classification head for mid-price direction. Schematic.

million), LOBERT predicts every field of the next message more often correctly (Table 45): the whole message, all fields right, 26.4% of the time against 6.1%, and 27.8% with the book snapshot.

Table 45: Next-message prediction: share of next messages whose field was predicted correctly, on the test days. From Linna et al. (2025), Table 1.

Model	Type	Side	Price class	Size class	Whole message
S5 (state space)	50.8%	52.2%	18.6%	32.1%	6.1%
LOBERT	60.6%	65.1%	51.8%	72.1%	26.4%
LOBERT with the book	61.9%	65.4%	53.8%	72.2%	27.8%

On mid-price direction, LOBERT and DeepLOB, re-trained on the same data, are compared using *selective prediction*: a model makes a forecast only when its highest class probability exceeds a confidence threshold, and is scored on the cases it chooses to forecast; *coverage* is the share of cases it forecasts. Table 46 shows the two ends of the range. When both forecast every case, LOBERT is ahead at 10 messages (macro-F1 0.51 against 0.44), slightly ahead at 50 (0.56 against 0.53) and level at 100 (0.55 each). As the threshold rises, LOBERT’s score climbs much faster—to 0.88 at 100 messages—but on fewer and fewer cases: 10% of them, against DeepLOB’s 0.61 on 28%. Scores on different subsets of cases are not a like-for-like comparison; what the numbers show is that LOBERT’s confidence separates easy cases from hard ones more sharply.

Table 46: Mid-price direction, macro-F1 and coverage, averaged over four stocks, at the lowest and highest confidence thresholds reported. From Linna et al. (2025), Table 3.

Threshold, model	$h = 10$		$h = 50$		$h = 100$	
	F1	coverage	F1	coverage	F1	coverage
0.3, LOBERT	0.51	1.00	0.56	1.00	0.55	1.00
0.3, DeepLOB	0.44	1.00	0.53	1.00	0.55	1.00
0.9, LOBERT	0.82	0.29	0.84	0.14	0.88	0.10
0.9, DeepLOB	0.48	0.50	0.60	0.40	0.61	0.28

Limitations. LOBERT is a preprint, tested on four stocks over twenty test days of 2015, and only against one state-space model and DeepLOB. Its inference—making predictions once trained—is 53% slower than DeepLOB’s (about 282 predictions per second on one V100 GPU with batch size one, 47% of DeepLOB’s throughput). The authors list as limitations that realistic long generated sequences are not established, that the model may lose track of order size and location when price levels move, that it does not model individual order identifiers, and that the data’s millisecond time

stamps cover time only partly. What it shows is the pattern of foundation models applied to the book: learn the structure of the message stream once, without labels, then adapt it cheaply to many tasks.

10.10.2 Confidence and uncertainty

Point forecasts and uncertainty. The deep models of this section are almost all point predictors: they output a direction or a price change, but not how far that output should be trusted. Edward Manoharan et al. (2026) make the point by surveying the main lines of work: models that predict the next message and then feed their own prediction back in to predict the one after (*autoregressive* roll-forward, where errors compound with the horizon) (Nagy et al., 2023); direction classifiers, from DeepLOB to message-level encoders (Zhang et al., 2019a; Tran et al., 2019; Wallbridge, 2020; Xiao et al., 2025; Linna et al., 2025); and regressions on the size of the price change (Yang et al., 2025b). In all of these, they observe, the output carries no measure of confidence. They argue that this is a serious gap for trading. How predictable the price is varies across market regimes and even between consecutive windows, and every action costs fees and slippage—the gap between the price expected and the price actually obtained—so the question that matters is rarely only which way the price will move, but whether this particular forecast is reliable enough to act on. That is the setting of *selective prediction*: classification with the option to abstain, studied since Chow (1970), with the trade-off between how often a model predicts (its coverage) and how often it is wrong when it does (its *risk*, here an error rate, not a financial risk) characterised by El-Yaniv and Wiener (2010) and applied to deep networks by Geifman and El-Yaniv (2017). Acting on it needs an uncertainty that depends on the current input and is *calibrated*: an interval said to contain the outcome 68% of the time should contain it about 68% of the time. The selective evaluation of LOBERT above, which scores only the forecasts made above a confidence threshold, is an example of the same idea.

Uncertainty layers. UQ-LOB (Edward Manoharan et al., 2026) is a plug-in module that can be attached to any pretrained order-book encoder. To make a forecast, it looks not only at the current window but also at a set of recent windows whose outcomes are already known, and uses how the encoder fared on those to judge the current case (the idea of *attentive neural processes*). It outputs a forecast together with a single confidence number: in its regression version, a bell-shaped (normal) distribution for the coming price change in ticks; in its classification version, probabilities for down, stationary and up. The paper tests it with one encoder, a bilinear network with attention, on Level-2 and Level-3 data from the Kraken cryptocurrency exchange for seven assets, about 5.2 billion events. Its intervals said to contain the outcome 68% of the time did so close to 68% of the time: the confidence was *calibrated*.

For this survey the point is the change of target: from a direction to a direction *with a calibrated confidence*. That is precisely the input an execution policy needs, because the decision to place, keep or cancel a passive order depends on how sure the model is, not only on which way it leans (Sections 1.4 and 11.1).

General time-series foundation models. A parallel line of work pretrains forecasters on very large collections of generic time series and applies them *zero-shot*, without training on the target series. Chronos (Ansari et al., 2024) scales each series by its mean absolute value, bins the values into 4096 tokens and trains a language-model architecture with a cross-entropy loss on about 84 billion observations. TimesFM (Das et al., 2024) is a 200-million-parameter decoder over patches of 32 time points, trained on about 100 billion points, most of them Wikipedia page views. Moirai

(Woo et al., 2024) is an encoder that outputs a full probability distribution, trained on 27 billion observations of which 0.1% are economic or financial. Lag-Llama (Rasul et al., 2023) feeds a decoder with past values at fixed delays and outputs a heavy-tailed distribution. None of these is trained or evaluated on order-book data.

Finance-specific versions change the corpus. Kronos (Shi et al., 2026) is pretrained on over 12 billion candlestick bars (open, high, low, close, volume) from 45 exchanges and reports large gains over general models in zero-shot price forecasting; its data are bars from five minutes to a day, not messages. TradeFM (Kawawa-Beaudan et al., 2026), a 524-million-parameter preprint from J.P. Morgan AI Research, tokenises individual trade-flow events (inter-arrival time, price depth, volume, side, add or cancel) from more than 9,000 US equities and generates order flow inside a deterministic matching simulator. Its rollouts reproduce heavy tails (large moves more frequent than a normal distribution allows) and volatility clustering (calm and turbulent periods), stylised facts of Section 3, with a smaller distance to the observed 10-second return distribution than a compound Hawkes model (0.013 against 0.039), but the Hawkes model matches the distribution of spreads better. The evidence for general models on financial prediction is weak: on daily excess returns from 94 countries, Chronos and TimesFM used zero-shot have negative out-of-sample R^2 (−1.37% and −2.80%)—worse than always predicting the average—and lose to gradient-boosted trees, a standard ensemble of decision trees, and pretraining from scratch on financial data closes much of the gap (Rahimikia et al., 2025). We found no fully reported study of zero-shot general models on intraday or tick-level order-book data.

Model card: Foundation encoders and uncertainty layers

What you calibrate. An encoder pretrained on large volumes of message data; then a small task head—and, for UQ-LOB, an uncertainty module—fine-tuned on the target task.

How. Self-supervised pretraining (for example, predicting masked or next messages), followed by supervised fine-tuning; for the uncertainty layer, a context set of recently completed windows.

What you do with it. Start new tasks from a pretrained representation, forecast only when confident (as in LOBERT’s selective evaluation), and pass a calibrated confidence to an execution policy that can threshold it. General time-series foundation models serve as baselines rather than as LOB models.

How to picture it. A reader who has studied a library of order flow before being asked a specific question, and who also says how sure the answer is.

10.11 Multi-asset and universal models

Two questions arise once more than one instrument is involved. Can one model, with the same weights, be trained on many assets and used on assets it has not seen? And does the order flow of one asset help predict another? The published models connect assets in three ways (Figure 76): by sharing weights across assets without exchanging any information between them; by feeding several assets’ data into one network that attends across them; or by a graph whose edges link assets that are related.

One model for many stocks. Sirignano (2019) introduced a “spatial” network that exploits the ordering of price levels to use information deep in the book with few parameters, trained on 489 US stocks over twenty months on a GPU cluster; it outperforms logistic regression and a standard network, especially on rare, large moves (the tails of the distribution). Sirignano and Cont (2019) then train a single LSTM on pooled data from 464 stocks to predict the direction of the next mid-price move. On 25 stocks it never saw in training, the pooled model beat models trained on each stock’s own data in all 25 cases, by 1.45 percentage points of accuracy on average, and was

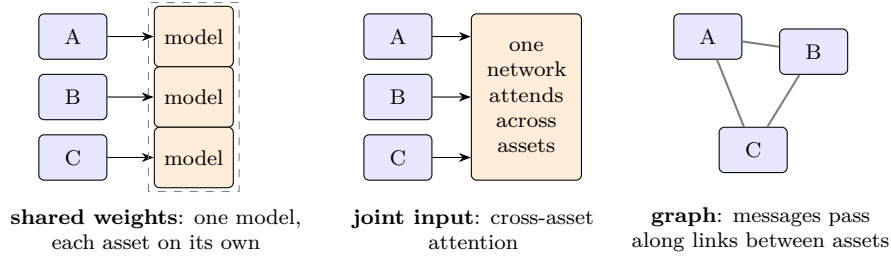


Figure 76: Three ways deep models connect several assets. Schematic.

about as good (-0.15 points) as a pooled model that had seen those stocks. Longer pooled history helped: training on nineteen months beat training on one month by 7.2 points on average. They read this as evidence of universal features of price formation. The result is for the next mid-price move, with no trading costs. Later work qualifies it. With the same architecture, [Lucchese et al. \(2024\)](#) find that a pooled model detects predictability for two of five unseen Nasdaq stocks (AAL and ATVI) but none beyond ten events for the other three, and that pooled models need the most detailed input—volumes on a price grid from full order data—to work at all. On FI-2010, a bilinear network trained on four stocks and applied to the fifth scores 65.62 F1 on average against 66.27 for one trained on all five, and a low-rank adaptation to the new stock—adjusting the weights by a small learned correction built from two thin matrices, instead of retraining them all—reaches 69.10 ([Shabani et al., 2023](#)). In the LOBench benchmark, a pretrained encoder that is frozen (Section 10.7.1), with only its decoder retrained on part of the new stock’s data, transferred best across five Shenzhen stocks (mean recall 0.726, against at most 0.681 for models trained end to end) ([Zhong et al., 2025b](#)).

Direction of transfer between liquid and illiquid markets. The one test we found of which way transfer works comes from electricity: in continuous intraday electricity markets, models trained on the more liquid market transfer well to the less liquid one, while the reverse raises the average quantile loss—the error measure for percentile forecasts; lower is better—from 3.30 to 23.84 ([Yu et al., 2025](#)). Fine-tuning a model trained on one stock on 30% of another’s data recovered much of its accuracy in a study of reconstructing deeper book levels from top-of-book data, though that study used a randomly shuffled split ([Shi et al., 2021](#)).

Attending across assets. OF-MATNet ([Bandealineaeni et al., 2025](#)) feeds multi-level order-flow imbalance from many Nasdaq assets into a Transformer that attends along time, assets and levels, choosing peer assets by rolling Granger-causality tests—whether the past of one asset improves forecasts of another—; its abstract reports improvements in out-of-sample R^2 in over 90% of cases on 110 assets, which we could not verify beyond the abstract. A CNN-LSTM applied jointly to nine Eurodollar futures contracts (interest-rate futures), with a learned estimate of how their forecast errors move together, beat a multilayer perceptron on mean squared error by 2.5% on average, but did not report a comparison with single-contract models ([Spears et al., 2021](#)).

Graphs over assets. The clearest verified evidence that links between assets help comes from volatility, not price direction. A graph Transformer over about 500 S&P 500 stocks, using top-of-book data, forecast ten-minute realised volatility—the variability of the price, measured from returns within the interval—better when relations between stocks were added to the same model: the root mean squared percentage error fell from 0.2498 to 0.2287 ([Chen and Robert, 2022](#)). The largest

single gain came from a relation in time rather than between stocks (0.2358 alone), relations between stocks alone gave 0.2382 to 0.2422, and without relations the graph model was worse than the simple HAR volatility model (a linear model of volatility on its own daily, weekly and monthly averages) at a twenty-minute horizon.

Summary of the evidence on multi-asset models. Pooling many assets into one model is supported for the direction of the next move, with qualifications for longer horizons and for some stocks. That one asset’s order flow improves deep forecasts of another asset’s *price* is, as far as we could verify, not yet shown with reported numbers; the verified cross-asset gains are for volatility, and the linear evidence on cross-asset order flow is reviewed in Section 4.6.

Model card: Multi-asset and universal deep models

What you calibrate. One set of network weights shared across assets; for joint or graph models, also how assets interact—cross-asset attention weights or the links of a graph.

How. Train on pooled data from many assets; test on assets and periods not used in training; for transfer, freeze the shared part and adapt a small part to the new asset.

What you do with it. Forecast for assets with little history of their own; test whether patterns are common across assets; forecast volatility using related assets.

How to picture it. One trader who has watched hundreds of stocks reading a new one, rather than a specialist who has only ever watched that one.

10.12 Benchmark and re-test results

Individual model papers report results on their own terms. Two studies re-test many models under common conditions, and two more vary the input or the evaluation while holding the model fixed; their findings are more informative than any single paper.

LOBench. Zhong et al. (2025b) benchmark nine encoders—including DeepLOB, TransLOB, an LSTM, iTransformer and TimesNet, re-implemented at 15 to 49 million parameters, far larger than the originals—on a year of three-second snapshots of five Shenzhen stocks, for three tasks: rebuilding the input (reconstruction), filling in missing values (imputation) and predicting the direction (trend prediction). Transformer-based encoders reconstructed the book best; in transfer across stocks DeepLOB had the best mean among models trained end to end, and a frozen pretrained encoder did best overall. Its test sets are balanced by down-sampling, which differs from the class frequencies a trader faces.

LOBCAST. Prata et al. (2024) re-implemented fifteen published models and tested them on FI-2010 and on new Nasdaq data from 2021 and 2022. Their headline is that “all models exhibit a significant performance drop when exposed to new data”: F1 on the new data lies between 48% and 61% for every model. The best model on the new data, BiNCTABL, a bilinear network of the TABL family, loses about a fifth of its FI-2010 score. Five of the six best models use attention. A simple MLP ranks near the bottom but degrades the least. Parameter counts range from about 10^4 to 10^7 , with no clear relation between size and degradation. A trading simulation shows that profitability “is far from guaranteed”, and a learned combination of the fifteen models never beat the best single one (Section 8).

Simple models on the same inputs. Briola et al. (2020) compare a random classifier, logistic regression, an MLP, LSTMs and a CNN–LSTM on identical inputs for one large-tick stock (one whose spread is usually a single tick). The MLP matches or exceeds the CNN–LSTM at all three horizons, while logistic regression is clearly worse. Linear models are not generally competitive with deep ones on raw book inputs; simple non-linear models often are.

Representation. Lucchese et al. (2024) hold the DeepLOB architecture fixed and change only the input, on ten Nasdaq stocks. Models fed order-flow inputs or a new volume-on-a-price-grid representation “considerably” outperform those fed the raw level-by-level book, and predictability is detected up to about 50 order-book events ahead at the 99% confidence level, using model confidence sets (Hansen et al., 2011) to compare models. The authors stress that mid-to-mid returns “are not tradable in practice”: a trader buys at the ask and sells at the bid, not at the mid. Kolm et al. (2023), on 115 Nasdaq stocks, find that networks trained on order flow “significantly outperform most models trained directly on order books”, and that the effective horizon of stock-specific forecasts is about two average price changes. In a later practitioner-oriented account, as reported in coverage of the authors’ 2026 award, very deep models “don’t really help” for this task, and careful time-stamping and stationary transformations of inputs—using changes rather than levels, so that their statistics do not drift—mattered more (Kolm and Westray, 2025). Wu et al. (2021) show that level-based inputs are fragile: adding orders in empty price ticks drops DeepLOB’s accuracy from about 77% to about 51%. The reason is the level-by-level input: a new order in a gap becomes a new “level”, shifting every deeper level by one column, so the same book looks completely different to the network. They propose representations on a fixed price grid as an alternative.

Trading with forecasts after costs. Briola et al. (2025a) retrain DeepLOB on fifteen Nasdaq stocks grouped by tick size and introduce a transaction-level score,

$$\text{TO} = \frac{N_{\text{correct}}}{N_{\text{possible}} + N_{\text{predicted}} - N_{\text{correct}}}, \quad (67)$$

where N_{possible} counts the round-trip trades (open, then close) that the true price path would have allowed, $N_{\text{predicted}}$ those the forecasts would have triggered, and N_{correct} those in both sets.

In words: TO, which its authors call p_T , is the overlap between the trades the model would make and the trades that were actually there to be made, as a fraction of their union—a score of 1 means the model traded exactly the available opportunities, 0 means no overlap.

The values are sobering: between about 0.01 and 0.19 across stocks and horizons at the lowest confidence threshold, and close to zero for small-tick stocks at thresholds of 0.5 and above; even where F1 is high, as at the highest threshold, the score can be zero. Predictability, by both F1-type and transaction-level measures, is higher for large-tick stocks—those whose average spread is below about 1.5 ticks—than for small-tick stocks, whose spread exceeds about 3 ticks. The authors conclude that “high forecasting power does not necessarily correspond to actionable trading signals”.

10.13 Representation versus architecture

The benchmark evidence suggests that the choice and treatment of inputs can matter as much as the network—a synthesis across studies, since none compares both systematically. Four decisions should be made, and reported, before capacity is increased.

Representation

raw levels, order flow, volume on a price grid, or individual messages (Lucchese et al., 2024; Kolm et al., 2023; Wu et al., 2021; Linna et al., 2025).

Table 47: Selected deep LOB models. F1 values are as reported by the authors on the data stated and are not comparable across rows (Section 2.4).

Model	Architecture	Data	Reported result	Independent re-test
CNN (Tsantekidis et al., 2017a)	convolutions over time $\times$ levels	FI-2010	F1 55–59%	–
LSTM (Tsantekidis et al., 2017b)	40-unit LSTM	FI-2010	F1 61–66%	–
DeepLOB (Zhang et al., 2019a)	CNN + Inception + LSTM, $\sim 60k$ params	LSE, 5 stocks, 2017	F1 61–70%	re-trained on Nasdaq (Briola et al., 2025a): p_T 0.01–0.19
TABL (Tran et al., 2019)	bilinear + temporal attention, $\sim 10k$ params	FI-2010	F1 73–74%	70% on FI-2010, 60% on 2021 data (Prata et al., 2024)
TransLOB (Wallbridge, 2020)	dilated causal CNN + Transformer	FI-2010	F1 81–92%	59% on FI-2010 (Prata et al., 2024)
Axial-LOB (Kisiel and Gorse, 2022)	axial attention, $< 10k$ params	FI-2010	F1 76–86%	73% on FI-2010 (Prata et al., 2024)
TLOB (Berti and Kasneci, 2025)	dual spatial/temporal attention	FI-2010, Nasdaq, BTC	F1 to 93% (FI-2010); 50–80% (INTC)	–
LiT (Xiao et al., 2025)	structured patches + Transformer	Binance, 2024	F1 59–66%	–
OF-MATNet (Bandealinaeini et al., 2025)	multi-asset, multi-axis attention on OFI	110 Nasdaq assets	R^2 gains in $> 90\%$ of cases	–

Normalisation

fixed statistics from the training set, rolling-window statistics, or a learned normalisation (Passalis et al., 2020a); the statistics of LOB data drift over days.

Target

label construction, threshold relative to the spread, and horizon (Berti and Kasneci, 2025).

Clock

event time, a fixed number of book updates, or calendar time; and how events are time-stamped (Kolm and Westray, 2025).

In one further comparison, order-flow-imbalance inputs beat raw book inputs for an LSTM on 146 of 149 test sets of Chinese stocks (Yang et al., 2025a); an independent survey of representations concludes that the evidence “does not identify a universally preferred architecture” (Barbosa De Oliveira, 2026).

Evaluation rule (strongest-baseline comparison). A complex architecture should be compared not only against another complex architecture, but against the strongest reasonable low-capacity model on the best available representation.

Table 48: The comparisons a deep-model study should include. Each row should be reported with out-of-sample log loss and out-of-sample net P&L under the same execution model. The comparison that matters is the Transformer on raw snapshots against the logistic model on OFI and queue imbalance, not the Transformer against DeepLOB. Log loss is the cross-entropy of Equation (62) on the test data; net P&L is profit and loss after costs.

Model	Input	Parameters	Typical inference latency
Logistic	$\text{OFI}_t^{(w)}$, $w = 1, \dots, 5$ s; QI_t (§4)	~ 10	ns
CNN-LSTM (DeepLOB-type)	10-level snapshots	$\sim 10^5$	μs –ms
Transformer	raw snapshots	$\sim 10^6$	ms
Transformer	message stream + OFI	$\sim 10^6$	ms

10.14 Fill-probability models

Deep learning has also been applied to the execution problem directly: how long will a resting limit order wait before it is filled? The natural answer is a *survival curve*, $S(u) = \mathbb{P}(\tau_F > u \mid X_t)$, the probability that the order is still unfilled after time u , where τ_F is the time until it is filled; the curve starts at one and can only go down. Training such a model has one complication. Many orders are cancelled before they are filled, and a cancelled order tells us only that it was not filled *before* it was cancelled, not when it would have been. Such observations are called *censored*, and throwing them away would bias the model towards fast fills.

An early example is Maglaras et al. (2022), who use a recurrent network to estimate the distribution of the time until a limit order is filled, given current market conditions, and report that it outperforms several benchmarks on historical data and lowers costs in a simplified, simulated trading task; we read only the abstract. Arroyo et al. (2024) estimate the full survival curve with an encoder that combines convolutions and attention over recent market activity, and a decoder constrained to output a curve that only goes down. Cancelled orders enter as censored observations, and models are trained and compared by the likelihood that accounts for censoring. That likelihood is a *proper scoring rule*: a score that is best only when the predicted probabilities are the true ones.

The authors explain why two common alternatives, the Brier score and the concordance index, are unsuitable for limit orders, because their assumptions fail. In their study, queue position enters through the design of the data—hypothetical orders placed at the back of the queue—rather than as an input. KANFormer (Zhong et al., 2025a) adds the order’s queue position and features of the market participants placing orders as inputs, on futures on the French CAC 40 stock index, and reports a large improvement over the earlier architecture both in likelihood and in how well it ranks quickly filled orders ahead of slowly filled ones, which it attributes to the participant information and queue position. The study covers one futures contract, but it is consistent with the claim of Section 11.2 that queue position is a first-class variable.

Model card: Fill-probability (survival) models

What you calibrate. A survival curve $\mathbb{P}(\tau_F > u \mid X_t)$ for a resting order: for deep models, the weights of an encoder and a decoder constrained to output a monotone curve.

How. Maximise the right-censored log-likelihood, treating cancelled orders as censored observations; queue position enters as an input or through the design of hypothetical orders.

What you do with it. Choose between a limit and a market order, set how long to wait before repricing, and weight expected profit by the probability of being filled.

How to picture it. A waiting-room clock for the order, which runs faster when the queue ahead is short and the other side is aggressive.

10.15 Generative models

Generators. A growing literature generates synthetic order flow rather than predicting one quantity. Three kinds of generator are used. An *autoregressive* model generates one token at a time, each from the ones before it, as language models do. A *generative adversarial network* (GAN) trains two networks against each other: a generator that produces fake data, and a discriminator that tries to tell fake from real, until the discriminator can no longer tell. A *diffusion model* learns to remove noise step by step, and generates by starting from pure noise.

Nagy et al. (2023) tokenise LOB messages and train an autoregressive model that generates message streams, processed through a simulator; LOB-Bench (Nagy et al., 2025) standardises the evaluation of such generators. Conditional GANs act as “world agents” that produce realistic order flow, reacting to the user’s orders, inside the ABIDES simulator, a simulated market populated by many programmed traders (an *agent-based* simulator) (Coletta et al., 2021, 2022; Byrd et al., 2020). Diffusion models generate future volume snapshots (Wang and Ventre, 2025), order flow conditioned on the book (Berti et al., 2025), and books conditioned on a chosen future regime, to ask “what if” (Wang and Ventre, 2026). M3 (Zhang et al., 2026) trains an autoregressive model on a joint sequence of order events and book states for 800 Chinese stocks and about 32 billion events. It reports a scaling law—larger models do predictably better—the stylised facts of Section 3, and, when large orders executed in pieces are injected, a price impact that grows with the square root of the order’s size, as observed in real markets. Neural extensions of queue-reactive models (Bodor and Carlier, 2025) reproduce square-root impact and the correlations between queues on Bund futures (Section 9.3).

Evaluation of generators. A generator is useful for research only if it preserves the joint structure of orders, queues, executions and prices—not just marginal statistics, the distribution of one quantity on its own, such as the average spread. The evaluation criteria in this literature are: marginal distributions (spreads, depths, inter-event times); temporal dependence (clustering, long memory;

Section 7); cross-event dependence (how cancellations follow trades); price dynamics (volatility, and signature plots—measured volatility as a function of the sampling interval, Section 7.11); liquidity dynamics (queue depletion and refill); responsiveness to injected orders (market impact); and downstream utility—whether a model trained on generated data performs well on real data (Nagy et al., 2025; Jain et al., 2024b). Consistent with Principle 1 (“the market is observed, not generated”, Part I), this survey treats generators as tools for simulation and stress testing and evaluates trading models only on observed data.

Model card: Generative models of order flow

What you calibrate. The weights of an autoregressive, adversarial or diffusion network that produces messages or book states.

How. Next-token likelihood for autoregressive models, an adversarial game against a discriminator for GANs, a loss for removing added noise for diffusion models; then the generator is checked against the criteria above.

What you do with it. Stress tests, counterfactual scenarios and training environments; not as evidence that a trading model works.

How to picture it. A flight simulator for the order book: good for practice and rare scenarios, not a substitute for flying.

10.16 Contribution of deep learning relative to the classical models

This section has described many architectures. This closing subsection asks what, taken together, they add to the classical models of Sections 4–7. It is our synthesis of the evidence reviewed in this part, not a conclusion stated in any one source.

Fixed structure in classical and deep models. It is tempting to say that classical models describe the book while deep models learn about it from data. But classical models are also fitted to data: the slope of the OFI regression, the kernels of a Hawkes process and the rates of the queue-reactive model are all estimated. The difference is what is decided *before* the data are seen (Table 49). A classical model specifies a mechanism—the price moves in proportion to order flow, events excite further events, queues deplete—and fitting estimates a handful of parameters. A deep model specifies only its inputs, its architecture and its loss, and fitting estimates everything else. Section 10.2 showed that the three inputs a deep model can be shown correspond to three families of classical models (Table 37); a deep model given one of those inputs is a flexible estimator working on the same information as the matching classical model.

Table 49: What is fixed before seeing the data, and what is learned from it.

	Fixed in advance	Learned from data
Classical	the mechanism: a formula for how the book evolves or how the price responds	a few parameters (a slope, kernel heights and decays, a table of rates)
Deep	the input (snapshots, order flow or messages) and the loss	the whole function from input to output, through many weights
Structural hybrid (§9)	part of the mechanism (e.g. exponential waiting times, event types)	the parts that were fixed formulas (e.g. the rates)

Deep models as analogues of the signal models. Most deep LOB models, from DeepLOB on, are *discriminative*: they map a window of the book to a label such as up, down or unchanged.

The queue and Hawkes models are *generative* mechanisms—in a broader sense than the generators of Section 10.15: they can be run forward, an order can be inserted into them, and they answer questions such as “what is the chance that an order at this queue position is filled within one second?”. A direction classifier cannot answer that question at all. A discriminative deep model is therefore the counterpart of the signal models—OFI, queue imbalance, the micro-price—while the counterparts of the mechanistic models are the generative ones: neural point processes, the deep queue-reactive model (Section 9.3) and the message generators of Section 10.15.

Tasks with little gain from deep learning. For the task that dominates the literature—predicting the direction of the mid-price from one stock’s book over a short horizon—the evidence of this part points one way. With a label measured from the current price, DeepLOB is right 54.6% of the time on a year of Apple data, and 53.6% using only the best level, so the nine deeper levels add about a point (Lee, 2026). With well-chosen inputs, neural networks perform about as well as standard non-deep methods—linear regressions with a penalty on large coefficients (the lasso and ridge regression) and ensembles of decision trees (boosted trees) (Aït-Sahalia et al., 2026)—and logistic regression is slightly more accurate than a random forest, another tree ensemble, on the same inputs (Easley et al., 2021). Holding the network fixed, the input matters more than the architecture (Lucchese et al., 2024; Kolm et al., 2023); a plain multilayer perceptron matches a CNN–LSTM (Briola et al., 2020); Transformers do not beat the DeepLOB family in head-to-head comparisons (Bilokon and Qiu, 2023; Zhong et al., 2025b); published scores fall on new data, and a learned combination of fifteen deep models never beats the best of them (Prata et al., 2024). For this task, most of the predictable signal lies in quantities that the classical models already define, and the gains from architecture are small and do not hold up reliably on new data.

Tasks with measured gains from deep learning. The gains are elsewhere, and they are mostly gains in *estimation*—in what can be fitted—rather than new mechanisms:

- **Richer conditioning, made estimable.** The queue-reactive model’s rates are a table indexed by queue size; with 25 inputs such a table could not be filled by counting. A network can estimate rates that depend on all of them, and the deep queue-reactive model reproduces queue correlations, queue-size distributions and a concave impact curve—impact that grows more slowly than the order’s size—that the classical model does not (Bodor and Carlier, 2025).
- **More flexible event models.** A neural Hawkes model predicts the next event’s type better than a classical state-dependent Hawkes model—49.7% against 42.7% on MSFT—and has a higher likelihood (Shi and Cartlidge, 2022), because the effects of past events need not add up or decay in a fixed shape.
- **Pooling across instruments.** Classical models are fitted one instrument at a time. One network trained on 464 stocks predicted the next mid-price move better than each stock’s own model on 25 stocks it had not seen (Sirignano and Cont, 2019), a claim of shared structure across stocks that the classical toolkit was not set up to test; later work finds it for two of five unseen stocks (Lucchese et al., 2024) (Section 10.11).
- **Full distributions with many inputs.** Survival models estimate the whole distribution of the time until a resting order is filled, treating cancelled orders correctly, from many market variables at once (Arroyo et al., 2024; Zhong et al., 2025a) (Section 10.14).

Attention as a new modelling component. One component is conceptually new. A Hawkes kernel weights past events by how long ago they happened; a lead–lag model, in which one

instrument’s moves tend to precede another’s (Section 4.6), weights other instruments by fixed coefficients. Attention weights each past event, price level or instrument by how its *content* compares with the present (Section 10.7): it is a kernel computed from the data rather than fixed in advance, and nothing in the classical toolkit does this. On order-book data it has not yet shown an advantage: Transformers did not beat simpler networks in the head-to-head comparisons above; five of the six best models in the LOBCAST re-test use attention, and the best of them on new data lost about a fifth of its score there (Prata et al., 2024); and among neural point processes, the attention-based and the recurrent models are mixed against each other (Shi and Cartlidge, 2022).

Summary. For predicting the direction of the price from one book, the foundations were laid by the classical models, and deep learning has added mainly new machinery for fitting them. Its measurable contributions are richer conditioning, pooling across instruments, more flexible event models and full distributions—advances in what can be estimated. Attention is the one new idea about *how* past information should be weighted, and on order-book data its value is still an open question. Section 9 described the designs that try to combine the two traditions directly.

Part III

Simulation and Hardware Acceleration

About this part. Part II asked what the book predicts. Part III is about the infrastructure needed to turn a prediction into a tested trading decision. It sets out what a simulator must capture for execution questions—queue position, adverse selection and latency (Section 11); surveys the order-book simulators available and compares them with the Kaspar system, including how an order-book model is added to Kaspar as an actor that publishes its predictions to a strategy (Section 12); explains why order-book models are slow at inference and what hardware can accelerate (Section 13); and closes with open problems and the survey’s conclusion (Sections 14 and 15).

11 Mechanics a simulator must capture

A simulator for execution research has to reproduce the mechanics that decide whether an order trades and what the trade is worth. This section sets out those mechanics: the difference between crossing the spread and waiting in the queue, how queue position determines whether a passive order is filled, why the fills it does get are biased towards bad ones, and how latency reduces the value of a signal. Section 12 then compares simulators on how well they model each of them.

11.1 Passive and aggressive execution

An order either crosses the spread and trades at once (aggressive) or rests in the book and waits to be filled (passive). This section writes the profit of each and splits the passive profit into a fill probability and a value per fill.

Consider one unit traded on side d at time t . For an aggressive order, which executes immediately, a simplified realised profit is

$$\Pi^{\text{agg}} = d \Delta m_{t,H} - C_{\text{spread}} - C_{\text{fees}} - C_{\text{impact}}, \quad (68)$$

where $C_{\text{spread}} = s_t/2$ is the half spread paid to cross, C_{fees} the exchange and clearing fees, and C_{impact} the additional price concession from walking the book.

In words: profit from trading immediately is the price move in our favour—multiplying by d flips the sign for a sell, so a falling price is good for a seller—minus what it cost to get in: half the spread, the fees, and any extra paid for consuming more than the best level.

A passive order placed at t trades only if it is filled, at time t_F and price p^{fill} . Define its post-fill mark-out over horizon h as

$$R_h^{\text{fill}} = d \left(m_{t_F+h} - p^{\text{fill}} \right). \quad (69)$$

In words: once our order has traded, look at the mid h seconds later and ask how far it is from the price we got, counted positive when it is in our favour. A large positive mark-out means we bought cheaply; a negative one means the market moved against us right after we traded.

For a passive buy filled at the bid, the mark-out is the half spread at the fill time, $s_{t_F}/2$, plus the mid move after the fill. The passive profit is

$$\Pi^{\text{pass}} = \mathbb{1}_{\text{Fill}} \left(R_h^{\text{fill}} + R_{\text{rebate}} \right), \quad (70)$$

where R_{rebate} is any liquidity-provision rebate net of fees.

In words: if the order is filled, the profit is its mark-out plus any rebate the exchange pays for providing liquidity; if it is never filled, the indicator is zero and so is the profit.

The passive profit exists only conditional on execution, which gives the decomposition

$$\mathbb{E}[\Pi^{\text{pass}}] = \mathbb{P}(\text{Fill}) \mathbb{E}[\Pi^{\text{pass}} \mid \text{Fill}]. \quad (71)$$

In words: the average profit of a passive order equals the chance it is filled times the average profit when it is. This is the identity $\mathbb{E}[Z \mathbb{1}_A] = \mathbb{P}(A) \mathbb{E}[Z \mid A]$ of Section 1.3, with A the fill.

Equation (71) separates two quantities that a directional forecast conflates: how likely the order is to trade, and how good the trade is when it does. The rest of this section is about the two factors. Market-making models (Avellaneda and Stoikov, 2008; Cartea et al., 2015) and optimal-execution models (Almgren and Chriss, 2001) give the surrounding control problems; here we focus on the inputs those problems need from the book.

11.1.1 Signal value and execution value

The decomposition gives tradeability a precise meaning. A directional model estimates the *signal value*

$$V^{\text{sig}}(X_t) = \mathbb{E}[\Delta m_{t,H} \mid X_t], \quad (72)$$

the expected price move given what is known. A passive execution decision is worth its *execution value*

$$V^{\text{exec}}(X_t, a_t) = \mathbb{P}(\text{Fill} \mid X_t, a_t) \mathbb{E}[\Pi^{\text{pass}} \mid \text{Fill}, X_t, a_t], \quad (73)$$

which depends on the action through where the order is placed and therefore on its queue position.

In words: signal value asks “which way will the price go?”; execution value asks “if I act on that view with this order, how likely am I to trade, and how good will the trade be when I do?” A model can be excellent at the first and useless at the second.

The two can even point in opposite directions. A forecast of a falling price is most confident when the book is already one-sided—a thin bid, heavy selling. That is exactly when a resting buy order is most likely to be filled and least likely to be worth having: the fill and the adverse move arrive together (Section 6). Conditioning on the fill reverses the apparent value of the signal. This

is why the survey treats $\mathbb{P}(\text{Fill} \mid \cdot)$ and $\mathbb{E}[\cdot \mid \text{Fill}]$ as separate modelling targets, and why evaluation must use the conditional quantities rather than unconditional accuracy.

Who earns the execution value in practice is documented. A large high-frequency market maker studied by [Menkveld \(2013\)](#) traded passively in about four of every five trades, earning the spread while losing on its inventory; across high-frequency firms, differences in relative speed explain large differences in trading performance ([Baron et al., 2019](#)).

11.2 Queue position and fill probability

A passive order is filled only when everything ahead of it in its queue has been traded or cancelled. This section shows why that makes queue position a variable in its own right: two orders at the same price, with the same forecast behind them, can have very different chances of being filled.

11.2.1 Residual queue position

For a resting order, the displayed size at its price does not determine its fill probability. Let Q_t^{ahead} be the volume ahead of the order at the same price. Under price-time priority the order cannot execute until that volume has been removed by executions or cancellations. Two orders at the same price can therefore have very different values, and queue position has been modelled explicitly as a source of value ([Moallemi and Yuan, 2017](#)).

For a passive order with q ahead of it, the probability of a fill within horizon H is $\mathbb{P}(\text{Fill} \mid Q_t^{\text{ahead}} = q, X_t)$. It must be distinguished from $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$. A model can predict direction excellently and still have poor trading value if the predicted moves happen mainly when the passive order is unlikely to execute.

Example 11.1 (Same forecast, different orders). Two resting bids at the same price have 20 and 480 lots ahead of them. Net depletion runs at 40 lots per second. The naive time to reach the front is 0.5 s for the first and 12 s for the second. Both carry the same $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$. Over a five-second horizon the first almost surely fills and the second almost surely does not.

11.3 Adverse selection

Being filled is not always good news. A passive order tends to be filled exactly when someone better informed wants to trade against it, so the fills it gets are, on average, followed by price moves against it. This section defines that cost precisely and shows how it depends on queue position.

11.3.1 Conditioning on the fill

Using the mark-out R_h^{fill} of (69), define the conditional adverse-selection component for an order that had q ahead of it when placed as

$$\text{AS}(q, h) = -\mathbb{E} \left[R_h^{\text{fill}} - \frac{s_{t_F}}{2} \mid \text{Fill}, Q_t^{\text{ahead}} = q \right], \quad (74)$$

the expected mid move against the order after it is filled, excluding the half spread it earns.

In words: among orders that started with q lots ahead of them *and were filled*, how far, on average, did the price then move against us? It is a form of the winner’s curse: in an auction, the times your low bid wins are disproportionately the times when others knew something you did not. A passive order is most likely to be filled just when an informed trader wants to trade the other way.

The conditioning on the fill is essential. Without it, the quantity measures the unconditional drift of the market rather than the information in the event that the order actually traded. Classical

models make the mechanism explicit: a liquidity provider who trades with a better-informed counterparty loses on average (Glosten and Milgrom, 1985; Kyle, 1985).

11.3.2 Queue position as an information variable

Queue position affects both the probability of execution and the information content of execution. Moallemi and Yuan (2017) model both effects and find that a position at the front of the queue is always worth more than one at the back: in their model adverse selection increases with queue position, because an order at the end of a long queue tends to be filled only by a large aggressive order, which is more likely to be informed. For some large-tick US stocks they estimate the front–back value difference to be comparable to the bid–ask spread. The direction of the effect is therefore supported by a model; its size is an empirical object that can depend on asset, tick regime, spread, order-flow state, horizon and cancellation behaviour, and execution-aware benchmarks should report it directly (Section 14).

Example 11.2 (The decomposition at two positions). All values in ticks; fill window $H = 10$ s, mark-out horizon $h = 10$ s, one-tick spread, no rebate. Here “ $| q$ ” abbreviates $| Q_t^{\text{ahead}} = q$, and

$$\mathbb{E}[\Pi^{\text{pass}} | \text{Fill}, q] = s_{t_F}/2 - \text{AS}(q, h) :$$

Position	$\mathbb{P}(\text{Fill} q)$	$s_{t_F}/2$	$-\text{AS}(q, h)$	$\mathbb{E}[\Pi^{\text{pass}} \text{Fill}, q]$	$\mathbb{E}[\Pi^{\text{pass}} q]$
Front	0.80	+0.5	−0.20	+0.30	+0.24
Back	0.25	+0.5	−0.45	+0.05	+0.0125

The table shows the arithmetic of (71); the ordering of the two rows matches the direction found by Moallemi and Yuan (2017), with stylised magnitudes. A metric that looked only at direction or only at fill rate would rank the two positions without seeing the product.

11.4 Latency and signal decay

A prediction at horizon H is useful only if its information survives until the decision is executed. This section splits the delay between seeing the data and acting on it into stages and shows how it reduces the value of a signal. Write the total latency as

$$T_{\text{lat}} = T_{\text{signal}} + T_{\text{decision}} + T_{\text{network}} + T_{\text{exchange}}, \quad (75)$$

the times taken to compute the signal, to decide on an action, to transmit the order, and for the exchange to process it.

In words: total delay from seeing the data to the order being live at the exchange is the sum of the stage delays, just as the latency of a processor pipeline is the sum of its stages.

A signal whose value decays materially before T_{lat} is not economically equivalent to one of the same accuracy with longer persistence. The relevant object is not $\mathbb{E}[\Delta m_{t,H} | X_t]$ but

$$\mathbb{E}[\Pi | X_t, T_{\text{lat}}, \text{execution policy}], \quad (76)$$

the expected profit given the information, the latency, and the rule that maps forecasts to orders.

In words: the quantity that matters is not “will the price go up?” but “how much do we expect to make, given what we knew, how slow we are, and what we do with the forecast?”

Trading firms compete on latency, and that competition has itself been studied as an economic phenomenon (Budish et al., 2015; Hasbrouck and Saar, 2013).

Example 11.3 (Half-life against latency). Let a signal’s value decay exponentially with a half-life of 2 ms, so a fraction $2^{-T_{\text{lat}}/2\text{ms}}$ of it remains after a delay T_{lat} . At $100\text{ }\mu\text{s}$ total latency $2^{-0.05} \approx 97\%$ of its value remains; at 5 ms only $2^{-2.5} \approx 18\%$ does. The same forecast accuracy yields very different economics.

12 Order-book simulators

An order-book simulator estimates what would have happened to a researcher’s orders. Simulators differ in where the market comes from and in how faithfully they model the mechanics of Section 11. This section first shows why a naive backtest is not a simulator for passive orders and what a queue-exact replay requires; it then surveys the simulators in use for research and compares them on those requirements, describes the Kaspar system in detail, and shows how an order-book model is added to it.

12.1 Naive backtests and queue-exact replay

The questions of Section 11—will the order fill, and is the fill a good one—cannot be answered by a backtest that assumes orders fill when the price touches them. This section shows how such a backtest misleads, lists what a simulation must model to give trustworthy answers, and states what even an exact replay cannot see.

12.1.1 Assumptions of a naive backtest

A common backtest of a passive signal assumes that a limit order placed at the bid is filled when the signal fires, or when the price touches the order’s level:

```
naive:   signal at t -> assume filled at bid -> P&L = m(t+H) - bid
```

This fills every order, including those the market would never have reached. The orders it wrongly fills are exactly the ones where the price moved away before the queue was consumed—the fills a real passive order does not get, and which would have been profitable. The naive backtest is therefore biased toward adverse-selection-free fills that do not exist.

12.1.2 Requirements of a queue-exact replay

A replay that can answer the execution questions of this part needs:

1. market-by-order data, so that queue position is defined at all;
2. an order-level reconstructed book in which the simulated order occupies a place in the first-in-first-out queue;
3. explicit, independent latencies for outbound orders, outbound cancels and inbound market data;
4. an explicit fill rule stated in terms of recorded events;
5. a stated assumption about market impact.

```
replay:  strategy sees the book at market time t
          (the feed reached it a feed delay after the exchange event)
          -> order arrives at the exchange at t + feed delay + order delay
          -> appended at the tail of the real FIFO queue at its price
```

-> filled only when recorded executions reach an order behind it -> P&L from the actual (simulated) fill time and price
--

The fill rule rests on price–time priority: if a recorded aggressor reached a resting order that arrived later than the simulated order at the same price, it must have passed through the simulated order first. The standard simplifying assumption is *no market impact*: the recorded market is replayed as it happened, as though the simulated order had not been there. Every cost measured this way is a cost before the market’s reaction to the order.

Replay and generative simulation answer different questions. Generative simulators produce new market trajectories and are judged on how realistic they are; a replay simulator keeps the recorded market fixed and produces only the counterfactual outcome of the researcher’s own orders. GPU simulators such as JAX-LOB (Frey et al., 2023) move the matching engine onto accelerators to speed up reinforcement-learning training; the concern here is not throughput but the fidelity of the fill inference. Section 12.3 describes one implementation of a queue-exact replay built to the five requirements above.

12.1.3 Limits of replay fidelity

A queue-exact replay is exact only with respect to what the feed shows. Two gaps remain.

Hidden liquidity. Iceberg orders make executable liquidity exceed displayed liquidity, so a model trained on displayed volume misestimates depletion, impact and fill probability. In E-mini S&P 500 futures, one detection study estimates that about 9% of book volume is iceberg orders (Christensen and Woodmansey, 2013); when participants detect an iceberg they trade against it actively, and icebergs create an adverse-selection cost for other limit orders (Frey and Sandås, 2017). In a replay, an iceberg’s hidden size is invisible until it trades, so the queue ahead of a simulated order can be longer than the reconstructed one, and fills are overstated by an amount that cannot be measured from displayed data alone.

Feed and timestamp errors. At high frequency, data quality is part of the model: timestamp synchronisation, packet loss, sequence gaps, book reconstruction, exchange-specific event semantics, stale snapshots, duplicates, and clock differences between market-data and execution systems.

Example 12.1 (A sequence gap). On a feed with sequence numbers, a missed packet leaves the reconstructed book wrong until the handler recovers from a snapshot. A model trained on rows from that interval learns reconstruction error, not market behaviour, and a replay that runs through it fills orders against a book that never existed. No number of parameters compensates for it; the gap has to be detected and the interval excluded or repaired.

12.2 Existing simulators compared

This section compares existing simulators in two steps. Table 50 covers the simulators most used in research, of all four kinds described below; Table 51 then narrows to historical-replay tools only, the kind Kaspar belongs to, and includes every such tool we could classify. A few tools, such as hftbacktest and NautilusTrader, appear in both.

Four kinds of simulator. Simulators differ first in where the market comes from.

- **Historical replay.** The recorded market is played back event by event, and the researcher’s orders are matched against it. The market is real but cannot react to those orders.

- **Agent-based.** Many simulated traders, each following a rule, send orders to a simulated exchange. The market reacts, but it is only as realistic as the agents.
- **Model-based.** A fitted stochastic model of the book—such as the queue-reactive model of Section 6.3—generates order flow. The market reacts mechanically, through the model’s rates.
- **Generative.** A network trained on order data generates order flow, often conditioned on the researcher’s orders (Section 10.15).

Jain et al. (2024b) review the model-based and generative approaches and the stylised facts used to validate them. For execution questions the type matters less than five properties: whether the simulated order has an exact position in a queue of individual orders; whether latency is modelled; whether the simulated order changes the market; whether a run can be repeated exactly; and whether the same code can trade live.

Simulators of all four kinds. Table 50 compares the simulators most used in research, as documented in their papers and code.

- **ABIDES** (Byrd et al., 2020) and its reinforcement-learning interface **ABIDES-Gym** (Amrouni et al., 2021) are discrete-event agent-based simulators in Python. The exchange keeps every order in first-in-first-out queues, and messages between agents and the exchange carry modelled network and processing delays.
- **MAXE** (Belcak et al., 2022) is an agent-based simulator in C++ with a Python interface, with price–time and pro-rata matching and message and processing delays.
- **PyMarketSim** (Mascioli et al., 2024) is an agent-based environment in Python for reinforcement learning.
- **JAX-LOB** (Frey et al., 2023) replays Nasdaq order messages (LOBSTER) on a GPU, so that many simulations run in parallel for reinforcement-learning training. It starts each episode from a snapshot in which each price level is a single synthetic order, so the agent’s position relative to orders that were resting at the start is approximate, and it re-submits the agent’s orders at each step.
- **mbt_gym** (Jerome et al., 2023) simulates the price and fills from parametric models, without a book; fills depend on the distance of the quote from the mid.
- **hftbacktest** (nkaz001, 2022) replays market-by-price or market-by-order data, mainly from cryptocurrency venues. With market-by-order data it keeps every order in a first-in-first-out queue and fills the simulated order when an execution reaches an order behind it—the same rule as Kaspar; with price-level data it estimates the volume ahead. It models feed latency from the data’s timestamps and order latency from recorded live latencies, and the same Rust strategy code can trade live on two cryptocurrency exchanges.
- **NautilusTrader** (Nautech Systems, 2018) is an event-driven backtester and live-trading platform in Rust and Python. It tracks an estimate of the displayed quantity ahead of a simulated order, models order-entry delays, and runs the same strategy code in backtest and live trading.
- **MarS** (Li et al., 2025) generates order flow with a large generative model trained on Chinese equity orders and matches it in an order-level exchange; the generated market responds to injected orders.
- **TRADES** (Berti et al., 2025), released in the DeepMarket framework, generates orders with a diffusion model inside ABIDES.

- **The queue-reactive model** (Huang et al., 2015) and its neural extension (Bodor and Carlier, 2025) are model-based simulators of the queues at the first five levels (Section 6.3).

Table 50: Order-book simulators compared on the properties execution research needs, from their papers, documentation and code; citations are in the list above. Queue position: *exact (recorded)* means the simulated order sits in a reconstructed queue of the recorded individual orders; *exact (simulated)* means a queue of individual orders in a simulated market; *estimated* means a model of the volume ahead. “–”: not provided.

Simulator	Market from	Own-order queue position	Latency model	Market reacts	Repeatable runs	Same code live
ABIDES	agents	exact (simulated)	network and processing delays	yes	same seed, same run (ties within a nanosecond in arbitrary order)	–
ABIDES-Gym	agents	exact (simulated)	pairwise delays	yes	seeded	–
MAXE	agents	exact (simulated); price–time or pro-rata	message and processing delays	yes	not seeded	–
PyMarketSim	agents	price–time book; agents withdraw orders on arrival	–	yes	partly seeded	–
JAX-LOB	replay (Nasdaq)	approximate: initial levels are one synthetic order each; agent orders re-queued each step	one observation-to-action delay (default 0)	–	random keys; no claim	–
mbt_gym	model	no book; fill probability from quote depth	–	parametric impact	seeded	–
hftbacktest	replay (mainly crypto; CME via vendor files)	exact (recorded) with order-level data; estimated with price-level data	feed latency from data; order latency constant or from recorded live latencies	–	no claim found	yes (two crypto venues)
NautilusTrader	replay	estimated volume ahead	order-entry delays	–	deterministic runtime claimed; seeded fill model	yes
MarS	generative model	exact (simulated)	computation and communication delays	yes	seeded sampling	–
TRADES	generative model in ABIDES	exact (simulated)	– (as shipped)	yes	seeded	–
Queue-reactive	model	one tagged order, under stated assumptions	–	yes, through queue sizes	–	–
Kaspar (Section 12.3)	replay (any venue with a feed handler; CME MDP 3.0 implemented)	exact (recorded)	separate order, cancel and feed delays	–	single thread, market-time clock, seeded; argued, test not reported	live path shares all code up to one branch

What the comparison shows. Agent-based, model-based and generative simulators let the market react to the simulated orders, which replay cannot; but in them the queue is exact only inside a simulated market, so fill rates are only as realistic as the simulated order flow. Among replay tools, exact queue position in the recorded market requires order-level data and an order-level book: in Table 50, hftbacktest with order-level data and Kaspar provide it, JAX-LOB approximates it, and NautilusTrader estimates it. Only hftbacktest, NautilusTrader and Kaspar run the same strategy code in simulation and live trading; the venues each of them connects to are compared at the end of this section.

Historical-replay tools only. Kaspar is a historical-replay tool, so Table 51 compares it with the other tools that replay recorded order-book data and fill a researcher’s orders against it. It lists every such tool we found with enough documentation or code to classify it, including several not in Table 50. The list below groups them by how they place the simulated order in the queue.

- **Exact position in the recorded queue, recorded market unchanged.** Besides hftbacktest and Kaspar, **lobsim** (kpetridis24, 2026), a C++20 library, replays order-level data and counts exactly how much recorded quantity is ahead of each simulated order; it has optional order latency, states that it is deterministic, and has no live-trading path. Moallemi and Yuan (2017) use the same rule as a method on Nasdaq ITCH data, without software: a simulated order counts as filled once a later order at its price is filled.
- **Exact position, but the simulated orders change the recorded book.** Two open-source engines insert the researcher’s orders into the replayed order-level book and match them there, so the orders consume liquidity that the recorded market then lacks (chasemetoyer, 2026; koteyevlev, 2021). Neither models latency.
- **Estimated position.** Most replay tools work from price-level data and estimate the volume ahead: the market-making simulators of Spooner et al. (2018) and Sadighian (2019), one open-source backtester for cryptocurrency data (everher, 2020), the CME execution model of Dixon (2018b)—which notes that order-level data would make the position exact—and Vyetrenko and Xu (2019), who add latency taken from historical profiles and a synthetic market response. A note by Rigtorp (2013) describes one such estimation rule. Among commercial tools, TT’s simulation environment states that queue position is estimated for all markets, counting trades ahead of the order but not cancellations (Trading Technologies International, Inc., n.d.c,n); Deltix documents order-level simulators whose fills remove liquidity from the replayed book, but not how they place the order in the queue (Deltix, n.d.a,n).
- **No queue at all.** Some tools fill an order only when the price crosses it (Grosner, 2018). Retail platforms replay the depth of book for display but fill without a queue: Sierra Chart fills a limit order when the opposite best quote reaches its price, and states that its queue tracking does not apply in replay (Sierra Chart, n.d.b); ATAS fills replayed orders at once and in full (ATAS, n.d.b). NinjaTrader, Bookmap and MultiCharts replay depth with simulated trading but do not document how replayed orders are filled (NinjaTrader, LLC, n.d.; Bookmap Ltd., n.d.; MultiCharts, LLC, n.d.).

Bar-level backtesters, which see only open, high, low and close prices over fixed intervals or the best quotes, are outside the scope of this comparison: for example LEAN (QuantConnect, 2026), Jesse (Jesse.Trade, 2026) and the backtesters of Hummingbot (Hummingbot Foundation, 2026) and Barter (Barter contributors, 2026).

Where Kaspar sits. Of the replay tools found, three place the simulated order in the recorded queue without changing the recorded market: hftbacktest, lobsim and Kaspar. Of these, lobsim has no live-trading path, and hftbacktest trades live on two cryptocurrency exchanges. NautilusTrader, which estimates queue position, connects to several venues and data vendors. Both reach CME data only through a data vendor’s files. Kaspar is the only one we found that decodes an exchange’s own market-by-order feed (CME MDP 3.0, from live multicast or packet captures), models order, cancel and feed delays separately, and runs the same code in replay, paper and live trading against that exchange. We found no open-source or documented commercial backtester that decodes MDP 3.0 itself. None of the commercial documentation we could read describes filling a simulated order by

Table 51: Historical-replay backtesters at order-book level, from their papers, documentation and code. “Recorded market unchanged”: the simulated orders do not alter the replayed book (no market impact). “–”: not provided; “not stated”: documentation silent. Commercial entries (below the line) are from vendor documentation; the Deltix knowledge base requires a login and was not read, and no readable documentation of a replay fill simulator was found for CQG.

Tool	Data	Own-order queue position	Latency	Recorded market unchanged	Same code live	Feeds
Kaspar	order-level	exact, in the recorded queue	order, cancel and feed delays	yes	yes	CME MDP 3.0 (own handler)
hftbacktest	order- or price-level	exact with order-level data; estimated otherwise	feed from data; order constant or recorded	yes	yes (crypto)	crypto; CME via vendor files
lobsim	order-level	exact count of recorded quantity ahead	optional order delay	yes	–	Coinbase, LOBSTER files
NautilusTrader	order- or price-level	estimated	order-entry delays	yes	yes	several venues; CME via vendor files
Moallemi–Yuan method	order-level (ITCH)	exact (fill when a later order fills)	–	yes	–	Nasdaq
Backtesting-Engine	order-level	exact; own orders matched in the replayed book	–	no	not stated	crypto (CoinAPI)
MarketMakingML- Algo	order log	exact; recorded trades re-matched	–	no	–	one equity order log
Spooner et al. (2018)	price-level, 5 levels	estimated: back of queue, proportional cancels	coded but unused	yes	–	European equities
Sadighian (2019)	price-level snapshots	estimated: volume ahead less trades	–	yes	–	crypto
Dixon (2018b)	price-level	estimated (exact with order-level data)	zero in backtest	not stated	–	CME ES
Vyetrenko and Xu (2019)	price-level	estimated: back of queue	historical latency profiles	no: synthetic response	–	futures
hft-backtesting	price-level	estimated, probabilistic	single delay	yes	–	BitMEX
TT Backtesting	book and trades; price- or order-level not stated	estimated: quantity ahead reduced by trades, not by cancels	not stated	not stated	yes (TT algos)	TT-connected exchanges
Deltix QuantOffice	price- or order-level	not stated	configurable	no: fills remove replayed liquidity	yes	several asset classes
NinjaTrader Playback	price-level; bid and ask sizes set to 1	not stated	–	not stated	yes	data-provider dependent
Bookmap replay	depth and best quotes	estimate displayed; use in fills not stated	not stated	not stated	yes	recorded feeds; CME futures
MultiCharts	levels 1 and 2	not stated	not stated	not stated	yes	data-provider dependent
Sierra Chart replay	trades with recorded best quotes	none: fills when the opposite quote reaches the price	not stated	not stated	yes	CME and other exchanges
ATAS replay	trades and order book	none: filled at once, in full	–	not stated	manual trading only	subscription dependent

its exact place in a recorded order-level queue: the tools estimate the position, fill without a queue, or do not say.

Running many days at once. A Kaspar replay processes one trading day per run, from one packet capture or one decoded file, on one thread. Days are independent, so many days can be simulated at the same time as separate processes on one large machine—for example a large cloud instance—with one process per day and no licence limit on the number of runs. The other tools differ on this point (Table 52). The open-source replay tools allow the same: hftbacktest’s examples run backtests in parallel processes (nkaz001, 2022), NautilusTrader recommends one process per backtest (Nautech Systems, 2018), and ABIDES ships a script that starts one replay process per date (Byrd and Balch, n.d.); JAX-LOB instead runs up to a thousand environments in parallel on one GPU (Frey et al., 2023). Among the commercial tools, TT’s backtests run only on TT’s servers, are charged by run time, and cover at most one trading day each (Trading Technologies International, Inc., n.d.b,n); Bookmap and ATAS allow one running copy or session per licence or account (Bookmap Ltd., 2026; ATAS, n.d.a); and a MultiCharts licence runs online on one computer at a time (MultiCharts, LLC, 2017). Sierra Chart is the exception: its licence allows many copies on one computer at the same time (Sierra Chart, n.d.a). For NinjaTrader’s depth replay, Deltix and lobsim the documentation does not say. Compared with the commercial replay tools, therefore, the ability to run many days in parallel on one’s own hardware is an advantage of Kaspar shared only by Sierra Chart; among open-source tools it is common.

Table 52: Running many replay simulations at once on one’s own hardware, from documentation, licence terms and code.

Tool	Parallel runs on own hardware	Restriction or evidence
Kaspar	yes	one process per day; no licence limit
hftbacktest	yes	examples use parallel processes
NautilusTrader	yes	one backtest per process
ABIDES	yes	script starts one replay process per date
JAX-LOB	on one GPU	up to 1,000 parallel environments; GPU memory
lobsim	not documented	open licence; no restriction found
TT Backtesting	no	runs on TT’s servers only; one trading day per backtest; charged by run time
Bookmap	no	one running copy per licence
ATAS	no	one active session per account
MultiCharts	limited	online on one computer; a second computer offline for backtesting
Sierra Chart	yes	many copies on one computer under one licence
NinjaTrader (depth replay)	not documented	one replay connection per session described
Deltix QuantOffice	not documented	cloud product lists a parallel runner; licence terms behind a login

12.3 Kaspar HFT

This section describes Kaspar, the replay system used as the worked example in this survey: its data path, how it fills simulated orders, which design choices make its replays repeatable, how the same code runs live, a model-free benchmark strategy, and how it measures latency and execution cost.

Kaspar is a trading and simulation system written in C++20 on an *actor* framework (Maciejewski, 2026b): every component is an actor, which owns its state and communicates with other actors

only by messages. Its order book, simulated order manager, strategy actors and replay are not tied to one exchange: a venue is connected through a market-data handler, which turns the venue’s feed into book updates, and an order-entry gateway. The handlers implemented today are for CME—the MDP 3.0 market-by-order feed and the iLink 3 order-entry protocol—and are used for ES, NQ and Treasury futures. Another market—equities, other futures exchanges, cryptocurrency venues—needs only its own pair of handlers; the rest of the system is reused unchanged. Every statement below describes the design and code; the few numbers quoted come from previously published measurements, attributed where they appear.

The data path is

exchange multicast (CME MDP 3.0 today) → socket reader → message buffer → decoder → feed handler → order book → strategy → simulated order manager → [exchange gateway | simulated fill].

Multicast means the exchange sends each packet once and every subscribed host receives it. The strategy actors are called *lights* in the code and in the figures. Actors can be placed in a *group*, which runs several actors on one thread, processing one message at a time in arrival order. The simulated order manager (SOM) sits between the strategy and the venue. In live mode it forwards orders to the exchange gateway (CME iLink 3); in simulation mode it sends them into the reconstructed book, where they are matched against recorded order flow.

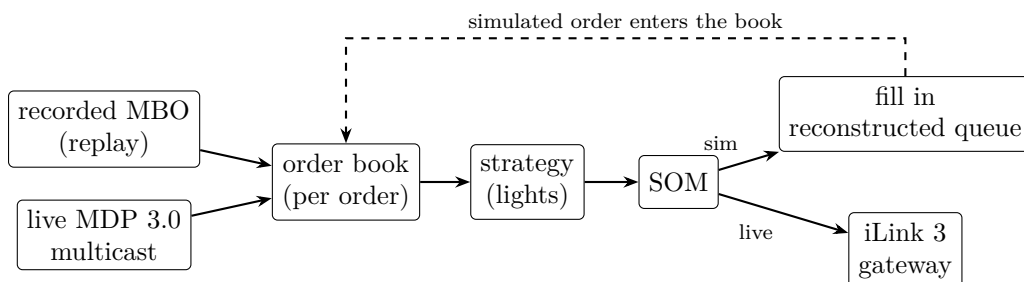


Figure 77: Kaspar data and order paths. Everything downstream of the market-data source is the same code in replay and live operation; the branch between simulated and live execution is a single decision in the simulated order manager.

12.3.1 Queue-exact simulated fills

The central requirement of Section 12.1 is that a simulated order should be filled only if a real order in its place would have been. This section explains how Kaspar meets it: simulated orders sit in the same per-order queue as real ones, reach the book after a modelled delay, and are filled only by real executions that reach them. Here and throughout, *queue-exact* means exact with respect to the displayed order sequence carried by the input MBO feed and the documented matching rules; it does not imply knowledge of hidden liquidity or unobserved participants, and it does not model the market impact of the simulated orders.

The book holds every order. The simulator reconstructs a full market-by-order book from recorded exchange data. Each price level is a queue that holds every resting order individually, in arrival order, and simulated orders are held in the same queue alongside real ones. There is no estimated “volume ahead” variable: a simulated order’s queue position is its position in the list. This is the property that makes a fill inference definable at all (Maciejewski, 2026d).

Placement and latency. In simulation mode the SOM turns a new order into a book *add* message tagged with a simulated venue identifier and sends it to the order-book actor. The book does not insert it immediately. It holds the order on a delay queue until its modelled arrival time at the matching engine:

```
auto order_engine_arrive_time =
    order_leave_time
    + uint64_t(feed_delay) * 1000
    + uint64_t(std::max(40, eff_delay)) * 1000; // 40 us floor
```

The outbound order delay, the outbound cancel delay and the inbound feed delay are independent parameters. The feed delay is added because the strategy saw the market late by that amount, so a round trip is feed plus order, not order alone. When the order arrives it is appended at the tail of the queue at its price, behind all depth already resting there.

The fill rule. A recorded cancellation of a real order simply unlinks it, and every order behind it moves up. A recorded execution triggers simulated fills only when the executing real order is the first real order in the queue and a simulated order sits ahead of it:

```
// fill sim orders only if the executing order is at the front
if (o == q->get_head_no_sim())
{
    fill_prev_sim_order();
    ...
}
```

Any remaining executed quantity is carried forward to earlier simulated orders. This is the price-time-priority argument of Section 12.1 in code: an aggressor that reached a real order which arrived after the simulated order must have passed through the simulated order. Simulated orders are excluded from the best bid and offer that the strategy sees, so a strategy cannot react to its own quote. Market impact is not modelled (Maciejewski, 2026d).

Example 12.2 (A fill, step by step). Bid queue at 5000.00 over seven successive steps #0–#6, sizes in lots, real orders R1–R4 and one simulated order S:

#0 resting:	R1(10) R2(5) R3(20)	
#1 S(3) arrives:	R1(10) R2(5) R3(20) S(3)	appended at tail
#2 R2 cancels:	R1(10) R3(20) S(3)	S moves up
#3 exec 10 vs R1:	R3(20) S(3)	none ahead of R1: no fill
#4 exec 20 vs R3:	S(3)	S is behind R3: no fill
#5 R4(8) joins:	S(3) R4(8)	
#6 exec 5 vs R4:	R4 is head real, S ahead ->	S filled 3 @ 5000.00

At step #6 the aggressor reached R4, which arrived after S, so it must have passed through S. A “touch” backtest would have filled S at step #3, three recorded events earlier and before the queue ahead of it was consumed.

Market impact: a limitation of replay, and a possible extension. Because the recorded market behaves exactly as it did, whatever the simulated orders do, replay answers “would my order have been filled against this market?” but not “how would the market have moved because of my order?”. A simulated order never consumes liquidity that real traders would have reacted to. Market impact is therefore outside what this architecture measures, and it is not implemented.

One way to add a reacting market, which we describe as possible future work rather than a feature, is a second market-data source: a queue-reactive simulator (Section 6.3) in place of the

recorded feed. The separation between the source of market data and the book, order manager and strategy actors means the rest of the pipeline would not change. The generator produces queue counts rather than individual orders, so its events would have to be turned into synthetic order-level messages—order identifiers and first-in-first-out placement—for the existing book. The researcher’s orders would then change queue sizes, the simulator’s rates would respond, and the price response to the strategy’s own trading could be measured.

What such an extension would estimate is limited, and should be stated with it.

- It would capture *mechanical* impact: queues consumed by the strategy’s orders, refilling at state-dependent rates, and price moves when a queue empties. Neural extensions of the queue-reactive model reproduce the square-root law of impact on Bund futures (Bodor and Carlier, 2025), so this part can be realistic.
- It would not capture *informational* impact: other participants inferring that a large buyer is present and repricing. In the queue-reactive model this sits in the random resets of the book, which do not respond to the strategy.
- Its impact would be a property of the simulator, and would need checking against impact measured on real trading—post-fill mark-outs from paper or live fills, or published impact laws—before being relied on.

The resulting division of labour would be: queue-exact replay for fill realism on the real market, without impact; a reactive simulator for mechanical impact and for comparing tactics in a market that responds; and paper and live trading as the ground truth for both.

12.3.2 Deterministic replay

In the replay configuration the file reader, the order book, the timer, the SOM, the strategy actors and the position manager all run in one actor group on one thread. The reader is added to the group last, so all consumers exist before the first market-data message is delivered. Three design choices then make a replay repeatable:

1. **One message sequence.** A single thread processes every message—market data, orders, acknowledgements, fills, timer events—in one total order.
2. **Market time.** The timer actor is driven by market-data timestamps, not the wall clock, so timeouts fire at the same point in the data on every run.
3. **Seeded randomness.** Any randomised strategy decision uses a per-actor seeded generator set in configuration.

The property this implies, and the test that would check it, is that replaying the same recorded file twice with the same configuration yields byte-identical fill records. We argue the property from the design; we do not report the test here.

Running the same program in research and in production also removes one source of difference between them: a result cannot change because the production code was written separately.

12.3.3 Replay, paper and live trading from the same code

Two choices are independent: where market data come from (recorded file or live multicast), and where orders go (reconstructed book or exchange). Everything downstream of the message buffer—the decoder, feed handler, order book, strategy and SOM—is the same code whichever source feeds

it. The branch between simulated and live execution is a single condition in the SOM’s new-order handler; both paths apply the same pre-trade reject checks, with one additional rule in live mode for cancels of orders not yet acknowledged by the exchange. The combination “live data, simulated execution” is paper trading: orders enter the live reconstructed book and are filled by the live order flow under the same rule as in replay.

The live order-entry path implements CME’s iLink 3 protocol: binary message encoding, HMAC request signing (a keyed checksum that authenticates each message), sequence management, and failover between primary and secondary gateways. The project documentation states that both the market-data handler and the order-entry session have passed CME’s automated certification. Strategies can be written as in-process actors in C++ or, through an in-process foreign-function interface, in Rust.

Two qualifications keep the claim precise. Switching between simulated and live execution is a single flag in the SOM, but in the distributed build that flag is set in the program source rather than in a configuration file, and the distributed configuration runs in simulation mode; the equivalence demonstrated day to day is therefore between replay and paper trading, with the live path sharing all code up to the SOM branch. And equivalence of code is not equivalence of outcomes: replay assumes no market impact (Section 12.3.1) and models latency with configured delays, so a strategy that is profitable in replay can still behave differently live.

12.3.4 A model-free execution benchmark

SHADOW-PPOV is the passive execution algorithm used in the case study (Maciejewski, 2026d). It makes no prediction about book evolution. When a third-party limit order is added within a configured distance of the touch, the algorithm may place its own order at the same price, records that participant’s exchange order identifier, and cancels when that order leaves the book. The placement decision is randomised at a configured rate; cancellation can be delayed by a grace period measured in events or time. The design comment for the grace period states the trade-off of Section 11.3 directly: cancel too quickly and fills are given up; cancel too slowly and the order bears the adverse selection.

Its role in this survey is as a null model for passive placement. It defines the bottom rung of a hierarchy:

model-free placement $\rightarrow$ predictive placement $\rightarrow$ full predictive execution policy.

A predictive model that claims value for passive execution should beat the model-free benchmark *in the same replay, under the same fill rule*, on slippage and on post-fill mark-outs. Any improvement measured against a naive backtest instead is confounded by the fill-inference bias of Section 12.1.

12.3.5 Latency and slippage measurement

Latency claims need their endpoints. In the published measurement of Kaspar’s market-data handler, the start point is a software timestamp at the socket read and the end point is the order book being published; time spent in the network card and kernel before the read is outside the measurement, which is therefore socket-to-book, not wire-to-book. Measured on live CME multicast, the median is $8.0\ \mu\text{s}$, with book-message medians of 7.1 , 7.3 and $7.1\ \mu\text{s}$ for ES, NQ and ZN and 99th percentiles of 18.5 , 13.6 and $57.0\ \mu\text{s}$ (Maciejewski, 2026c). These numbers are quoted as an example of stating endpoints, not as a result of this paper.

The simulator’s slippage instrumentation measures execution cost (*slippage*) for each fill against four references: the mid at decision time, the interval volume-weighted average price, same-side

passive fills by other participants, and the touch. Comparing against the interval VWAP separates market drift from adverse selection. Post-fill mark-outs at fixed times and after a fixed number of book events provide the mark-out R_h^{fill} of (69) (the event-count version is its analogue on an event clock) and the adverse-selection quantity of Section 11.3.

12.4 Adding an order-book model to Kaspar

Any model of Part II can be added to Kaspar as one more actor between the order book and the strategy. The actor subscribes to the book, computes its prediction each time the book changes, and sends the prediction to the strategy actors as a message (Figure 78). Because it is an actor like the others, the same model runs in replay, paper trading and live trading. The description below follows the code; the parts marked *new* do not exist in the repository and would have to be written.

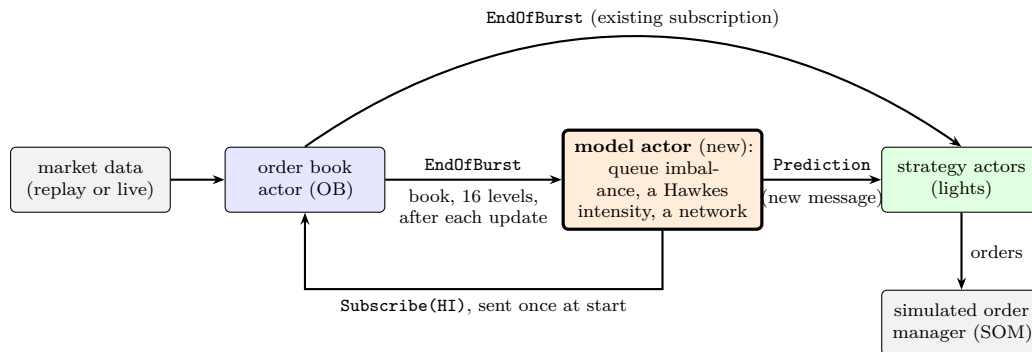


Figure 78: An order-book model added to Kaspar as an actor. It subscribes to the order book, receives the book after each burst of updates, and sends a prediction message to each strategy actor. Message names are those in the code; the model actor, the `Prediction` message and its handler in the strategy are new.

1. **Define the message.** A new message type derives from `actors::MessageT<Prediction>`, which assigns it a unique identifier automatically; it carries the instrument, the market time of the book update it was computed from, and the predicted value.
2. **Define the actor.** The model actor derives from `actors::Actor` and registers two handlers with `MESSAGE_HANDLER`: one for `Start`, sent by the framework when the system starts, and one for `EndOfBurst`, the message the order book sends after each burst of updates. In its `Start` handler it sends `Subscribe(HI)` to the order book—the same call the strategy actors and the simulator’s slippage probe already make.
3. **Compute and publish.** On each `EndOfBurst` it reads the prices and sizes of up to 16 levels from the message, computes the prediction, and sends a *new* `Prediction` message to each strategy actor; the framework does not allow one message object to be sent twice.
4. **Use the prediction in the strategy (new).** The strategy actor registers a handler for `Prediction`, stores the latest value, and uses it in its placement decision—for example as an extra condition before it places an order.
5. **Wire it in.** In the replay simulator the actor is added to the single actor group before the market-data reader, so that it exists before the first message arrives; in the trading binary it is added alongside the strategy actors of the same order book.

In outline (the full sketch, with the file and line of every call used, is in the notes accompanying this survey):

```

namespace ob_msg = frame::ob::msg;
namespace mda_msg = frame::mda::msg;

// New message type: the model's output for one book update.
struct Prediction : actors::MessageT<Prediction> {
    uint    sym;           // instrument
    uint64_t market_time; // time of the book update it came from
    double  value;        // e.g. queue imbalance in [-1, 1]
    Prediction(uint s, uint64_t t, double v)
        : sym(s), market_time(t), value(v) {}
};

// New actor: subscribes to the book, publishes one prediction per update.
class ImbalanceModel : public actors::Actor {
    actor_ptr ob;           // the order book it reads
    std::vector<actor_ptr> lights; // the strategy actors it feeds

public:
    ImbalanceModel(actor_ptr ob_, std::vector<actor_ptr> lights_)
        : ob(ob_), lights(std::move(lights_)) {
        MESSAGE_HANDLER(actors::msg::Start, on_start);
        MESSAGE_HANDLER(ob_msg::EndOfBurst, on_book);
    }

    void on_start(const actors::msg::Start*) {
        ob->send(new mda_msg::Subscribe(mda_msg::Subscribe::HI), this);
    }

    void on_book(const ob_msg::EndOfBurst* m) {
        const auto& pl = m->payload;           // the book after the burst
        // skip bad data and bursts sent while the book is being rebuilt
        // after a gap, as the strategy actors do
        if (!pl || pl->point_.baddata || pl->recovery) return;
        double b = pl->point_.bid_sz[0];       // size at the best bid
        double a = pl->point_.ask_sz[0];       // size at the best ask
        if (a + b <= 0) return;
        double imb = (b - a) / (b + a);
        for (auto l : lights)                  // a new message per recipient
            l->send(new Prediction(pl->sym, pl->txtim_epoch, imb), this);
    }
};

```

Three properties of the present code matter for anyone doing this.

- **Ordering and determinism.** In the replay simulator every actor runs in one group, on one thread, in one message order, so a replay with the model actor added is still deterministic. From reading the code (not tested), a prediction sent with `send` for a book update reaches the strategy after the strategy has acted on that update and before the next one; the framework's synchronous `fast_send` would deliver it first. In the trading binary each actor has its own thread, and the order in which the book update and the prediction reach the strategy is not fixed.
- **Which book.** The model actor should subscribe to the price-level book (OB). The order-level book (TachBook) accepts high-priority subscribers only under specific names, and in the trading binary it receives data only in a measurement mode that starves the other consumers.
- **What is missing.** There is no general publish–subscribe bus: a model actor must hold a pointer to the book and to each strategy actor. The strategy has no input for an external signal today;

its only numeric input sets a position target.

13 Hardware acceleration of order-book models

Hardware bears directly on the second organising principle, Occam’s razor. The economic value of a signal depends on the latency and the determinism of the infrastructure that evaluates it—how long evaluation takes, and how much that time varies from one event to the next (*jitter*)—and richer models tend to raise both latency and jitter:

$$\text{model complexity} \uparrow \Rightarrow \text{inference latency} \uparrow \Rightarrow \text{signal value at execution} \downarrow. \quad (77)$$

In words: a bigger model takes longer to evaluate, and by the time it answers, some of what it predicted has already happened.

This section first follows one market event through a model-driven trading system and shows where the time is spent; it then explains why recurrent networks and Transformers are slow at that task, collects the published and the independently audited latency measurements, and lists the stages worth accelerating. The remaining subsections review the hardware—field-programmable gate arrays (FPGAs: chips whose circuits are configured after manufacture to implement one design), graphics processors (GPUs), accelerators between the two, and the network—and close with a guide to placing each workload and a limit on what hardware can achieve.

13.1 Stages of the per-event pipeline

This section follows one market event through the system, separates the stages fixed by the exchange protocol from the stage that depends on the model, and summarises what is known about the cost of each.

Between a market-data packet arriving and an order leaving, a model-driven system runs a pipeline (Figure 79): decode the exchange message; update the book; compute the model’s inputs; evaluate the model; decide; send the order. Each stage has a different cost structure, and each is accelerated in a different way.

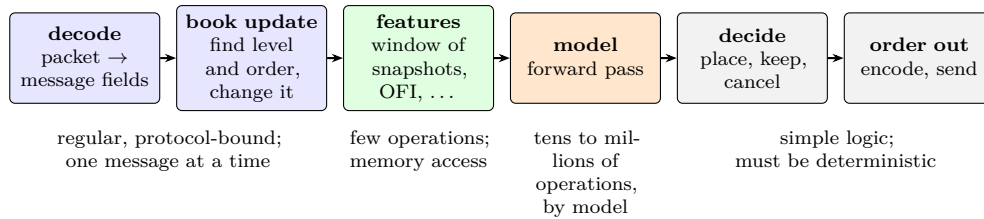


Figure 79: The per-event pipeline of a model-driven trading system. The model stage is the only one whose cost depends on the modeller’s choices. Schematic.

Book building and inference. The stages fall into two groups that are often conflated. *Book building*—decoding market data and maintaining the order book—is fixed by the exchange protocol, highly regular, and the same for every model. *Inference*—evaluating a model on the book—depends on the model. A latency figure for one group should not be compared directly with a figure for the other.

Decoding and book update. FPGAs have long been used for book building. Decoding market-data feeds on FPGAs is well documented (Leber et al., 2011; Morris et al., 2009): Leber et al. (2011) decode Ethernet, IP, UDP and the FAST protocol in hardware with a fourfold latency reduction over software, and Lockwood et al. (2012) report a fixed end-to-end latency of 1 μ s at 10 Gb/s line rate for a library of networking and protocol-parsing blocks. The book update is a search in an ordered set of price levels, followed by a change to one level or one order; with standard ordered containers each search takes time proportional to the logarithm of the number of levels, and an industry preprint reports a specialised structure 9 to 15 times faster than the C++ standard library’s ordered map on real market data (Krapivensky, 2025). We found no peer-reviewed measurement of the cost of book updates on CPUs. On FPGAs the book itself has been built in hardware: a depth-of-book design for options averages about 253 ns per update, estimated from clock-cycle counts (Dvorak and Korenek, 2014); a later design reports 26 to 209 ns, 3.5 to 37.6 times faster than its software version (Liu et al., 2024); and a complete trading system written in high-level synthesis (C++ compiled into FPGA logic)—network stack, parsing, book and strategy—measured an on-board round trip below 870 ns (Boutros et al., 2017). For Kaspar’s software path, socket-to-book latency is reported in Section 12.3.5.

The model. This is the stage that grows with model complexity. A linear score on a few order-flow features takes tens of multiply-add operations; DeepLOB has about 60,000 weights (Table 41), and each forward pass uses each of them at least once. Placing models on the same FPGA as the book is more recent: tool flows for fast neural-network inference on FPGAs (Duarte et al., 2018) make small, quantised networks feasible with very low and fixed latency, but model capacity is bounded by the size of the FPGA’s programmable logic. The next subsection explains why larger models are slower than their operation count suggests.

13.2 Sources of per-event latency in recurrent networks and Transformers

Training processes many examples at once, and graphics processors are built for that. A trading system needs one prediction for the event that has just arrived—a *batch* of one—and as soon as possible. This section explains four effects that make that case slow and then collects the published inference times of order-book models.

1. Recurrence is sequential. A recurrent network computes its memory at step t from its memory at step $t-1$ (Section 10.3), so the 100 steps of a window must be computed one after another; this “inherently sequential nature precludes parallelization within training examples” (Vaswani et al., 2017). A layer needs a number of sequential steps that grows with the sequence length n , where self-attention needs a constant number (Vaswani et al., 2017). On an FPGA, a recurrent layer built from one reused block has a latency that grows linearly with the sequence length; unrolling it over time steps removes that growth but multiplies the hardware used by the sequence length (Khoda et al., 2023).

2. Attention grows with the square of the window. Self-attention compares every element of the sequence with every other, so its cost per layer grows as n^2d , where n is the sequence length and d the size of each element’s vector (Vaswani et al., 2017); “the time and memory complexity of self-attention are quadratic in sequence length” (Dao et al., 2022). Doubling the window from 100 to 200 events doubles the work of most layers but quadruples that of attention.

3. Small batches are limited by memory, not arithmetic. A processor can only compute as fast as it is fed with data. The *roofline* model makes this precise: attainable speed is the smaller of the peak arithmetic rate and the memory bandwidth times the number of operations performed per byte loaded (Williams et al., 2009). With a batch of one, each weight loaded from memory is used for about one multiplication and one addition, so the computation is limited by memory bandwidth (Kim et al., 2023); for large language models “at small batch sizes ... the time to load weights dominates” (Pope et al., 2023). Attention is itself mostly limited by memory traffic, which FlashAttention reduces by keeping blocks of the computation in on-chip memory (Dao et al., 2022). These measurements are for large models; whether weight loading dominates for a model as small as DeepLOB, whose weights fit in on-chip memory, has not been measured. One measured case shows the batch effect directly: an LSTM on a V100 GPU processed 660 events per second at batch one, about 7,700 at batch ten and about 30,000 at batch 100 (Khoda et al., 2023).

4. Fixed overheads per operation. Each operation sent to a GPU runs as a separate program, called a *kernel*, and each kernel launch costs an overhead of the order of microseconds. In NVIDIA’s measurement a kernel that ran for 2.9 μ s cost 9.6 μ s per launch when each launch waited for the previous one, 3.8 μ s when launches overlapped, and 3.4 μ s when the sequence was recorded once as a *CUDA graph* (CUDA is NVIDIA’s GPU programming platform) and replayed (Gray, 2019); graphs pay off only when the same sequence is reused many times (NVIDIA, 2024b). A network of many small layers is many small kernels. Moving the input to the GPU and the result back crosses the PCIe bus, the link between the processor and add-in cards such as GPUs and network cards, whose bandwidth is far below the GPU’s own memory bandwidth (NVIDIA, 2024a); a single small transfer over PCIe has been measured at about half a microsecond (for a network card, not a GPU) (Neugebauer et al., 2018).

Measured latencies of order-book models. Table 53 collects the published inference times we found. They range from about 25 μ s to about 23 ms—from about 3 to about 3,000 times the roughly 8 μ s socket-to-book latency of Section 12.3.5; about half are below one millisecond and half above. They are not comparable across rows: devices and software differ, and some sources do not state the batch size. Latency also does not follow operation counts: in one study a gradient-boosted model with 23 times more structural work than another was 15 times faster (Hedges, 2026).

Table 53: Published inference times of order-book models. Each row is as reported by the source.

Source	Models and times	Conditions
Zhang et al. (2019a)	DeepLOB 0.253 ms	batch size and device for the timing not stated
Tran et al. (2019) Liu (2026)	C(TABL) 0.025, CNN 0.061, LSTM 0.229 ms 8-bit integer 1D-CNN on FI-2010: 57 μ s	one sample, Intel i7-4790 CPU FPGA (KU040), inference core only; from the abstract
Berti and Kasneci (2025) Hedges (2026)	MLP 0.08, LSTM 0.21, DeepLOB 1.31, TLOB 2.24, MLPLOB 4.79 ms median at batch one: MLPLOB 2.1 ms (CPU), 2.8 ms (GPU); TLOB 22.8 ms (CPU), 5.1 ms (GPU)	RTX 3090 GPU; batch size not stated Apple M3 Max, PyTorch, 32-bit floating point; forward pass only, no preprocessing or transfer

13.3 Audited benchmarks

The times in Table 53 come from research papers, each under its own conditions. Most published latency figures for trading hardware come from vendors. The exception is the benchmarks of STAC (the Securities Technology Analysis Center), run by an independent organisation to a published specification and audited, of which three bear on this survey (Table 54). Their public summaries are chosen by the vendor that submitted the system, and the full reports require registration, so the figures below are those summaries.

- **STAC-T0** measures network input and output only: the time from the last bit of the market-data packet needed for a decision to the first bit of the order, with trivial decision logic.
- **STAC-T1** measures a whole *tick-to-trade* path—from a market-data packet arriving to the resulting order leaving—on CME E-mini data: decode the feed, keep the state, decide, and send the order.
- **STAC-ML Markets (Inference)** measures model inference alone, for three sizes of LSTM—the smallest under 200,000 parameters, according to one vendor—and later for gradient-boosted trees. It excludes reading the data and computing the features. One of its two suites sends each window as a whole; the other, *Tacana*, slides the window by one step and allows earlier computation to be reused, which is the incremental case discussed in Section 13.4 (STAC, 2026b; Buckland and STAC, 2025).

Table 54: Audited STAC results relevant to order-book models, from STAC’s public summaries. The three benchmarks measure different paths and must not be compared with one another. ML: 99th-percentile latency, one model instance, sliding-window (Tacana) suite.

Benchmark	System	Result
STAC-T0	LDA network stack on an FPGA card (2020) (STAC, 2020)	minimum 24.2 ns; mean 27.9–29.2 ns
STAC-T0	Exablaze ExaNIC V5P (2019) (STAC, 2019a)	31 to 44 ns
STAC-T1	ADHOC appliance, AMD Versal FPGA (2024) (STAC, 2024b)	mean 115 ns; 99th percentile 140 ns
STAC-T1	Exegy Xero FPGA card (2019) (STAC, 2019b)	mean 0.552 μ s; 99th percentile 0.609 μ s
STAC-ML	NVIDIA A100 GPU (2023) (STAC, 2023b)	smallest LSTM 35.2 μ s; largest 640 μ s
STAC-ML	Myrtle VOLLO, Intel Agilex FPGAs (2023) (STAC, 2023a)	smallest LSTM 5.07 μ s; largest 31.0 μ s
STAC-ML	NVIDIA GH200 (2025) (STAC, 2025)	smallest LSTM 4.70 μ s; largest 15.8 μ s
STAC-ML	Myrtle VOLLO, AMD Versal FPGA (2026) (STAC, 2026a)	smallest LSTM below 2 μ s; largest below 8 μ s
STAC-ML	Xelera Silva, AMD Alveo V80 (2026) (STAC, 2026c)	gradient-boosted trees (different suite): at most 1.95 μ s for the two smaller models, 2.88 μ s for the largest

Three points follow. A complete tick-to-trade path on an FPGA, with simple decision logic, runs in about a tenth to half a microsecond. Inference of a small LSTM, in the sliding-window setting, now takes single-digit microseconds on both FPGAs and GPUs: in these results a GH200 GPU was slightly faster than the 2023 FPGA system and slower than the 2026 one. And no audited benchmark yet combines building the book with evaluating a learned model on it, or uses a published order-book architecture.

13.4 Stages that need acceleration

Combining the pipeline of Section 13.1, the four effects of Section 13.2 and the measurements above gives a short list of what to accelerate.

1. **Decoding and the book update**, for every model. They are regular and protocol-bound, and FPGAs handle them well (Section 13.6).
2. **The input window, incrementally**. A window of the last 100 snapshots changes by one row per event. Keeping it as a ring buffer—a fixed array in which the newest row overwrites the oldest—and updating only the new row, rather than rebuilding it, removes work that does not depend on the model. (Our observation; we found no paper that measures it.)
3. **The sequential part of recurrent models**. Either shorten the sequence, keep the recurrent state from the previous event so that only one new step is computed per event (possible when the window slides by one), or move the recurrence to hardware that runs it as a pipeline (Khoda et al., 2023).
4. **Attention over long windows**. Shorter windows, attention that reuses the keys and values of earlier events rather than recomputing them, and memory-aware kernels such as FlashAttention (Dao et al., 2022).
5. **Memory traffic and launch overhead at batch one**. Smaller, lower-precision weights: integer quantisation (storing weights as low-precision integers instead of floating-point numbers) can cut memory and latency by a factor of 4 to 8 in practice (Gholami et al., 2022; Jacob et al., 2018), and pruning plus quantisation compressed a speech LSTM twentyfold with negligible loss of accuracy (Han et al., 2017); fusing operations into fewer kernels and recording them as a CUDA graph (Gray, 2019); or keeping the whole model on the device that receives the data, to avoid PCIe transfers.

We found no published FPGA implementation of DeepLOB, TransLOB or TLOB with a measured latency. The closest are an 8-bit convolutional network for the FI-2010 data at 57 μ s on an FPGA (Liu, 2026), a state-space model designed for an FPGA from the start, reported at 247 μ s end to end on cryptocurrency order books (Popov and Huber, 2026), and the audited LSTM results of Table 54, which reach a few microseconds for small generic LSTMs.

13.5 Determinism as a risk-control property

The figures above are averages and percentiles; the variation of latency matters as well as its typical value. General-purpose processors, even with *kernel-bypass* networking (packets passed between the network card and the application without going through the operating system), introduce jitter from caches, branch prediction, interrupts and scheduling. A strategy that is profitable at its average latency can lose money if latency spikes at the moments when its signal is acted on. Deterministic, fixed-cycle execution is therefore a risk-control property, not only a speed optimisation.

13.6 Commercial FPGA products for trading

Most production hardware for trading is commercial, and most of its published figures are vendor claims rather than audited measurements. This section lists representative products and how to read their figures. Table 55 lists representative offerings with the figures their makers publish; audited results are in Section 13.3, and the STAC-T0 result of an Exegy framework on an AMD card is shown here for reference (STAC, 2024a). Two cautions apply when reading such numbers. They

measure different things—a transceiver loopback (a signal sent out and straight back through the circuit that puts bits on the wire), a trigger, a book build, a model inference—that must not be compared directly. And the AMD transceiver figure explicitly excludes protocol processing and programmable logic, so it is a lower bound on a component, not a tick-to-trade time.

Table 55: Representative commercial hardware for trading and the figures published by the vendors. Except for the STAC result, figures are vendor claims without independent audit.

Offering	What it is	Published figure
AMD Alveo UL3524 / UL3422 (AMD, 2023, 2024)	FPGA cards built for trading, 64 low-latency transceivers	under 3 ns transceiver latency, raw loopback excluding logic and protocol
Exegy (incl. Enyx, acquired 2022) (Exegy, 2022 ; STAC, 2024a)	FPGA feed handling, order entry and trigger frameworks	STAC-T0 audited trigger latency of the order of ten nanoseconds
Algo-Logic (Algo-Logic Systems and AMD, n.d.)	CME tick-to-trade system on Alveo U50, with feed handler, book and TCP order entry	under 500 ns tick-to-trade
Magmio (Magmio, n.d.)	C++ strategies compiled to FPGA by high-level synthesis	under 100 ns for triggers; under 350 ns with book building
Xelera Silva (Xelera Technologies, n.d.)	tree-ensemble and small-network inference on FPGA	about 250 ns (99th percentile) for a gradient-boosted model inline on UL3524; 1.1–1.3 μ s as card offload; audited STAC-ML: at most 1.95 μ s (Table 54)
Napatech + Xelera (Napatech, 2025)	the same inference on FPGA network cards	1.55 μ s for a 1000-tree model with 100 features

13.7 Accelerators between FPGAs and GPUs

A class of accelerators aims at the gap between FPGA determinism and GPU flexibility. This section gives three examples. LightTrader ([Yoo et al., 2023](#)), from KAIST and the chip designer Rebellions, combines an FPGA for market data and orders with dedicated AI processors and a scheduler that manages workload and power; back-tested on CME data, it reports speed-ups of about 14 times over a GPU-based and 7 times over an FPGA-based system for AI inference. Corsair ([Srivastava et al., 2025](#)) is an in-memory-computing design for large-language-model inference, built from chiplets (small dies combined in one package); it is not a trading product, but it illustrates the direction of the hardware industry. Network cards with programmable logic carry inference into the network path itself ([Napatech, 2025](#)).

13.8 Transformers on FPGAs

Mapping attention to FPGAs is an active research topic outside finance; this section summarises two designs and what they imply for order-book Transformers. FAMOUS ([Kabir et al., 2024](#)) accelerates dense multi-head attention on a data-centre FPGA, at 0.6 to 0.94 ms per attention layer for sequences of 64 elements; its comparison with a V100 GPU uses a figure taken from another paper rather than measured. ELiTeFormer ([Agostinelli et al., 2026](#)) co-designs a ternary-weight Transformer with linear attention for an adaptive system-on-chip (a chip that combines processor cores with programmable logic). Neither is a trading system, and we found no measured FPGA implementation of an order-book Transformer. Small, quantised attention models are candidates for programmable logic, with quantisation as the enabling technique ([Jacob et al., 2018](#)); full spatial-temporal Transformers remain GPU workloads.

13.9 GPUs for training, simulation and single-event inference

Graphics processors are built for data-parallel workloads: training, large-scale simulation, and models whose memory footprint or operator mix exceeds practical FPGA budgets. JAX-LOB (Frey et al., 2023) (Section 12.2) moves order-book matching onto accelerators for reinforcement-learning training. Monte Carlo simulation of order-book dynamics parallelises well across paths; one vendor demonstration reports speed-ups of 14 to 114 times over a CPU (Bennett, 2025), though for simulated paths rather than a historical matching-engine backtest. Specialised processors also accelerate training: a multi-horizon DeepLOB variant trained in about 15% of the GPU time on intelligence processing units (Zhang and Zohren, 2021). For single-event inference, GPUs used naively are slow (Section 13.2), but with vendor-optimised code and a sliding window that reuses earlier computation, a GH200 reached 4.70 μ s at the 99th percentile on the smallest audited LSTM (Table 54).

13.10 Networks and distance

Latency is not only computation. This section covers the time spent in the operating system’s network stack, in the fibre, and in network devices that compute. A peer-reviewed measurement of the latency budget of a server gives medians of 4.5 μ s through the operating system’s network stack, 946 ns with kernel bypass, and 572 ns for one small read across PCIe (Zilberman et al., 2017). The operating system’s network stack adds microseconds; kernel-bypass libraries deliver packets directly to the application (DPDK Project, n.d.). Published figures for such stacks include half-round-trip times of about 3 μ s on older 10-gigabit hardware (Solarflare Communications and Arista Networks, n.d.) and about 1 μ s for UDP (Mellanox Technologies, 2017). Physical distance sets a floor no software can remove: light in optical fibre takes about 5 μ s per kilometre (about 4.9 ns per metre) (Corning, 2013; ITU-T, 2003), which is why trading systems are colocated in the exchange’s data centre and why differences in relative speed translate into differences in trading performance (Baron et al., 2019; Budish et al., 2015).

Computation can also move into the network itself. Programmable switches have filtered a Nasdaq ITCH feed by symbol, delivering all messages of a recorded trace within 50 μ s against 300 μ s for the software baseline (Jepsen et al., 2018, 2020), and machine-learning models have been run inside switches on ITCH data (Zheng et al., 2024a). We found no measured order book or matching engine built inside a switch. At the extreme, application-specific chips fix a design in silicon, trading the reprogrammability of an FPGA for speed and power (Kuon and Rose, 2007).

13.11 Placing workloads on hardware

Table 56 gathers the preceding subsections into a qualitative guide to which hardware each workload belongs on.

The guiding rule is Occam’s razor (Principle 2) again: place a computation on the most constrained, and therefore most deterministic, hardware that still delivers the required economic performance. Moving a model to faster hardware is justified only when the reduction in latency and jitter measurably improves post-cost performance.

13.12 Hardware cannot extend signal lifetime

Simpler models have a structural advantage on deterministic hardware: fewer parameters and simpler data flow map onto fixed-point, branch-free pipelines. Acceleration can rescue part of a complex model’s value by removing engineering latency, but it cannot extend the statistical lifetime of the signal.

Table 56: A qualitative guide to workload placement. It reflects the general division of labour between platforms, not measured results.

Workload	Preferred form	plat-	Rationale
Market-data parsing, book maintenance	FPGA		regular, protocol-bound, deterministic
Linear OFI, queue imbalance, simple triggers	FPGA or CPU		few operations; easy to pipeline
Small quantised networks or tree ensembles on the critical path	FPGA		feasible when capacity is constrained by design
Pre-trade risk checks, mass cancel	FPGA		must be deterministic and next to order generation
Multi-horizon or cross-sectional deep models, full Transformers	GPU or other accelerator		memory and operator mix exceed FPGA budgets
Large-scale simulation, training, hyper-parameter search	GPU		throughput dominates
Strategy logic that changes often	CPU with kernel bypass		flexibility outweighs nanoseconds

Example 13.1 (Second-order latency). A logistic score on five OFI windows and queue imbalance takes a few tens of multiply-add operations. If the signal’s half-life is 5 ms, evaluating it in $50\ \mu\text{s}$ rather than 500 ns keeps $2^{-0.01} \approx 99.3\%$ of its value rather than essentially all of it. The difference is second-order; the binding constraint is the quality and horizon of the signal itself.

The design sequence that follows is: identify the horizon and value of the signal under idealised execution; choose the simplest model that captures that value; only then choose the hardware tier whose latency and jitter preserve it after costs.

14 Research agenda

The survey has identified places where the literature stops short of what a trader needs. They are grouped below by the part of the survey they come from.

14.1 Models

The open problems in this group concern what the models of Part II predict and how they are built.

Execution-aware objectives. Most prediction models stop at an estimate of $\mathbb{P}(\Delta m_{t,H} > 0 \mid X_t)$. The more useful objective is an estimate of $\mathbb{E}[\Pi \mid a_t, X_t]$, the expected profit of each available action, combining prediction, fill probability, queue position and adverse selection. Calibrated confidence (Edward Manoharan et al., 2026) is a step toward it: it tells an execution policy when to act.

Message-level foundation models for execution. Pretrained message-level encoders have so far been evaluated mainly on mid-price and next-message prediction (Linna et al., 2025). Fine-tuning them for fill probability and conditional mark-outs, evaluated in queue-exact replay, is an open direction.

State-space and long-context models for event streams. State-space sequence models are well suited to long, irregular event streams, yet apart from one hardware-oriented design (Popov and Huber, 2026) we found no established LOB forecaster built on them. Testing them against simple baselines on the same inputs—not against each other—is a clear next step.

Near-criticality and learned models. Estimated branching ratios of order flow are high, from about 0.7–0.8 to about one depending on method. How deep sequence models behave on near-critical, long-memory inputs, and whether Hawkes structure built into them (Section 9) improves robustness, is untested.

A parsimony audit. For each widely cited deep LOB model, re-estimate its incremental economic value and deflated performance against the best low-capacity baseline on the same data and fill model. Such a meta-study would directly test the central claim of this survey.

14.2 Execution and simulation

The open problems in this group concern how predictions become orders and how simulators measure the result (Sections 11 and 12).

Queue-aware benchmarks. Queue position should be a first-class state variable. Benchmarks should report performance conditional on queue rank, residual volume, spread and expected depletion time, and should measure the size of the queue-rank effect on fill probability and on conditional mark-outs across assets and regimes, rather than take a single model estimate as given.

End-to-end signal and execution. Most models output a forecast and leave execution to a separate rule. Models whose output is the decision itself—quote, keep, cancel, cross—and whose training objective is execution value would close the gap that this survey describes. Reinforcement learning in LOB simulators (Lalor and Swishchuk, 2025; Frey et al., 2023) is one route, ideally validated in queue-exact replay; its results must be judged against model-free benchmarks in the same simulator.

Causal rather than predictive questions. The questions that matter for execution are causal: they ask what happens to the queue and the price *if* an order is placed at a given price. Replay with a no-impact assumption answers this only for small orders. Models of the market’s response to one’s own orders, and ways to validate them without live experiments, remain open.

Generative evaluation. Generators should be evaluated by market invariants and downstream utility rather than by visual or marginal similarity alone (Nagy et al., 2025).

Cross-asset information after costs. An open question is when cross-asset information remains valuable after costs, latency, multiple testing and parameter instability.

Simulators against live fills. The simulators of Table 50 have not been compared on the same orders. Running one strategy through several of them and through paper trading, and comparing fill rates and post-fill mark-outs with the live fills, would measure how much each simulator’s assumptions—estimated queue position, simulated order flow, no market impact—cost in accuracy.

Replay with a reacting market. Replay cannot measure the market’s response to one’s own orders; a model-based simulator can, mechanically (Section 12.3.1). Combining the two—recorded flow for the rest of the market, a fitted model for the response to the strategy’s orders—and validating the response against live trading is open.

14.3 Hardware

The open problems in this group follow from Section 13.

- Training objectives that penalise expected inference latency and resource use on a target device, so that models trade accuracy against deployability explicitly.
- Which components of spatial-temporal LOB models survive mapping to programmable logic without losing their economic value.
- Hawkes intensities in hardware: exponential kernels update in constant time and suit pipelines, and a power-law kernel can be approximated by a few exponentials, each with its own running sum (Bochud and Challet, 2007); how many are needed at the precision of fixed-point hardware, and how state-dependent kernels map to pipelines, are open.
- Whether the statistical properties of a strategy developed on one hardware and latency profile survive deployment on another—the research–production gap of Section 12.3.2, at the hardware layer.
- Public, auditable benchmarks that cover book building and inference together: STAC-T1 audits a whole tick-to-trade path with simple logic, and STAC-ML audits inference alone, excluding data ingestion and features; neither evaluates a learned model on a book it has built.

15 Conclusion

This section summarises the survey’s argument, the two kinds of infrastructure needed to test it, and the order of operations it recommends to a reader who must choose a model.

The literature on limit-order-book modelling has moved from low-dimensional queue and order-flow models to expressive sequence architectures and pretrained message-level encoders. Statistical prediction has improved. Statistical prediction and executable trading remain different problems. The translation between them is

LOB state $\rightarrow$ representation $\rightarrow$ prediction $\rightarrow$ decision $\rightarrow$ queue $\rightarrow$ execution $\rightarrow$ P&L.

In words: the book is turned into model inputs, the inputs into a forecast, the forecast into an order, the order waits in a queue and may be filled, and the fill produces a profit or loss. Each link can destroy information or add uncertainty.

Order-flow imbalance, queue imbalance, Hawkes intensities and first-passage models give interpretable low-dimensional descriptions of market dynamics. Deep and pretrained models give representational capacity and, increasingly, calibrated confidence; most of them are learned analogues

of the classical signal models rather than of the mechanistic ones (Section 10.16). Meta-models and structural hybrids combine the two, either by feeding classical outputs to a learner or by building classical structure inside a network. Much of the accuracy reported for deep models depends on how the label and the data split are built: with a label that averages past prices, a re-test of DeepLOB on Apple was right 65.9% of the time and 54.6% with a label measured from the current price (Lee, 2026), and in our synthetic example a network reached 57.6% on a market with nothing to predict (Section 10.4). The choice of input matters as much: with the network fixed, order-flow inputs were among the best in 89% of cases and raw snapshots in 11% (Lucchese et al., 2024). The conclusion is not that simple models are superior. It is that complexity is an empirical hypothesis: a more complex model is justified when it delivers incremental out-of-sample economic value that survives realistic execution, temporal validation, and the multiple-testing burden of its own selection.

Two kinds of infrastructure make that standard testable. The first is the simulator. Agent-based, model-based and generative simulators let the market react to the researcher’s orders, but their queues are only as realistic as the simulated order flow; a replay keeps the real market but cannot react. Exact queue position in the recorded market needs order-level data and an order-level book; among the replay tools compared, hftbacktest (with order-level data), lobsim and Kaspar provide it without altering the recorded market (Section 12.2). In a replay built from actors, a model of Part II becomes one more actor that reads the book and sends its predictions to the strategy, and the same code runs in replay, paper and live trading (Section 12.4).

The second is hardware. Book building takes about 8 μ s from socket to book in Kaspar’s software path and hundreds of nanoseconds or less in published FPGA designs; published inference times of order-book models range from about 25 μ s to about 23 ms, about half of them above one millisecond (Table 53). Section 13.2 identifies four causes—recurrence that must run step by step, attention whose cost grows with the square of the window, memory traffic at a batch of one, and fixed overheads per operation—and Section 13.4 lists the candidate remedies: shorter or incremental windows, smaller and lower-precision models, fused kernels, and moving the regular stages to FPGAs. Hardware decides how much of a signal’s value survives to the moment of execution, but it cannot rescue a signal whose life is shorter than the pipeline that computes it.

For a reader who must choose a model, the survey’s guidance reduces to an order of operations. Decide first what the output is for—aggressive trading, passive execution, market making or simulation—and therefore which of the targets of Section 10 matters. Establish the strongest simple baseline on the best available representation: order-flow imbalance, queue imbalance, the micro-price, the queue race, and for passive orders the volume ahead. Evaluate it with proper scores and economic metrics in a queue-exact replay, counting every configuration tried. Only then add capacity—a shallow network, a sequence model, a hybrid—and keep it only if the increment survives the same test.

A useful future literature should therefore report not only whether a model predicts the market, but what information it extracts, what decision it changes, how that decision is executed, and whether the incremental value survives a properly constructed null.

Table 57: Evidence status of the main claims in this survey.

Claim	Status
Short-horizon mid changes are approximately linear in OFI, with slope inversely related to depth (Cont et al., 2014)	established
Hawkes stationarity when $\rho(\Gamma) < 1$ and $\bar{\lambda} = (I - \Gamma)^{-1}\mu$	established
Input representation can matter as much as the architecture	synthesis
Naive “touch” backtests overstate passive fills relative to queue-exact replay	synthesis; argued from price–time priority
Front-of-queue positions are worth more than back-of-queue positions, with adverse selection increasing in queue position (Moallemi and Yuan, 2017)	established in a model; size across regimes open
Hybrid inputs derived from the same stream add little to a sufficiently large network	open hypothesis
Labels that average past prices inflate reported direction accuracy (Lee, 2026)	one re-test on real data; reproduced on synthetic data here
Order-flow inputs beat raw snapshots with the network held fixed (Lucchese et al., 2024)	established in one study (10 stocks)
Published inference times of LOB models range from tens of microseconds to tens of milliseconds, above book-update times	measured in five studies under different conditions
Exact own-order queue position in the recorded market requires order-level data and an order-level book	synthesis; argued from price–time priority

A Symbols and acronyms

A.1 Symbols

Global symbols are defined in Section 1.2; the others are defined where they first appear. The table lists every symbol used in more than one place, and the main local ones, with the section that defines them.

Symbol	Meaning	Defined in
<i>Time, book and prices</i>		
$t, t_n, t-$	time; time of event n ; just before t	§1.2
n	index of book events	§1.2
H, h	forecast or fill horizon; mark-out horizon after a fill	§1.2
δ	tick size	§1.2
L, ℓ	number of levels; level index	§1
$p_{\ell,t}^b, v_{\ell,t}^b, p_{\ell,t}^a, v_{\ell,t}^a$	price and size at bid / ask level ℓ	§1
m_t, s_t	mid-price; spread	§1
$\Delta m_{t,H}$	mid change over the next H	§1.2
u	elapsed time after a reference time	§6
<i>Information, actions and outcomes</i>		
$X_t, Y_{t+H}, \hat{Y}_t$	observable information; target; prediction	§1.2
a_t, d	action; order side (+1 buy, -1 sell)	§1.2
Q_t^{ahead}, q	volume ahead of an order in its queue; a value of it	§1.2
$\tau_F, t_F, p^{\text{fill}}$	time to fill; fill time; fill price	§1.2
Fill, $\mathbb{K}_{\text{Fill}}$	event $\{\tau_F \leq H\}$; its indicator	§1.2
Π	profit per lot	§1.2
$\Pi^{\text{agg}}, \Pi^{\text{pass}}$	aggressive and passive profit	§11.1
$R_h^{\text{fill}}, R_{\text{rebate}}$	post-fill mark-out; rebate	§11.1
$C_{\text{spread}}, C_{\text{fees}}, C_{\text{impact}}$	costs of crossing	§11.1
$\text{AS}(q, h)$	adverse selection given queue position	§11.3
$V^{\text{sig}}, V^{\text{exec}}$	signal value; execution value	§11.1.1
$T_{\text{lat}}, T_{\text{move}}$	total latency; time to the next one-tick mid move	§11.4, §10
θ	move threshold in prediction targets	§10
<i>Signals</i>		
$\Delta v_n^b, \Delta v_n^a, e_n$	change in best sizes from event n ; its OFI score	§4
$w, \text{OFI}_t^{(w)}$	window length; order-flow imbalance over it	§4
β, ε_t	price-impact coefficient; residual	§4
QI_t, ι_t	queue imbalance in $[-1, 1]$; rescaled to $[0, 1]$	§4, §4.5
$m_t^{\text{wmid}}, m_t^{\text{micro}}$	weighted mid; micro-price	§4, §4.5
$\eta_k, g^{(k)}, g^*$	times of successive mid changes; expected adjustments; their sum	§4.5
n_ι, n_s	numbers of imbalance buckets and spread values	§4.5
N_{paths}	number of simulated paths in a Monte Carlo estimate	§4.5
$\mathbf{U}, \mathbf{J}^{(1)}, \mathbf{J}^{(2)}, \mathbf{B}, \mathbf{\Delta}$	micro-price transition matrices and jump sizes	§4.5
$\beta_{\kappa\kappa'}, m_t^{(\kappa)}$	cross-impact coefficient; mid of instrument κ	§4.6
<i>Queues</i>		
τ^a, τ^b	depletion time of the best ask / bid	§6
$r_{\text{in}}, r_{\text{out}}, r_{\text{out}}(x)$	rates at which a queue fills and drains; drain rate at size x	§6, §6.1
$\tau_x, \mathcal{L}_x(\varpi)$	time for a queue to go from x to $x - 1$ orders; its Laplace transform	§6.1
τ^{fill}	time until an order at the back of the best bid is filled	§6.1
r_L, r_M, r_C	limit-order, market-order and cancellation rates	§6.2
$\bar{\Theta}$	average product of new best-queue sizes after a price change	§6.2
$\bar{\ell}$	distance in ticks from the opposite best quote	§6.1

Symbol	Meaning	Defined in
$\mathcal{Q}_{\bar{\ell}}^{\text{bid}}, \mathcal{Q}_{\bar{\ell}}^{\text{ask}}$	orders at distance $\bar{\ell}$ from the opposite quote	§6.1
$r_L(\bar{\ell}), r_c(\bar{\ell}), r_0, \Xi$	limit-order rate; per-order cancellation rate; power-law scale and exponent	§6.1
$N_L(\bar{\ell}), T_{\text{obs}}$	count of queue increases; observed trading time	§6.1
ϖ	Laplace-transform variable	§6.1
x	number of orders (or units) in a queue	§6.1, §6.3
$r_L(x), r_C(x), r_M(x)$	queue-reactive rates at queue size x	§6.3
$a_k, \ell_k, v_k, \mathbf{y}_k$	type, level, size and state before event k (deep queue-reactive model)	§9.3
$\mathbf{W}, r_{a,\ell}^{\mathbf{W}}(\mathbf{y}), R^{\mathbf{W}}(\mathbf{y})$	network weights; learned rate of type a at level ℓ ; their sum	§9.3
$\hat{Y}_t^{(m)}, a_{m,t}$	forecast of component model m ; its attention weight	§8.5.1
$J_k, \bar{J}, \sigma_J, p$	size of the k -th mid move in a compound Hawkes model; its long-run average; its variance constant; probability a move repeats the previous direction	§7.11
$R(x), p^{\mathbf{W}}(v \mid \cdot)$	total queue-reactive rate; learned order-size probability	§9.3
$c(t), \bar{c}, d, h_i(t)$	memory cell of a continuous-time LSTM; its target; its decay rate; output for type i	§9.5
$\text{sgn}_i, J^{\text{mid}}$	direction of the mid move caused by event type i ; size of the move in ticks	§9.5
$Q_t, \text{Cash}_t, \varkappa$	market maker's inventory; cash; inventory penalty	§9.5
$p_{\text{ref}}, \pi_{\text{move}}, \pi_{\text{reinit}}$	reference price; probability it moves when a best queue empties; probability the book is redrawn	§6.3
<i>Point processes</i>		
$N(t), N_i(t)$	counting process; for event type i	§7
$\lambda(t), \lambda_i(t), \bar{\lambda}$	conditional intensity; for type i ; stationary mean	§7
μ, μ_i, μ_0	Poisson or background rate; for type i ; per side in the price model	§7
ϕ, ϕ_{ij}, K	kernel; between types; number of event types	§7
$\Gamma, \rho(\Gamma), I$	branching matrix (scalar: branching ratio); spectral radius; identity	§7
α, ω	exponential-kernel jump and decay rate (decay often written β elsewhere)	§7
$X(t), \pi_j(x' \mid x), \phi_{ij}(u, x)$	state of the book in a state-dependent Hawkes model; probability it moves from x to x' after a type- j event; kernel when the triggering event occurred in state x	§7.16
$N^{\pm}(t), \lambda^{\pm}(t), \delta_m, \Gamma_{\times}$	up/down mid-move counts; their intensities; mid jump; cross-excitation weight	§7
$\sigma^2(w), \Lambda$	variance per unit time at window w ; total move rate	§7
$\lambda^{L,b}, \lambda^{C,b}, \lambda^{M,b};$ $\lambda^{L,a}, \lambda^{C,a}, \lambda^{M,a}$	intensities of limit orders and cancellations at the bid and ask, and of market buys (M, b) and sells (M, a)	§7.20.2
$\bar{v}$	average event size	§7.20.2
ζ	power-law kernel tail exponent	§7
ψ_k	state of a discrete-time linear system (stability analogy)	§7
$D(w), \mathcal{H}$	Fano factor; Hurst exponent	§7.8
$\Delta t_n, \Psi_n$	gap between arrivals; running sum for the likelihood	§7.4, §7.12
$\Omega, \mathcal{A}$	sample space; collection of events with probabilities	§7.3
$\mathcal{I}_t, \mathcal{I}_{t-}$	information up to and including t ; strictly before t	§7.3
$N_w, N_{w,v}, \Upsilon$	count in a window of length w ; in window v ; number of windows	§7.1, §7.8
u_0	time scale of the power-law kernel	§7.5
$t_{\text{end}}, N_{\text{end}}$	end of the observation window; number of arrivals in it	§7.12
<i>Models and evaluation</i>		
$M, M_0, M_1, V(M), \Delta V$	model; baseline; candidate; out-of-sample value; its difference	§1.2
TO	transaction-level overlap score (called p_T by its authors)	§10.12
$\mathbf{z}_n^{\text{qry}}, \mathbf{z}_n^{\text{key}}, \mathbf{z}_n^{\text{val}}, \text{dim}_{\text{emb}}$	attention query, key and value; their dimension	§10
$\text{att}_{nn'}$	attention weight of event n on event n'	§10

Symbol	Meaning	Defined in
$\mathbf{z}^{\text{hist}}(t), \mathbf{o}_i$	history vector of a neural point process; output weights for type i	§7.19
$\text{softplus}(y)$	$\ln(1 + e^y)$, a smooth positive function; y any real number	§7.19
ν_t, S_t	gate; Hawkes-intensity input	§9
$\mathbb{P}, \mathbb{E}$	probability; expectation	§1.3
$A, B, Z, \mathbb{K}_A$	generic events and random number in the primer; indicator	§1.3
R^2	coefficient of determination	§4
$f_{n,k}, \beta_k, \varrho$	input k at observation n ; its regression weight; lasso penalty strength	§8.3
$e, \ln$	Euler’s number; natural logarithm	–

A.2 Acronyms

AIC	Akaike information criterion	LOB	limit order book
AUC	area under the ROC curve	LSTM	long short-term memory network
BIC	Bayesian information criterion	MBO / MBP	market by order / by price
CNN	convolutional neural network	MCC	Matthews correlation coefficient
CRPS	continuous ranked probability score	MDP	CME Market Data Platform
DSR	deflated Sharpe ratio	MLP	multilayer perceptron
FIFO	first in, first out	OFI	order-flow imbalance
FPGA	field-programmable gate array	OOS	out of sample
GPU	graphics processing unit	P&L	profit and loss
HFT	high-frequency trading	QI	queue imbalance
IOC / FOK	immediate-or-cancel / fill-or-kill	RNN	recurrent neural network
ITCH	Nasdaq market-by-order data protocol	SOM	simulated order manager
LAMD	LLM-assisted model discovery	SSM	state-space model
LLM	large language model	VC	Vapnik–Chervonenkis

B Glossary

Terms are grouped by subject and listed alphabetically within each group. Each entry gives a plain-language definition and, where useful, a reference for further reading. Introductory textbooks for each area are: market microstructure [Harris \(2002\)](#); [O’Hara \(1995\)](#); [Hasbrouck \(2007\)](#); futures [Hull \(2022\)](#); statistical learning [Hastie et al. \(2009\)](#); deep learning [Goodfellow et al. \(2016\)](#); point processes [Daley and Vere-Jones \(2003\)](#).

B.1 Markets and trading

Adverse selection The tendency of a passive order to be filled exactly when the price is about to move against it, because the counterparty often knows more ([Glosten and Milgrom, 1985](#)). Section 11.3.

Aggressive / passive An aggressive order trades immediately against waiting orders; a passive order waits in the book for someone else to trade with it.

Alpha The return of a strategy or signal beyond an appropriate benchmark, after the relevant risk and cost adjustments; in this survey, usually the part of a short-horizon forecast that is profitable after costs.

Backtest Running a trading strategy on historical data to estimate how it would have done.

Basis point (bp) One hundredth of a percent, 0.01%. A parameter of 300 bp is 3%.

Capacity How much a strategy can trade before its own price impact erases its profit.

Clearing fees Fees paid to the clearing house, the institution that settles trades and guarantees both sides.

Counterparty The trader on the other side of a trade.

Cross-sectional Across many instruments at the same moment, rather than over time for one.

Depth The total size waiting in the book near the best prices; a *deep* book has a lot.

Drift The average direction in which the price moves over a period, independent of one's own trades.

Edge Expected profit per trade, before or after costs as stated.

Equity / fixed income Stocks / bonds and other interest-rate products.

ETF Exchange-traded fund: a share, traded like a stock, that holds a basket of assets.

Fill The execution of an order, in full or in part.

Fragmentation The same asset trading on several venues, so its liquidity is split among them.

Futures contract A standardised exchange-traded agreement to buy or sell an asset at a set future date and price (Hull, 2022). ES, NQ and ZN are futures on the S&P 500 index, the Nasdaq-100 index and the 10-year US Treasury note.

Iceberg / hidden order An order that shows only part (iceberg) or none (hidden) of its size.

Implementation cost / slippage The difference between the price obtained and a reference price such as the mid at the moment of decision (Harris, 2002).

Index constituents The individual stocks that make up an index such as the S&P 500.

Informed trader A trader who knows something about the next price move that others do not.

Instrument One specific tradable product, for example the March ES contract.

Inventory / position The net quantity held after one's fills, long (positive) or short (negative); it carries risk until offset.

Large-tick asset An instrument whose tick is large relative to its typical price moves, so the spread is almost always one tick and queues are long.

Lifting the offer / hitting the bid Buying aggressively at the ask / selling aggressively at the bid.

Liquidity How easily one can trade without moving the price. Resting limit orders supply it; aggressive orders consume it.

Mark-out The gain or loss of a fill measured against the mid a fixed time later. Equation (69).

Market impact How much one's own trading moves the price against oneself (Bouchaud et al., 2018).

Market making Continuously posting both a bid and an ask in order to earn the spread, while managing inventory (Avellaneda and Stoikov, 2008; Cartea et al., 2015). A posted bid or ask is a *quote*.

Mass cancel A single command cancelling all of a trader's open orders.

Meta-order A large order executed as a sequence of smaller orders over time.

Micro-price An estimate of the “fair” price that adjusts the mid toward the side with the smaller queue ([Stoikov, 2018](#)).

Optimal execution Splitting a large order over time to minimise its total cost and risk ([Almgren and Chriss, 2001](#)).

P&L Profit and loss.

Paper trading Running a strategy on live data with simulated, not real, orders.

Pre-trade checks Automatic checks that block an order breaking size, price or position limits before it is sent.

Price point The contract’s own price unit. One ES point is four ticks, worth \$50 per contract.

Quote skew Shifting both a market maker’s bid and ask in the direction of expected price movement or to reduce inventory.

Rebate A payment some venues make to passive orders for supplying liquidity, funded by fees on aggressive orders.

Regime A stretch of time with stable market behaviour, such as calm or volatile.

Return The relative change in value over a period.

Session One period during which the exchange is open, usually one trading day.

Sharpe ratio Average return divided by the standard deviation of return: profit per unit of risk ([Sharpe, 1966, 1994](#)).

Spoofing / layering Placing orders one intends to cancel before they trade—one large order (spoofing) or several stacked across prices (layering)—to create a false impression of supply or demand. Illegal in US futures markets ([Commodity Futures Trading Commission, 2013](#)).

Square-root impact law The empirical regularity that the price impact of a meta-order grows roughly with the square root of its size ([Bouchaud et al., 2018](#)).

Stress testing Checking a system or strategy under extreme or rare scenarios.

The touch The current best bid and best ask.

Venue A trading platform or exchange.

Volatility How much the price fluctuates, usually the standard deviation of returns.

VWAP Volume-weighted average price: the average trade price over an interval, each trade weighted by its size ([Harris, 2002](#)).

Zero-intelligence model A model in which orders arrive at random with no strategy; the matching rules alone generate the book’s behaviour ([Smith et al., 2003](#)).

B.2 Exchanges and market data

CME Group The Chicago Mercantile Exchange group, the main US futures exchange operator; its electronic platform is CME Globex.

Event clock / calendar clock Sampling the market after a fixed number of events / a fixed number of seconds.

Feed / feed handler An exchange’s real-time stream of market-data messages / the software that decodes the feed and maintains the book.

iLink 3 CME’s order-entry protocol, used to send, modify and cancel orders (CME Group, n.d.a).

ITCH Nasdaq’s market-by-order data protocol (Nasdaq, 2015); Section 1.

L1, L2, L3 Feed detail levels: best bid and ask only; total size at each price level (market by price, MBP); every individual order (market by order, MBO).

LOBSTER A service that reconstructs Nasdaq order books from ITCH data for research (Huang and Polak, 2011).

Matching engine The exchange computer that holds the book and matches buy and sell orders.

MDP 3.0 CME’s Market Data Platform protocol: binary messages over UDP multicast (CME Group, n.d.c).

Order acknowledgement The exchange’s confirmation that it has accepted an order.

Snapshot recovery A periodic full-book message that lets a feed handler resynchronise after lost packets.

B.3 Statistics and evaluation

Ablation Removing or adding one component of a model to measure what it contributes.

Akaike / Bayesian information criterion (AIC / BIC) Scores that trade goodness of fit against the number of parameters; BIC penalises parameters more heavily in large samples (Akaike, 1974; Schwarz, 1978).

AUC Area under the ROC curve: the probability that a randomly chosen positive case receives a higher score than a randomly chosen negative one (Fawcett, 2006).

Balanced accuracy The average of the recall (true-positive rate) of each class.

Brier score The mean squared difference between predicted probabilities and 0/1 outcomes (Brier, 1950).

Calibration error A summary of how far predicted probabilities are from observed frequencies (Guo et al., 2017).

Calibration Agreement between predicted probabilities and observed frequencies: events given 70% happen about 70% of the time (Guo et al., 2017).

Coefficient of determination (R^2) The fraction of the variance of a target explained by a fitted model.

Concordance (C-index) The fraction of pairs whose predicted order of event times matches the actual order.

Confidence interval A range that contains the true value with a stated probability over repeated experiments; it can be computed by resampling (Efron and Tibshirani, 1993).

Confounding Mixing of two effects so that the cause of an observed difference cannot be isolated.

Counterfactual What would have happened under an action that was not actually taken.

CRPS Continuous ranked probability score: a distance between a predicted distribution and the realised outcome; lower is better (Gneiting and Raftery, 2007).

Decile One tenth of a sorted data set.

Deflated Sharpe ratio A Sharpe ratio corrected for the number of strategies tried and for non-normal returns (Bailey and López de Prado, 2014).

Double descent The observation that test error can fall again as model capacity grows past the point of fitting the training data exactly (Belkin et al., 2019).

Epps effect The fall in measured correlation between two assets when it is computed over very short intervals.

F1 / macro-F1 F1 is the harmonic mean of precision and recall; macro-F1 averages F1 over the classes with equal weight.

False-discovery rate The expected fraction of claimed discoveries that are false (Benjamini and Hochberg, 1995).

Granger causality A test of whether past values of one series improve forecasts of another beyond its own past.

In-sample / out-of-sample (OOS) Measured on the data used to build a model / on data it has never seen.

Interval coverage The fraction of outcomes that fall inside predicted intervals, compared with the intended (*nominal*) fraction.

Cross-validation (random folds) Splitting data into several parts, training on all but one and testing on the remaining one, in rotation (Hastie et al., 2009). Inappropriate for time-ordered market data (Section 10.5).

Lead–lag A relation in which one instrument’s price moves first and a related one follows after a short delay.

Lag A past value of a variable at a fixed offset in time.

Lasso Regression with a penalty on the sum of absolute coefficients, which sets unhelpful coefficients exactly to zero (Tibshirani, 1996).

Log loss The average of $-\ln$ of the probability a classifier gave to the true class (cross-entropy); lower is better (Hastie et al., 2009).

Log-likelihood The logarithm of the probability a model assigns to the observed data; higher means a better fit.

Matthews correlation coefficient (MCC) A classification score between -1 and 1 that remains informative when classes are unbalanced.

Model confidence set The set of models that cannot be statistically distinguished from the best, at a stated confidence level (Hansen et al., 2011).

MSE / MAE Mean squared error / mean absolute error.

Multiple testing Trying many hypotheses inflates the chance that some look good by luck (White, 2000; Harvey et al., 2016).

Non-normal returns Returns whose distribution has fatter tails or more skew than a Gaussian.

Non-stationary Having statistical properties that change over time.

Null hypothesis / significance The null is the assumption of no effect. A result is *significant at 5%* if it would occur less than 5% of the time under the null.

Principal component A direction in the data along which variance is largest; the first principal component is the best one-number summary of several correlated variables.

Probability of backtest overfitting The estimated probability that the configuration that looked best in sample performs below the median out of sample (Bailey et al., 2017).

Proper scoring rule A score that a forecaster optimises by reporting true beliefs, such as log loss, the Brier score and the CRPS (Gneiting and Raftery, 2007).

Purging / embargo Removing training samples whose information overlaps the test period / leaving a gap after each test period, both to prevent leakage of future information (López de Prado, 2018).

Rademacher complexity A measure of how well a family of models can fit random noise on the observed data; it bounds the gap between training and test error (Bartlett and Mendelson, 2002).

Reality check A resampling test of whether the best of many strategies truly beats a benchmark (White, 2000).

Risk–coverage curve The error rate plotted against the fraction of predictions kept, as the least confident are discarded (Geifman and El-Yaniv, 2017).

Selective prediction Acting only on predictions whose confidence exceeds a threshold (Geifman and El-Yaniv, 2017).

Signature plot Measured volatility plotted against the sampling interval; higher measured volatility at short intervals reveals bid–ask bounce.

Standard error / annualised The standard deviation of an estimate / scaled to a one-year period.

Survival model A model of the time until an event that correctly handles cases where the event has not yet happened (*censoring*) (Cox, 1972).

Time-rescaling test A goodness-of-fit test for point-process models: rescaling time by the fitted intensity should turn the gaps between events into independent unit-exponential variables (Brown et al., 2002).

VC dimension The largest number of points a family of classifiers can label in every possible way; a measure of capacity (Vapnik, 1998).

Walk-forward evaluation Train on a past window, test on the next, roll forward and repeat (López de Prado, 2018).

B.4 Machine learning

Attention A mechanism that weights each input element by its learned relevance to the current one (Vaswani et al., 2017).

Attentive neural process A model that predicts a distribution for a new input by attending to a *context set* of example input–outcome pairs (Kim et al., 2019).

Autoregressive model A model that generates a sequence one element at a time, each conditioned on those before it.

Axial attention Attention applied separately along each axis of a multi-dimensional input, such as time and price level.

Backbone The main feature-extracting network beneath a task-specific output layer.

BERT A Transformer encoder pretrained by predicting masked-out tokens (Devlin et al., 2019).

CNN / RNN / LSTM Convolutional networks apply learned sliding filters (LeCun et al., 2015); recurrent networks carry a hidden state along a sequence; the LSTM is a recurrent network with gates for long-range memory (Hochreiter and Schmidhuber, 1997).

Context length How many past elements a sequence model sees at once.

Cross-entropy loss The negative log-likelihood of the true class under a classifier’s predicted probabilities; the standard training loss for classification.

Data augmentation Enlarging a training set with modified or synthetic examples.

Differential evolution A derivative-free optimisation method that evolves a population of candidate parameter vectors.

Diffusion model A generative model that learns to reverse a gradual process of adding noise to data.

Embedding A learned vector representation of an input.

Encoder A network that maps an input to a learned vector representation.

Fine-tuning Further training a pretrained model on a specific task.

Foundation model A large model pretrained on broad data and then adapted to many tasks.

Generative adversarial network (GAN) A generator trained against a discriminator that tries to tell generated data from real data.

Generative model A model that produces new samples resembling its training data.

Inception module A network block that applies several convolutions of different widths in parallel and concatenates the results.

Inference Running a trained model to obtain a prediction; *inference latency* is how long that takes.

Logit / logistic regression The log-odds $\ln(\mathbb{P}(A)/(1 - \mathbb{P}(A)))$ of an event A ; logistic regression models it as a linear function of the inputs (Hastie et al., 2009).

Meta-model A model whose inputs are the outputs of other models; here, a learner that combines the outputs of classical order-book models.

Model capacity How complex a function a model can represent; roughly, its number of parameters.

Out-of-sample predictions (for stacking) Predictions of a first-stage model on data it was not fitted on; a second-stage model must be trained on these, not on in-sample predictions, or it learns from the first stage’s overfitting (Breiman, 1996).

Pretraining Training a model on a generic task before adapting it to a specific one.

Query, key, value The three vectors an attention layer computes for each element: the query is matched against every key to set the weights, and the output is the weighted average of the values.

Quantisation Using low-precision integers instead of floating point for a network’s weights and arithmetic, for speed or to fit hardware (Jacob et al., 2018).

Reinforcement learning Learning a decision policy by trial and error from rewards.

Selective state-space model A state-space model, such as Mamba, whose recurrence parameters depend on the current input (Gu and Dao, 2023).

Softmax A function that turns a vector of scores into probabilities that are positive and sum to one.

Softplus The function $\ln(1 + e^y)$: smooth, always positive, and close to y when y is large; used to keep predicted rates positive.

Stacking Training a second-stage model on the predictions of several first-stage models (Wolpert, 1992; Breiman, 1996).

State-space model A model with a hidden state updated by a (learned) linear recurrence (Kalman, 1960); recent neural versions such as Mamba handle long sequences efficiently (Gu and Dao, 2023).

Token / tokenisation The unit of input a sequence model reads / splitting data into such units.

Transformer A sequence network built from attention layers; the architecture behind large language models (Vaswani et al., 2017).

Tree ensemble Many decision trees combined, as in random forests or gradient boosting (Hastie et al., 2009).

World model A model’s internal representation of how the state of a system evolves.

B.5 Probability and stochastic processes

Birth–death process A counting process that moves up by one at a birth rate and down by one at a death rate that may depend on its current value (Cont et al., 2010).

Branching ratio In a Hawkes process, the expected number of events directly triggered by one event (the spectral radius of the branching matrix in several dimensions); below one, every cascade dies out.

Branching process A model in which each individual independently produces a random number of offspring; a Hawkes process is one (Hawkes and Oakes, 1974).

Cox–Ingersoll–Ross process A continuous-time random process that is always positive and reverts toward a mean; used for interest rates and volatility.

Diffusive limit The continuous random process that a jumpy process resembles when viewed at a large scale of time and size.

Ergodic Having a unique long-run distribution, so that averages over a long history converge to fixed values.

Expectation–maximisation (EM) An iterative fitting method for models with hidden quantities: guess the hidden quantities probabilistically from the current parameters, re-estimate the parameters, and repeat (Veen and Schoenberg, 2008).

Exogenous / endogenous Caused from outside / triggered by earlier events of the same process.

First-passage time The first time a random quantity reaches a given level.

Hawkes process A self-exciting point process in which each event temporarily raises the rate of future events (Hawkes, 1971; Bacry et al., 2015). Section 7.

Heston model A stochastic-volatility model in which variance follows a Cox–Ingersoll–Ross process.

Hurst exponent A measure of long memory: 0.5 for no memory, above 0.5 for persistence.

Infimum (inf) The greatest lower bound of a set; for a set of times, the earliest one. By convention the infimum of an empty set is $+\infty$.

Intensity The instantaneous expected rate of events.

Kalman filter A recursive estimator of a hidden state that evolves linearly and is observed with noise (Kalman, 1960).

Laplace transform For a waiting time τ , the function $\varpi \mapsto \mathbb{E}[e^{-\varpi\tau}]$; it turns sums of independent waiting times into products, which is why it is used for first-passage problems.

Markov A process whose future depends only on its present state, not on its history.

Martingale A process whose expected future value, given the present, equals its present value.

Point process A random sequence of event times (Daley and Vere-Jones, 2003).

Queue-reactive model A model of the book in which the rate of each event type at a queue depends on the current queue sizes (Huang et al., 2015).

Queueing model A probabilistic model of arrivals to and departures from queues.

Rough volatility Volatility whose paths are rougher than those of Brownian motion, with a Hurst parameter below 0.5.

Stationary Having statistical properties that do not change over time. For a Hawkes process, the cascade of triggered events does not explode.

Stochastic Random.

Thinning Simulating a point process by proposing candidate events at an upper-bound rate and keeping each with probability equal to the true intensity divided by the bound (Ogata, 1981).

Zumbach effect Past volatility over long horizons predicts future short-horizon volatility better than the reverse; a time asymmetry of volatility (Zumbach, 2009).

B.6 Systems and hardware

Actor model A model of concurrency in which independent actors own their state and communicate only by messages (Hewitt et al., 1973).

ASIC Application-specific integrated circuit: a chip designed for one task.

CGRA Coarse-grained reconfigurable array: a programmable chip whose building blocks are larger than an FPGA's, often used for AI inference.

Chiplet One of several small dies packaged together to act as a single processor.

Colocation Placing trading servers in the exchange's data centre to minimise network distance.

FPGA Field-programmable gate array: a chip whose logic can be reconfigured to implement a specific circuit; slower and larger than an ASIC for the same function but reprogrammable (Kuon and Rose, 2007). Its configurable resources are called the *fabric*.

High-level synthesis (HLS) Compiling a program written in a language such as C++ into FPGA logic.

Jitter Variation in latency from one event to the next.

Kernel bypass Receiving and sending network packets directly from user space, skipping the operating system’s network stack (DPDK Project, n.d.).

Multicast One-to-many UDP delivery: the sender transmits once and every subscribed host receives the packet.

Operator mix The kinds of computation a model needs, such as matrix multiplication or softmax.

SmartNIC A network card with programmable logic or processors that can compute on packets as they arrive.

STAC An industry body that runs audited benchmarks of trading technology.

Tick-to-trade The time from a market-data packet arriving to the resulting order leaving.

Transceiver The circuit that converts between parallel data and the serial signal on a network link.

Trigger Pre-computed order logic, often in hardware, that sends an order as soon as a market condition is detected.

B.7 Economics

Abduction Forming a new hypothesis that would explain a surprising observation, as opposed to searching among hypotheses already given (Zahavy, 2026).

Agent-based model A simulation of many artificial traders following rules and interacting.

Econometric / reduced-form model A statistical model fitted to economic data that describes relationships without modelling traders’ decisions.

Equilibrium model A theoretical model in which prices result from rational traders’ optimal strategies.

Glosten–Milgrom and Kyle models Classic models in which market makers lose to better-informed traders and set prices or spreads to recover those losses (Glosten and Milgrom, 1985; Kyle, 1985).

Latency arms race Competition among fast traders to be first, and its economic costs (Budish et al., 2015).

Price formation How trading and information are turned into the market price.

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, Savannah, GA, 2016. USENIX Association. ISBN 978-1-931971-33-1. URL <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.

Joseph Abate and Ward Whitt. Computing Laplace transforms for numerical inversion via continued fractions. *INFORMS Journal on Computing*, 11(4):394–405, 1999. doi: 10.1287/ijoc.11.4.394.

- Frédéric Abergel, Marouane Anane, Anirban Chakraborti, Aymen Jedidi, and Ioane Muni Toke. *Limit Order Books*. Physics of Society: Econophysics and Sociophysics. Cambridge University Press, 2016. ISBN 9781107163980. doi: 10.1017/CBO9781316683040.
- Eduardo Abi Jaber and Omar El Euch. Multifactor approximation of rough volatility models. *SIAM Journal on Financial Mathematics*, 10(2):309–349, 2019. doi: 10.1137/18M1170236.
- Massil Achab, Emmanuel Bacry, Stéphane Gaïffas, Iacopo Mastromatteo, and Jean-François Muzy. Uncovering causality from multivariate Hawkes integrated cumulants. *Journal of Machine Learning Research*, 18(192):1–28, 2018. URL <http://jmlr.org/papers/v18/17-284.html>.
- Victor Agostinelli, Nicolas Bohm Agostini, and Antonino Tumeo. ELiTeFormer: An efficient transformer for FPGAs, 2026. URL <https://arxiv.org/abs/2607.03652>.
- Yacine Aït-Sahalia, Jianqing Fan, Lirong Xue, and Xiaonan Zhu. How and when are high-frequency stock returns predictable? *Management Science*, 72(9):7792–7815, 2026. doi: 10.1287/mnsc.2022.02435. Earlier version: NBER Working Paper 30366 (2022), doi:10.3386/w30366.
- Hirotsugu Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974. doi: 10.1109/TAC.1974.1100705.
- Aurélien Alfonsi and Pierre Blanc. Dynamic optimal execution in a mixed-market-impact Hawkes price model. *Finance and Stochastics*, 20(1):183–218, 2016. doi: 10.1007/s00780-015-0282-y. Published online 6 Nov 2015.
- Algo-Logic Systems and AMD. AMD Alveo U50 with HLS: FPGA accelerated CME tick-to-trade (solution brief). Solution brief, n.d. URL <https://www.amd.com/content/dam/amd/en/documents/products/accelerators/alveo/u50/algo-logic-t2t-product-brief.pdf>.
- Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *The Journal of Risk*, 3(2):5–39, 2001. doi: 10.21314/JOR.2001.041.
- AMD. AMD Alveo UL3524 accelerator card: FPGA accelerator for ultra-low latency trading (product brief). Product brief, 2023. URL https://www.xilinx.com/content/dam/xilinx/publications/product-briefs/2233051_Product_Brief_UL3524_Alveo_Accelerator_Card.pdf. Product page: <https://www.amd.com/en/products/accelerators/alveo/ul3524.html>; launch press release 2023-09-27.
- AMD. AMD Alveo UL3422 accelerator card (product brief). Product brief, 2024. URL <https://www.amd.com/content/dam/amd/en/documents/products/accelerators/alveo/ul3422/alveo-ul3422-product-brief.pdf>. Launch press release 2024-10-14.
- Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetrenko, Tucker Balch, and Manuela Veloso. ABIDES-Gym: Gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF ’21)*, pages 1–9, New York, NY, USA, 2021. ACM. doi: 10.1145/3490354.3494433.
- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=gerNCVqqtR>.

- Nino Antulov-Fantulin, Tian Guo, and Fabrizio Lillo. Temporal mixture ensemble models for probabilistic forecasting of intraday cryptocurrency volume. *Decisions in Economics and Finance*, 44(2):905–940, 2021. doi: 10.1007/s10203-021-00344-9.
- Álvaro Arroyo, Álvaro Cartea, Fernando Moreno-Pino, and Stefan Zohren. Deep attentive survival analysis in limit order books: Estimating fill probabilities with convolutional-transformers. *Quantitative Finance*, 24(1):35–57, 2024. doi: 10.1080/14697688.2023.2286351.
- ATAS. Authorization window and platform version selection. <https://help.atas.net/en/support/solutions/articles/72000602319-authorization-window-and-platform-version-selection>, n.d.a. ATAS Help Center. Accessed 2026-10-04.
- ATAS. Replay trading simulator in ATAS. <https://help.atas.net/en/support/solutions/articles/72000662219-replay-trading-simulator-in-atas>, n.d.b. ATAS help center. Accessed 2026-10-04.
- Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008. doi: 10.1080/14697680701381228.
- Emmanuel Bacry and Jean-François Muzy. Hawkes model for price and trades high-frequency dynamics. *Quantitative Finance*, 14(7):1147–1166, 2014. doi: 10.1080/14697688.2014.897000.
- Emmanuel Bacry and Jean-François Muzy. First- and second-order statistics characterization of Hawkes processes and non-parametric estimation. *IEEE Transactions on Information Theory*, 62(4):2184–2202, 2016. doi: 10.1109/TIT.2016.2533397.
- Emmanuel Bacry, Sylvain Delattre, Marc Hoffmann, and Jean-François Muzy. Modelling microstructure noise with mutually exciting point processes. *Quantitative Finance*, 13(1):65–77, 2013. doi: 10.1080/14697688.2011.647054.
- Emmanuel Bacry, Iacopo Mastromatteo, and Jean-François Muzy. Hawkes processes in finance. *Market Microstructure and Liquidity*, 1(1):1550005, 2015. doi: 10.1142/S2382626615500057.
- Emmanuel Bacry, Thibault Jaisson, and Jean-François Muzy. Estimation of slowly decreasing Hawkes kernels: Application to high-frequency order book dynamics. *Quantitative Finance*, 16(8):1179–1201, 2016. doi: 10.1080/14697688.2015.1123287.
- Emmanuel Bacry, Martin Bompairé, Philip Deegan, Stéphane Gaïffas, and Søren V. Poulsen. tick: a Python library for statistical learning, with an emphasis on Hawkes processes and time-dependent models. *Journal of Machine Learning Research*, 18(214):1–5, 2018. URL <http://jmlr.org/papers/v18/17-381.html>.
- David H. Bailey and Marcos López de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014. doi: 10.3905/jpm.2014.40.5.094.
- David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. The probability of backtest overfitting. *The Journal of Computational Finance*, 20(4):39–69, 2017. doi: 10.21314/JCF.2016.322.
- Hamidreza Banderalinaeini, Mohammad Sharifkhani, and Erfan Salavati. Attention-based multi-asset order flow networks for enhanced mid-price prediction. In *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF ’25)*, pages 525–533, 2025. doi: 10.1145/3768292.3770430.

- Alisson Franchin Barbosa De Oliveira. Limit order book representation for predictive modeling: A critical survey of depth, time, and empirical evidence. SSRN Working Paper, 2026.
- Avinash Barnwal, Hari Pad Bharti, Aasim Ali, and Vishal Singh. Stacking with neural network for cryptocurrency investment, 2019. arXiv preprint.
- Matthew Baron, Jonathan Brogaard, Björn Hagströmer, and Andrei Kirilenko. Risk and return in high-frequency trading. *Journal of Financial and Quantitative Analysis*, 54(3):993–1024, 2019. doi: 10.1017/S0022109018001096.
- Barter contributors. Barter: Rust framework for live-trading, paper-trading and back-testing systems. <https://github.com/barter-rs/barter-rs>, 2026. Rust, MIT. Accessed 2026-10-03.
- Peter L. Bartlett and Shahar Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3:463–482, 2002. URL <https://jmlr.org/papers/volume3/bartlett02a/bartlett02a.pdf>.
- J. M. Bates and C. W. J. Granger. The combination of forecasts. *Operational Research Quarterly*, 20(4):451–468, 1969. doi: 10.2307/3008764. Also registered as doi:10.1057/jors.1969.103.
- Peter Belcak, Jan-Peter Calliess, and Stefan Zohren. Fast agent-based simulation framework with applications to reinforcement learning and the study of trading latency effects. In *Multi-Agent-Based Simulation XXII*, Lecture Notes in Computer Science, pages 42–56, Cham, 2022. Springer. doi: 10.1007/978-3-030-94548-0_4.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019. doi: 10.1073/pnas.1903070116.
- Zied Ben Bouallègue, Jonathan A. Weyn, Mariana C. A. Clare, Jesper Dramsch, Peter Dueben, and Matthew Chantry. Improving medium-range ensemble weather forecasts with hierarchical ensemble transformers. *Artificial Intelligence for the Earth Systems*, 3(1), 2024. doi: 10.1175/AIES-D-23-0027.1.
- Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.
- Mark J. Bennett. GPU-accelerate algorithmic trading simulations by over 100x with Numba. NVIDIA Technical Blog, March 2025. URL <https://developer.nvidia.com/blog/gpu-accelerate-algorithmic-trading-simulations-by-over-100x-with-numba/>.
- Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.
- Leonardo Berti and Gjergji Kasneci. TLOB: A novel transformer model with dual attention for price trend prediction with limit order book data, 2025.
- Leonardo Berti, Bardh Prenkaj, and Paola Velardi. TRADES: Generating realistic market simulations with diffusion models. In *ECAI 2025*, Frontiers in Artificial Intelligence and Applications, pages 3703–3710, 2025. doi: 10.3233/FAIA251249.

- Bartosz Bieganski and Robert Ślepaczuk. Explainable patterns in cryptocurrency microstructure, 2026.
- Guilherme A. Bileki, Flávio Barboza, Luiz Henrique C. Silva, and Vanderlei Bonato. Order book mid-price movement inference by CatBoost classifier from convolutional feature maps. *Applied Soft Computing*, 116:108274, 2022. doi: 10.1016/j.asoc.2021.108274.
- Paul Bilokon and Yitao Qiu. Transformers versus LSTMs for electronic trading, 2023. Preprint.
- Pierre Blanc, Jonathan Donier, and Jean-Philippe Bouchaud. Quadratic Hawkes processes for financial prices. *Quantitative Finance*, 17(2):171–188, 2017. doi: 10.1080/14697688.2016.1193215.
- Thierry Bochud and Damien Challet. Optimal approximations of power laws with exponentials: Application to volatility models with long memory. *Quantitative Finance*, 7(6):585–589, 2007. doi: 10.1080/14697680701278291.
- Hamza Bodor and Laurent Carlier. Deep learning meets queue-reactive: A framework for realistic limit order book simulation, 2025.
- Bookmap Ltd. Bookmap system requirements. <https://bookmap.com/knowledgebase/docs/KB-IntroductionToBookmap-SystemRequirements>, 2026. Bookmap Knowledge Base, last updated 27 July 2026. Accessed 2026-10-04.
- Bookmap Ltd. Bookmap knowledge base: Select run mode; export and import bookmap files; API general info. <https://bookmap.com/knowledgebase/docs/KB-GettingStarted-SelectRunMode>, n.d. Accessed 2026-10-04.
- Jean-Philippe Bouchaud, Marc Mézard, and Marc Potters. Statistical properties of stock order books: Empirical results and models. *Quantitative Finance*, 2(4):251–256, 2002. doi: 10.1088/1469-7688/2/4/301.
- Jean-Philippe Bouchaud, Yuval Gefen, Marc Potters, and Matthieu Wyart. Fluctuations and response in financial markets: The subtle nature of ‘random’ price changes. *Quantitative Finance*, 4(2):176–190, 2004. doi: 10.1080/14697680400000022.
- Jean-Philippe Bouchaud, J. Dooyne Farmer, and Fabrizio Lillo. How markets slowly digest changes in supply and demand. In *Handbook of Financial Markets: Dynamics and Evolution*, pages 57–160. Elsevier, 2009. ISBN 9780123742582. doi: 10.1016/B978-012374258-2.50006-3.
- Jean-Philippe Bouchaud, Julius Bonart, Jonathan Donier, and Martin Gould. *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press, 2018. doi: 10.1017/9781316659335.
- Victor D. Bourgin, Ioana Bica, and Mihaela van der Schaar. Model-attentive ensemble learning for sequence modeling. arXiv preprint 2102.11500, 2021.
- Andrew Boutros, Brett Grady, Mustafa Abbas, and Paul Chow. Build fast, trade fast: FPGA-based high-frequency trading using high-level synthesis. In *2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pages 1–6. IEEE, 2017. doi: 10.1109/RECONFIG.2017.8279781.

- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. <http://github.com/jax-ml/jax>, 2018. Version 0.3.13.
- Michael W. Brandt and Kenneth A. Kavajecz. Price discovery in the U.S. treasury market: The impact of orderflow and liquidity on the yield curve. *The Journal of Finance*, 59(6):2623–2654, 2004. doi: 10.1111/j.1540-6261.2004.00711.x.
- Leo Breiman. Stacked regressions. *Machine Learning*, 24(1):49–64, 1996. doi: 10.1007/BF00117832.
- Pierre Brémaud and Laurent Massoulié. Stability of nonlinear Hawkes processes. *The Annals of Probability*, 24(3):1563–1588, 1996. doi: 10.1214/aop/1065725193.
- Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.
- Antonio Briola, Jeremy Turiel, and Tomaso Aste. Deep learning modeling of the limit order book: A comparative perspective, 2020.
- Antonio Briola, Silvia Bartolucci, and Tomaso Aste. Deep limit order book forecasting: A microstructural guide. *Quantitative Finance*, 25(7):1101–1131, 2025a. doi: 10.1080/14697688.2025.2522911.
- Antonio Briola, Silvia Bartolucci, and Tomaso Aste. HLOB–information persistence and structure in limit order books. *Expert Systems with Applications*, 266:126078, 2025b. doi: 10.1016/j.eswa.2024.126078.
- Emery N. Brown, Riccardo Barbieri, Valérie Ventura, Robert E. Kass, and Loren M. Frank. The time-rescaling theorem and its application to neural spike train data analysis. *Neural Computation*, 14(2):325–346, 2002. doi: 10.1162/08997660252741149.
- Stuart Buckland and STAC. Testing the limits of machine learning inference in finance. <https://stacresearch.com/news/testing-the-limits-of-machine-learning-inference-in-finance/>, 2025. Published 2025-10-28. Accessed 2026-10-04.
- Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015. doi: 10.1093/qje/qjv027.
- Thanh Dan Bui. HAELET: A hybrid attentive ensemble learning transformer framework for high-frequency stock price forecasting. arXiv preprint 2506.13981, 2025.
- Prabir Burman, Edmond Chow, and Deborah Nolan. A cross-validatory method for dependent data. *Biometrika*, 81(2):351–358, 1994. doi: 10.1093/biomet/81.2.351.
- David Byrd and Tucker Balch. ABIDES: Agent-based interactive discrete event simulation. <https://github.com/abides-sim/abides>, n.d. Software repository, BSD-3-Clause; files config/parallel.py, scripts/parallel.sh, scripts/marketreplay.sh. Commit c4bf157, 2020-11-19. Accessed 2026-10-04.
- David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS ’20)*, pages 11–22, 2020. doi: 10.1145/3384441.3395986.

- Charles Cao, Oliver Hansch, and Xiaoxin Wang. The information content of an open limit-order book. *Journal of Futures Markets*, 29(1):16–41, 2009. doi: 10.1002/fut.20334.
- Álvaro Cartea, Sebastian Jaimungal, and Jason Ricci. Buy low, sell high: A high frequency trading perspective. *SIAM Journal on Financial Mathematics*, 5(1):415–444, 2014. doi: 10.1137/130911196.
- Álvaro Cartea, Sebastian Jaimungal, and José Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015. ISBN 978-1-107-09114-6.
- Álvaro Cartea, Sebastian Jaimungal, and Jason Ricci. Algorithmic trading, stochastic control, and mutually exciting processes. *SIAM Review*, 60(3):673–703, 2018. doi: 10.1137/18M1176968. SIGEST republication of cartea2014buylow.
- Álvaro Cartea, Sebastian Jaimungal, and Yixuan Wang. Spoofing and price manipulation in order-driven markets. *Applied Mathematical Finance*, 27(1–2):67–98, 2020. doi: 10.1080/1350486X.2020.1726783.
- Vitor Cerqueira, Luis Torgo, and Igor Mozetič. Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning*, 109(11):1997–2028, 2020. doi: 10.1007/s10994-020-05910-7.
- Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, Cambridge, 2006. doi: 10.1017/CBO9780511546921.
- Raffaele G. Cestari, Filippo Barchi, Riccardo Busetto, Daniele Marazzina, and Simone Formentin. Univariate Hawkes-based cryptocurrency forecasting via limit order book data. In *2025 European Control Conference (ECC)*, pages 1943–1948, 2025. doi: 10.23919/ECC65951.2025.11187051.
- chasemetoyer. Backtesting-engine: L3 order book replay and backtesting. <https://github.com/chasemetoyer/Backtesting-Engine>, 2026. Rust/Python, MIT. Accessed 2026-10-03.
- Feng Chen and Peter Hall. Inference for a nonstationary self-exciting point process with an application in ultra-high frequency financial data modeling. *Journal of Applied Probability*, 50(4):1006–1024, 2013. doi: 10.1239/jap/1389370096.
- Hao Chen, Nanyan Li, Mingyong Li, Zhigang Sun, Fanlin Meng, and Jinya Su. Ensemble learning with additive attention mechanism for short-term load forecasting. In *Proceedings of the 43rd Chinese Control Conference (CCC)*, pages 7113–7118. IEEE, 2024. doi: 10.23919/CCC63176.2024.10661985.
- Jiadong Chen, Xiao He, Hengyu Ye, Fuxin Jiang, Tieying Zhang, Jianjun Chen, and Xiaofeng Gao. Online ensemble transformer for accurate cloud workload forecasting in predictive auto-scaling. arXiv preprint 2508.12773, 2025.
- Junxiao Chen and Paul Glasserman. Do LLMs understand limit order book dynamics?, 2026.
- Qinkai Chen and Christian-Yann Robert. Multivariate realized volatility forecasting with graph neural network. In *Proceedings of the Third ACM International Conference on AI in Finance (ICAIF ’22)*, pages 156–164. ACM, 2022. doi: 10.1145/3533271.3561663.
- Etienne Chevalier, Yadh Hafsi, and Vathana Ly Vath. Uncovering market disorder and liquidity trends detection, 2023.

- François Chollet et al. Keras. <https://keras.io>, 2015.
- C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- Hugh L. Christensen and Robert Woodmansey. Prediction of hidden liquidity in the limit order book of GLOBEX futures. *The Journal of Trading*, 8(3):68–95, 2013. doi: 10.3905/jot.2013.8.3.068.
- Guhyuk Chung, Yongjae Lee, and Woo Chang Kim. Neural marked Hawkes process for limit order book modeling. In *Advances in Knowledge Discovery and Data Mining (PAKDD 2024)*, Lecture Notes in Computer Science, pages 197–209. Springer, 2024. doi: 10.1007/978-981-97-2259-4_15.
- Gerda Claeskens, Jan R. Magnus, Andrey L. Vasnev, and Wendun Wang. The forecast combination puzzle: A simple theoretical explanation. *International Journal of Forecasting*, 32(3):754–762, 2016. doi: 10.1016/j.ijforecast.2015.12.005.
- Simon Clinet and Yoann Potiron. Statistical inference for the doubly stochastic self-exciting process. *Bernoulli*, 24(4B), 2018. doi: 10.3150/17-BEJ966.
- CME Group. iLink binary order entry. CME Group Client Systems Wiki, n.d.a. URL <https://cmegroupclientsite.atlassian.net/wiki/display/EPICSANDBOX/iLink+Binary+Order+Entry>.
- CME Group. CME Globex matching algorithm steps and supported matching algorithms. CME Group Client Systems Wiki, n.d.b. URL <https://cmegroupclientsite.atlassian.net/wiki/spaces/EPICSANDBOX/pages/457218521>.
- CME Group. CME MDP 3.0 market data. CME Group Client Systems Wiki, n.d.c. URL <https://cmegroupclientsite.atlassian.net/wiki/spaces/EPICSANDBOX/pages/457219613/CME+MDP+3.0+Market+Data>.
- Andrea Coletta, Matteo Prata, Michele Conti, Emanuele Mercanti, Novella Bartolini, Aymeric Moulin, Svitlana Vyetenko, and Tucker Balch. Towards realistic market simulations: A generative adversarial networks approach. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF ’21)*, pages 1–9, 2021. doi: 10.1145/3490354.3494411.
- Andrea Coletta, Aymeric Moulin, Svitlana Vyetenko, and Tucker Balch. Learning to simulate realistic limit order book markets from data as a world agent. In *Proceedings of the Third ACM International Conference on AI in Finance (ICAIF ’22)*, pages 428–436, 2022. doi: 10.1145/3533271.3561753.
- Commodity Futures Trading Commission. Antidisruptive practices authority. *Federal Register*, 78(102):31890–31897, May 2013. URL <https://www.federalregister.gov/documents/2013/05/28/2013-12365/antidisruptive-practices-authority>. Interpretive guidance and policy statement; 78 FR 31890 (FR Doc. 2013-12365).
- Commodity Futures Trading Commission. CFTC charges U.K. resident Navinder Singh Sarao and his company Nav Sarao Futures Limited PLC with price manipulation and spoofing. CFTC Press Release No. 7156-15, April 2015. URL <https://www.cftc.gov/PressRoom/PressReleases/7156-15>.
- Commodity Futures Trading Commission. CFTC orders JPMorgan to pay record \$920 million for spoofing and manipulation. CFTC Press Release No. 8260-20, September 2020. URL <https://www.cftc.gov/PressRoom/PressReleases/8260-20>.

- Rama Cont and Adrien de Larrard. Order book dynamics in liquid markets: Limit theorems and diffusion approximations, 2012.
- Rama Cont and Adrien de Larrard. Price dynamics in a Markovian limit order market. *SIAM Journal on Financial Mathematics*, 4(1):1–25, 2013. doi: 10.1137/110856605.
- Rama Cont, Sasha Stoikov, and Rishi Talreja. A stochastic model for order book dynamics. *Operations Research*, 58(3):549–563, 2010. doi: 10.1287/opre.1090.0780.
- Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014. doi: 10.1093/jjfinec/nbt003.
- Rama Cont, Mihai Cucuringu, and Chao Zhang. Cross-impact of order flow imbalance in equity markets. *Quantitative Finance*, 23(10):1373–1393, 2023. doi: 10.1080/14697688.2023.2236159.
- Corning. Corning SMF-28 Ultra optical fiber product information (PI1424). Product information sheet, 2013. URL <https://www.thelightconnection.com/pdf/CorningUltra.pdf>.
- David R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):187–202, 1972. doi: 10.1111/j.2517-6161.1972.tb00899.x.
- José Da Fonseca and Riadh Zaatour. Hawkes process: Fast calibration, application to trade clustering, and diffusive limit. *Journal of Futures Markets*, 34(6):548–579, 2014. doi: 10.1002/fut.21644.
- José Da Fonseca and Riadh Zaatour. Correlation and lead–lag relationships in a Hawkes microstructure model. *Journal of Futures Markets*, 37(3):260–285, 2017. doi: 10.1002/fut.21800.
- D. J. Daley and D. Vere-Jones. *An Introduction to the Theory of Point Processes, Volume I: Elementary Theory and Methods*. Probability and Its Applications. Springer, New York, 2nd edition, 2003. ISBN 0387955410. doi: 10.1007/b97277.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 16344–16359, 2022.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10148–10167. PMLR, 2024.
- Philippe Debie, Cornelis Gardebroek, Stephan Hageboeck, Paul van Leeuwen, Lorenzo Moneta, Axel Naumann, Joost M. E. Pennings, Andres A. Trujillo-Barrera, and Marjolein E. Verhulst. Unravelling the JPMorgan spoofing case using particle physics visualization methods. *European Financial Management*, 29(1):288–326, 2023. doi: 10.1111/eufm.12353.
- Deltix. Exchange simulator. <https://web.archive.org/web/20240208125153/https://www.deltixlab.com/products/exchange-simulator/>, n.d.a. Archived vendor page (2024-02-08). Accessed 2026-10-04.
- Deltix. QuantOffice product data sheet. <http://www.deltixlab.com/wp-content/uploads/2015/08/QuantOfficeProductSheet.pdf>, n.d.b. Read via web.archive.org. Accessed 2026-10-04.
- Thomas Deschatre and Pierre Gruet. Electricity intraday price modelling with marked Hawkes processes. *Applied Mathematical Finance*, 29(4):227–260, 2022. doi: 10.1080/1350486X.2023.2180399.

- Thomas Deschatre, Pierre Gruet, and Antoine Lotz. A non-local estimator for locally stationary Hawkes processes, 2025.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.
- Han Ding, Yinheng Li, Junhao Wang, Hang Chen, Doudou Guo, and Yunbai Zhang. Large language model agent in financial trading: A survey, 2024. Journal reference on arXiv: International Conference on Computers in Management and Business 2026.
- Matthew Dixon. Sequence classification of the limit order book using recurrent neural networks. *Journal of Computational Science*, 24:277–286, 2018a. doi: 10.1016/j.jocs.2017.08.018.
- Matthew Dixon. A high-frequency trade execution model for supervised learning. *High Frequency*, 1(1):32–52, 2018b. doi: 10.1002/hf2.10016.
- Dobrislav Dobrev and Ernst Schaumburg. High-frequency cross-market trading: Model free measurement and applications. Working paper (version of october 15, 2017), Federal Reserve Board of Governors, 2017. URL <https://www.aeaweb.org/conference/2018/preliminary/paper/QdkHkBSZ>.
- R. Glen Donaldson and Mark Kamstra. Forecast combining with neural networks. *Journal of Forecasting*, 15(1):49–61, 1996. doi: 10.1002/(SICI)1099-131X(199601)15:1<49::AID-FOR604>3.0.CO;2-2.
- Yifei Dong, Fengyi Wu, Kunlin Zhang, Yilong Dai, Sanjian Zhang, Wanghao Ye, Sihan Chen, and Zhi-Qi Cheng. Large language model agents in finance: A survey bridging research, practice, and real-world deployment. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17889–17907. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.972.
- DPDK Project. DPDK: Data plane development kit. Project website and documentation, n.d. URL <https://www.dpdk.org/>. Documentation: <https://doc.dpdk.org/guides/>.
- Nan Du, Hanjun Dai, Rakshit Trivedi, Utkarsh Upadhyay, Manuel Gomez-Rodriguez, and Le Song. Recurrent marked temporal point processes: Embedding event history to vector. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1555–1564, 2016. doi: 10.1145/2939672.2939875.
- J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, and Z. Wu. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation*, 13(07):P07027, 2018. doi: 10.1088/1748-0221/13/07/P07027.
- Milan Dvorak and Jan Korenek. Low latency book handling in FPGA for high frequency trading. In *17th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS)*, pages 175–178. IEEE, 2014. doi: 10.1109/DDECS.2014.6868785.
- David Easley, Marcos López de Prado, Maureen O’Hara, and Zhibai Zhang. Microstructure in the machine age. *The Review of Financial Studies*, 34(7):3316–3363, 2021. doi: 10.1093/rfs/hhaa078.

- Derrick Gilchrist Edward Manoharan, Eljas Linna, Kestutis Baltakys, Hao Dong, and Juho Kanninen. UQ-LOB: Uncertainty-aware limit order book mid-price forecasting, 2026.
- Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman & Hall, New York, 1993. ISBN 0412042312. doi: 10.1007/978-1-4899-4541-9.
- Jared F. Egginton, Bonnie F. Van Ness, and Robert A. Van Ness. Quote stuffing. *Financial Management*, 45(3):583–608, 2016. doi: 10.1111/fima.12126.
- Zoltán Eisler, Jean-Philippe Bouchaud, and Julien Kockelkoren. The price impact of order book events: Market orders, limit orders and cancellations. *Quantitative Finance*, 12(9):1395–1419, 2012. doi: 10.1080/14697688.2010.528444.
- Omar El Euch, Masaaki Fukasawa, and Mathieu Rosenbaum. The microstructural foundations of leverage effect and rough volatility. *Finance and Stochastics*, 22(2):241–280, 2018. doi: 10.1007/s00780-018-0360-z.
- Omar El Euch, Jim Gatheral, and Mathieu Rosenbaum. Roughening Heston. *Risk*, pages 84–89, May 2019. Online at risk.net 15 Apr 2019; SSRN 3116887. Page range from Gatheral’s Baruch faculty page, not from the issue itself.
- Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.
- Graham Elliott and Allan Timmermann. Optimal forecast combination under regime switching. *International Economic Review*, 46(4):1081–1102, 2005. doi: 10.1111/j.1468-2354.2005.00361.x.
- evgerher. Hft-backtesting. <https://github.com/evgerher/hft-backtesting>, 2020. Python, Apache-2.0. Accessed 2026-10-03.
- Exegy. Exegy acquires Enyx, expands global FPGA leadership. Press release, May 2022. URL <https://www.exegy.com/exegy-acquires-enyx/>.
- Timothée Fabre and Damien Challet. Learning the spoofability of limit order books with interpretable probabilistic neural networks, 2025.
- Timothée Fabre and Ioane Muni Toke. Neural Hawkes: Non-parametric estimation in high dimension and causality analysis in cryptocurrency markets. *Quantitative Finance*, 25(5):671–698, 2025. doi: 10.1080/14697688.2025.2477673.
- Timothée Fabre and Ioane Muni Toke. High-frequency market manipulation detection with a Markov-modulated Hawkes process. *The European Journal of Finance*, 32(3):309–341, 2026. doi: 10.1080/1351847X.2026.2624485. arXiv:2502.04027.
- J. Doynne Farmer, Paolo Patelli, and Ilija I. Zovko. The predictive power of zero intelligence in financial markets. *Proceedings of the National Academy of Sciences*, 102(6):2254–2259, 2005. doi: 10.1073/pnas.0409157102.
- Alexis Fauth and Ciprian A. Tudor. Modeling first line of an order book with multivariate marked point processes, 2012. arXiv preprint (v1, 17 Nov 2012); journal publication not verified.
- Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.

- Giovanni Felici and Antonio M. Sudoso. Optimizing accuracy and diversity: a multi-task approach to forecast combinations. *Annals of Operations Research*, 362(1–3):493–520, 2026. doi: 10.1007/s10479-026-07291-x. arXiv:2310.20545.
- Vladimir Filimonov and Didier Sornette. Quantifying reflexivity in financial markets: Toward a prediction of flash crashes. *Physical Review E*, 85:056108, 2012.
- Vladimir Filimonov and Didier Sornette. Apparent criticality and calibration issues in the Hawkes self-excited point process model: Application to high-frequency financial data. *Quantitative Finance*, 15(8):1293–1314, 2015.
- Tobias Sebastian Finn. Self-attentive ensemble transformer: Representing ensemble interactions in neural networks for earth system models. arXiv preprint 2106.13924; ICML 2021 workshop “Tackling Climate Change with Machine Learning”, 2021.
- Michael J. Fleming, Bruce Mizrach, and Giang Nguyen. The microstructure of a U.S. Treasury ECN: The BrokerTec platform. *Journal of Financial Markets*, 40:2–22, 2018. doi: 10.1016/j.finmar.2017.05.004.
- Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. doi: 10.1006/jcss.1997.1504.
- Sascha Yves Frey, Kang Li, Peer Nagy, Silvia Sapora, Christopher Lu, Stefan Zohren, Jakob Foerster, and Anisoara Calinescu. JAX-LOB: A GPU-accelerated limit order book simulator to unlock large scale reinforcement learning for trading. In *Proceedings of the 4th ACM International Conference on AI in Finance (ICAIF ’23)*, pages 583–591, 2023. doi: 10.1145/3604237.3626880.
- Stefan Frey and Patrik Sandås. The impact of iceberg orders in limit order books. *Quarterly Journal of Finance*, 7(3):1750007, 2017. doi: 10.1142/S2010139217500070.
- Anningzhe Gao, Shan Dai, and Yan Hu. Mamba Hawkes process, 2024.
- Robert Garrison, Pankaj Jain, and Mark Paddrik. Cross-asset market order flow, liquidity, and price discovery. OFR Working Paper 19-04, Office of Financial Research, 2019. URL https://www.financialresearch.gov/working-papers/files/OFRwp-19_04_cross-asset-market-order-flow-liquidity-and-price-discovery.pdf.
- Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6):933–949, 2018. doi: 10.1080/14697688.2017.1393551.
- Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.
- John Geweke and Gianni Amisano. Optimal prediction pools. *Journal of Econometrics*, 164(1):130–141, 2011. doi: 10.1016/j.jeconom.2011.02.017.
- Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*, pages 291–326. Chapman and Hall/CRC, 2022. doi: 10.1201/9781003162810-13.

- Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985. doi: 10.1016/0304-405X(85)90044-3.
- Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.
- Tilmann Gneiting and Roopesh Ranjan. Combining predictive distributions. *Electronic Journal of Statistics*, 7:1747–1782, 2013. doi: 10.1214/13-EJS823.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Adaptive Computation and Machine Learning. MIT Press, 2016. ISBN 9780262035613.
- Martin D. Gould and Julius Bonart. Queue imbalance as a one-tick-ahead price predictor in a limit order book. *Market Microstructure and Liquidity*, 2(2):1650006, 2016. doi: 10.1142/S2382626616500064.
- Martin D. Gould, Mason A. Porter, Stacy Williams, Mark McDonald, Daniel J. Fenn, and Sam D. Howison. Limit order books. *Quantitative Finance*, 13(11):1709–1742, 2013. doi: 10.1080/14697688.2013.803148.
- Clive W. J. Granger and Ramu Ramanathan. Improved methods of combining forecasts. *Journal of Forecasting*, 3(2):197–204, 1984. doi: 10.1002/for.3980030207.
- Alan Gray. Getting started with CUDA graphs. NVIDIA Technical Blog, September 2019. URL <https://developer.nvidia.com/blog/cuda-graphs/>.
- Michael Grosner. tribeca: A high frequency, market making cryptocurrency trading platform. <https://github.com/michaelgrosner/tribeca>, 2018. TypeScript, ISC. Accessed 2026-10-03.
- Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 19622–19635, 2023. doi: 10.52202/075280-0861.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
- Tian Guo, Albert Bifet, and Nino Antulov-Fantulin. Bitcoin volatility forecasting with a glimpse into buy and sell orders. In *2018 IEEE International Conference on Data Mining (ICDM)*, pages 989–994, 2018. doi: 10.1109/ICDM.2018.00123.
- Yoshihiro Gytoku, Ioane Muni Toke, and Nakahiro Yoshida. Deep learning of point processes for modeling high-frequency data, 2025.
- Donatien Hainaut. A bivariate Hawkes process for interest rate modeling. *Economic Modelling*, 57: 180–196, 2016. doi: 10.1016/j.econmod.2016.04.016.

- Song Han, Junlong Kang, Huizi Mao, Yiming Hu, Xin Li, Yubin Li, Dongliang Xie, Hong Luo, Song Yao, Yu Wang, Huazhong Yang, and William J. Dally. ESE: Efficient speech recognition engine with sparse LSTM on FPGA. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)*, pages 75–84. ACM, 2017. doi: 10.1145/3020078.3021745.
- Peter R. Hansen, Asger Lunde, and James M. Nason. The model confidence set. *Econometrica*, 79(2):453–497, 2011. doi: 10.3982/ECTA5771.
- Stephen J. Hardiman and Jean-Philippe Bouchaud. Branching-ratio approximation for the self-exciting Hawkes process. *Physical Review E*, 90(6):062807, 2014. doi: 10.1103/PhysRevE.90.062807.
- Stephen J. Hardiman, Nicolas Bercot, and Jean-Philippe Bouchaud. Critical reflexivity in financial markets: A Hawkes process analysis, 2013.
- Paul G. Harrald and Mark Kamstra. Evolving artificial neural networks to combine financial forecasts. *IEEE Transactions on Evolutionary Computation*, 1(1):40–52, 1997. doi: 10.1109/4235.585891.
- Larry Harris. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, New York, 2002. ISBN 9780195144703. doi: 10.1093/oso/9780195144703.001.0001.
- Campbell R. Harvey, Yan Liu, and Heqing Zhu. . . . and the cross-section of expected returns. *The Review of Financial Studies*, 29(1):5–68, 2016. doi: 10.1093/rfs/hhv059.
- Joel Hasbrouck. Intraday price formation in U.S. equity index markets. *The Journal of Finance*, 58(6):2375–2400, 2003. doi: 10.1046/j.1540-6261.2003.00609.x.
- Joel Hasbrouck. *Empirical Market Microstructure: The Institutions, Economics, and Econometrics of Securities Trading*. Oxford University Press, New York, 2007. ISBN 9780195301649. doi: 10.1093/oso/9780195301649.001.0001.
- Joel Hasbrouck and Gideon Saar. Low-latency trading. *Journal of Financial Markets*, 16(4):646–679, 2013. doi: 10.1016/j.finmar.2013.05.003.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2nd edition, 2009. ISBN 9780387848570. doi: 10.1007/978-0-387-84858-7.
- Alan G. Hawkes. Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, 58(1):83–90, 1971. doi: 10.1093/biomet/58.1.83.
- Alan G. Hawkes and David Oakes. A cluster process representation of a self-exciting process. *Journal of Applied Probability*, 11(3):493–503, 1974.
- C. Evans Hedges. The inference-compute frontier and a latency-efficient architecture for limit order book prediction, 2026. Preprint.
- Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973. URL <https://www.ijcai.org/Proceedings/73/Papers/027B.pdf>.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

- Marc Hoffmann, Mathieu Rosenbaum, and Nakahiro Yoshida. Estimation of the lead-lag parameter from non-synchronous data. *Bernoulli*, 19(2):426–461, 2013. doi: 10.3150/11-BEJ407.
- Fengrui Hua, Hengyi Yang, Xinlei Hao, Haohan Zhang, Bokai Cao, Yiyang Qi, Jia Li, and Jian Guo. Agentic quantitative trading: A survey of workflows, systems, and evaluation, 2026.
- Ruihong Huang and Tomas Polak. LOBSTER: Limit order book reconstruction system. SSRN working paper, 2011.
- Weibing Huang, Charles-Albert Lehalle, and Mathieu Rosenbaum. Simulating and analyzing order book data: The queue-reactive model. *Journal of the American Statistical Association*, 110(509): 107–122, 2015. doi: 10.1080/01621459.2014.982278.
- John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 11th edition, 2022. ISBN 9780136939917.
- Hummingbot Foundation. Hummingbot. <https://github.com/hummingbot/hummingbot>, 2026. Python/Cython, Apache-2.0. Accessed 2026-10-03.
- Nicolas Huth and Frédéric Abergel. High frequency lead/lag relationships – empirical facts. *Journal of Empirical Finance*, 26:41–58, 2014. doi: 10.1016/j.jempfin.2014.01.003.
- Sukmin Hwang, Sungho Lee, Chanyeong Kim, Yongjae Lee, and Woo Chang Kim. Shock-biased attention: Enhancing transformer Hawkes processes with amplitude-driven temporal kernels. In *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF ’25)*, pages 915–923. ACM, 2025. doi: 10.1145/3768292.3770431.
- ITU-T. Recommendation G.114: One-way transmission time. International Telecommunication Union, May 2003. URL <https://www.itu.int/rec/T-REC-G.114-200305-I>.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713. IEEE, 2018. doi: 10.1109/CVPR.2018.00286.
- Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.
- Konark Jain, Nick Firoozye, Jonathan Kochems, and Philip Treleaven. Limit order book dynamics and order size modelling using compound Hawkes process. *Finance Research Letters*, 69:106157, 2024a. doi: 10.1016/j.frl.2024.106157.
- Konark Jain, Nick Firoozye, Jonathan Kochems, and Philip Treleaven. Limit order book simulations: A review, 2024b.
- Thibault Jaisson and Mathieu Rosenbaum. Limit theorems for nearly unstable Hawkes processes. *The Annals of Applied Probability*, 25(2), 2015. doi: 10.1214/14-AAP1005.
- Thibault Jaisson and Mathieu Rosenbaum. Rough fractional diffusions as scaling limits of nearly unstable heavy tailed Hawkes processes. *The Annals of Applied Probability*, 26(5), 2016. doi: 10.1214/15-AAP1164.

- Theo Jepsen, Masoud Moshref, Antonio Carzaniga, Nate Foster, and Robert Soulé. Packet subscriptions for programmable ASICs. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 176–183. ACM, 2018. doi: 10.1145/3286062.3286092.
- Theo Jepsen, Ali Fattaholmanan, Masoud Moshref, Nate Foster, Antonio Carzaniga, and Robert Soulé. Forwarding and routing with packet subscriptions. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, pages 282–294. ACM, 2020. doi: 10.1145/3386367.3431315.
- Joseph Jerome, Leandro Sánchez-Betancourt, Rahul Savani, and Martin Herdegen. Mbt-gym: Reinforcement learning for model-based limit order book trading. In *Proceedings of the 4th ACM International Conference on AI in Finance (ICAIF ’23)*, pages 619–627. ACM, 2023. doi: 10.1145/3604237.3626873.
- Jesse.Trade. Jesse: Algorithmic trading framework. <https://github.com/jesse-ai/jesse>, 2026. Python, MIT; live trading via a separate licensed plugin. Accessed 2026-10-03.
- Michael I. Jordan and Robert A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994. doi: 10.1162/neco.1994.6.2.181.
- Sobin Joseph and Shashi Jain. Non-parametric estimation of multi-dimensional marked Hawkes processes, 2024.
- Sobin Joseph, Lekhapriya Dheeraj Kashyap, and Shashi Jain. Shallow neural Hawkes: Non-parametric kernel estimation for Hawkes processes. *Journal of Computational Science*, 63:101754, 2022. doi: 10.1016/j.jocs.2022.101754.
- Jiwon Jung and Kiseop Lee. Attention-based reading, highlighting, and forecasting of the limit order book. *Quantitative Finance*, 25(7):1015–1027, 2025. doi: 10.1080/14697688.2025.2522914.
- Paul Jusselin. Optimal market making with persistent order flow. *SIAM Journal on Financial Mathematics*, 12(3):1150–1200, 2021. doi: 10.1137/20M1376054.
- Ehsan Kabir, Md. Arafat Kabir, Austin R. J. Downey, Jason D. Bakos, David Andrews, and Miaoqing Huang. FAMOUS: Flexible accelerator for the attention mechanism of transformer on UltraScale+ FPGAs. In *2024 International Conference on Field Programmable Technology (ICFPT)*, pages 1–2, 2024. doi: 10.1109/ICFPT64416.2024.11113430.
- R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- Maxime Kawawa-Beaudan, Srijan Sood, Kassiani Papasotiriou, Daniel Borrajo, and Manuela Veloso. TradeFM: A generative foundation model for trade-flow and market microstructure, 2026.
- Alec N. Kercheval and Yuan Zhang. Modelling high-frequency limit order book dynamics with support vector machines. *Quantitative Finance*, 15(8):1315–1329, 2015. doi: 10.1080/14697688.2015.1032546.
- Elham E. Khoda, Dylan Rankin, Rafael Teixeira de Lima, Philip Harris, Scott Hauck, Shih-Chieh Hsu, Michael Kagan, Vladimir Loncar, Chaitanya Paikara, Richa Rao, Sioni Summers, Caterina Vernieri, and Aaron Wang. Ultra-low latency recurrent neural network inference on FPGAs for physics applications with hls4ml. *Machine Learning: Science and Technology*, 4(2):025004, 2023. doi: 10.1088/2632-2153/acc0d7.

- Hyunjik Kim, Andriy Mnih, Jonathan Schwarz, Marta Garnelo, Ali Eslami, Dan Rosenbaum, Oriol Vinyals, and Yee Whye Teh. Attentive neural processes. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=SkE6PjC9KX>.
- Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, Yakun Sophia Shao, and Amir Gholami. Full stack optimization of transformer inference: a survey, 2023.
- Damian Kisiel and Denise Gorse. Axial-LOB: High-frequency trading with axial attention. In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1327–1333, 2022. doi: 10.1109/SSCI51031.2022.10022284.
- Petter N. Kolm and Nicholas Westray. Deep learning alpha signals from limit order books: Practical insights and lessons learned. Risk.net, Cutting Edge, August 2025. URL <https://www.risk.net/cutting-edge/7961982/deep-learning-alpha-signals-from-limit-order-books-practical-insights-and-lessons-learned>.
- Petter N. Kolm, Jeremy Turiel, and Nicholas Westray. Deep order flow imbalance: Extracting alpha at multiple horizons from the limit order book. *Mathematical Finance*, 33(4):1044–1081, 2023. doi: 10.1111/mafi.12413.
- Gary Koop and Dimitris Korobilis. Forecasting inflation using dynamic model averaging. *International Economic Review*, 53(3):867–886, 2012. doi: 10.1111/j.1468-2354.2012.00704.x.
- koteyevlev. Marketmakingmlalgo: Market making backtest on a full order log. <https://github.com/koteyevlev/MarketMakingMLAlgo>, 2021. Python, MIT. Accessed 2026-10-03.
- kpetridis24. lobsim: L3 limit order book replay and paper execution simulator. <https://github.com/kpetridis24/lobsim>, 2026. C++20/Python, Apache-2.0. Accessed 2026-10-03.
- Viktor Krapivensky. glass: ordered set data structure for client-side order books, 2025.
- Pankaj Kumar. Deep Hawkes process for high-frequency market making. *Journal of Banking and Financial Technology*, 8(1):11–28, 2024. doi: 10.1007/s42786-024-00049-8.
- Ian Kuon and Jonathan Rose. Measuring the gap between FPGAs and ASICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 26(2):203–215, 2007. doi: 10.1109/TCAD.2006.884574.
- Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315, 1985. doi: 10.2307/1913210.
- Luca Lalor and Anatoliy Swishchuk. Event-based limit order book simulation under a neural Hawkes process: Application in market-making. *Applied Mathematical Finance*, 32(2):128–155, 2025. doi: 10.1080/1350486X.2025.2548448.
- Jeremy Large. Measuring the resiliency of an electronic limit order book. *Journal of Financial Markets*, 10(1):1–25, 2007. doi: 10.1016/j.finmar.2006.09.001.
- Gregory Laughlin, Anthony Aguirre, and Joseph Grundfest. Information transmission between financial markets in Chicago and New York. *Financial Review*, 49(2):283–312, 2014. doi: 10.1111/fire.12036.

- Christian Leber, Benjamin Geib, and Heiner Litz. High frequency trading acceleration using FPGAs. In *2011 21st International Conference on Field Programmable Logic and Applications (FPL)*, pages 317–322, 2011. doi: 10.1109/FPL.2011.64.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.
- Eun Jung Lee, Kyong Shik Eom, and Kyung Suh Park. Microstructure-based manipulation: Strategic behavior and performance of spoofing traders. *Journal of Financial Markets*, 16(2):227–252, 2013. doi: 10.1016/j.finmar.2012.05.004.
- Kyungsub Lee. Recurrent neural network based parameter estimation of Hawkes model on high-frequency financial data. *Finance Research Letters*, 55:103922, 2023. doi: 10.1016/j.frl.2023.103922.
- Kyungsub Lee. Price predictability in limit order book with deep learning model. *Applied Economics Letters*, 33(8):1105–1111, 2026. doi: 10.1080/13504851.2024.2409330. Published online 2024.
- Kyungsub Lee and Byoung Ki Seo. Marked Hawkes process modeling of price dynamics and volatility estimation. *Journal of Empirical Finance*, 40:174–200, 2017. doi: 10.1016/j.jempfin.2016.08.004.
- Erik Lewis and George Mohler. A nonparametric EM algorithm for multiscale Hawkes processes. Preprint, dated May 9, 2011, 2011. URL http://paleo.sscnet.ucla.edu/Lewis-Molher-EM_Preprint.pdf.
- Junjie Li, Yang Liu, Weiqing Liu, Shikai Fang, Lewen Wang, Chang Xu, and Jiang Bian. MarS: A financial market simulation engine powered by generative foundation model. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yang Li and Zhi Chen. ByteGen: A tokenizer-free generative model for orderbook events in byte space, 2025.
- Yuanzhe Li, Yue Wu, Muyao Zhong, Shengcai Liu, and Peng Yang. SimLOB: Learning representations of limited order book for financial market simulation, 2024. Preprint.
- Fabrizio Lillo and J. Doyne Farmer. The long memory of the efficient market. *Studies in Nonlinear Dynamics & Econometrics*, 8(3), 2004. doi: 10.2202/1558-3708.1226.
- Fabrizio Lillo, Szabolcs Mike, and J. Doyne Farmer. Theory for long memory in supply and demand. *Physical Review E*, 71(6):066122, 2005. doi: 10.1103/PhysRevE.71.066122.
- Jessica Lin, Eamonn Keogh, Stefano Lonardi, and Bill Chiu. A symbolic representation of time series, with implications for streaming algorithms. In *Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery (DMKD '03)*, pages 2–11, 2003. doi: 10.1145/882082.882086.
- Jessica Lin, Eamonn Keogh, Li Wei, and Stefano Lonardi. Experiencing SAX: A novel symbolic representation of time series. *Data Mining and Knowledge Discovery*, 15(2):107–144, 2007. doi: 10.1007/s10618-007-0064-z.
- Eljas Linna, Kestutis Baltakys, Alexandros Iosifidis, and Juho Kanninen. LOBERT: Generative AI foundation model for limit order book messages, 2025.
- Nick Littlestone and Manfred K. Warmuth. The weighted majority algorithm. *Information and Computation*, 108(2):212–261, 1994. doi: 10.1006/inco.1994.1009.

- Siyuan Liu. Microsecond-latency, fully on-chip limit order book prediction on an edge FPGA: A lightweight INT8 1D-CNN hardware–software co-design. *IEEE Access*, 14:141192–141215, 2026. doi: 10.1109/ACCESS.2026.3732942.
- Yao Liu, Shiyang Chen, Long Ma, Guolong Yang, and Kun Wan. Real-time order book building and snapshot generating for high frequency trading on FPGA. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pages 71–76. IEEE, 2024. doi: 10.1109/ASAP61560.2024.00025.
- John W. Lockwood, Adwait Gupte, Nishit Mehta, Michaela Blott, Tom English, and Kees Vissers. A low-latency library in FPGA hardware for high-frequency trading (HFT). In *2012 IEEE 20th Annual Symposium on High-Performance Interconnects (HOTI)*, pages 9–16, 2012. doi: 10.1109/HOTI.2012.15.
- Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018. ISBN 978-1-119-48208-6.
- Xiaofei Lu and Frédéric Abergel. High-dimensional Hawkes processes for limit order books: Modelling, empirical analysis and numerical calibration. *Quantitative Finance*, 18(2):249–264, 2018. doi: 10.1080/14697688.2017.1403142.
- Lorenzo Lucchese, Mikko S. Pakkanen, and Almut E. D. Veraart. The short-term predictability of returns in order book markets: A deep learning perspective. *International Journal of Forecasting*, 40(4):1587–1621, 2024. doi: 10.1016/j.ijforecast.2024.02.001.
- Shaohui Ma and Robert Fildes. Retail sales forecasting with meta-learning. *European Journal of Operational Research*, 288(1):111–128, 2021. doi: 10.1016/j.ejor.2020.05.038.
- Vincent Maciejewski. Packets, transactions and queues: Design principles for HFT systems from a measurement study of CME market data, 2026a.
- Vincent Maciejewski. Adapting the actor model of concurrency for high-frequency trading: Synchronous message delivery (`fast_send`) and a tick-to-book latency study, 2026b.
- Vincent Maciejewski. The queue is inside the packet: Market-data latency measured one message at a time. Technical note, M2 Technologies, 2026c.
- Vincent Maciejewski. Model-free passive execution via order-level shadowing, 2026d.
- Costis Maglaras, Ciamac C. Moallemi, and Muye Wang. A deep learning approach to estimating fill probabilities in a limit order book. *Quantitative Finance*, 22(11):1989–2003, 2022. doi: 10.1080/14697688.2022.2124189.
- Magmio. Magmio: FPGA system for ultra-low latency trading. Vendor website, n.d. URL <https://www.magmio.com/product>. Accessed 2026-10-02.
- Martin Magris, Mostafa Shabani, and Alexandros Iosifidis. Bayesian bilinear neural network for predicting the mid-price dynamics in limit-order book markets. *Journal of Forecasting*, 42(6):1407–1428, 2023. doi: 10.1002/for.2955.
- Ymir Mäkinen, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Forecasting jump arrivals in stock prices: New attention-based network architecture using limit order book data. *Quantitative Finance*, 19(12):2033–2050, 2019. doi: 10.1080/14697688.2019.1634277.

- Chris Mascioli, Anri Gu, Yongzhao Wang, Mithun Chakraborty, and Michael Wellman. A financial market simulation environment for trading agents using deep reinforcement learning. In *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF '24)*, pages 117–125. ACM, 2024. doi: 10.1145/3677052.3698639.
- Hongyuan Mei and Jason Eisner. The neural Hawkes process: A neurally self-modulating multivariate point process. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 6754–6764, 2017.
- Mellanox Technologies. Accelerating electronic trading (solution brief). Solution brief, 2017. URL https://network.nvidia.com/pdf/applications/SB_HighFreq_Trading.pdf.
- Albert J. Menkveld. High frequency trading and the new market makers. *Journal of Financial Markets*, 16(4):712–740, 2013. doi: 10.1016/j.finmar.2013.06.006.
- Ciamac C. Moallemi and Kai Yuan. A model for queue position valuation in a limit order book. Working paper, Columbia Business School (initial version December 2016, revised June 2017), 2017. URL <https://moallemi.com/ciamac/papers/queue-value-2016.pdf>.
- Pablo Montero-Manso, George Athanasopoulos, Rob J. Hyndman, and Thiyanga S. Talagala. FFORMA: Feature-based forecast model averaging. *International Journal of Forecasting*, 36(1): 86–92, 2020. doi: 10.1016/j.ijforecast.2019.02.011.
- Maxime Morariu-Patrichi and Mikko S. Pakkanen. State-dependent Hawkes processes and their application to limit order book modelling. *Quantitative Finance*, 22(3):563–583, 2022. doi: 10.1080/14697688.2021.1983199.
- Gareth W. Morris, David B. Thomas, and Wayne Luk. FPGA accelerated low-latency market data feed processing. In *2009 17th IEEE Symposium on High Performance Interconnects (HOTI)*, pages 83–89. IEEE, 2009. doi: 10.1109/HOTI.2009.17.
- Luca Mucciante and Alessio Sancetta. Estimation of an order book dependent Hawkes process for large datasets. *Journal of Financial Econometrics*, 22(4):1098–1129, 2024. doi: 10.1093/jjfinec/nbad021. Published online 11 Aug 2023.
- MultiCharts, LLC. How to use your license on two computers at the same time. https://www.multicharts.com/trading-software/index.php/How_to_use_your_license_on_two_computers_at_the_same_time, 2017. MultiCharts wiki, last modified 30 May 2017. Read via web.archive.org snapshot 2018-10-08. Accessed 2026-10-04.
- MultiCharts, LLC. How to use simulated trading; data playback. https://www.multicharts.com/trading-software/index.php?title=How_to_Use_Simulated_Trading, n.d. MultiCharts wiki, read via web.archive.org snapshot 2024-09-10. Accessed 2026-10-04.
- Ioane Muni Toke. “Market making” in an order book model and its impact on the spread. In *Econophysics of Order-driven Markets*, New Economic Windows, pages 49–64. Springer, Milano, 2011. doi: 10.1007/978-88-470-1766-5_4. arXiv title: “Market making” behaviour in an order book model and its impact on the bid-ask spread.
- Ioane Muni Toke and Fabrizio Pomponio. Modelling trades-through in a limit order book using Hawkes processes. *Economics: The Open-Access, Open-Assessment E-Journal*, 6(2012-22), 2012. doi: 10.5018/economics-ejournal.ja.2012-22.

- Ioane Muni Toke and Nakahiro Yoshida. Analyzing order flows in limit order books with ratios of Cox-type intensities. *Quantitative Finance*, 20(1):81–98, 2020. doi: 10.1080/14697688.2019.1637927.
- Peer Nagy, Sascha Frey, Silvia Sapora, Kang Li, Anisoara Calinescu, Stefan Zohren, and Jakob Foerster. Generative AI for end-to-end limit order book modelling: A token-level autoregressive generative model of message flow using a deep state space network. In *Proceedings of the 4th ACM International Conference on AI in Finance (ICAIF '23)*, pages 91–99, 2023. doi: 10.1145/3604237.3626898.
- Peer Nagy, Sascha Frey, Kang Li, Bidipta Sarkar, Svitlana Vyetrenko, Stefan Zohren, Ani Calinescu, and Jakob Foerster. LOB-Bench: Benchmarking generative AI for finance – an application to limit order book data. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 45437–45460, 2025.
- Napatech. Napatech turbocharges Xelera software to deliver ultra-low latency AI inference for financial trading. Press release, May 2025. URL <https://www.napatech.com/media/press-releases/napatech-turbocharges-xelera-software-to-deliver-ultra-low-latency-ai-inference-for-financial-trading/>.
- Nasdaq. Nasdaq TotalView-ITCH 5.0 interface specification, 2015. URL https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCHSpecification_5.0.pdf. Version 5.0, 03/06/2015.
- Nautech Systems. NautilusTrader. https://github.com/nautechsystems/nautilus_trader, 2018. LGPL-3.0. Documentation at <https://nautilustrader.io/docs/>. Accessed 2026-10-03 (develop, v2.0.0rc6).
- Rolf Neugebauer, Gianni Antichi, José Fernando Zazo, Yury Audzevich, Sergio López-Buedo, and Andrew W. Moore. Understanding PCIe performance for end host networking. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*, pages 327–341. ACM, 2018. doi: 10.1145/3230543.3230560.
- Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pages 625–632, 2005. doi: 10.1145/1102351.1102430.
- NinjaTrader, LLC. NinjaTrader 8 help guide: Playback connection; simulator. <https://ninjatrader.com/support/helpGuides/nt8/playback.htm>, n.d. Read via web.archive.org 2025 snapshot. Accessed 2026-10-03.
- nkaz001. HftBacktest: High-frequency trading and market-making backtesting and trading bot. <https://github.com/nkaz001/hftbacktest>, 2022. MIT License. Accessed 2026-10-03 (commit 5f3ec40).
- Paraskevi Nousi, Avraam Tsantekidis, Nikolaos Passalis, Adamantios Ntakaris, Juho Kannianen, Anastasios Tefas, Moncef Gabbouj, and Alexandros Iosifidis. Machine learning for forecasting mid-price movements using limit order book data. *IEEE Access*, 7:64722–64736, 2019. doi: 10.1109/ACCESS.2019.2916793.
- Adamantios Ntakaris and Gbenga Ibikunle. Minimal batch adaptive learning policy engine for real-time mid-price forecasting in high-frequency trading, 2024. Preprint.

- Adamantios Ntakaris, Martin Magris, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018. doi: 10.1002/for.2543.
- Adamantios Ntakaris, Giorgio Mirone, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Feature engineering for mid-price prediction with deep learning. *IEEE Access*, 7:82390–82412, 2019. doi: 10.1109/ACCESS.2019.2924353.
- Adamantios Ntakaris, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Mid-price prediction based on machine learning methods with technical and quantitative indicators. *PLOS ONE*, 15(6):e0234107, 2020. doi: 10.1371/journal.pone.0234107.
- NVIDIA. *CUDA C++ Best Practices Guide, Release 12.4*. NVIDIA Corporation, 2024a. URL <https://docs.nvidia.com/cuda/archive/12.4.0/cuda-c-best-practices-guide/index.html>.
- NVIDIA. *CUDA C++ Programming Guide, Release 12.4, Section 3.2.8.7 (CUDA Graphs)*. NVIDIA Corporation, 2024b. URL <https://docs.nvidia.com/cuda/archive/12.4.0/cuda-c-programming-guide/index.html>.
- Yosihiko Ogata. On Lewis’ simulation method for point processes. *IEEE Transactions on Information Theory*, 27(1):23–31, 1981. doi: 10.1109/TIT.1981.1056305.
- Yosihiko Ogata. Statistical models for earthquake occurrences and residual analysis for point processes. *Journal of the American Statistical Association*, 83(401):9–27, 1988. doi: 10.1080/01621459.1988.10478560.
- Maureen O’Hara. *Market Microstructure Theory*. Blackwell Publishers, Cambridge, MA, 1995. ISBN 9781557864437.
- Takahiro Omi, Yoshito Hirata, and Kazuyuki Aihara. Hawkes process model with a time-dependent background rate and its application to high-frequency financial data. *Physical Review E*, 96(1):012303, 2017. doi: 10.1103/PhysRevE.96.012303.
- Takahiro Omi, Naonori Ueda, and Kazuyuki Aihara. Fully neural network based model for general temporal point processes. In *Advances in Neural Information Processing Systems 32*, 2019.
- T. Ozaki. Maximum likelihood estimation of Hawkes’ self-exciting point processes. *Annals of the Institute of Statistical Mathematics*, 31(1):145–155, 1979.
- Deepan Palguna and Ilya Pollak. Mid-price prediction in a limit order book. *IEEE Journal of Selected Topics in Signal Processing*, 10(6):1083–1092, 2016. doi: 10.1109/JSTSP.2016.2593060.
- Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3760–3765, 2020a. doi: 10.1109/TNNLS.2019.2944933.
- Nikolaos Passalis, Anastasios Tefas, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Temporal logistic neural bag-of-features for financial time series forecasting leveraging limit order book data. *Pattern Recognition Letters*, 136:183–189, 2020b. doi: 10.1016/j.patrec.2020.06.006.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner,

- Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.
- Dhruvit Patel and Alexander Wikner. Attention-based ensemble pooling for time series forecasting. arXiv preprint 2310.16231, 2023.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85): 2825–2830, 2011. URL <http://jmlr.org/papers/v12/pedregosa11a.html>.
- John C. Platt. Probabilities for SV machines. In Alexander J. Smola, Peter L. Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, MA, 2000. doi: 10.7551/mitpress/1113.003.0008. Preprint (1999): “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods”.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems 5 (MLSys 2023)*, pages 606–624, 2023.
- Andrei Popov and Martina Huber. Scaling temporal feature extraction for high-frequency crypto trading via hardware-aware state space design. *Computer Science Bulletin*, 9(1):177–187, 2026. doi: 10.71465/csb211.
- Matteo Prata, Giuseppe Masi, Leonardo Berti, Viviana Arrigoni, Andrea Coletta, Irene Cannistraci, Svitlana Vyetenko, Paola Velardi, and Novella Bartolini. LOB-based deep learning models for stock price trend prediction: A benchmark study. *Artificial Intelligence Review*, 57(5), 2024. doi: 10.1007/s10462-024-10715-4.
- QuantConnect. LEAN algorithmic trading engine. <https://github.com/QuantConnect/Lean>, 2026. C#/Python, Apache-2.0. Accessed 2026-10-03.
- Jeff Racine. Consistent cross-validatory model-selection for dependent data: hv-block cross-validation. *Journal of Econometrics*, 99(1):39–61, 2000. doi: 10.1016/S0304-4076(00)00030-0.
- Davide Raffaelli, Raffaele Giuseppe Cestari, Daniele Marazzina, and Simone Formentin. Forecasting Bitcoin price movements using multivariate Hawkes processes and limit order book data. *Decisions in Economics and Finance*, 2026. doi: 10.1007/s10203-026-00570-z. Online 17 April 2026.
- Adrian E. Raftery, Miroslav Kárný, and Pavel Ettler. Online prediction under model uncertainty via dynamic model averaging: Application to a cold rolling mill. *Technometrics*, 52(1):52–66, 2010. doi: 10.1198/TECH.2009.08104.
- Eghbal Rahimikia and Ser-Huang Poon. Machine learning for realised volatility forecasting. SSRN Working Paper 3707796, 2020.

- Eghbal Rahimikia and Ser-Huang Poon. Alternative data for realised volatility forecasting: Limit order book and news stories. *International Journal of Finance & Economics*, 2026. doi: 10.1002/ijfe.70239. Online 8 June 2026; volume/pages not yet assigned in Crossref.
- Eghbal Rahimikia, Hao Ni, and Weiguan Wang. Re(visiting) time series foundation models in finance, 2025.
- Marcello Rambaldi, Paris Pennesi, and Fabrizio Lillo. Modeling foreign exchange market activity around macroeconomic news: Hawkes-process approach. *Physical Review E*, 91(1):012819, 2015. doi: 10.1103/PhysRevE.91.012819.
- Marcello Rambaldi, Emmanuel Bacry, and Fabrizio Lillo. The role of volume in order book dynamics: A multivariate Hawkes process analysis. *Quantitative Finance*, 17(7):999–1020, 2017. doi: 10.1080/14697688.2016.1260759.
- Marcello Rambaldi, Vladimir Filimonov, and Fabrizio Lillo. Detection of intensity bursts using Hawkes processes: An application to high-frequency financial data. *Physical Review E*, 97(3):032318, 2018. doi: 10.1103/PhysRevE.97.032318.
- Eduardo Ramos-Pérez, Pablo J. Alonso-González, and José Javier Núñez-Velázquez. Forecasting volatility with a stacked model based on a hybridized artificial neural network. *Expert Systems with Applications*, 129:1–9, 2019. doi: 10.1016/j.eswa.2019.03.046. arXiv:2006.16383.
- Roopesh Ranjan and Tilmann Gneiting. Combining probability forecasts. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(1):71–91, 2010. doi: 10.1111/j.1467-9868.2009.00726.x.
- Jakob Gulddahl Rasmussen. Lecture notes: Temporal point processes and the conditional intensity function, 2018.
- Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, Marin Biloš, Sahil Garg, Anderson Schneider, Nicolas Chapados, Alexandre Drouin, Valentina Zantedeschi, Yuriy Nevmyvaka, and Irina Rish. Lag-Llama: Towards foundation models for probabilistic time series forecasting, 2023.
- Carl Remlinger, Clémence Alasseur, Marie Brière, and Joseph Mikael. Expert aggregation for financial forecasting. *The Journal of Finance and Data Science*, 9:100108, 2023. doi: 10.1016/j.jfds.2023.100108.
- Kylie-Anne Richards, William T. M. Dunsmuir, and Gareth W. Peters. Score test for marks in Hawkes processes. *International Journal of Data Science and Analytics*, 20(3):3037–3052, 2025. doi: 10.1007/s41060-024-00644-4. Published online 10 October 2024.
- Erik Rigtorp. Estimating order queue position. <https://rigtorp.se/2013/06/08/estimating-order-queue-position.html>, 2013. Blog post, dated 2013-06-08.
- François Roueff and Rainer von Sachs. Time-frequency analysis of locally stationary Hawkes processes. *Bernoulli*, 25(2), 2019. doi: 10.3150/18-BEJ1023.
- Jonathan Sadighian. Deep reinforcement learning in cryptocurrency market making, 2019.

- Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978. doi: 10.1214/aos/1176344136.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, 2016. doi: 10.18653/v1/P16-1162.
- Marcus A. K. September, Francesco Sanna Passino, Leonie Goldmann, and Anton Hinel. Extended deep adaptive input normalization for preprocessing time series data for neural networks. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238 of *Proceedings of Machine Learning Research*, pages 1891–1899, 2024.
- Emmanouil Sfendourakis and Ioane Muni Toke. LOB modeling using Hawkes processes with a state-dependent factor. *Market Microstructure and Liquidity*, 6(01n04):2050014, 2020. doi: 10.1142/S2382626620500148.
- Mostafa Shabani, Dat Thanh Tran, Martin Magris, Juho Kannianen, and Alexandros Iosifidis. Multi-head temporal attention-augmented bilinear network for financial time series prediction. In *2022 30th European Signal Processing Conference (EUSIPCO)*, pages 1487–1491. IEEE, 2022. doi: 10.23919/EUSIPCO55093.2022.9909957.
- Mostafa Shabani, Dat Thanh Tran, Juho Kannianen, and Alexandros Iosifidis. Augmented bilinear network for incremental multi-stock time-series classification. *Pattern Recognition*, 141:109604, 2023. doi: 10.1016/j.patcog.2023.109604.
- William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1, Part 2):119–138, 1966. doi: 10.1086/294846.
- William F. Sharpe. The Sharpe ratio. *The Journal of Portfolio Management*, 21(1):49–58, 1994. doi: 10.3905/jpm.1994.409501.
- Oleksandr Shchur, Marin Biloš, and Stephan Günnemann. Intensity-free learning of temporal point processes. In *International Conference on Learning Representations (ICLR)*, 2020.
- Oleksandr Shchur, Ali Caner Türkmen, Tim Januschowski, and Stephan Günnemann. Neural temporal point processes: A review. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)*, pages 4585–4593, 2021. doi: 10.24963/ijcai.2021/623.
- Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-MoE: Billion-scale time series foundation models with mixture of experts. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yu Shi, Zongliang Fu, Shuo Chen, Bohan Zhao, Wei Xu, Changshui Zhang, and Jian Li. Kronos: A foundation model for the language of financial markets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 25366–25373, 2026. doi: 10.1609/aaai.v40i30.39730. Vol. 40, no. 30.
- Zijian Shi and John Cartlidge. State dependent parallel neural Hawkes process for limit order book event stream prediction and simulation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’22)*, pages 1607–1615, 2022. doi: 10.1145/3534678.3539462.

- Zijian Shi, Yu Chen, and John Cartlidge. The LOB recreation model: Predicting the limit order book from TAQ history using an ordinary differential equation recurrent neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 548–556, 2021. doi: 10.1609/aaai.v35i1.16133.
- Sierra Chart. Purchase information: Licensing / number of systems. <https://www.sierrachart.com/index.php?page=doc/PurchaseInformation.php>, n.d.a. Sierra Chart documentation. Accessed 2026-10-04.
- Sierra Chart. Replaying charts. <https://www.sierrachart.com/index.php?page=doc/ReplayChart.html>, n.d.b. Accessed 2026-10-04.
- Justin Sirignano and Rama Cont. Universal features of price formation in financial markets: Perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019. doi: 10.1080/14697688.2019.1622295.
- Justin A. Sirignano. Deep learning for limit order books. *Quantitative Finance*, 19(4):549–570, 2019. doi: 10.1080/14697688.2018.1546053.
- Eric Smith, J. Doyne Farmer, László Gillemot, and Supriya Krishnamurthy. Statistical theory of the continuous double auction. *Quantitative Finance*, 3(6):481–514, 2003. doi: 10.1088/1469-7688/3/6/307.
- Solarflare Communications and Arista Networks. 10Gb Ethernet: The foundation for low-latency, real-time financial services applications and other, latency-sensitive applications. Solution brief (Solarflare/Arista joint paper), n.d. URL https://www.arista.com/assets/data/pdf/JointPapers/Arista_Solarflare_Low_Latency_10GbE.pdf.
- Trent Spears, Stefan Zohren, and Stephen Roberts. Investment sizing with deep learning prediction uncertainties for high-frequency Eurodollar futures trading. *The Journal of Financial Data Science*, 3(1):57–73, 2021. doi: 10.3905/jfds.2020.1.049.
- Thomas Spooner, John Fearnley, Rahul Savani, and Andreas Koukorinis. Market making via reinforcement learning. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS 2018)*, pages 434–442, Stockholm, Sweden, 2018. IFAAMAS.
- Satyam Srivastava, Akhil Arunkumar, Nithesh Kurella, Amrit Panda, Gaurav Jain, Purushotham Kamath, Mark Wutzke, Arun Tiruvur, Mayank Mike Gupta, Ilya Soloveychik, Vamsi Darsi, Malav Dalal, Vinayak Patankar, Sasidhar Dudyala, Senthil Duraisamy, Santhosh Ramchandran, Raghav Venkatasubramanian, Yuwei Qin, Xin Wang, Jayaprakash Balachandran, Ali Murat Gok, Piotr Wojciechowski, Saliya Ekanayake, Chris Ng, Ranju Sarma, Shubhankit Rathore, Tristan Trouwen, Siwei Zhuang, Chris Nicol, and Sudeep Bhoja. Corsair: An in-memory computing chiplet architecture for inference-time compute acceleration. *IEEE Micro*, 45(5):30–42, 2025. doi: 10.1109/MM.2025.3593444.
- STAC. Stac reports: Exab blaze under stac-t0 (tick-to-trade network i/o) and stac-ts.pse (sut id exa181106). STAC news post (Internet Archive copy), audited STAC-T0, 2019a. URL <https://web.archive.org/web/20191021141855/https://stacresearch.com/news/2019/10/08/EXA181106>. Posted October 7, 2019. Accessed 2026-10-04.

- STAC. Stac report: Cme tick-to-trade with exegy xero fpga card (sut id exg191010). STAC news post, audited STAC-T1, 2019b. URL <https://docs.stacresearch.com/news/EXG191010>. Posted October 15, 2019. Accessed 2026-10-04.
- STAC. Stac report: New lda/xilinx solution under stac-t0 (tick-to-trade network i/o) (sut id xlx200514). STAC news post, audited STAC-T0, 2020. URL <https://docs.stacresearch.com/news/XLX200514>. Posted May 25, 2020. Accessed 2026-10-04.
- STAC. STAC report: STAC-ML Tacana suite on Myrtle.ai VOLLO (SUT ID MRTL230426). <https://stacresearch.com/news/mrtl230426/>, 2023a. Audited, Tacana suite. Accessed 2026-10-04.
- STAC. STAC reports: STAC-ML on an NVIDIA A100 in a Supermicro server (SUT ID NVDA221118). <https://stacresearch.com/news/nvda221118/>, 2023b. Audited; SUT pages <https://docs.stacresearch.com/NVDA221118a>, [NVDA221118b](https://docs.stacresearch.com/NVDA221118b), [NVDA221118c](https://docs.stacresearch.com/NVDA221118c). Accessed 2026-10-04.
- STAC. STAC report: New STAC-T0 results with an Exegy/AMD FPGA solution. STAC Research news, 2024a. URL <https://docs.stacresearch.com/news/AMD240422>.
- STAC. Stac report: Stac-t1 with adhoc teknoloji hfft-02a (versal premium) based ultra low latency hft solution (sut id adhc240918). STAC news post, audited STAC-T1, 2024b. URL <https://docs.stacresearch.com/news/ADHC240918>. Posted October 16, 2024. Accessed 2026-10-04.
- STAC. STAC-ML markets (inference) on a Supermicro ARS-111GL-NHR with NVIDIA GH200 Grace Hopper superchip (SUT ID SMC250910). <https://docs.stacresearch.com/SMC250910>, 2025. Audited. Accessed 2026-10-04.
- STAC. STAC-ML pack for myrtle.ai VOLLO (rev c) with Silicom Artena (AMD VP1802 FPGA) on Supermicro AS-2015CS-TNR (SUT ID MRTL260323). <https://docs.stacresearch.com/MRTL260323>, 2026a. Audited, Tacana suite. Accessed 2026-10-04.
- STAC. STAC-ML working group. <https://docs.stacresearch.com/ml>, 2026b. Accessed 2026-10-04.
- STAC. STAC-ML pack for Xelera Silva with AMD Alveo V80 on an HPE ProLiant DL385 Gen10 plus v2 server (SUT ID XLRA260312). <https://docs.stacresearch.com/XLRA260312>, 2026c. Audited, GBT (El Popo) suite. Accessed 2026-10-04.
- James H. Stock and Mark W. Watson. Combination forecasts of output growth in a seven-country data set. *Journal of Forecasting*, 23(6):405–430, 2004. doi: 10.1002/for.928.
- Sasha Stoikov. The micro-price: A high-frequency estimator of future prices. *Quantitative Finance*, 18(12):1959–1966, 2018. doi: 10.1080/14697688.2018.1489139.
- Anatoliy Swishchuk and Aiden Huffman. General compound Hawkes processes in limit order books. *Risks*, 8(1):28, 2020. doi: 10.3390/risks8010028.
- Anatoliy Swishchuk, Bruno Remillard, Robert Elliott, and Jonathan Chavez-Casillas. Compound Hawkes processes in limit order books. In *Financial Mathematics, Volatility and Covariance Modelling*, pages 191–214. Routledge, 2019. doi: 10.4324/9781315162737-9.

- Sabera Talukder, Yisong Yue, and Georgia Gkioxari. TOTEM: Tokenized time series EMbeddings for general time series analysis. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=Q1TLkH6xRC>.
- Daigo Tashiro, Hiroyasu Matsushima, Kiyoshi Izumi, and Hiroki Sakaji. Encoding of high-frequency order information and prediction of short-term stock price by deep learning. *Quantitative Finance*, 19(9):1499–1506, 2019. doi: 10.1080/14697688.2019.1622314.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.
- Robert J. Tibshirani and Bradley Efron. Pre-validation and inference in microarrays. *Statistical Applications in Genetics and Molecular Biology*, 1(1):Article 1, 2002. doi: 10.2202/1544-6115.1000.
- Kai Ming Ting and Ian H. Witten. Issues in stacked generalization. *Journal of Artificial Intelligence Research*, 10:271–289, 1999. doi: 10.1613/jair.594.
- Mehdi Tomas, Iacopo Mastromatteo, and Michael Benzaquen. How to build a cross-impact model from first principles: Theoretical requirements and empirical results. *Quantitative Finance*, 22(6): 1017–1036, 2022. doi: 10.1080/14697688.2021.2020328.
- Trading Technologies International, Inc. TT Backtesting: Getting started. <https://library.tradingtechnologies.com/tt-backtesting/backtesting-algos/getting-started-3/>, n.d.a. TT Help Library. Accessed 2026-10-04.
- Trading Technologies International, Inc. TT Backtesting: Running a backtest. <https://library.tradingtechnologies.com/tt-backtesting/backtesting-algos/running-a-backtest/>, n.d.b. TT Help Library. Accessed 2026-10-04.
- Trading Technologies International, Inc. Position in queue (PIQ) overview. <https://library.tradingtechnologies.com/trade/viewing-market-data/position-in-queue-piq/position-in-queue-piq/>, n.d.c. Accessed 2026-10-03.
- Trading Technologies International, Inc. Changing trading environments. <https://library.tradingtechnologies.com/trade/overview/workspace-windows/task-workspace-windows/changing-trading-environments/>, n.d.d. TT Help Library. Accessed 2026-10-04.
- Dat Thanh Tran, Alexandros Iosifidis, Juho Kannianen, and Moncef Gabbouj. Temporal attention-augmented bilinear network for financial time-series data analysis. *IEEE Transactions on Neural Networks and Learning Systems*, 30(5):1407–1418, 2019. doi: 10.1109/TNNLS.2018.2869225.
- Dat Thanh Tran, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Data normalization for bilinear structures in high-frequency financial time-series. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 7287–7292. IEEE, 2021a. doi: 10.1109/ICPR48806.2021.9412547.
- Dat Thanh Tran, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Bilinear input normalization for neural networks in financial forecasting, 2021b. Preprint; extended version of the ICPR 2020 BiN paper.
- Abhinava Tripathi, Vipul, and Alok Dixit. Limit order books: A systematic review of literature. *Qualitative Research in Financial Markets*, 12(4):505–541, 2020. doi: 10.1108/QRFM-07-2019-0080.

- Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis. Forecasting stock prices from the limit order book using convolutional neural networks. In *2017 IEEE 19th Conference on Business Informatics (CBI)*, pages 7–12, 2017a. doi: 10.1109/CBI.2017.23.
- Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis. Using deep learning to detect price change indications in financial markets. In *2017 25th European Signal Processing Conference (EUSIPCO)*, pages 2511–2515, 2017b. doi: 10.23919/EUSIPCO.2017.8081663.
- Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanninen, Moncef Gabbouj, and Alexandros Iosifidis. Using deep learning for price prediction by exploiting stationary limit order book features. *Applied Soft Computing*, 93:106401, 2020. doi: 10.1016/j.asoc.2020.106401.
- Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017.
- Vladimir N. Vapnik. *Statistical Learning Theory*. Adaptive and Learning Systems for Signal Processing, Communications, and Control. Wiley, 1998. ISBN 978-0-471-03003-4.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5998–6008, 2017.
- Alejandro Veen and Frederic P. Schoenberg. Estimation of space–time branching process models in seismology using an EM-type algorithm. *Journal of the American Statistical Association*, 103(482):614–624, 2008. doi: 10.1198/016214508000000148.
- Ekaterina Vinkovskaya. *A Point Process Model for the Dynamics of Limit Order Books*. PhD thesis, Columbia University, 2014.
- Svitlana Vyetenko and Shaojie Xu. Risk-sensitive compact decision trees for autonomous execution in presence of simulated market response, 2019.
- James Wallbridge. Transformers for limit order books, 2020.
- Haochuan Wang. Exploring microstructural dynamics in cryptocurrency limit order books: Better inputs matter more than stacking another hidden layer, 2025.
- Ting Wang, Mark Bebbington, and David Harte. Markov-modulated Hawkes process with stepwise decay. *Annals of the Institute of Statistical Mathematics*, 64(3):521–544, 2012. doi: 10.1007/s10463-010-0320-7.
- Zhuohan Wang and Carmine Ventre. DiffVolume: Diffusion models for volume generation in limit order books. In *Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF ’25)*, pages 587–595, 2025. doi: 10.1145/3768292.3770413.
- Zhuohan Wang and Carmine Ventre. DiffLOB: Diffusion models for counterfactual generation in limit order books. In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-26)*, pages 6564–6572, 2026. doi: 10.24963/ijcai.2026/730.
- Alexander Wehrli, Spencer Wheatley, and Didier Sornette. Scale-, time- and asset-dependence of Hawkes process estimates on high frequency price changes. *Quantitative Finance*, 21(5):729–752, 2021. doi: 10.1080/14697688.2020.1838602.

- Spencer Wheatley, Alexander Wehrli, and Didier Sornette. The endo–exo problem in high frequency financial price fluctuations and rejecting criticality. *Quantitative Finance*, 19(7):1165–1178, 2019. doi: 10.1080/14697688.2018.1550266.
- Aaron Wheeler and Jeffrey D. Varner. MarketGPT: Developing a pre-trained transformer (GPT) for modeling financial time series, 2024.
- Halbert White. A reality check for data snooping. *Econometrica*, 68(5):1097–1126, 2000. doi: 10.1111/1468-0262.00152.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.
- David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 53140–53164. PMLR, 2024.
- Peng Wu, Marcello Rambaldi, Jean-François Muzy, and Emmanuel Bacry. Queue-reactive Hawkes models for the order flow, 2019.
- Yufei Wu, Mahmoud Mahfouz, Daniele Magazzeni, and Manuela Veloso. Towards robust representation of limit orders books for deep learning models, 2021.
- Xelera Technologies. Xelera Silva: Low-latency inference for gradient-boosted trees and neural networks. Product page, n.d. URL <https://www.xelera.io/products/silva>. Accessed 2026-10-02.
- Yue Xiao, Carmine Ventre, Yuhang Wang, Haochen Li, Yuxi Huan, and Buhong Liu. LiT: Limit order book transformer. *Frontiers in Artificial Intelligence*, 8:1616485, 2025. doi: 10.3389/frai.2025.1616485.
- Ke Xu, Martin D. Gould, and Sam D. Howison. Multi-level order-flow imbalance in a limit order book. *Market Microstructure and Liquidity*, 4(3–4):1950011, 2018. doi: 10.1142/S2382626619500114. Issue cover-dated 2018 per Crossref; published online 8 January 2020; arXiv July 2019.
- Vaddarapu Karthik Yadav, Vallabhaneni Nagarjuna, and Anisha M. Lal. When not to trade: Hawkes intensity burst detection for regime-aware limit order book prediction. In *Artificial Intelligence Applications and Innovations (AIAI 2026)*, IFIP Advances in Information and Communication Technology, pages 223–236. Springer, 2026. doi: 10.1007/978-3-032-30809-2_16.
- Jiahao Yang, Ran Fang, Ming Zhang, and Jun Zhou. An efficient deep learning model to predict stock price movement based on limit order book, 2025a. Preprint.
- Mingyu Yang, Woo Chang Kim, Gukyuk Chung, and Chanyeong Kim. Type-based representation of limit order book events for predicting mid-price change. SSRN working paper 5269651, 2025b. <https://ssrn.com/abstract=5269651>.

- Yingxiang Yang, Jalal Etesami, Niao He, and Negar Kiyavash. Online learning for multivariate Hawkes processes. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Curran Associates, Inc., 2017. URL <https://papers.nips.cc/paper/2017/hash/92a0e7a415d64ebafcb16a8ca817cde4-Abstract.html>.
- Hanxue Yao and Irena Koprinska. Attention score-based dynamic ensemble for time series forecasting with an importance greedy diversity model selection. *Information Fusion*, 137:104656, 2027. doi: 10.1016/j.inffus.2026.104656. Crossref issue date January 2027.
- Sungyeob Yoo, Hyunsung Kim, Jinseok Kim, Sunghyun Park, Joo-Young Kim, and Jinwook Oh. LightTrader: A standalone high-frequency trading system with deep learning inference accelerators and proactive scheduler. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1017–1030, 2023. doi: 10.1109/HPCA56546.2023.10070930.
- Runyao Yu, Ruochen Wu, Yongsheng Han, and Jochen L. Cremer. Orderbook feature learning and asymmetric generalization in intraday electricity markets, 2025. Accepted to PSCC 2026 (per arXiv comment).
- Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 694–699, 2002. doi: 10.1145/775047.775151.
- Tom Zahavy. LLMs can’t jump. Position paper (preprint), Google DeepMind, 2026. URL <https://www.tomzahavy.com/files/llms-cant-jump.pdf>.
- Ilia Zaznov, Julian Kunkel, Alfonso Dufour, and Atta Badii. Predicting stock price changes based on the limit order book: A survey. *Mathematics*, 10(8):1234, 2022. doi: 10.3390/math10081234.
- Qiang Zhang, Aldo Lipani, Omer Kirnap, and Emine Yilmaz. Self-attentive Hawkes process. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 11183–11193, 2020.
- Ting Zhang, Chi Xie, and Gang-Jin Wang. Do order imbalances predict intraday returns? new evidence from the Chinese stock market. *Asia-Pacific Journal of Accounting & Economics*, pages 1–15, 2025. doi: 10.1080/16081625.2025.2604824. Published online 19 December 2025.
- Yanzhi Zhang, Yu Ma, Yilin Cheng, Jian Li, and Yitong Duan. M3: A state-event generative foundation model for market microstructure dynamics, 2026.
- Zihao Zhang and Stefan Zohren. Multi-horizon forecasting for limit order books: Novel deep learning approaches and hardware acceleration using intelligent processing units, 2021.
- Zihao Zhang, Stefan Zohren, and Stephen Roberts. BDLOB: Bayesian deep convolutional neural networks for limit order books, 2018. Third Workshop on Bayesian Deep Learning, NeurIPS 2018.
- Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019a. doi: 10.1109/TSP.2019.2907260.
- Zihao Zhang, Stefan Zohren, and Stephen Roberts. Extending deep learning models for limit order books to quantile regression, 2019b. Time Series Workshop, ICML 2019.

- Zihao Zhang, Bryan Lim, and Stefan Zohren. Deep learning for market by order data. *Applied Mathematical Finance*, 28(1):79–95, 2021. doi: 10.1080/1350486X.2021.1967767.
- Shi Zhao and Ying Feng. For2For: Learning to forecast from forecasts, 2020. arXiv preprint.
- Ban Zheng, Eric Moulines, and Frédéric Abergel. Price jump prediction in a limit order book. *Journal of Mathematical Finance*, 3(2):242–255, 2013. doi: 10.4236/jmf.2013.32024.
- Changgang Zheng, Mingyuan Zang, Xinpeng Hong, Liam Perreault, Riyad Bensoussane, Shay Vargaftik, Yaniv Ben-Itzhak, and Noa Zilberman. Planter: Rapid prototyping of in-network machine learning inference. *ACM SIGCOMM Computer Communication Review*, 54(1):2–21, 2024a. doi: 10.1145/3687230.3687232.
- Zetao Zheng, Guoan Li, Deqiang Ouyang, Decui Liang, and Jie Shao. Limit order book event stream prediction with diffusion model, 2024b. Journal version: Zheng, Li, Liang and Shao, Data Science and Engineering 11(3):706–723, 2026, doi:10.1007/s41019-025-00328-4 (not read).
- Jinfeng Zhong, Emmanuel Bacry, Agathe Guilloux, and Jean-François Muzy. KANFormer for predicting fill probabilities via survival analysis in limit order books, 2025a.
- Muyao Zhong, Yushi Lin, and Peng Yang. Representation learning of limit order book: A comprehensive study and benchmarking, 2025b. Preprint.
- Noa Zilberman, Matthew Grosvenor, Diana Andreea Popescu, Neelakandan Manihatty-Bojan, Gianni Antichi, Marcin Wójcik, and Andrew W. Moore. Where has my time gone? In *Passive and Active Measurement (PAM 2017)*, Lecture Notes in Computer Science, pages 201–214. Springer, 2017. doi: 10.1007/978-3-319-54328-4_15.
- Gilles Zumbach. Time reversal invariance in finance. *Quantitative Finance*, 9(5):505–515, 2009. doi: 10.1080/14697680802616712.
- Simiao Zuo, Haoming Jiang, Zichong Li, Tuo Zhao, and Hongyuan Zha. Transformer Hawkes process. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 11692–11702, 2020.